

УНИВЕРСАЛЬНОЕ ПОСОБИЕ ПО АЛГОРИТМАМ И СТРУКТУРАМ
ПОЛНЫЙ ИСХОДНЫЙ КОД ПРОЕКТА (ВСЕ ФАЙЛЫ, ВСЕ

Дата генерации: 12.09.2026, 18:46:33
Файлов исходного кода: 83
Суммарный объём кода: 1.18 МВ
Глав/билетов в пособии: 30

ОГЛАВЛЕНИЕ: ГЛАВЫ И БИЛЕТЫ

1. 1. Дерево отрезков с операциями на отрезках [id: segment-tree]
2. 2. Двумерная разреженная матрица (Sparse Table)
3. 3. Декартово дерево (Treap) [id: treap]
4. 4. Splay-дерево [id: splay-tree]
5. 5. Двумерная матрица префиксных сумм [id: prefix-sums]
6. Планарность: K5, K3,3 и формула Эйлера [id: planarity]
7. 6. Графы. Представление, DFS, BFS [id: graph-basics]
8. 7. Графы. Планарные. Покраска [id: graph-planarity]
9. 8. Графы. Компоненты связности [id: graph-components]
10. 9. Графы. Топологическая сортировка [id: topological-sort]
11. 10. Графы. Компоненты сильной связности (SCC)
12. 11. Графы. Мосты [id: graph-bridges]
13. 12. Графы. Точки сочленения [id: graph-articulation-points]
14. 13. Графы. Эйлеров цикл [id: graph-euler]
15. Билет 14. Дейкстра [id: dijkstra]
16. Билет 15. Форд-Беллман [id: bellman-ford]
17. Билет 16. Флойд-Уоршелл [id: floyd]
18. 17. Графы. Ускорения поиска [id: pathfinding-optimizations]
19. 17. Графы. Алгоритм Джонсона (Фокус с перевзвешиванием)
20. 18. Графы. Остовное дерево. Краскал [id: mst-kruskal]
21. 19. Графы. Остовное дерево. Прима [id: mst-prims]
22. 20. Графы. Остовное дерево. Борувка [id: mst-boruvka]
23. 21. Алгоритм Кнута-Морриса-Пратта (KMP) [id: kmp]
24. 22. Z-функция [id: string-z-function]
25. Билет 23. Алгоритм Ахо-Корасик [id: aho-corasick]

Билеты (учебный контент)

- 23. src/data/tickets/ticket1_5.ts
- 24. src/data/tickets/ticket14_20.ts
- 25. src/data/tickets/ticket21_24.ts
- 26. src/data/tickets/ticket6_13.ts

Интерактивные компоненты и симуляторы

- 27. src/components/ArticulationPointsViz.tsx
- 28. src/components/BellmanFordViz.tsx
- 29. src/components/BfsViz.tsx
- 30. src/components/ChapterImage.tsx
- 31. src/components/ChapterNav.tsx
- 32. src/components/ChapterView.tsx
- 33. src/components/DfsBridgesSimulator.tsx
- 34. src/components/DijkstraViz.tsx
- 35. src/components/DPVisualizer.tsx
- 36. src/components/EulerSimulator.tsx
- 37. src/components/ExpressQuiz.tsx
- 38. src/components/FloydViz.tsx
- 39. src/components/GraphTraversalViz.tsx
- 40. src/components/HeapViz.tsx
- 41. src/components/hints/demoKit.tsx
- 42. src/components/hints/demosExtra.tsx
- 43. src/components/hints/demosGraph.tsx
- 44. src/components/hints/demosStruct.tsx
- 45. src/components/hints/MiniDemo.tsx
- 46. src/components/hints/SimulatorModal.tsx
- 47. src/components/hints/TermPopover.tsx
- 48. src/components/JohnsonViz.tsx
- 49. src/components/KosarajuViz.tsx
- 50. src/components/KruskalSimulator.tsx
- 51. src/components/MnemonicCards.tsx
- 52. src/components/MobileToc.tsx
- 53. src/components/Navbar.tsx
- 54. src/components/PlanarityDemo.tsx
- 55. src/components/QueueViz.tsx
- 56. src/components/SalmonAutomatonWidget.tsx
- 57. src/components/SegmentTreeVisualizer.tsx

РАЗДЕЛ: КОНФИГУРАЦИЯ ПРОЕКТА

ФАЙЛ 1/83: add.md

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 1224 | РАЗМЕР:

Изменённые файлы

```text

```
A .github/workflows/deploy-pages.yml
M .github/workflows/rebuild-pdf.yml
M index.html
M public/export/full_code_all_pages.pdf
M public/export/full_code_all_pages.txt
A public/favicon.svg
M src/App.tsx
M src/components/Navbar.tsx
A src/components/PythonCompiler.tsx
M vite.config.ts
```
```

` .github/workflows/deploy-pages.yml`

Полное содержимое после изменений

```yaml

name: Deploy app to GitHub Pages

on:

push:

branches:

- main

# Позволяет проверить Pages прямо из рабочей ветки

- "arena/\*\*"

workflow dispatch:

```
 path: ./dist
```

```
 deploy:
```

```
 name: Publish to GitHub Pages
```

```
 needs: build
```

```
 runs-on: ubuntu-latest
```

```
 environment:
```

```
 name: github-pages
```

```
 url: ${ steps.deployment.outputs.page_url }
```

```
 steps:
```

```
 - name: Deploy
```

```
 id: deployment
```

```
 uses: actions/deploy-pages@v5
```

```
....
```

```
Patch относительно `main`
```

```
````diff
```

```
diff --git a/.github/workflows/deploy-pages.yml b/.github/workflows/deploy-pages.yml
```

```
new file mode 100644
```

```
index 00000000..17418e3
```

```
--- /dev/null
```

```
+++ b/.github/workflows/deploy-pages.yml
```

```
@@ -0,0 +1,60 @@
```

```
+name: Deploy app to GitHub Pages
```

```
+
```

```
+on:
```

```
+  push:
```

```
+    branches:
```

```
+      - main
```

```
+      # Позволяет проверить Pages прямо из рабочей ветки
```

```
+      - "arena/**"
```

```
+  workflow_dispatch:
```

```
+
```

```
+permissions:
```

```
+  contents: read
```

```
+  pages: write
```

```
+  id-token: write
```

Универсальное пособие • полный исходный код

```
-----
+   runs-on: ubuntu-latest
+   environment:
+     name: github-pages
+     url: ${ steps.deployment.outputs.page_url }
+   steps:
+     - name: Deploy
+       id: deployment
+       uses: actions/deploy-pages@v5
+....
```

`.github/workflows/rebuild-pdf.yml`

Полное содержимое после изменений

```yaml

name: Rebuild code PDF

# Пайплайн пересборки PDF при каждом коммите:

```
#
исходный код → full_code_all_pages.txt (script)
|
→ page-0001.png ... page-NNNN.png (
|
→ full_code_all_page
#
```

# Готовые TXT и PDF коммитятся обратно в ветку (их отда  
# промежуточные PNG сохраняются как artifacts запуска.

on:

push:

branches:

- "\*\*"

paths-ignore:

# чтобы коммит самого workflow не запускал бескон

- "public/export/\*\*"

- "\*\*/\*.md"

workflow\_dispatch:

inputs:

- ```
    for policy in /etc/ImageMagick-6/policy.xml /
      if [ -f "$policy" ]; then
        sudo sed -i 's/rights="none" pattern="\{P
      fi
    done
    convert -version
```
- name: Install dependencies
run: npm ci --no-audit --no-fund
 - name: Step 1 – код → txt
run: npm run export:txt
 - name: Step 2 – txt → png
env:
 EXPORT_DENSITY: \${github.event.inputs.density}
run: npm run export:png
 - name: Step 3 – png → pdf
run: npm run export:pdf
 - name: Type-check приложения
run: npx tsc --noEmit
continue-on-error: true
 - name: Summary
run: |
 {
 echo "### Экспорт пересобран"
 echo ""
 echo "| Артефакт | Размер |"
 echo "| --- | --- |"
 echo "| \`public/export/full\_code\_all\_pages`"
 echo "| \`public/export/full\_code\_all\_pages`"
 echo "| PNG-страниц | \$(ls tmp/export-pages) |"
 } >> "\$GITHUB_STEP_SUMMARY"
 - name: Upload artifacts (PDF + TXT)

```
    steps:
      - name: Checkout
      - uses: actions/checkout@v4
      + uses: actions/checkout@v6
        with:
          fetch-depth: 0
          persist-credentials: true

      - name: Setup Node.js
      - uses: actions/setup-node@v4
      + uses: actions/setup-node@v7
        with:
          - node-version: "22"
          + node-version: "24"
            cache: npm

      - name: Install ImageMagick + DejaVu fonts
@@ -97,7 +97,7 @@ jobs:
        } >> "$GITHUB_STEP_SUMMARY"

      - name: Upload artifacts (PDF + TXT)
      - uses: actions/upload-artifact@v4
      + uses: actions/upload-artifact@v7
        with:
          name: full-code-export
          path: |
@@ -107,7 +107,7 @@ jobs:
          retention-days: 30

      - name: Upload artifacts (PNG-страницы)
      - uses: actions/upload-artifact@v4
      + uses: actions/upload-artifact@v7
        with:
          name: full-code-pages-png
          path: tmp/export-pages/*.png
    ....
```

Полное содержимое после изменений

````xml

```
<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 64 64"
 <rect width="64" height="64" rx="16" fill="#4f46e5"/>
 <path d="M18 17h28v8H26v8h16v8H26v8h20v8H18z" fill="#4f46e5"/>
 <circle cx="47" cy="21" r="4" fill="#34d399"/>
</svg>
```

### Patch относительно `main`

````diff

```
diff --git a/public/favicon.svg b/public/favicon.svg
new file mode 100644
index 00000000..3bbbba0
--- /dev/null
+++ b/public/favicon.svg
@@ -0,0 +1,5 @@
+<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 64 64"
+  <rect width="64" height="64" rx="16" fill="#4f46e5"/>
+  <path d="M18 17h28v8H26v8h16v8H26v8h20v8H18z" fill="#4f46e5"/>
+  <circle cx="47" cy="21" r="4" fill="#34d399"/>
+</svg>
```

`src/App.tsx`

Полное содержимое после изменений

````tsx

```
import { useCallback, useEffect, useMemo, useState } from 'react';
import { chapters } from '../data/content';
import { Navbar } from '../components/Navbar';
import { Sidebar } from '../components/Sidebar';
import { MobileToc } from '../components/MobileToc';
import { ChapterView } from '../components/ChapterView';
```



```

return (
 <div className="min-h-screen bg-slate-950 text-slate-50">
 <Navbar activeTab={activeTab} setActiveTab={setActiveTab}>
 {activeTab === "guide" ? (
 <>
 <MobileToc
 selectedChapterId={activeChapterId}
 setSelectedChapterId={goTo}
 currentTitle={activeChapter.title}
 index={index}
 total={chapters.length}
 />

 <div className="flex flex-col lg:flex-row max-w-7xl mx-auto" >
 {/* Сайдбар только на десктопе – на мобильном скрывается */}
 <div className="hidden lg:block lg:w-80 shrink-0" >
 <Sidebar selectedChapterId={activeChapterId} />
 </div>

 <main id="main-content" className="flex-1 min-w-0" >
 {/* Шапка главы с быстрыми переходами */}
 <div className="flex items-center justify-between" >
 <div className="min-w-0" >
 <div className="text-[10px] uppercase" >
 {activeChapter.category || "Универсальное"}
 </div>
 <h2 className="text-base sm:text-xl font-medium" >
 {activeChapter.title}
 </h2>
 </div>
 <ChapterNav prev={prev} next={next} index={index} />
 </div>

 <div className="h-1 w-full bg-slate-800 rounded" >
 <div
 className="h-full bg-gradient-to-r from-slate-800 to-slate-700"
 style={{ width: `${progress}%` }}
 />
 </div>
 </main>
 </div>
 </>
) : null}
 </Navbar>
 </div>
)

```

-----  
````  

Patch относительно `main`

````diff

diff --git a/src/App.tsx b/src/App.tsx

index 2091f33..4583756 100644

--- a/src/App.tsx

+++ b/src/App.tsx

@@ -7,9 +7,10 @@ import { ChapterView } from "../components/ChapterView";

import { ChapterNav } from "../components/ChapterNav";

import { SimulatorModal } from "../components/hints/SimulatorModal";

import { SimulatorHub } from "../components/SimulatorHub";

+import { PythonCompiler } from "../components/PythonCompiler";

export default function App() {

- const [activeTab, setActiveTab] = useState<"guide" | "simulator">();

+ const [activeTab, setActiveTab] = useState<"guide" | "simulator">();

const [activeChapterId, setActiveChapterId] = useState<string>();

const [modalVizId, setModalVizId] = useState<string>();

@@ -95,7 +96,7 @@ export default function App() {

</main>

</div>

</>

- ) : (

+ ) : activeTab === "simulator" ? (

<main className="px-4 sm:px-6 lg:px-10 py-6 lg:py-8">

<SimulatorHub

onOpen={setModalVizId}

@@ -105,6 +106,8 @@ export default function App() {

}}

/>

</main>

+ ) : (

+ <PythonCompiler />

}}

```

 </div>
 <div className="min-w-0 flex-1">
 <h1 className="text-xl font-extrabold text-white">
 Универсальное пособие
 </h1>
 <p className="text-xs text-indigo-400 font-mono">
 Подготовка к экзаменам: Алгоритмы, Структуры
 </p>
 </div>
 </div>

 <div className="flex items-center w-full lg:w-auto">
 >
 <button
 id="tab-guide-btn"
 onClick={() => setActiveTab('guide')}
 className={`flex-1 lg:flex-none flex items-center justify-center
 duration-200 whitespace-nowrap ${
 activeTab === 'guide'
 ? 'bg-indigo-600 text-white shadow-lg shadow-indigo-500'
 : 'text-slate-400 hover:text-white'
 }`}
 >
 <BookOpen className="w-4 h-4" /> Учебное пособие
 </button>
 <button
 id="tab-simulator-btn"
 onClick={() => setActiveTab('simulator')}
 className={`flex-1 lg:flex-none flex items-center justify-center
 duration-200 whitespace-nowrap ${
 activeTab === 'simulator'
 ? 'bg-indigo-600 text-white shadow-lg shadow-indigo-500'
 : 'text-slate-400 hover:text-white'
 }`}
 >
 <Cpu className="w-4 h-4" /> Тренажёры
 </button>
 <button
 id="tab-projects-btn"
 onClick={() => setActiveTab('projects')}
 className={`flex-1 lg:flex-none flex items-center justify-center
 duration-200 whitespace-nowrap ${
 activeTab === 'projects'
 ? 'bg-indigo-600 text-white shadow-lg shadow-indigo-500'
 : 'text-slate-400 hover:text-white'
 }`}
 >
 <Code className="w-4 h-4" /> Проекты
 </button>
 </div>
</div>

```

```

 disabled={busy}
 className="flex items-center justify-center
er:bg-amber-600/20 border border-amber-500/
 title="Скачать всё пособие одним автономным
 >
 {busy ? <Loader2 className="w-4 h-4 animate
 </button>
 </div>
 </div>
 </header>
);
};
....

```

### Patch относительно `main`

```

````diff
diff --git a/src/components/Navbar.tsx b/src/components
index a528778..6f8cd36 100644
--- a/src/components/Navbar.tsx
+++ b/src/components/Navbar.tsx
@@ -1,11 +1,11 @@
  import React, { useState } from 'react';
- import { BookOpen, Cpu, Sparkles, FileDown, FileText,
+ import { BookOpen, Cpu, Sparkles, FileDown, FileText,
  import { chapters } from '../data/content';
  import { downloadBookHtml } from '../utils/exportHtml'

  interface NavbarProps {
-   activeTab: 'guide' | 'simulator';
-   setActiveTab: (tab: 'guide' | 'simulator') => void;
+   activeTab: 'guide' | 'simulator' | 'compiler';
+   setActiveTab: (tab: 'guide' | 'simulator' | 'compile
  }

  export const Navbar: React.FC<NavbarProps> = ({ active
@@ -60,9 +60,20 @@ export const Navbar: React.FC<Navbar
  >

```

Универсальное пособие • полный исходный код

import { CheckCircle2, ExternalLink, Info, Loader2, Play

```
const PYODIDE_VERSION = "0.27.7";  
const PYODIDE_MODULE_URL = `https://cdn.jsdelivr.net/pyo  
const PYODIDE_INDEX_URL = `https://cdn.jsdelivr.net/pyo
```

```
const STARTER_CODE = `# Первый запуск загрузит Python в  
name = "алгоритмы"
```

```
for number in range(1, 4):  
    print(f"{number}. Учим {name}!")  
`;  
`;
```

```
type PyodideRuntime = {  
  runPythonAsync: (source: string) => Promise<unknown>;  
  setStdout: (options: { batched: (message: string) => ;  
  setStderr: (options: { batched: (message: string) => ;  
  setStdin: (options: { stdin: () => string | number | ;  
};
```

```
type PyodideModule = {  
  loadPyodide: (options: { indexURL: string }) => Promi  
};
```

```
let runtimePromise: Promise<PyodideRuntime> | null = nu
```

```
function getPythonRuntime() {  
  if (!runtimePromise) {  
    runtimePromise = import(/* @vite-ignore */ PYODIDE_I  
      (module as PyodideModule).loadPyodide({ indexURL:  
    });  
  }  
  return runtimePromise;  
}
```

```
function errorText(error: unknown) {  
  if (error instanceof Error) return error.message;  
  return String(error);  
}
```

```

        setRunning(false);
    }
}, [code, stdin, running, runtimeReady]);

const reset = () => {
    setCode(STARTER_CODE);
    setStdin("");
    setOutput("Нажмите «Запустить», чтобы увидеть резул
};

return (
    <main className="max-w-screen-2xl mx-auto px-4 sm:p
    <div className="max-w-6xl mx-auto">
        <div className="flex flex-col sm:flex-row sm:it
            <div>
                <div className="inline-flex items-center gap
                    <Terminal className="w-4 h-4" /> Боковая
                </div>
                <h2 className="text-2xl sm:text-3xl font-ex
                <p className="text-slate-400 mt-2 max-w-2xl
                    Пишите небольшой код и запускайте его пря
                    который переводит CPython в WebAssembly.
                </p>
            </div>
            <a
                href="https://pyodide.org/"
                target="_blank"
                rel="noreferrer"
                className="inline-flex items-center gap-1.5
            >
                Как это работает <ExternalLink className="w
            </a>
        </div>

        <div className="grid xl:grid-cols-[minmax(0,1.33fr)
            <section className="rounded-2xl border border
                <div className="flex flex-wrap items-center
                    <div className="flex items-center gap-2">

```

```

        -none focus:ring-2 focus:ring-inset focus:ring-slate-400
        placeholder="Напишите Python-код..."
    />

    <div className="border-t border-slate-800 border-b border-slate-800 border-l border-slate-800 border-r border-slate-800" >
        <label className="block px-4 pt-3 text-slate-400" >
            Ввод для input() • по одной строке на клавишу
        </label>
        <textarea
            id="python-stdin"
            value={stdin}
            onChange={(event) => setStdin(event.target.value)}
            spellCheck={false}
            rows={2}
            placeholder={'Например:\nАлиса'}
            className="block w-full resize-y bg-transparent border border-slate-800"
        />
    </div>
</section>

<section className="rounded-2xl border border-slate-800" >
    <div className="flex items-center justify-between" >
        <div className="flex items-center gap-2" >
            <Terminal className="w-4 h-4 text-emerald-500" />
        </div>
        <span className={`inline-flex items-center gap-2`} >
            <CheckCircle2 className="w-4 h-4 text-emerald-500" />
            {runtimeReady ? "готов" : "не загружен"}
        </span>
    </div>
    <pre className="min-h-[360px] max-h-[560px] px-4 py-4 leading-6 text-slate-300" >
        {output}
    </pre>
    <div className="border-t border-slate-800 border-b border-slate-800 border-l border-slate-800 border-r border-slate-800" >
    </div>
</section>
</div>

```

```
+name = "алгоритмы"
+
+for number in range(1, 4):
+    print(f"{number}. Учим {name}!")
+`;
+
+type PyodideRuntime = {
+  runPythonAsync: (source: string) => Promise<unknown>
+  setStdout: (options: { batched: (message: string) =>
+  setStderr: (options: { batched: (message: string) =>
+  setStdin: (options: { stdin: () => string | number |
+});
+
+type PyodideModule = {
+  loadPyodide: (options: { indexURL: string }) => Prom
+});
+
+let runtimePromise: Promise<PyodideRuntime> | null = n
+
+function getPythonRuntime() {
+  if (!runtimePromise) {
+    runtimePromise = import(/* @vite-ignore */ PYODIDE
+      (module as PyodideModule).loadPyodide({ indexURL
+    });
+  }
+  return runtimePromise;
+}
+
+function errorText(error: unknown) {
+  if (error instanceof Error) return error.message;
+  return String(error);
+}
+
+export function PythonCompiler() {
+  const [code, setCode] = useState(STARTER_CODE);
+  const [stdin, setStdin] = useState("");
+  const [output, setOutput] = useState("Нажмите «Запус
+  const [running, setRunning] = useState(false);
```



```

+   setOutput("Нажмите «Запустить», чтобы увидеть резу.
+   };
+
+   return (
+     <main className="max-w-screen-2xl mx-auto px-4 sm:p
+     <div className="max-w-6xl mx-auto">
+       <div className="flex flex-col sm:flex-row sm:i
+       <div>
+         <div className="inline-flex items-center g
+           <Terminal className="w-4 h-4" /> Боковая
+         </div>
+         <h2 className="text-2xl sm:text-3xl font-e
+         <p className="text-slate-400 mt-2 max-w-2x
+           Пишите небольшой код и запускайте его пр
+           который переводит CPython в WebAssembly.
+         </p>
+       </div>
+       <a
+         href="https://pyodide.org/"
+         target="_blank"
+         rel="noreferrer"
+         className="inline-flex items-center gap-1.5
+       >
+         Как это работает <ExternalLink className="
+       </a>
+     </div>
+
+     <div className="grid xl:grid-cols-[minmax(0,1.
+     <section className="rounded-2xl border borde
+     <div className="flex flex-wrap items-cente
+       <div className="flex items-center gap-2">
+         <span className="w-2.5 h-2.5 rounded-fr
+         <span className="text-sm font-bold tex
+         <span className="text-[11px] text-slate
+       </div>
+       <div className="flex items-center gap-2">
+         <button
+           type="button"

```

```

+         </label>
+         <textarea
+             id="python-stdin"
+             value={stdin}
+             onChange={(event) => setStdin(event.target.value)}
+             spellCheck={false}
+             rows={2}
+             placeholder={'Например:\nАлиса'}
+             className="block w-full resize-y bg-tran
ceholder:text-slate-700"
+         />
+     </div>
+ </section>
+
+     <section className="rounded-2xl border border-slate-200"
+         <div className="flex items-center justify-between p-4"
+             <div className="flex items-center gap-2"
+                 <Terminal className="w-4 h-4 text-emerald-500" />
+             </div>
+             <span className={`inline-flex items-center gap-2`}
+                 {runtimeReady && <CheckCircle2 className="text-emerald-500" />}
+                 {runtimeReady ? "готов" : "не загружен"}
+             </span>
+         </div>
+         <pre className="min-h-[360px] max-h-[560px] leading-6 text-slate-300">
+             {output}
+         </pre>
+         <div className="border-t border-slate-800 p-4" />
+     </section>
+ </div>
+
+ <div className="mt-5 grid md:grid-cols-3 gap-3"
+     <div className="flex gap-2 rounded-xl border border-slate-200 p-4"
+         <Info className="w-4 h-4 shrink-0 text-indigo-500" />
+         <span>Ctrl или Cmd + Enter запускает код. l
+     </div>
+     <div className="flex gap-2 rounded-xl border

```

```
    },
  },
  server: {
    host: "0.0.0.0",
    port: 5173,
    strictPort: true,
    // предпросмотр может открываться через внешний про
    allowedHosts: true,
  },
  preview: {
    host: "0.0.0.0",
    port: 4173,
    strictPort: true,
    allowedHosts: true,
  },
});
```
```

### Patch относительно `main`

```
````diff
diff --git a/vite.config.ts b/vite.config.ts
index c0456e7..e19d945 100644
--- a/vite.config.ts
+++ b/vite.config.ts
@@ -10,6 +10,9 @@ const __dirname = path.dirname(__file)

// https://vite.dev/config/
export default defineConfig({
+ // На GitHub Pages приложение живёт в /algv0/, локал
+ // VITE_BASE можно переопределить, если репозиторий
+ base: process.env.VITE_BASE || "/",
  plugins: [react(), tailwindcss(), viteSingleFile()],
  resolve: {
    alias: {
```
```

## Универсальное пособие • полный исходный код

- Определение/правило в центре (`border-blue-500` или
  - Обязательная сетка аналогий (2 колонки: неформальн
  - "Душные" детали (код, формулы, сложные доказательства
4. **\*\*Не ломай верстку:\*\*** Не придумывай свои CSS-классы в файлах `src/data/content.ts`. Не используй чистый markd

ФАЙЛ 4/83: index.html

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 13 | РАЗМЕР: 512

```
<!doctype html>
<html lang="ru" class="dark bg-slate-950 text-slate-100" >
 <head>
 <meta charset="UTF-8" />
 <meta name="viewport" content="width=device-width, initial-scale=1" />
 <title> Дискретка для тупых – Интерактивный гид по дискретке
 </head>
 <body class="bg-slate-950 text-slate-100 min-h-screen" >
 <div id="root"></div>
 <script type="module" src="/src/main.tsx"></script>
 </body>
</html>
```

ФАЙЛ 5/83: metadata.json

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 8 | РАЗМЕР: 312

```
{
 "name": "Remix Remix: ExamPrep Reader",
 "description": "Удобная web-читалка для подготовки к экзаменам",
 "requestFramePermissions": [],
 "majorCapabilities": [
 "MAJOR_CAPABILITY_SERVER_SIDE_GEMINI_API"
]
}
```

## Универсальное пособие • полный исходный код

```

 "@types/react-dom": "19.2.3",
 "@vitejs/plugin-react": "5.1.1",
 "esbuild": "^0.28.2",
 "tailwindcss": "4.1.17",
 "typescript": "5.9.3",
 "vite": "7.3.2",
 "vite-plugin-singlefile": "2.3.0"
 },
 "description": "Интерактивное пособие по алгоритмам и
}
```

-----  
ФАЙЛ 7/83: README.md

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 115 | РАЗМЕР: 1.5 КБ

-----  
# algv0 – Универсальное пособие по алгоритмам и структурам данных

Интерактивная web-читалка (React + Vite + Tailwind) с синтаксисом Markdown и автоматическим экспортом всего исходного кода в PDF.

## Быстрый старт

```
```bash  
npm install  
npm run dev          # предпросмотр на http://0.0.0.0:5174  
npm run build        # сборка в dist/ (single-file)  
```
```

## Экспорт кода: код → txt → png → pdf

Пайплайн собирает **\*\*весь\*\*** исходный код репозитория в один файл

| Команда              | Шаг       | Результат                     |
|----------------------|-----------|-------------------------------|
| ---                  | ---       | ---                           |
| `npm run export:txt` | код → txt | `public/export/full-code.txt` |
| `npm run export:png` | txt → png | `tmp/export-pages/...         |

## Универсальное пособие • полный исходный код

- \* **\*\*Всплывающие подсказки по терминам\*\*** – текст главы а известные термины (BFS, куча, бор, суффиксная ссылка, подчёркиваются, а по наведению/тапу показывается кар мини-анимацией и кнопкой «Показать симулятор» (симуля Словарь – `src/data/glossary.ts`, мини-анимации – `src`
- \* **\*\*Реестр интерактивов\*\*** – `src/components/vizRegistry` и для панели под главой, и для модалки из подсказки.

### ## Структура

...

|                    |                                  |
|--------------------|----------------------------------|
| src/               |                                  |
| App.tsx            | – оболочка, привязка глав к визу |
| components/        | – интерактивные симуляторы (Дейк |
| data/              | – контент пособия (главы/билеты  |
| scripts/export/    | – пайплайн код → txt → png → pdf |
| public/export/     | – сгенерированные TXT и PDF (обн |
| .github/workflows/ | – автоматизация                  |

...

### # Структура приложения и парсинг элементов

Если вы хотите парсить HTML конспект внешними инструмен  
азуемые теги и классы.

### ## Заголовки

- \* Заголовки секций: Используют стандартный тег `

## ` и
- \* Подзаголовки: Используют тег `

### `.

### ## Текст и параграфы

- \* Обычный текст содержится в тегах `

`.
- \* Блоки "Шиза" (Аналогии) и другая текстовая инфографика  
раграфы `

`.

### ## Математические формулы

В приложении используется MathJax.

Универсальное пособие • полный исходный код

## Уровень 1. Нет вообще ничего: ни аналогии, ни 2D

|                    |           |                      |
|--------------------|-----------|----------------------|
| Глава              | id        | Что делать           |
| ---                | ---       | ---                  |
| Алгоритмы на карте | `alg-map` | Страница-заглушка (8 |

## Уровень 2. Темы, которых в пособии НЕТ ВООБЩЕ (дыры)

Это самая большая проблема: визуализаторы для них написаны, но соответствующих глав в `src/data/content.ts` нет — и

|               |                                |                                        |
|---------------|--------------------------------|----------------------------------------|
| Билет         | Тема                           | Статус                                 |
| ---           | ---                            | ---                                    |
| <b>**1**</b>  | Дерево отрезков (Segment Tree) | Главы нет. Контент вхолостую.          |
| <b>**7**</b>  | Планарность и раскраска графов | Номерной главы нет; аналогии**.        |
| <b>**11**</b> | Мосты в графах                 | Номерной главы нет; есть без аналогии. |
| <b>**13**</b> | Эйлеровы пути и циклы          | Номерной главы нет; есть без аналогии. |

Дополнительно — «осиротевшие» визуализаторы без глав (контент вхолостую)

- \* `stack-dfs` → `StackViz.tsx` — **\*\*Стек и DFS\*\***
- \* `queue-bfs` → `QueueViz.tsx` + `BfsViz.tsx` — **\*\*Очередь и BFS\*\***
- \* `heap-beam-search` → `HeapViz.tsx` — **\*\*Куча и лучевой поиск\*\***
- \* `segment-trees` → `SegmentTreeVisualizer.tsx` — **\*\*Дерево отрезков\*\***
- \* `dynamic-programming` → `DPVisualizer.tsx` — **\*\*Динамическое программирование\*\***

## Уровень 3. Есть 2D, но НЕТ бытовой аналогии («сухая математика»)

|       |     |             |
|-------|-----|-------------|
| Глава | id  | Комментарий |
| ---   | --- | ---         |

## Универсальное пособие • полный исходный код

|    |                                      |   |   |   |
|----|--------------------------------------|---|---|---|
| 16 | 16. Графы. Кратчайший путь. Флойд    | - | - | - |
| 17 | 17. Графы. Алгоритм Джонсона         | - | - | - |
| 18 | 18. Графы. Остовное дерево. Краскал  | 1 | - | - |
| 19 | 19. Графы. Остовное дерево. Прима    | 1 | - | - |
| 20 | 20. Графы. Остовное дерево. Борувка  | 1 | - | - |
| 21 | 21. Префикс-функция (п), КМП         | 4 | - | - |
| 22 | 22. Z-функция                        | 4 | - | - |
| 23 | 23. Алгоритм Ахо-Корасик             | 2 | - | - |
| 24 | 24. Классы сложности, сведение задач | 4 | - | - |
| -  | Обзор: Кратчайшие пути и Графы       | - | - | - |
| -  | Алгоритмы на карте                   | - | - | - |
| -  | Эйлер: Путь ≠ Цикл                   | - | - | - |
| -  | Мосты в графах: код + визуализация   | 1 | - | - |

## ## Рекомендуемый порядок работ

1. **\*\*Вернуть потерянные билеты 1, 7, 11, 13\*\*** и подключить ``segment-trees``, ``stack-dfs``, ``queue-bfs``, ``heap-be``.  
это 5 готовых компонентов, которые сейчас просто не
2. **\*\*Дописать аналогии\*\*** в 5 главах из «Уровня 3» (по ш
3. **\*\*Добавить 2D\*\*** к 4 главам «Уровня 4» – минимум `inli`
4. **\*\*Решить судьбу `alg-map`\*\*** – раскрыть или удалить.

ФАЙЛ 9/83: tsconfig.json

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 32 | РАЗМЕР: 6

```
{
 "compilerOptions": {
 "target": "ES2020",
 "useDefineForClassFields": true,
 "lib": ["ES2020", "DOM", "DOM.Iterable"],
 "module": "ESNext",
 "skipLibCheck": true,
```



```

const __filename = fileURLToPath(import.meta.url);
const __dirname = path.dirname(__filename);

// https://vite.dev/config/
export default defineConfig({
 plugins: [react(), tailwindcss(), viteSingleFile()],
 resolve: {
 alias: {
 "@": path.resolve(__dirname, "src"),
 },
 },
 server: {
 host: "0.0.0.0",
 port: 5173,
 strictPort: true,
 // предпросмотр может открываться через внешний про
 allowedHosts: true,
 },
 preview: {
 host: "0.0.0.0",
 port: 4173,
 strictPort: true,
 allowedHosts: true,
 },
});
```

=====

## РАЗДЕЛ: ЯДРО ПРИЛОЖЕНИЯ

=====

-----

ФАЙЛ 11/83: src/App.tsx

КАТЕГОРИЯ: Ядро приложения | СТРОК: 118 | РАЗМЕР: 4.9 КБ

-----

```
import { useCallback, useEffect, useMemo, useState } from "react";
import { chapters } from "../data/content";
```

```

const progress = useMemo(() => ((index + 1) / chapter

return (
 <div className="min-h-screen bg-slate-950 text-slate-50">
 <Navbar activeTab={activeTab} setActiveTab={setActiveTab}>
 {activeTab === "guide" ? (
 <>
 <MobileToc
 selectedChapterId={activeChapterId}
 setSelectedChapterId={goTo}
 currentTitle={activeChapter.title}
 index={index}
 total={chapters.length}
 />

 <div className="flex flex-col lg:flex-row max-w-7xl mx-auto" >
 {/* Сайдбар только на десктопе – на мобильном скрывается */}
 <div className="hidden lg:block lg:w-80 shrink-0" >
 <Sidebar selectedChapterId={activeChapterId} />
 </div>

 <main id="main-content" className="flex-1 min-w-0" >
 {/* Шапка главы с быстрыми переходами */}
 <div className="flex items-center justify-between" >
 <div className="min-w-0">
 <div className="text-[10px] uppercase" >
 {activeChapter.category || "Универсальное"}
 </div>
 <h2 className="text-base sm:text-xl font-medium" >
 {activeChapter.title}
 </h2>
 </div>
 <ChapterNav prev={prev} next={next} index={index} />
 </div>

 <div className="h-1 w-full bg-slate-800 rounded-t" />
 </main>
 </div>
 </>
) : null}
 </Navbar>
 </div>
)

```

}

ФАЙЛ 12/83: src/hooks/useTermHints.ts

КАТЕГОРИЯ: Ядро приложения | СТРОК: 124 | РАЗМЕР: 5.0 КБ

```
import { useEffect } from "react";
import { GLOSSARY_LABELS } from "../data/glossary";

/**
 * Ограничители, чтобы текст не рябил, но и чтобы подсказки
 *
 * Считаем не «сколько раз встретился термин», а «сколько
 * написание». Иначе в главе про splay первые же «Splay
 * лимит, и слова «Zig-Zag», «вращений», «AVL-дереве» о
 */
const MAX_PER_SURFACE = 2;
const MAX_PER_TERM = 8;

const SKIP_SELECTOR = "code, pre, script, style, a, text";

const escapeRe = (s: string) => s.replace(/[\.*+?^${}()|]/g, "\\$&");

/** Одна большая регулярка из всех меток: длинные раньше
function buildRegex(): RegExp {
 const alternation = GLOSSARY_LABELS.map((l) => escapeRe(l));
 // границы «не буква/цифра» – \b не работает с кириллицей
 return new RegExp(`(?<![\\p{L}\\p{N}_-])(${alternation})`);
}

let cachedRegex: RegExp | null = null;
const labelToId = new Map(GLOSSARY_LABELS.map((l) => [l, ...]));

/**
 * Проходит по тексту главы и оборачивает известные термины
 * , чтобы на
```

```

 if (!id) continue;

 const usedTerm = perTerm.get(id) ?? 0;
 const usedSurface = perSurface.get(surface) ?? 0;
 if (usedTerm >= MAX_PER_TERM || usedSurface >= MAX_SURFACE) continue;
 perTerm.set(id, usedTerm + 1);
 perSurface.set(surface, usedSurface + 1);

 if (m.index > last) frag.appendChild(document.createElement("div"));
 const span = document.createElement("span");
 span.className = "term-hint";
 span.dataset.termId = id;
 span.tabIndex = 0;
 span.setAttribute("role", "button");
 span.setAttribute("aria-label", `Подсказка: ${m[1]}`);
 span.textContent = m[1];
 frag.appendChild(span);
 last = m.index + m[1].length;
 }

 if (last === 0) continue;
 if (last < text.length) frag.appendChild(document.createElement("div"));
 textNode.parentNode?.replaceChild(frag, textNode);
}

/**
 * Навешивает разметку терминов на контейнер при смене
 *
 * Дополнительно перепроверяет разметку после каждого рендера
 * (или что-то ещё) перезаписал innerHTML главы, подсказки
 * а не пропадают до перезагрузки страницы.
 */
export function useTermHints(ref: React.RefObject<HTMLDivElement>) {
 const run = () => {
 const el = ref.current;
 if (!el) return;
 try {

```

```
}
.no-scrollbar::-webkit-scrollbar {
 display: none;
}
.no-scrollbar {
 scrollbar-width: none;
}
}

@keyframes pulse-gold {
 0%,
 100% {
 stroke: #eab308;
 stroke-width: 5;
 filter: drop-shadow(0 0 2px rgba(234, 179, 8, 0.5));
 }
 50% {
 stroke: #fef08a;
 stroke-width: 8;
 filter: drop-shadow(0 0 12px rgba(234, 179, 8, 0.9));
 }
}

.animate-pulse-gold {
 animation: pulse-gold 1.5s infinite;
}

@keyframes tocIn {
 from {
 transform: translateX(-100%);
 }
 to {
 transform: translateX(0);
 }
}

@keyframes hintIn {
 from {
```

```
.chapter-body {
 max-width: 100%;
 overflow-wrap: anywhere;
}

.chapter-body :where(section, div, p, li) {
 min-width: 0;
}

.chapter-body :where(pre, code) {
 white-space: pre-wrap;
 word-break: break-word;
}

.chapter-body pre {
 overflow-x: auto;
 max-width: 100%;
 font-size: clamp(11px, 2.9vw, 13px);
}

.chapter-body table {
 display: block;
 width: 100%;
 overflow-x: auto;
 border-collapse: collapse;
 font-size: clamp(11px, 3vw, 14px);
}

.chapter-body summary {
 cursor: pointer;
 padding: 0.35rem 0;
}

@media (max-width: 767px) {
 .chapter-body :where(.grid) {
 grid-template-columns: minmax(0, 1fr) !important;
 }
 .chapter-body :where(h2) {
```

```
@keyframes demoProgress {
 from {
 width: 0%;
 }
 to {
 width: 100%;
 }
}
```

---

ФАЙЛ 14/83: src/main.tsx

КАТЕГОРИЯ: Ядро приложения | СТРОК: 11 | РАЗМЕР: 230 В

---

```
import { StrictMode } from "react";
import { createRoot } from "react-dom/client";
import "./index.css";
import App from "./App";

createRoot(document.getElementById("root")!).render(
 <StrictMode>
 <App />
 </StrictMode>
);
```

---

ФАЙЛ 15/83: src/types.ts

КАТЕГОРИЯ: Ядро приложения | СТРОК: 9 | РАЗМЕР: 153 В

---

```
export interface Chapter {
 id: string;
 title: string;
 type: "html" | "markdown";
 content: string;
 description?: string;
```

```

import { Chapter } from "../types";

export const additionalTickets: Chapter[] = [
 {
 id: "sparse-table",
 title: "2. Двумерная разреженная матрица (Sparse Table)",
 type: "html",
 description: "Разреженная таблица для идемпотентных операций",
 category: "Продвинутые структуры",
 content: `
<section id="sparse-table" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-600">
 <h3 class="text-xl font-bold text-blue-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic">
 <p class="text-xl font-mono text-white">
 ух отрезков: $\min(ST[L][k], ST[R - 2^k + 1][k])$
 </div>
 <div class="grid grid-cols-1 xl:grid-cols-2 gap-4">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 right-4 border border-emerald-500 shadow-md">
 Аналогия 1: Школьные линейки
 </div>
 <p class="text-slate-300 text-sm mb-4">
 зок длины 7. Ты берешь две заготовленные заготовки
 ь отрезок длины 7 (перекрывая друг друга пополам)
 ничего складывать, просто пересекаем куски.
 </div>
 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 right-4 border border-rose-500 shadow-md">
 Аналогия 2: Ступени
 </div>
 </div>
 </div>
 </div>
 </div>
 </div>
 </div>
`
 }
]

```



```
<p class="text-lg text-blue-300 italic">
 <p class="text-xl font-mono text-white">
 двоичная куча. Пара – точка на декартовой
 </div>
```

```
<p class="text-slate-300 text-sm mb-4">Постр
 платит $\approx n \cdot \log n$ сравнений, counting по ма
 точка вставляется спуском от корня: $x \leq \text{уз}$
 единственно. Весь процесс – в симуляторе под
 «соединить сам» – кликаешь две точки,
 ь родитель», «получился бы цикл», «равные »,
 «Миф $2k/2k+1$ » и «Поиск $\neq$ сортировка»
 ждый запрещённый ход.</p>
```

```
<div class="text-center my-6">
 <p class="text-slate-400 text-xs uppercase">
 <p class="my-1 font-bold leading-none">
 И
 </p>
 <p class="text-slate-300 font-mono text">
</div>
```

```
<div class="grid grid-cols-1 xl:grid-cols-2">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 l
 rder border-emerald-500 shadow-md">
 Аналогия 1: Split – таможня
 </div>
 <p class="text-slate-300 text-sm">S
 х»». Спуск от корня: узел, который вместе
 скрытия – досматривается только одна ве
 </div>
```

```
 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 l
 border-rose-500 shadow-md">
 Аналогия 2: Merge – стыковка
```

```

 auto [L, R] = split(t->right, x);
 t->right = L;
 return {t, R};
 } else {
 auto [L, R] = split(t->left, x);
 t->left = R;
 return {L, t};
 }
}
}

```

</div>  
</details>

<details class="bg-slate-900/40 rounded-lg l  
<summary class="p-4 font-bold text-slate  
-b border-slate-700/50">

Код: вставка, merge, erase (Скрыт

</summary>

<div class="p-5 text-sm text-slate-300  
<pre class="bg-slate-950 p-4 rounded

```

def insert(root, key, pri):
 if root is None:
 return Node(key, pri)
 if key <= root.key: # равные x — левый
 root.left = insert(root.left, key, pri)
 else:
 root.right = insert(root.right, key, pri)
 return root

```

```

for key, pri in pairs_sorted_by_pri_desc: # порядок э
 root = insert(root, key, pri)

```

<pre class="bg-slate-950 p-4 rounded

```

def merge(a, b): # все x в a < все x в b
 if not a or not b: return a or b
 if a.pri > b.pri:
 a.right = merge(a.right, b); return a
 b.left = merge(a, b.left); return b

```

```

def erase(root, key): # удалить произвольн

```

```
>0(log n)</code> на операцию; память — <code>
border-slate-800">0(n)</code> (ключ + 2 сс
</div>
```

```
<div class="grid grid-cols-1 lg:grid-cols-3
 <div class="bg-slate-900 rounded-lg p-4
 <p class="text-xs uppercase tracking
 <p class="text-white font-bold">мож
 <p class="text-slate-400 text-xs mt
 </div>
```

```
<div class="bg-slate-900 rounded-lg p-4
 <p class="text-xs uppercase tracking
 <p class="text-emerald-300 font-bol
 <p class="text-slate-400 text-xs mt
</div>
```

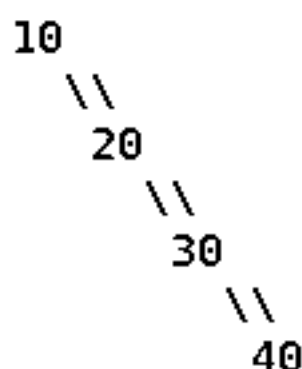
```
<div class="bg-slate-900 rounded-lg p-4
 <p class="text-xs uppercase tracking
 <p class="text-white font-bold">0(n
 <p class="text-slate-400 text-xs mt
</div>
```

```
</div>
```

```
</div>
```

```
<!-- 1. Зачем -->
```

```
<div class="bg-slate-800/60 p-6 rounded-xl bord
 <h3 class="text-lg font-bold text-white mb-3
 <p class="text-slate-300 text-sm mb-4">В об
 ючи по возрастанию — и дерево вырождается в
 <pre class="bg-slate-950 p-4 rounded-lg ove
 ading-relaxed">вставили 10, 20, 30, 40, 50
```



высота = 5, поиск 50 = 5 сравнений  
это уже не дерево, а односвязный

```

<h3 class="text-lg font-bold text-white mb-3">Поворот
<p class="text-slate-300 text-sm mb-4">Поворот 20
 движение, три перевешенные ссылки и
<pre class="bg-slate-950 p-4 rounded-lg overflow-x-auto font-mono text-sm text-white leading-relaxed">
 40
 / \ \
 20 C поворот 20
 / \ \ ----->
 A B <-----<
 поворот 40
 / \ \
 A B C

```

обход слева направо ДО: A 20 B 40 C

обход слева направо ПОСЛЕ: A 20 B 40 C ← тот же сам

```

<div class="grid grid-cols-1 md:grid-cols-2 gap-4">
 <div class="bg-slate-900 rounded-lg p-4">
 <p class="text-emerald-400 font-bold">Поворот 20
 <p class="text-slate-300 text-xs">Поворот 40
 </div>
 <div class="bg-slate-900 rounded-lg p-4">
 <p class="text-amber-400 font-bold">Поворот 20
 <p class="text-slate-300 text-xs">Поворот 40
 </div>
</div>

```

<!-- 4. Три случая -->

```

<div class="bg-slate-800/60 p-6 rounded-xl border border-slate-700">
 <h3 class="text-lg font-bold text-white mb-3">Обозначения
 <p class="text-slate-300 text-sm mb-4">Обозначения
 узел, который поднимаем,
 <g></g> – дед (родитель родителя). Смотри

 </p>
 <div class="overflow-x-auto -mx-2 px-2">
 <table class="w-full text-xs sm:text-sm">
 <thead>

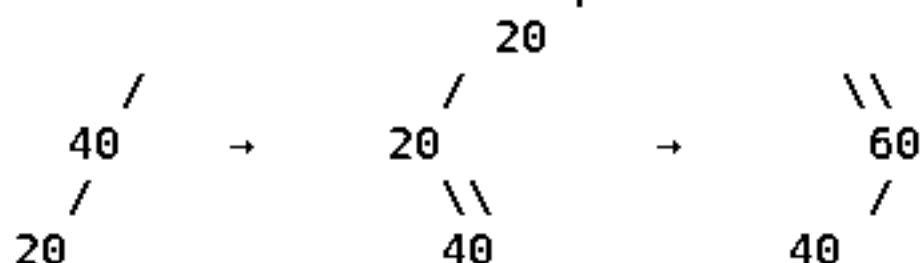
```

отличает splay от «просто поворотов».

Нисходящий узел (с грузом  $\alpha$ ) одёжём в корень разложится на атомарные кадры (с грузом  $\beta$ ), о, какой груз переедет) → **поворот** (с грузом  $\gamma$ ) (схема р→х с  $\alpha/\beta/\gamma$ ), а под демо – песочница

5. Главная ловушка

Это

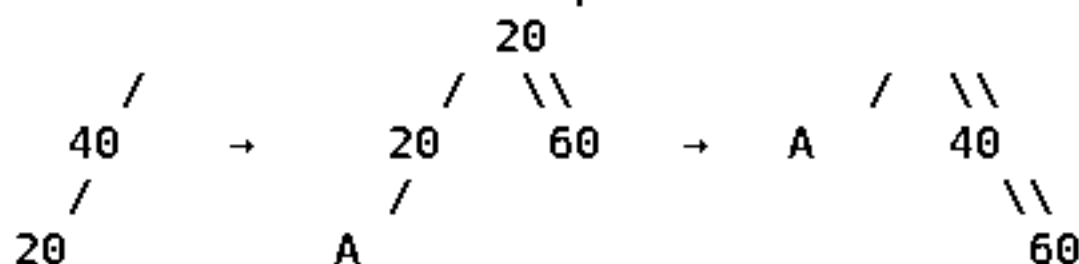


было: бамбук влево

стало: бамбук вправо

глубина никого не сократилась!

мортизации не получится.



```
<p class="text-slate-300 text-sm mb-3">Стро
одерева) по всем узлам, и Zig-Zig/Zig-Zag
ах идея такая:</p>
```

```
<div class="grid grid-cols-1 md:grid-cols-3
 <div class="bg-slate-900 rounded-lg p-4
 <p class="text-white font-bold mb-1
 <p class="text-slate-400 text-xs">c
 </div>
```

```
<div class="bg-slate-900 rounded-lg p-4
 <p class="text-white font-bold mb-1
 <p class="text-slate-400 text-xs">H
</div>
```

```
<div class="bg-slate-900 rounded-lg p-4
 <p class="text-white font-bold mb-1
 <p class="text-slate-400 text-xs">д
</div>
```

```
</div>
<p class="text-slate-400 text-xs mt-4">Отсю
сти запросов splay-дерево не хуже (с точнос
дерево. Оно как бы само подстраивается под
```

```
</div>
```

```
<!-- 8. Операции -->
```

```
<div class="bg-slate-800/60 p-6 rounded-xl bord
 <h3 class="text-lg font-bold text-white mb-3
 <p class="text-slate-300 text-sm mb-4">Крас
лается в корне.</p>
```

```
<div class="space-y-2 text-sm">
```

```
 <div class="bg-slate-900 rounded-lg p-3
 — спус
маем последний узел на пути (соседний по зн
```

```
 <div class="bg-slate-900 rounded-lg p-3
) — Spl
росто отрезаем ссылки.</div>
```

```
 <div class="bg-slate-900 rounded-lg p-3
k) — sp
/div>
```

```
 <div class="bg-slate-900 rounded-lg p-3
```

```

 if (!g) { // ZIG: деда не
 rotate(x);
 } else if ((g.left === p) === (p.left === x)) {
 rotate(p); // ZIG-ZIG: сн
 rotate(x);
 } else {
 rotate(x); // ZIG-ZAG: сн
 rotate(x);
 }
 }
 return x; // теперь x —
}

// поиск: спустились и подняли найденное наверх
function find(root, key) {
 let cur = root, last = null;
 while (cur) {
 last = cur;
 if (key === cur.key) break;
 cur = key < cur.key ? cur.left : cur.right;
 }
 return last ? splay(last) : null; // splay делае
}
</pre>

```

<p class="text-xs text-slate-400">Вся р  
 ate(x);</span> против <span class="font-mon
 т свою оценку.</p>

</div>

</details>

<!-- 10. Плюсы/минусы -->

<div class="grid grid-cols-1 md:grid-cols-2 gap

<div class="bg-emerald-950/30 rounded-xl p-

<p class="text-emerald-300 font-bold mb

<ul class="list-disc list-inside text-s

<li>Простой код: нет высот, цветов,

<li>Меньше памяти, чем у AVL и крас

<li>Сам подстраивается под «горячие

```

 <div>
 <strong class="text-white block mb-6">
 <p class="text-slate-400">Треар дер
 play не хранит вообще ничего и держит балан
 </div>
 </div>
 </details>

</div>
</section>`,
 },
 {
 id: "prefix-sums-2d",
 title: "5. Двумерная матрица префиксных сумм",
 type: "html",
 description: "Сжатие суммы подматрицы за $O(1)$ ",
 category: "Алгоритмы матрицы",
 content: `
<section id="prefix-sums-2d" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-emerald-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-6">
 <p class="text-lg text-emerald-300 italic">
 <p class="text-xl font-mono text-white">
 , y1...y2) = P[x2][y2] - P[x1-1][y2] - P[x2
 </div>
 <div class="grid grid-cols-1 xl:grid-cols-2">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 l
 rder border-emerald-500 shadow-md">
 ≈ Аналогия 1: Вырезание прямоу
 </div>
 </div>
 </div>
 </div>
</section>
`
 }
]

```



```

 </div>
 </div>
 </div>
 </div>
</section>`,
 },
 {
 id: "top-sort",
 title: "9. Графы. Топологическая сортировка",
 type: "html",
 description: "Разложение задач по порядку выполнения",
 category: "Графы",
 content: `
<section id="top-sort" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 9
 <h2 class="text-2xl sm:text-3xl font-bold text-white">Графы</h2>
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-600">
 <h3 class="text-xl font-bold text-blue-400">Топологическая сортировка</h3>
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic">Топологическая сортировка – это расписание, которое можно выполнить, если все задачи имеют порядок выполнения.</p>
 <p class="text-xl font-mono text-white">
 перед.</p>
 </div>
 <div class="grid grid-cols-1 xl:grid-cols-2">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 Аналогия 1: Учеба в вузе
 </div>
 <p class="text-slate-300 text-sm mb-4">Топологическая сортировка – это расписание, которое можно выполнить, если все задачи имеют порядок выполнения.</p>
 </div>
 </div>
 </div>
 </div>
</section>`
 }
]

```

```

 БЫХ направлениях по односторонним улицам. К
 у на карте.</p>
 </div>
 </div>
</div>
</div>
</section>`,
 },
 {
 id: "graph-bridges",
 title: "11. Графы. Мосты",
 type: "html",
 description: "Рёбра, удаление которых расхреначивает",
 category: "Графы. Продвинутые",
 content: `
<section id="graph-bridges" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-rose-400">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-rose-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-rose-300 italic">
 <p class="text-xl font-mono text-white">
и fup[v] > tin[u].</p>
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border
rder border-emerald-500 shadow-md">
 Аналогия 1: Единственный мост
 </div>
 <p class="text-slate-300 text-sm mb-4">
этот мост, экономикой обоих кусков придет
 </div>
 </div>
 </div>
 </div>
 </div>
`
 }
]

```

```

{
 id: "graph-euler",
 title: "13. Графы. Эйлеров цикл",
 type: "html",
 description: "Пройти по всем ребрам ровно 1 раз.",
 category: "Графы",
 content: `
<section id="graph-euler" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 Графы
 <h2 class="text-2xl sm:text-3xl font-bold text-indigo-400">Эйлеров цикл
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-600">
 <h3 class="text-xl font-bold text-indigo-400">Определение
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-indigo-300 italic">Граф называется эйлеровым, если существует замкнутый путь, проходящий по каждому ребру ровно один раз.
 <p class="text-xl font-mono text-white">Эйлеров цикл
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 Аналогия 1: Рисование не отрывая
 </div>
 <p class="text-slate-300 text-sm mb-4">Если граф эйлеров, то сумма степеней вершин не сходится нечетное число линий, значит, из каждой вершины должно выходить и заходить одинаковое количество линий, а везде должно быть четное число (зашел-вышел)
 </div>
 </div>
 </div>
 </div>
</section>`,
},
{
 id: "pathfinding-accel",
 title: "17. Графы. Ускорения поиска",
 type: "html",

```

```

<section id="mst-kruskal" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-emerald-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">
 <p class="text-xl font-mono text-white">
 (проверяем через DSU) - берем в ответ.</p>
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border
 order border-emerald-500 shadow-md">
 Аналогия: Прокладка интернет
 </div>
 <p class="text-slate-300 text-sm mb-4">
 с самых дешевых дорог в стране". Он строит
 единую сеть.</p>
 </div>
 </div>
 </div>
 </div>
</section>`,
 },
 {
 id: "mst-prima",
 title: "19. Графы. Остовное дерево. Прима",
 type: "html",
 description: "",
 category: "Графы. Остовные",
 content: `
<section id="mst-prima" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

```

```

<div class="bg-slate-700/50 p-6 rounded-xl border-1 border-emerald-500" >
 <h3 class="text-xl font-bold text-emerald-400" >
 <div class="bg-slate-900 p-4 rounded-lg mb-4" >
 <p class="text-lg text-emerald-300 italic" >
 <p class="text-xl font-mono text-white" >
 ается с соседом.</p>
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg" >
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 Аналогия: Племена
 </div>
 <p class="text-slate-300 text-sm mb-4" >
 изкому племени для объединения. К вечеру племена
 ства шлют гонцов к ближайшему чужому царству.
 </div>
 </div>
 </div>
 </div>
 </div>
 </section>`,
 },
 {
 id: "string-kmp",
 title: "21. Алгоритм Кнута-Морриса-Пратта (КМП)",
 type: "html",
 description: "",
 category: "Строки",
 content: `
<section id="string-kmp" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border-1 border-emerald-500" >
 <h3 class="text-xl font-bold text-blue-400" >
 <div class="bg-slate-900 p-4 rounded-lg mb-4" >

```

```

 id: "string-z-func",
 title: "22. Z-функция",
 type: "html",
 description: "",
 category: "Строки",
 content: `
<section id="string-z-func" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-blue-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic">
 <p class="text-xl font-mono text-white">
щегоря с i.</p>
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
 Аналогия 1: Копипаста
 </div>
 <p class="text-slate-300 text-sm mb">
в тексте кричит: "Эй! Начиная с этой буквы
т "окно" [L, R] самой дальней найденной коп
 </div>
 </div>
 <details class="bg-slate-900/40 rounded-lg">
 <summary class="p-4 font-bold text-slate">
 -b border-slate-700/50">
 Код (Скрыто)
 </summary>
 <div class="p-5 text-sm text-slate-300">
 <pre class="bg-slate-950 p-4 rounded
int l = 0, r = 0;

```

```

 <p class="text-slate-300 text-sm mb-4">
 пример сортировка чисел). Класс NP – это
 енную сетку (ответ), он БЫСТРО (за класс P)
 </div>
 <!-- Аналогия 2 -->
 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border
 rder border-purple-500 shadow-md">
 Аналогия 2: Машина-переводчик
 </div>
 <p class="text-slate-300 text-sm mb-4">
 ь отличный переводчик на английский (Сведения
 аче В, то В – это NP-Полная (сосредоточенная)
 </div>
 </div>
 </div>
</div>
</section>`,
 },
];

```

ФАЙЛ 18/83: src/data/content\_temp.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРОК: 385 | РАЗМЕР: 1.5 КБ

```

export interface Chapter {
 id: string;
 title: string;
 type: "html";
 content: string;
}

export const chapters: Chapter[] = [
 {
 id: "euler-path-vs-cycle",
 title: "Эйлер: Путь ≠ Цикл (ты путаешь!)",
 },
];

```

Путь: Нормальная прогулка

</div>

<p class="text-slate-300 text-sm mb-4">

Ты вышел из дома <strong>(нечётная)</strong>.   
Ты пришёл в бар <strong>(вторая нечётная)</strong>.   
Домой вернуться не смог, потому что

</p>

</div>

<div class="bg-slate-900 p-6 rounded-lg border">

<div class="absolute -top-3 left-4 bg-rose-500 shadow-md bg-slate-950">

Цикл: Гамильтонов синдром (болезнь)

</div>

<p class="text-slate-300 text-sm mb-4">

<strong>Гамильтон – это болезнь.</strong>   
<strong>Гамильтон</strong> ровно раз и вернуться домой.

Ты обходишь все бары (вершины) ровно раз.

Главное – зайти и выйти, не заходя дважды.

Никто не знает, как решать быстро.

<br><span class="text-xs text-rose-400">   
<strong>Гамильтон</strong> ровно раз и вернуться домой.

</p>

</div>

</div>

<details class="bg-slate-900/40 rounded-lg border">

<summary class="p-4 font-bold text-slate-400">   
<strong>Гамильтон</strong> ровно раз и вернуться домой.

Душный режим: Формальное доказательство

</summary>

<div class="p-5 text-sm text-slate-300 space">

<p class="text-blue-400">Теорема Эйлера

<p>

Связный граф содержит эйлеров цикл

<br>Эйлеров путь (не цикл)  $\Leftrightarrow$  ровно

</p>

<p class="text-rose-400">Почему Гамильтон



```
 Если из сына u и всех его потомков <str
 Единственная ниточка. Порвётся — граф р
 </p>
</div>

<div class="grid grid-cols-1 xl:grid-cols-2 gap-4">
 <div class="bg-slate-800 p-6 rounded-lg border-emerald-500 shadow-md">
 Мост: Единственная веревка
 </div>
 <p class="text-slate-300 text-sm mb-4">
 Представь, что ты альпинист. Ты спу
 Из пещеры и нет других выходов — то
 Если верёвка (ребро $v-u$) оборвётся
 Если бы из пещеры и был чёрный ход
 </p>
</div>

<div class="bg-slate-900 p-6 rounded-lg border-rose-500 shadow-md">
 Аналогия: Кровеносный сосуд
</div>
 <p class="text-slate-300 text-sm mb-4">
 Граф — это кровеносная система. Реб
 терали) — это не мост, живём.
 Если же перерезать единственную арте
 Алгоритм DFS ищет такие «критически
 </p>
</div>
</div>

<details class="bg-slate-900/40 rounded-lg border-slate-700/50">
 <summary class="p-4 font-bold text-slate-400">
 Полный код на C++ (Скрыто)
 </summary>
```

```

}

void find_bridges() {
 timer = 0;
 visited.assign(n, false);
 tin.assign(n, -1);
 low.assign(n, -1);

 for (int i = 0; i < n; ++i) {
 if (!visited[i]) dfs(i);
 }
}
}

</div>
<p class="text-xs text-slate-400">Занес
</div>
</details>
</div>
</section>`,
},
{
 id: "planarity-euler-formula",
 title: "Планарность: K5, K3,3 и формула Эйлера",
 type: "html",
 content: `<section id="planarity-euler-formula" cla
<div class="flex items-center mb-6">
 <span class="bg-blue-600 text-white px-4 py-1 r
 <h2 class="text-3xl font-bold text-white">Плана
</div>

<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl bord
 <h3 class="text-xl font-bold text-blue-400 r

 <div class="bg-slate-900 p-4 rounded-lg mb-
 <p class="text-lg text-blue-300 italic r
 <p class="text-3xl font-mono text-white
 <div class="text-sm text-slate-400 spac
 <p><span class="text-blue-400 font-

```

```

 Отсюда более строгое ограничение: <
 <code class="text-white">9 > 8</code>
 </p>
</div>
</div>

<details class="bg-slate-900/40 rounded-lg border
 <summary class="p-4 font-bold text-slate-400
 order-slate-700/50">
 Доказательство непланарности K5 (Скры
</summary>
 <div class="p-5 text-sm text-slate-300 space
 <p class="text-blue-400">Доказательство
 <p>
 Пусть K5 планарен. $V = 5$, $E = 10$.

 $F = E - V + 2 = 10 - 5 + 2 = 7$

Каждая грань ограничена минимум

Сумма длин граней $= 2E = 20$.

Отсюда: $2E \geq 3F \Rightarrow 20 \geq 21$ — <sp

Значит, K5 не может быть планарн
 </p>
 </div>
</details>
</div>
</section>`,
 },
 {
 id: "graph-articulation",
 title: "12. Графы. Точки сочленения",
 type: "html",
 content: `<section id="graph-articulation" class="m
 <div class="flex items-center mb-6">
 <span class="bg-rose-600 text-white px-4 py-1 r
 <h2 class="text-3xl font-bold text-white">Точки
 </div>

 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl bord

```

Телеком: Центральный свитч

</div>

<p class="text-slate-300 text-sm mb-4">

У тебя несколько компьютеров в одной абелем (мостом). Сами свитчи — это точки со

</p>

</div>

</div>

<details class="bg-slate-900/40 rounded-lg border

<summary class="p-4 font-bold text-slate-400 order-slate-700/50">

Душный мод: Код и корень DFS (Скрыто)

</summary>

<div class="p-5 text-sm text-slate-300 spac

<p class="text-blue-400 font-bold">Особ

<p>

Для стартовой вершины обхода правил то выше неё прыгать некуда). Поэтому для него <strong>больше 1 независимого ребенка

</p>

<div class="bg-slate-950 p-4 rounded-lg

<pre class="text-xs font-mono text-

```
void dfs(int v, int p = -1) {
```

```
 visited[v] = true;
```

```
 tin[v] = low[v] = timer++;
```

```
 int children = 0;
```

```
 for (int to : adj[v]) {
```

```
 if (to == p) continue;
```

```
 if (visited[to]) {
```

```
 low[v] = min(low[v], tin[to]);
```

```
 } else {
```

```
 dfs(to, v);
```

```
 low[v] = min(low[v], low[to]);
```

```
 if (low[to] >= tin[v] && p != -1)
```

```
 IS_CUTPOINT(v);
```

```
 ++children;
```

```

import { graphChapters } from "./graphContent";
import { additionalTickets } from "./additionalTickets";
import { tickets14to20 } from "./tickets/ticket14_20";
import { tickets21to24 } from "./tickets/ticket21_24";
import { tickets6to13 } from "./tickets/ticket6_13";
import { chapters as newChapters } from "./content_temp";

// We want to combine all of them, but overwrite 7, 11,
const merged = [
 ...graphChapters,
 ...additionalTickets,
 ...tickets6to13,
 ...tickets14to20,
 ...tickets21to24
].filter(c => c && c.id);

// Remove old 7, 11, 12, 13: graph-planar-colors, graph
const toRemove = ["graph-planar-colors", "graph-bridges"];

const dedup = new Map<string, Chapter>();
merged.forEach(c => {
 if (!toRemove.includes(c.id)) {
 // We prefer the newer versions (like tickets14_20)
 dedup.set(c.id, c);
 }
});

// Add the new ones
if (newChapters) {
 newChapters.forEach(c => dedup.set(c.id, c));
}

const finalChapters = Array.from(dedup.values());

finalChapters.sort((a, b) => {
 const numA = parseInt(a.title.match(/(\d+)/)?.[0] || 0);
 const numB = parseInt(b.title.match(/(\d+)/)?.[0] || 0);
 return numA - numB;
});

```

```

demo?: DemoKind;
/** id полного симулятора (см. vizRegistry) – кнопка
simulator?: string;
}

export const GLOSSARY: GlossaryEntry[] = [
 {
 id: "bfs",
 labels: ["BFS", "обход в ширину", "поиск в ширину",
 title: "BFS – обход в ширину",
 short:
 "Волна от источника: сначала все соседи, потом со
 formal:
 "Очередь (FIFO). Кладём старт, достаём вершину –
 у рёбер.",
 complexity: "O(V + E)",
 demo: "bfs",
 simulator: "queue-bfs",
 },
 {
 id: "dfs",
 labels: ["DFS", "обход в глубину", "поиск в глубину",
 title: "DFS – обход в глубину",
 short:
 "Прёшь вперёд до упора, упёрся – откатываешься на
 formal:
 "Стек (или рекурсия). Из DFS растут: времена вход
 complexity: "O(V + E)",
 demo: "dfs",
 simulator: "stack-dfs",
 },
 {
 id: "binsect",
 labels: ["binsect", "бинсект", "бинарный поиск", "д
 title: "Бинарный поиск (и его самозванец binsect)",
 short:
 "Бинарный поиск НАХОДИТ готовый элемент в уже отс
 о не сортирует. «binsect» из конспекта – шутлив

```

```

labels: ["DSU", "СНМ", "система непересекающихся множеств"],
title: "DSU — система непересекающихся множеств",
short:
 "«Кто твой староста?» Каждый элемент знает своего старосту.",
formal:
 "find со сжатием путей + union по рангу/размеру. Инициализация O(n).",
complexity: "почти O(1) — обратная функция Аккермана",
demo: "dsu",
simulator: "mst-kruskal",
},
{
 id: "trie",
 labels: ["бор", "бора", "боре", "бору", "бором", "тот самый бор"],
 title: "Бор (trie, префиксное дерево)",
 short:
 "Дерево, где путь от корня по буквам и есть слово.",
 formal: "Из корня по символам; в вершине хранится фактическое слово.",
 complexity: "поиск слова — O(|слова|)",
 demo: "trie",
 simulator: "aho-corasick",
},
{
 id: "bfs-on-trie",
 labels: [
 "обход дерева",
 "обход бора",
 "обход в ширину по бору",
 "какой-то обход",
 "обходом дерева",
 "конечный автомат",
 "конечного автомата",
 "автомат",
 "автомата",
],
 title: "Обход бора в ширину (тот самый «какой-то обход»)",
 short:
 "В Ахо–Корасике бор обходят именно BFS: суффиксные ссылки не используют, а есть строго по уровням.",

```

```

 ан и Флойд.",
demo: "relax",
simulator: "dijkstra",
},
{
 id: "prefix-sum",
 labels: ["префиксные суммы", "префиксная сумма", "префикс"],
 title: "Префиксные суммы",
 short:
 "Заранее посчитанные «сколько всего набежало от начальной точки до i»",
 formal: "P[0]=0, P[i]=P[i-1]+a[i-1]. Сумма a[l..r] = P[r]-P[l-1]",
 complexity: "препроцессинг O(n), запрос O(1)",
 demo: "prefix-sum",
 simulator: "prefix-sums-2d",
},
{
 id: "sparse-table",
 labels: [
 "разреженная таблица", "разреженная таблица", "sparse-table",
 "разреженная", "двоичные подъёмы", "двоичный подъём",
],
 title: "Разреженная таблица (метод двоичных подъёмов)",
 short:
 "Заранее считаем ответ для всех отрезков длины 1, 2, 4, ..., m.",
 formal: "ST[i][j] = op(ST[i][j-1], ST[i+2^(j-1)][j-1])",
 complexity: "препроцессинг O(n log n), запрос O(1)",
 demo: "sparse-table",
 simulator: "sparse-table",
},
{
 id: "segment-tree",
 labels: ["дерево отрезков", "дерево отрезков", "дерево отрезков"],
 title: "Дерево отрезков",
 short:
 "Матрёшка из отрезков: корень отвечает за весь массив, а потом делится на две половины.",
 formal: "Строится за O(n), запрос и обновление — O(log n)",

```



```

 },
 {
 id: "amortized",
 labels: [
 "амортизированная", "амортизированный", "амортизи",
 "амортизированное", "амортизированного", "амортизи",
],
 title: "Амортизированная сложность",
 short:
 "Средняя цена операции «по абонементу»: одна опер",
 formal: "Метод потенциалов: дорогая операция оплачи",
 ассив.",
 demo: "amortized",
 },
 {
 id: "np",
 labels: [
 "NP-полная", "NP-полнота", "NP-трудная", "NP-полн",
 "класс P", "полином", "полиномиальное",
],
 title: "Класс NP и NP-полнота",
 short:
 "NP — «решение проверить легко, найти трудно» (су",
 "е в NP.",
 formal: "Задача NP-полна, если она в NP и к ней пол",
 demo: "np",
 },
 {
 id: "potential",
 labels: ["потенциал", "потенциалы", "потенциалов",
 title: "Потенциалы (перевзвешивание Джонсона)",
 short:
 "Трюк «поднять все дороги на одинаковую высоту»: ",
 formal: " $w'(u,v) = w(u,v) + h(u) - h(v)$, где h — ра",
 к путей сохраняется.",
 demo: "relax",
 simulator: "johnson-algo",
 },
],

```

```

 complexity: "O(n)",
 demo: "prefix-function",
 simulator: "string-z-func",
},
{
 id: "dp",
 labels: [
 "динамическое программирование", "динамики", "дин.
 "мемоизация", "мемоизацию", "подзадач", "подзадач
],
 title: "Динамическое программирование",
 short:
 "Решаем задачу через ответы на её меньшие копии и
 formal:
 "Нужны: оптимальная подструктура + перекрывающиеся
 ция).",
 demo: "dp",
 simulator: "dynamic-programming",
},
{
 id: "shortest-path",
 labels: ["кратчайший путь", "кратчайшие пути", "кра
 title: "Кратчайший путь",
 short:
 "Самый дешёвый маршрут из A в B. «Дешёвый» – по с
 formal:
 "Дейкстра – неотрицательные веса, $O(E \log V)$. Фор
 ы, $O(V^3)$.",
 demo: "relax",
 simulator: "dijkstra",
},
{
 id: "negative-cycle",
 labels: ["отрицательный цикл", "отрицательного цикл
 title: "Отрицательный цикл",
 short:
 "Круг, пройдя по которому вы становитесь «богаче»
 formal:

```

```

 simulator: "planarity-euler-formula",
},
{
 id: "splay",
 labels: [
 "splay", "splay-дерево", "AVL", "AVL-дерево", "вс
],
 title: "Splay и повороты",
 short:
 "Каждый раз, когда к узлу обратились, его «вытягивае
 formal:
 "Три случая: zig (родитель – корень), zig-zig (уз
 рацию.",
 demo: "amortized",
 simulator: "splay-tree",
},
{
 id: "adjacency",
 labels: ["матрица смежности", "список смежности", "
 title: "Способы хранить граф",
 short:
 "Матрица смежности – таблица «есть ли ребро», быс
 й, экономно для разреженных графов.",
 formal: "Матрица: проверка $O(1)$, память $O(V^2)$. Спис
},
{
 id: "dijkstra",
 labels: ["Дейкстра", "Дейкстры", "Дейкстре", "Дейкс
 title: "Алгоритм Дейкстры",
 short:
 "Жадный «навигатор»: всегда раскрываем ближайший
 льные.",
 formal:
 "Куча хранит пары (расстояние, вершина). Достали
 complexity: " $O(E \log V)$ с бинарной кучей",
 demo: "relax",
 simulator: "dijkstra",
},

```

```

 "Множество посещённых + куча рёбер на границе. До-
 усок.",
 complexity: "O(E log V)",
 demo: "mst",
 simulator: "mst-prima",
 },
 {
 id: "boruvka",
 labels: ["Борувка", "Борувки", "Борувку", "Boruvka"],
 title: "Алгоритм Борувки",
 short:
 "Все компоненты одновременно шлют «гонца» к ближай-
 formal:
 "Фаза: для каждой компоненты ищем минимальное исх-
 complexity: "O(E log V)",
 demo: "mst",
 simulator: "mst-boruvka",
 },
 {
 id: "kruskal",
 labels: ["Краскал", "Краскала", "Краскалу", "Kruska"],
 title: "Алгоритм Краскала",
 short:
 "Сортируем все рёбра по весу и берём подряд те, ч
 formal:
 "Сортировка O(E log E) + E операций union/find. С
 complexity: "O(E log E)",
 demo: "mst",
 simulator: "mst-kruskal",
 },
 {
 id: "components",
 labels: [
 "компонента связности", "компоненты связности", "
 "компоненте связности", "связности", "связный", "
],
 title: "Компоненты связности",
 short:

```

```

 title: "Treap = Tree + Heap",
 short:
 "Одно дерево живёт по двум правилам сразу: по ключу
 и по высоте. Случайность и держит его сбалансированным.",
 formal:
 "Базис — split(t, x) (разрезать по ключу) и merge(t1, t2)
 (соединить). Ожидаемая высота $O(\log n)$.",
 complexity: "ожидаемо $O(\log n)$ ",
 demo: "heap",
},
{
 id: "bst",
 labels: ["бинарное дерево поиска", "дерево поиска",
 title: "Бинарное дерево поиска",
 short:
 "Слева всё меньше корня, справа — всё больше. Поиск
 по ключу.",
 formal:
 "Без балансировки дерево может вырождаться в список.",
 complexity: " $O(\text{высоты})$: от $O(\log n)$ до $O(n)$ ",
 demo: "segment-tree",
},
{
 id: "inclusion-exclusion",
 labels: [
 "включений-исключений", "включения-исключения", "принцип
 включения-исключения", "вырезание прямоугольников",
],
 title: "Включения-исключения на префиксных суммах",
 short:
 "Чтобы получить сумму прямоугольника, берём большую
 сумму и вычитаем лишнее.",
 formal:
 "Sum(x1..x2, y1..y2) = P[x2][y2] - P[x1-1][y2] - P[x2][y1-1] + P[x1-1][y1-1]",
 complexity: "запрос $O(1)$ после $O(nm)$ предподсчёта",
 demo: "prefix-sum",
 simulator: "prefix-sums-2d",
},
{

```

```

 short:
 "Локальная перевеска двух узлов: ребёнок встаёт на

 дившемся месту. Порядок ключей при этом не ломается.",
 formal:
 "Правый поворот вокруг p: $x = p.left$; $p.left = x$.

 g и Zig-Zag.",
 complexity: "O(1)",
 demo: "zig",
 simulator: "splay-rotations",
},
{
 id: "zig",
 labels: ["Zig", "Zag", "зиг"],
 title: "Zig – одиночный поворот",
 short:
 "Самый простой случай: родитель x уже сидит в корне.

 канчивается, если глубина была нечётной.",
 formal: "Применяется ровно один раз за всю операцию",
 demo: "zig",
 simulator: "splay-rotations",
},
{
 id: "zig-zig",
 labels: ["Zig-Zig", "зиг-зиг", "ZigZig"],
 title: "Zig-Zig – x и родитель с одной стороны",
 short:
 "Дерево вытянулось в «бамбук»: $g \rightarrow p \rightarrow x$ идут в одну

 м (p, x). Именно это вдвое сокращает глубину всего дерева.",
 formal:
 "Если крутить наивно (два раза поднимать x), бамбук

 demo: "zig-zig",
 simulator: "splay-rotations",
},
{
 id: "zig-zag",
 labels: ["Zig-Zag", "зиг-заг", "ZigZag", "зигзаг"],
 title: "Zig-Zag – x и родитель с разных сторон",
 short:

```

```

<p class="text-slate-300 text-sm mb-4">Пред
оезда). Алгоритмы поиска кратчайшего пути п
<div class="mt-8 mb-4">
 <div id="slot-mnemonic-cards"></div>
</div>
</div>
</section>`
},
{
 id: "dijkstra",
 title: "Билет 14. Дейкстра",
 type: "html",
 category: "Графы",
 content: `<section id="dijkstra-content" class="mb-
<div class="flex items-center mb-6">
 <span class="bg-blue-600 text-white px-4 py-1 r
 <h2 class="text-3xl font-bold text-white">Дейкс
</div>

<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl bord
 <h3 class="text-xl font-bold text-emerald-4

 <div class="bg-slate-900 p-4 rounded-lg mb-1
 <p class="text-lg text-emerald-300 ital
 <p class="text-xl font-mono text-white":
 </div>

 <div class="grid grid-cols-1 xl:grid-cols-2
 <!-- Аналогия 1 -->
 <div class="bg-slate-800 p-6 rounded-lg
 <div class="absolute -top-3 left-4 l
 </div>
 <div class="border-emerald-500 shadow-md">
 Аналогия 1: Позитивное плани
 </div>
 <p class="text-slate-300 text-sm mb
 (нельзя поехать и заработать). Он всегда в

```

```
 <h2 class="text-3xl font-bold text-white">Формат
 </div>
```

```
<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border"
 <h3 class="text-xl font-bold text-rose-400"
 <div class="bg-slate-900 p-4 rounded-lg mb-4"
 <p class="text-lg text-rose-300 italic"
 <p class="text-xl font-mono text-white"
 </div>
```

```
<div class="grid grid-cols-1 xl:grid-cols-2"
 <!-- Аналогия 1 -->
 <div class="bg-slate-800 p-6 rounded-lg"
 <div class="absolute -top-3 left-4 l
 order border-emerald-500 shadow-md">
 ♂ Аналогия 1: Параноик
 </div>
 <p class="text-slate-300 text-sm mb-4"
 еряет ВСЕ возможные дороги V-1 раз подряд.
 <div id="slot-chapter-image-bellman"
 </div>
```

```
 <!-- Аналогия 2 -->
 <div class="bg-slate-900 p-6 rounded-lg"
 <div class="absolute -top-3 left-4 l
 border-rose-500 shadow-md">
 Отрицательный цикл
 </div>
 <p class="text-slate-300 text-sm mb-4"
 г станет ЕЩЕ короче — значит мы попали во в
 </div>
 </div>
```

```
 <div id="slot-bellman-viz" class="mt-8"></div>
 </div>
</div>
```



```

 Вре́мя работы
 </div>
 <p class="text-slate-300 text-sm mb-6">
 три вложенных цикла. Сложность $O(V^3)$. Если
 </div>
 </div>

 <div id="slot-floyd-viz" class="mt-8"></div>
 </div>
</div>
</section>`
 },
 {
 id: "aho-corasick",
 title: "Билет 23. Алгоритм Ахо-Корасик",
 type: "html",
 category: "Строки",
 content: `<section id="aho-corasick" class="mb-12">
<div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
</div>

<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border-l">
 <h3 class="text-xl font-bold text-blue-400 mb-4">

 <div class="bg-slate-900 p-4 rounded-lg mb-6 text-white">
 <p class="text-sm text-blue-300 italic mb-2">Осложнен
 <p class="text-lg font-mono text-white">Бор (Тр
 </div>
 <p class="text-slate-300 text-sm">Позволяет найти
 -mono text-amber-300 bg-black/30 px-1 rounded">
 y.</p>
 </div>

 <!-- Аналогии -->
 <div class="grid grid-cols-1 xl:grid-cols-2 gap-6">

```

```
// Расчет ссылок идет через BFS (level-order traversal)
// так как для вычисления link вершины v на глубине d,
// её предки на глубине d-1 уже должны иметь вычисленные
void buildLinks() {
 queue<int> q;
 // Корни и их дети
 for (auto const& [ch, child] : nodes[0].trie) {
 nodes[child].link = 0;
 q.push(child);
 }

 while (!q.empty()) {
 int v = q.front(); q.pop();
 for (auto const& [ch, child] : nodes[v].trie) {
 // Суффиксная ссылка ребёнка – это переход к
 nodes[child].link = nodes[nodes[v].link].go

 // Если суффиксная ссылка сама является словом
 // Иначе наследуем terminal_link
 if (nodes[nodes[child].link].is_terminal) {
 nodes[child].term_link = nodes[child].link;
 } else {
 nodes[child].term_link = nodes[nodes[ch].term_link];
 }

 q.push(child);
 }
 }
}

</div>
</div>
</details>
</div>
</section>`
},
{
 id: "alg-map",
```

```

 options: [
 "Есть эйлеров цикл (вернусь домой!)",
 "Есть эйлеров путь (из одной нечётной в другую)",
 "Ни пути, ни цикла нет — это полный провал"
],
 correctIndex: 1,
 explanation: "Если нечётных вершин ровно 2 — это эйлеров путь, заканчиваем в другой."
},
{
 question: "Почему гамильтонов цикл — это NP-полная задача?",
 options: [
 "Потому что Гамильтон — болезнь, и лечить её тяжело",
 "Потому что для эйлерова цикла есть простой критерий",
 "Потому что Эйлер был умнее Гамильтона"
],
 correctIndex: 1,
 explanation: "Для эйлерова цикла работает критерий на количество нечётных вершин. Это NP-полная задача, которую не решить за полиномиальное время."
},
{
 question: "Куда ведёт эйлеров путь, если нечётных вершин больше 2?",
 options: [
 "Всё ещё можно пройти по всем рёбрам ровно раз",
 "Такого графа не существует",
 "Эйлерова пути нет — нужно удалять лишние рёбра"
],
 correctIndex: 2,
 explanation: "Если нечётных вершин больше 2, эйлерова пути нет. Если ровно 2 нечётные или 0."
}
],
"bridges-code": [
 {
 question: "После DFS у нас low[u] = 4, а tin[v] = 2. Что это значит?",
 options: [
 "Да, так как low[u] (4) > tin[v] (2)",
 "Нет, low[u] должно быть меньше или равно tin[v]",
 "Нужно удалить рёбра, ведущие к v"
]
 }
]

```

```
 },
 {
 question: "Сколько граней у планарного графа с $V=7$?",
 options: [
 "3 грани",
 "5 граней",
 "10 граней"
],
 correctIndex: 1,
 explanation: " $F = E - V + 2 = 10 - 7 + 2 = 5$. Не",
 },
 {
 question: "Почему K_5 непланарен?",
 options: [
 "Потому что $10 > 3*5 - 6 = 9$ — нарушение",
 "Потому что Эйлер был против",
 "Потому что 5 вершин — несчастливое число"
],
 correctIndex: 0,
 explanation: "У K_5 : $V=5$, $E=10$, $3V - 6 = 9$. $10 > 9$ ",
 }
]
};
```

---

## РАЗДЕЛ: БИЛЕТЫ (УЧЕБНЫЙ КОНТЕНТ)

---

---

ФАЙЛ 23/83: src/data/tickets/ticket1\_5.ts

КАТЕГОРИЯ: Билеты (учебный контент) | СТРОК: 277 | РАЗМ

---

```
import { Chapter } from '../types';
```

```
export const tickets1to5: Chapter[] = [
 {
```

```

 </div>
 <p class="text-slate-300 text-sm mb-10">
 ше, и так далее. Если нам нужно покрасить п
 Внутри всё синее!" на большую коробку. Опус
 </div>
 </div>

 <details class="bg-slate-900/40 rounded-lg border-1
 <summary class="p-4 font-bold text-slate-300
 -b border-slate-700/50">
 Строгое доказательство / Код (Скрыто)
 </summary>
 <div class="p-5 text-sm text-slate-300">
 <pre class="bg-slate-950 p-4 rounded" style="font-family: monospace; font-size: 0.9em; margin: 0; border: 1px solid #334d5b; border-radius: 0.5em; padding: 1em 1.5em; color: #e6f2ff; line-height: 1.2; min-height: 150px; position: relative; background: linear-gradient(to top right, transparent 49%, #334d5b 49% 51%, #334d5b 51% 99%, transparent 99%);">
function push(node) {
 if (lazy[node] !== 0) {
 lazy[2 * node] += lazy[node];
 tree[2 * node] += lazy[node];
 lazy[2 * node + 1] += lazy[node];
 tree[2 * node + 1] += lazy[node];
 lazy[node] = 0;
 }
}
</pre>
 </div>
 </details>
 </div>
 </div>
</section>
`
 },
 {
 id: "sparse-table",
 title: "2. Двумерная разреженная матрица (Sparse Table)",
 type: "html",
 description: "Разреженная таблица для идемпотентных операций",
 category: "Продвинутые структуры",
 content: `
<section id="sparse-table" class="mb-12 scroll-mt-10">

```

```

<pre class="bg-slate-950 p-4 rounded">
int k = log2(R - L + 1);
int minimum = min(st[L][k], st[R - (1 << k) + 1][k]);</pre>
</div>
</details>
</div>
</div>
</section>`
},
{
 id: "treap",
 title: "3. Декартово дерево (Treap)",
 type: "html",
 description: "Дерево поиска по ключу и Куча по приоритету",
 category: "Продвинутые структуры",
 content: `
<section id="treap" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 Treap
 <h2 class="text-2xl sm:text-3xl font-bold text-white">Декартово дерево</h2>
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-600">
 <h3 class="text-xl font-bold text-blue-400">Описание</h3>
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic">Дерево поиска по ключу и Куча по приоритету</p>
 <p class="text-xl font-mono text-white">метода: Split и Merge.</p>
 </div>
 <div class="grid grid-cols-1 xl:grid-cols-2">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 Аналогия 1: Удар катаной (Split)
 </div>
 <p class="text-slate-300 text-sm mb-4">по значению X. Все, кто меньше X улетают в левое</p>
 </div>
 </div>
 </div>
 </div>
`

```

```

type: "html",
description: "Дерево поиска, которое самобалансируется",
category: "Продвинутые структуры",
content: `
<section id="splay-tree" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-blue-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic">
 <p class="text-xl font-mono text-white">
 (Zig, Zig-Zig, Zig-Zag), гарантируя амортизацию
 </div>
 <div class="grid grid-cols-1 xl:grid-cols-2">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 Аналогия 1: VIP-лифт
 </div>
 <p class="text-slate-300 text-sm mb-4">
 вится корнем дерева (VIP-статус). Если ты член VIP-клуба,
 ка почти не требуется времени!</p>
 </div>
 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-rose-500 shadow-md">
 Аналогия 2: Балансировка через бамбук
 </div>
 <p class="text-slate-300 text-sm mb-4">
 и становиться кривым как бамбук. Но как толстый бамбук,
 </p>
 </div>
 </div>
 </div>
 <details class="bg-slate-900/40 rounded-lg">

```

```

 </div>

 <div>
 <strong class="text-white block">
 <p>Хотя один поиск может стоить
емах обладают прекрасным свойством: они не
по пути. "Палочное" дерево прессуется в сб
осов.</p>
 </div>
 </div>
 </div>
</div>
</section>`
 },
 {
 id: "prefix-sums-2d",
 title: "5. Двумерная матрица префиксных сумм",
 type: "html",
 description: "Сжатие суммы подматрицы за $O(1)$ ",
 category: "Алгоритмы матрицы",
 content: `
<section id="prefix-sums-2d" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-emerald-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">
 <p class="text-xl font-mono text-white">
 , $y1...y2$) = $P[x2][y2]$ - $P[x1-1][y2]$ - $P[x2$
 </div>
 <div class="grid grid-cols-1 xl:grid-cols-2">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4">

```



```

 <p class="text-lg text-emerald-300 italic">
 <p class="text-xl font-mono text-white">
 </div>
 <div class="grid grid-cols-1 xl:grid-cols-2">
 <!-- Аналогия 1 -->
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border
rder border-emerald-500 shadow-md">
 Аналогия 1: Позитивное планирование
 </div>
 <p class="text-slate-300 text-sm mb-2">
(нельзя поехать и заработать). Он всегда в пути
 </div>
 <!-- Аналогия 2 -->
 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border
border-rose-500 shadow-md">
 Ограничения
 </div>
 <p class="text-slate-300 text-sm mb-2">
дный и не умеет пересматривать уже "зафиксированный
 </div>
 </div>
 <div id="slot-dijkstra-viz" class="mt-8"></div>
 </div>
 </div>
</section>`
 },
 {
 id: "bellman-ford",
 title: "15. Графы. Поиск кратчайшего пути. Форд-Беллман",
 type: "html",
 category: "Графы. Пути",
 content: `
<section id="bellman-ford-content" class="mb-12 scroll-my-20">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">

```

```

<div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-indigo-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-indigo-300 italic">
 <p class="text-xl font-mono text-white">
 </div>

 <div class="bg-slate-800 p-6 rounded-lg border">
 <div class="absolute -top-3 left-4 bg-slate-900
border-emerald-500 shadow-md">
 Суть фокуса (По шагам)
 </div>
 <p class="text-slate-300 text-sm mb-4">
 Главный герой – внешний цикл
 шаг <code class="bg-slate-900 text-pink-400">
 шу себе проходить через вершину k?»</i>
 </p>
 <ul class="text-sm text-slate-300 space-y-2">
 Шаг k=0: Ты знаешь только
 Шаг k=1: Разрешаем пересадку
е) через путь <code class="text-indigo-300">
 Шаг k=2: Разрешаем пересадку
нутри этих путей уже могут быть пересадки ч
 ...
 Шаг k=N: Ты проверил все

 </div>
</div>
</div>
</section>`
 },
 {

```

```

<div id="slot-johnson-viz" class="mt-8"></div>
</div>
</section>`
},
{
 id: "mst-kruskal",
 title: "18. Графы. Остовное дерево. Краскал",
 type: "html",
 category: "Графы. Остовные",
 content: `
<section id="mst-kruskal" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-emerald-400">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-emerald-500">
 <h3 class="text-xl font-bold text-emerald-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">
 <p class="text-xl font-mono text-white">
 (проверяем через DSU) - берем в ответ.</p>
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 Аналогия: Прокладка интернет
 </div>
 <p class="text-slate-300 text-sm mb-4">
 с самых дешевых дорог в стране". Он строит
 единую сеть.</p>
 </div>
 </div>
 </div>
 </div>
</div>
</section>`

```

```

 type: "html",
 category: "Графы. Остовные",
 content: `
<section id="mst-boruvka" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-slate-800">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-600">
 <h3 class="text-xl font-bold text-emerald-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">
 <p class="text-xl font-mono text-white">
 ается с соседом.</p>
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 Аналогия: Племена
 </div>
 <p class="text-slate-300 text-sm mb-4">
 изкому племени для объединения. К вечеру пле
 ства шлют гонцов к ближайшему чужому царству.
 </div>
 </div>
 </div>
 </div>
 </div>
</section>`
 }
];

```

```

</div>

<!-- Аналогия 2 -->
<div class="bg-slate-900 p-6 rounded-lg"
 <div class="absolute -top-3 left-4
border-rose-500 shadow-md">
 Аналогия 2: LLM стриминг
</div>
 <p class="text-slate-300 text-sm mb-4">
 Для LLM, которая стримит токены
 каждом шаге. Если мы частично совпали с запятой
 акого места этого слова мы можем продолжить
 </p>
</div>
</div>

<details class="bg-slate-900/40 rounded-lg"
 <summary class="p-4 font-bold text-slate-300"
 -b border-slate-700/50">
 Пример вычисления (Скрыто)
</summary>
 <div class="p-5 text-sm text-slate-300">
 <p>Тот же пример: abcabcd</p>
 <ul class="list-disc pl-5 space-y-2">
 a: Префиксов нет. <code></code>
 ab: Из начала ("a")
 >
 abc: Опять мимо. <code></code>
 abca: О! Начало ("a"
 </code>
 abcab: Начало ("ab")
 /li>
 abcabc: Начало ("abc
 e>
 abcabcd: "d" всё исп
 "abca" vs "abcd"). Конец "abcd" не совпадает

 </div>

```



```

<section id="aho-corasick" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>

 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border-l">
 <h3 class="text-xl font-bold text-blue-400 mb-4">

 <div class="bg-slate-900 p-4 rounded-lg mb-6 text">
 <p class="text-sm text-blue-300 italic mb-2">0c
 <p class="text-lg font-mono text-white">Бор (Tr
 </div>
 <p class="text-slate-300 text-sm">Позволяет найти
 го один проход.</p>
 </div>

 <!-- Аналогии -->
 <div class="grid grid-cols-1 xl:grid-cols-2 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg border bo">
 <div class="absolute -top-3 left-4 bg-slate-700
 emerald-500 shadow-md">
 Аналогия 1: Паутина (Бор)
 </div>
 <p class="text-slate-300 text-sm mb-4">Представ
 ет – мы падаем по суффиксной ссылке ("
 </div>

 <div class="bg-slate-900 p-6 rounded-lg border-2">
 <div class="absolute -top-3 left-4 bg-rose-900
 -500 shadow-md">
 Аналогия 2: Матёшка (Терминальные)
 </div>
 <p class="text-slate-300 text-sm mb-4">Представ
 нашли и "he", потому что оно спрятано внутри
 </div>
 </div>
 </div>
 </div>
```

```

</div>
<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border"
 <h3 class="text-xl font-bold text-purple-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-purple-300 italic">
 <p class="text-xl font-mono text-white">
 Сведение (Reduction) A->B: Если я умею реша
 </div>
 <div class="grid grid-cols-1 xl:grid-cols-2"
 <!-- Аналогия 1 -->
 <div class="bg-slate-800 p-6 rounded-lg"
 <div class="absolute -top-3 left-4 l
 rder border-emerald-500 shadow-md">
 Аналогия 1: Судoku (P vs NP)
 </div>
 <p class="text-slate-300 text-sm mb-4">
 пример сортировка чисел). Класс NP –
 енную сетку (ответ), он БЫСТРО (за класс P)
 </div>
 <!-- Аналогия 2 -->
 <div class="bg-slate-900 p-6 rounded-lg"
 <div class="absolute -top-3 left-4 l
 rder border-purple-500 shadow-md">
 Аналогия 2: Машина-переводчик
 </div>
 <p class="text-slate-300 text-sm mb-4">
 ь отличный переводчик на английский (Сведенн
 аче B, то B – это NP-Полная (сосредо
 </div>
 </div>
 </div>
 </div>
 </div>
 </div>
 </div>
 </div>
 </div>
</section>`
}
];

```



```

<div class="bg-slate-900 p-6 rounded-lg" style="border: 1px solid #000000; border-radius: 10px;">
 <div class="absolute -top-3 left-4 right-4 bottom-4 border border-rose-500 shadow-md">
 Аналогия: Списки = Контакты
 </div>
 <p class="text-slate-300 text-sm mb-0">
 т. Оптимально для почти всех задач.</p>
 </div>
</div>

```

```

{/* DFS vs BFS */}
<div class="bg-slate-700/50 p-6 rounded-xl border border-rose-500" style="border: 1px solid #000000; border-radius: 10px;">
 <h3 class="text-xl font-bold text-indigo-400">Аналогия DFS vs BFS</h3>

 <div class="grid grid-cols-1 xl:grid-cols-2 gap-4">
 <!-- Аналогия DFS -->
 <div class="bg-slate-800 p-6 rounded-lg" style="border: 1px solid #000000; border-radius: 10px;">
 <div class="absolute -top-3 left-4 right-4 bottom-4 border border-indigo-500 shadow-md">
 ♂ DFS (Поиск в глубину) = Жадный алгоритм
 </div>
 <p class="text-slate-300 text-sm mb-0">
 Что делает: Жадный алгоритм выбирает ближайшую вершину (например, минимальную по номеру) и пробует другой путь.
 </p>
 <ul class="text-xs text-indigo-300">
 Где используется:
 Топологическая сортировка (линейное время)
 Поиск мостов и точек сочленения
 Сильно связанные компоненты Киркмана
 Поиск циклов и просто проверка

 </div>
 </div>

```

```

<!-- Аналогия BFS -->
<div class="bg-slate-900 p-6 rounded-lg" style="border: 1px solid #000000; border-radius: 10px;">

```



```

 order border-emerald-500 shadow-md">
 Аналогия 1: Острова
 </div>
 <p class="text-slate-300 text-sm mb-6">
 карту – остались ли серые (неисследованные)
 ь через океан – столько и компонент.</p>
 </div>
 </div>
</div>
</section>`,
 },
 {
 id: "top-sort",
 title: "9. Графы. Топологическая сортировка",
 type: "html",
 description: "Разложение задач по порядку выполнения",
 category: "Графы",
 content: `
<section id="top-sort" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-blue-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic">
 <p class="text-xl font-mono text-white">
 перед.</p>
 </div>
 <div class="grid grid-cols-1 xl:grid-cols-2">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 l
 order border-emerald-500 shadow-md">
 Аналогия 1: Учеба в вузе
 </div>
 </div>
 </div>
 </div>
 </div>
 </div>
`
 }
]

```

```

 for (int i = 0; i < n; ++i) {
 if (!visited[i]) dfs(i);
 }
 reverse(ans.begin(), ans.end()); // Разворачиваем с
}

```

```

 </div>
 </div>
 </details>
 </div>
 </div>
</section>,
{
 id: "scc-kosaraju",
 title: "10. Графы. Компоненты сильной связности (SCC)",
 type: "html",
 description: "Разбиение орграфа на макро-вершины. Алгоритм Косарая",
 category: "Графы. Продвинутые",
 content: `
<section id="scc-kosaraju" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 Теория
 <h2 class="text-2xl sm:text-3xl font-bold text-emerald-400">10. Графы. Компоненты сильной связности (SCC)
 </div>

 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-700">
 <h3 class="text-xl font-bold text-emerald-400">Макро-вершины
 </div>

 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">Макро-вершиной (или макро-узлом) орграфа называют
 <div class="text-left font-mono text-sm">
 <p><strong class="text-white">Компонентой связности (или макро-вершиной)
 н <i>ориентированного</i> графа, в котором
 </div>
 </div>

 <p class="text-slate-300 text-sm mb-4">

```

```

 -b border-slate-700/50">
 Душный мод: Код Алгоритм Косарайю
 </summary>
 <div class="p-5 text-sm text-slate-300">
 <p class="text-emerald-400 font-bold">
 <ol class="list-decimal list-inside">
 Запускаем DFS на исходном графе

 Разворачиваем этот список (только вершины)

 Переворачиваем все ребра в графе

 Идем по <code>order</code>:

 </div>
 </details>
</div>
</div>
</section>`,
 },
 {
 id: "graph-bridges",
 title: "11. Графы. Мосты",
 type: "html",
 description: "Рёбра, удаление которых расхреначивает граф",
 category: "Графы. Продвинутое",
 content: `
<section id="graph-bridges" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-rose-400">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-rose-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-rose-300 italic">
 <p class="text-xl font-mono text-white">
 и $fup[v] > tin[u]$.</p>
 </div>
 </div>
 </div>
</section>

```

```

 <span class="text-rose-400 font
 <p class="text-xl font-bold tex
 <p class="text-xs text-slate-400
 </div>
 </div>
</div>

<p class="text-slate-300 text-sm">
 Важно: Это работает то
 епятся к точкам сочленения. То есть,
 ег - это тупик со степенью 1).
</p>
</div>

<div class="grid grid-cols-1 xl:grid-cols-2 gap
 <div class="bg-slate-800 p-6 rounded-lg bor
 <div class="absolute -top-3 left-4 bg-s
 rder-rose-500 shadow-md">
 Аналогия: Главный перекресток
 </div>
 <p class="text-slate-300 text-sm mb-4">
 Представь город, где два района сое
 её перекроют (удалят вершину) – районы буд
 g>хрупкость системы.
 </p>
 </div>

 <div class="bg-slate-900 p-6 rounded-lg bor
 <div class="absolute -top-3 left-4 bg-e
 er border-emerald-500 shadow-md">
 Телеком: Центральный свитч
 </div>
 <p class="text-slate-300 text-sm mb-4">
 У тебя несколько компьютеров в одно
 абелем (мостом). Сами свитчи – это точки со
 </p>
 </div>
</div>
```

```

 <div class="bg-indigo-950/60 p-6 rounded-xl border"
 <h4 class="text-lg font-bold text-indigo-400"
 <p class="text-slate-300 text-sm mb-4">
 Это глубокий дуальный мост между ориентир
 </p>
 <ul class="list-disc list-inside text-sm text"
 <strong class="text-rose-400">Точки
ong> и показывают физическую уязвимость свя
 <strong class="text-emerald-400">SCC
ависимые "конгломераты".
 Как точка сочленения рушит
Косарайю, свернув каждую SCC в одну супер-в
(сделаем неориентированным), то любая <strong
о и есть критическая SCC! Если удалить эту
. Одно слабое звено изолирует целые касты S

 </div>
 </div>
</section>`,
 },
 {
 id: "graph-euler",
 title: "13. Графы. Эйлеров цикл",
 type: "html",
 description: "Пройти по всем ребрам ровно 1 раз.",
 category: "Графы",
 content: `
<section id="graph-euler" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 <span class="bg-blue-600 text-white px-4 py-1 rounded"
 <h2 class="text-2xl sm:text-3xl font-bold text-white"
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border"
 <h3 class="text-xl font-bold text-indigo-400"
 <div class="bg-slate-900 p-4 rounded-lg mb-4"
 <p class="text-lg text-indigo-300 italic">

```

```

interface Edge {
 u: number;
 v: number;
}

const nodes: Node[] = [
 { id: 0, x: 250, y: 50, label: "0" },
 { id: 1, x: 100, y: 150, label: "1" },
 { id: 2, x: 250, y: 250, label: "2" },
 { id: 3, x: 400, y: 150, label: "3" },
 { id: 4, x: 550, y: 150, label: "4" },
 { id: 5, x: 700, y: 50, label: "5" },
 { id: 6, x: 700, y: 250, label: "6" },
];

const edges: Edge[] = [
 { u: 0, v: 1 },
 { u: 1, v: 2 },
 { u: 2, v: 0 },
 { u: 2, v: 3 },
 { u: 3, v: 4 }, // 3 and 4 are articulation points (3
 { u: 4, v: 5 },
 { u: 5, v: 6 },
 { u: 6, v: 4 },
];

// Adjacency list
const adj: number[][] = Array.from(
 { length: Object.keys(nodes).length },
 () => [],
);

edges.forEach((e) => {
 adj[e.u].push(e.v);
 adj[e.v].push(e.u);
});

interface StepInfo {

```



```

 recordStep(`Заходим в вершину ${v}. tin[${v}]=${c
for (const to of adj[v]) {
 if (to === p) continue;

 if (visited.has(to)) {
 currentLow[v] = Math.min(currentLow[v], curren
 recordStep(
 `Обратное ребро ${v}-${to}. Обновляем low[${v}
 v,
);
 } else {
 children++;
 dfs(to, v);
 currentLow[v] = Math.min(currentLow[v], curren

 recordStep(
 `Возврат в ${v} из ${to}. Обновляем low[${v}
 v,
);

 if (currentLow[to] >= currentTin[v] && p !==
 ap.add(v);
 recordStep(
 `Найдена точка сочленения: вершина ${v}
 v,
);
 }
 }
}

if (p === -1 && children > 1) {
 ap.add(v);
 recordStep(
 `Корень дерева DFS ${v} имеет >1 детей. Это
 v,
);
}

```

```

const togglePlay = () => setIsPlaying(!isPlaying);
const reset = () => {
 setIsPlaying(false);
 setCurrentStepIndex(0);
};

return (
 <div className="bg-slate-900 rounded-xl border border-
 <div className="bg-slate-800 p-4 border-b border-
 <h3 className="font-bold text-white flex items-
 <Info className="w-5 h-5 text-rose-400" />
 Моделирование поиска точек сочленения
 </h3>

 <div className="flex items-center gap-2 bg-slate-
 <button
 onClick={togglePlay}
 className="flex items-center gap-2 px-3 py-
 text-white"
 >
 {isPlaying ? (
 <Pause className="w-4 h-4 ml-1" />
) : (
 <Play className="w-4 h-4 ml-1" />
)}
 {isPlaying ? "Пауза " : "Старт "}
 </button>

 <button
 onClick={reset}
 className="flex items-center justify-center
 ors"
 >
 <RotateCcw className="w-4 h-4" />
 </button>
 </div>
 </div>

```

```

 ? "#10b981"
 : "#1e293b";
const stroke = isCurrent
 ? "#60a5fa"
 : isAp
 ? "#f43f5e"
 : isVisited
 ? "#34d399"
 : "#334155";
const r = isAp ? 24 : 20;

return (
 <g key={node.id}>
 <circle
 cx={node.x}
 cy={node.y}
 r={r}
 fill={fill}
 stroke={stroke}
 strokeWidth="3"
 className="transition-all duration-100"
 />
 <text
 x={node.x}
 y={node.y}
 dy=".3em"
 textAnchor="middle"
 fill="white"
 fontSize="14px"
 fontWeight="bold"
 className="select-none pointer-events-none"
 >
 {node.label}
 </text>

 {isVisited && (
 <>
 <text

```

```

 {currentStep.desc}
 </p>
 </div>

 <div className="space-y-4 flex-1 overflow-y-auto">
 <div className="mb-4">
 <h5 className="text-xs text-slate-500 font-medium">Шаг 1</h5>
 <div className="flex items-center gap-2">
 <div className="w-3 h-3 rounded-full bg-rose-500 border-1px solid black">
 Точка сочленения
 </div>
 <div className="flex items-center gap-2">
 <div className="w-3 h-3 rounded-full bg-rose-500 border-1px solid black">
 Посещена
 </div>
 <div className="flex items-center gap-2">
 <div className="w-3 h-3 rounded-full bg-rose-500 border-1px solid black">
 Текущая
 </div>
 <div className="flex items-center gap-2">
 <div className="w-8 h-1 bg-rose-500 border-1px solid black">
 Мост
 </div>
 </div>
 </div>
 </div>
 </div>

 <div>
 <h5 className="text-xs text-slate-500 font-medium">ЛОГ ДЕЙСТВИЙ:</h5>
 <div className="space-y-2">
 {steps
 .slice(0, currentStepIndex + 1)
 .reverse()
 .map((step, idx) => (
 <div
 key={idx}
 className={`text-xs p-2 rounded $
xt-slate-500`} `}

```

```

 log: string;
 hasNegativeCycle?: boolean;
 activeCodeLine: number;
}

export default function BellmanFordViz() {
 const [hasCycle, setHasCycle] = useState(false);
 const [steps, setSteps] = useState<Step[]>([]);
 const [currentStepIndex, setCurrentStepIndex] = useSt
 const [isPlaying, setIsPlaying] = useState(false);

 const nodes: Node[] = [
 { id: 'S', x: 60, y: 150, name: 'База (S)' },
 { id: 'A', x: 160, y: 60, name: 'Застава (A)' },
 { id: 'B', x: 160, y: 240, name: 'Топь 1 (B)' },
 { id: 'C', x: 260, y: 150, name: 'Мост (C)' },
 { id: 'D', x: 350, y: 60, name: 'Топь 2 (D)' },
 { id: 'E', x: 350, y: 240, name: 'Лес (E)' },
 { id: 'F', x: 450, y: 150, name: 'Склад (F)' },
];

 const getEdges = (cycle: boolean): Edge[] => {
 const defaultEdges = [
 { u: 'S', v: 'A', w: 4 },
 { u: 'S', v: 'B', w: 5 },
 { u: 'B', v: 'C', w: -2 },
 { u: 'A', v: 'C', w: 1 },
 { u: 'C', v: 'D', w: 3 },
 { u: 'C', v: 'E', w: 4 },
 { u: 'D', v: 'F', w: 2 },
 { u: 'E', v: 'F', w: 1 },
];
 if (cycle) {
 return [...defaultEdges, { u: 'D', v: 'A', w: -6 }];
 } else {
 return [...defaultEdges, { u: 'D', v: 'A', w: -2 }];
 }
 };
};

```

```

 let anyUpdate = false;
 for (const edge of edges) {
 const uDist = dist[edge.u];
 const vDist = dist[edge.v];

 let updated = false;
 if (uDist !== 999 && uDist + edge.w < vDist) {
 dist = { ...dist, [edge.v]: uDist + edge.w };
 prev = { ...prev, [edge.v]: edge.u };
 updated = true;
 anyUpdate = true;
 }

 simulationSteps.push({
 iteration: iter, checkingEdge: { u: edge.u, v: edge.v },
 distances: { ...dist }, previous: { ...prev },
 log: `Итерация ${iter}: Проверяем ${edge.u} → ${edge.v}.
 ${updated ? 'Релаксация успешна! dist[${edge.v}] = ' + (uDist + edge.w) : ''}
 `,
 activeCodeLine: updated ? 6 : 5,
 });
 }

 if (!anyUpdate && iter < vCount - 1) {
 simulationSteps.push({
 iteration: iter, checkingEdge: null, distances: { ...dist },
 log: `Ранний выход после итерации ${iter}.`,
 activeCodeLine: 10,
 });
 break;
 }
}

if (simulationSteps[simulationSteps.length - 1].iteration === vCount - 1) {
 let cycleFound = false;
 for (const edge of edges) {
 if (dist[edge.u] !== 999 && dist[edge.u] + edge.w < dist[edge.v]) {
 cycleFound = true;
 }
 }
}

```

```

return (
 <div className="bg-slate-800/90 p-6 rounded-2xl border"
 <div className="flex flex-wrap items-center justify"
 <div className="flex items-center gap-3">
 <div className="p-2.5 bg-blue-600 text-white"

 </div>
 </div>
 <h3 className="text-2xl font-bold text-white"
 <p className="text-sm text-blue-300">Полный
 </p>
 </div>
 </div>

 <div className="flex items-center gap-3">
 <div className="bg-slate-900 p-1 rounded-lg b
 <button onClick={() => setHasCycle(false)} c
 bg-blue-600 text-white' : 'text-slate-400'>
 <button onClick={() => setHasCycle(true)} c
 r gap-1 ${hasCycle ? 'bg-rose-600 text-white
 </div>
 <button onClick={() => { setIsPlaying(false);
 g-slate-600 text-white px-3 py-2 rounded-lg
 <button onClick={() => setIsPlaying(!isPlaying
 sition-colors text-white ${isPlaying ? 'bg-
 <button onClick={() => { setIsPlaying(false);
 ; }} className="flex items-center gap-1 bg-l
 s">⏮ar </button>
 </div>
 </div>

 <div className="flex flex-col lg:flex-row gap-6 m
 {/* Граф */}
 <div className="w-full lg:w-2/3 bg-blue-950/20
 y-center min-h-[400px]">
 <div className="absolute top-3 left-3 bg-blue
 ld shadow-sm">
 Итерация {currentStep.iteration} (⏮ar {curr
 </div>

```

```

 Checking ? '#f59e0b' : isNeg ? '#f43f5e' :
 <text x={({uNode.x + vNode.x) / 2} y={
8fafc'} fontSize="11" fontWeight="bold" tex
 </g>
);
}})

{nodes.map((node) => {
 const dist = currentStep.distances[node.id]
 const isCheckingNode = currentStep.checkin
node.id);

 return (
 <g key={node.id} className="transition-
 <circle cx={node.x} cy={node.y} r={is
ckingNode ? '#7dd3fc' : '#64748b'} strokeWi
 <text x={node.x} y={node.y + 5} fill=
 <text x={node.x} y={node.y - 28} fill
ight="bold" textAnchor="middle" className="
 <text x={node.x} y={node.y + 35} fill
split(' ')[0]}</text>
 </g>
);
}})
</svg>

<div className="w-full bg-slate-950/80 p-4 ro
 <p className="font-semibold text-amber-400
 <p className="min-h-[40px] flex items-cente
</div>
</div>

{/* Список смежности */}
<div className="w-full lg:w-1/3 bg-emerald-950/
 <h4 className="text-lg font-bold text-emerald
 <div className="space-y-2 text-sm font-mono t
 {Object.keys(adjList).map(u => {
 <div key={u} className="flex flex-wrap g

```



```
 {/* Код алгоритма */}
 <div className="w-full lg:w-1/2 bg-slate-900/60">
 <div>
 <h4 className="text-lg font-bold text-slate-100">
 <div className="bg-slate-950/80 rounded-lg p-4">
 {codeLines.map(item => (
 <div key={item.line} className={`py-1.5 px-4 ${
 e === item.line ? 'bg-blue-900/80 text-amber-400' : 'text-slate-400'
 }`} >
 {item.line}
 {item.code}
 </div>
)
)}
 </div>
 </div>
 </div>
</div>
</div>
);
}
```

---

ФАЙЛ 29/83: src/components/BfsViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

---

```
import { useState, useEffect, useCallback } from 'react';
import { Play, Pause, RotateCcw, StepForward } from 'lucide-react';
```

```
const NODES = [
 { id: 0, label: '0', x: 300, y: 50 },
 { id: 1, label: '1', x: 150, y: 150 },
 { id: 2, label: '2', x: 450, y: 150 },
 { id: 3, label: '3', x: 75, y: 250 },
 { id: 4, label: '4', x: 225, y: 250 },
 { id: 5, label: '5', x: 375, y: 250 },
 { id: 6, label: '6', x: 525, y: 250 },
];
```

```

 queue: [...queue],
 visited: Array.from(visited),
 currentNode: null,
 logs: "Добавляем стартовую вершину 0 в очередь и по
 });

```

```

while (queue.length > 0) {
 steps.push({
 activeLine: 4,
 queue: [...queue],
 visited: Array.from(visited),
 currentNode: null,
 logs: `Очередь не пуста, элементов: ${queue.length}
 });

```

```

const curr = queue.shift()!;
steps.push({
 activeLine: 5,
 queue: [...queue],
 visited: Array.from(visited),
 currentNode: curr,
 logs: `Достаем из очереди (${curr}).`
});

```

```

const neighbors = ADJ[curr];
if (neighbors.length > 0) {
 for (const n of neighbors) {
 if (!visited.has(n)) {
 steps.push({
 activeLine: 7,
 queue: [...queue],
 visited: Array.from(visited),
 currentNode: curr,
 logs: `Рассматриваем соседа: ${n}. Он еще не
 });

```

```

 visited.add(n);
 queue.push(n);

```

```

const step = STEPS[stepIdx];

const handleNext = useCallback(() => {
 setStepIdx(prev => Math.min(prev + 1, STEPS.length - 1));
}, []);

// @ts-expect-error зарезервировано под кнопку «назад»
const handlePrev = useCallback(() => {
 setStepIdx(prev => Math.max(prev - 1, 0));
}, []);

useEffect(() => {
 if (!isPlaying) return;
 if (stepIdx >= STEPS.length - 1) {
 setIsPlaying(false);
 return;
 }
 const timer = setTimeout(handleNext, 1200);
 return () => clearTimeout(timer);
}, [isPlaying, stepIdx, handleNext]);

return (
 <div className="bg-slate-900 border-2 border-slate-800 p-2">
 <div className="flex flex-col md:flex-row items-center justify-center gap-2">
 <div>
 <h3 className="text-2xl font-bold text-white">
 Алгоритм
 </h3>
 <p className="text-slate-400 text-sm mt-1">Видео
 </div>
 <div className="flex bg-slate-800 p-1.5 rounded">
 <button
 onClick={() => { setIsPlaying(false); setStepIdx(0); }}
 className="p-2 text-slate-400 hover:text-white"
 title="Сброс"
 >
 <RotateCcw className="w-5 h-5" />
 </button>
 </div>
 </div>
 </div>
);

```

```

 { /* Вершины */ }
 { NODES.map(n => {
 const isCurrent = step.currentNode === n.id
 const isInQueue = step.queue.includes(n.id)
 const isVisited = step.visited.includes(n.id)

 let fill = '#1e293b'; // slate-800
 let stroke = '#475569'; // slate-600
 let scale = 1;
 void scale;

 if (isCurrent) {
 fill = '#3b82f6'; // blue-500
 stroke = '#60a5fa'; // blue-400
 scale = 1.3;
 } else if (isInQueue) {
 fill = '#059669'; // emerald-600
 stroke = '#34d399'; // emerald-400
 scale = 1.1;
 } else if (isVisited) {
 fill = '#0f172a'; // slate-900
 stroke = '#3b82f6'; // blue-500
 }

 return (
 <g key={n.id} className="transition-tran
 {isCurrent && {
 <circle cx={n.x} cy={n.y} r="30" fi
 }}
 <circle
 cx={n.x} cy={n.y}
 r="20"
 fill={fill}
 stroke={stroke}
 strokeWidth="3"
 className="transition-colors duration
 />

```

```

 d border-l-2 border-blue-400 -ml-0.5' : 'te
 <span className="opacity-40 w-6 inl
 {c
 </div>
 });
 }}}
</div>
</div>

<div className="bg-slate-800/80 border border
 <div className="text-sm font-bold text-eme
 Состояние Очереди (Queue):
 <span className="text-xs text-slate-500
 </div>
 <div className="flex items-center gap-2 mi
 {step.queue.length === 0 ? {
 <span className="text-slate-500 text-x
 } : {
 step.queue.map(({qId, i}) => {
 <span key={`_${qId}-${i}`} className=
enter justify-center rounded-md shadow-lg s
 {qId}

 })
 })
 </div>
 <div className="mt-2 text-sm text-amber-30
ms-center justify-center">
 "{step.logs}"
 </div>
</div>
</div>
</div>
</div>
);
}
```

```
 <div>
 <div className={`rounded-2xl overflow-hidden border
 <img src={info.src} alt={info.alt} className="w
 </div>
 <p className={`text-center text-xs mt-2 font-bold
 </div>
 });
}
```

---

ФАЙЛ 31/83: src/components/ChapterNav.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

---

```
import React from "react";
import { ChevronLeft, ChevronRight } from "lucide-react";
import type { Chapter } from "../types";
```

```
interface Props {
 prev?: Chapter;
 next?: Chapter;
 index: number;
 total: number;
 onGo: (id: string) => void;
 compact?: boolean;
}
```

```
/**
```

```
 * Переходы «предыдущая / следующая тема» – как на CodeSandbox
```

```
 * Компактный вариант живёт в шапке главы, крупный – по умолчанию
```

```
 */
```

```
export const ChapterNav: React.FC<Props> = ({ prev, next, onGo, compact }) => {
 if (compact) {
 return (
 <div className="flex items-center gap-2 text-xs">
 <button
 disabled={!prev}
```

```

 </button>
) : (
 <div className="hidden sm:block" />
)}

 {next && (
 <button
 onClick={() => onGo(next.id)}
 className="group text-right p-4 rounded-xl bg
 transition-all sm:col-start-2"
 >
 <div className="flex items-center justify-end
 text-indigo-400 mb-1">
 Следующая тема <ChevronRight className="w-3
 </div>
 <div className="text-sm font-bold text-slate-1
 </button>
)}
 </nav>
);
};

```

ФАЙЛ 32/83: src/components/ChapterView.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРО

```

import React, { useCallback, useEffect, useRef, useState
import { Check, Download, Loader2, Maximize2, MousePoin
import type { Chapter } from "../types";
import { useTermHints } from "../hooks/useTermHints";
import { TermPopover, type HintAnchor } from "../hints/T
import { getViz } from "../vizRegistry";
import { ChapterNav } from "../ChapterNav";
import { downloadChapterHtml } from "../utils/exportHtml

```

```

interface Props {

```

```

 try {
 await downloadChapterHtml(chapter);
 setSaving("done");
 window.setTimeout(() => setSaving("idle"), 2500);
 } catch {
 setSaving("idle");
 }
 }, [chapter]);

const open = useCallback(
 (el: HTMLElement) => {
 const termId = el.dataset.termId;
 if (!termId) return;
 cancelHide();
 setAnchor({ termId, rect: el.getBoundingClientRect() });
 },
 [cancelHide]
);

const handlePointerOver = (e: React.MouseEvent) => {
 if (isTouch()) return;
 const el = (e.target as HTMLElement).closest<HTMLElement>(".term-link");
 if (el) open(el);
};

const handlePointerOut = (e: React.MouseEvent) => {
 if (isTouch()) return;
 const el = (e.target as HTMLElement).closest<HTMLElement>(".term-link");
 if (el) scheduleHide();
};

const handleClick = (e: React.MouseEvent) => {
 const el = (e.target as HTMLElement).closest<HTMLElement>(".term-link");
 if (!el) return;
 e.preventDefault();
 if (anchor && anchor.termId === el.dataset.termId)
 else open(el);
};

```



```
</div>
```

```
<div
```

```
 ref={contentRef}
```

```
 className="prose prose-invert max-w-none"
```

```
 onMouseOver={handlePointerOver}
```

```
 onMouseOut={handlePointerOut}
```

```
 onClick={handleClick}
```

```
 onFocus={handleFocus}
```

```
 dangerouslySetInnerHTML={{ __html: chapter.con
```

```
/>
```

```
<p className="mt-6 flex items-start gap-2 text-
 ed-lg px-3 py-2">
```

```
 <MousePointerClick className="w-4 h-4 shrink-1
```

```

```

```
 Подчёркнутые термины — интерактивные: наведи
 объяснение на пальцах, мини-анимацию и кноп
```

```

```

```
</p>
```

```
{viz && (
```

```
 <section id="chapter-viz" className="mt-8 scr
```

```
 <div className="flex items-center justify-b
```

```
 <h3 className="flex items-center gap-2 te
```

```
 <Sparkles className="w-4 h-4 text-indig
```

```
 Демонстрация: {viz.title}
```

```
 </h3>
```

```
 <button
```

```
 onClick={() => onOpenSimulator(chapter.
```

```
 className="flex items-center gap-1.5 te
```

```
 ate-300 hover:text-white hover:border-indig
```

```
 >
```

```
 <Maximize2 className="w-3.5 h-3.5" /> P
```

```
 </button>
```

```
 </div>
```

```
 {viz.hint && <p className="text-xs text-sla
```

```
 <viz.Component />
```

```

{ line: "", highlight: "normal" },
{ line: " for (int to : adj[v]) {", highlight: "normal" },
{ line: " if (to == p) continue;", highlight: "normal" },
{ line: "", highlight: "normal" },
{ line: " if (visited[to]) {", highlight: "normal" },
{ line: " low[v] = min(low[v], tin[to]);", highlight: "normal" },
{ line: " } else {", highlight: "normal" },
{ line: " dfs(to, v);", highlight: "normal" },
{ line: " low[v] = min(low[v], low[to]);", highlight: "normal" },
{ line: "", highlight: "normal" },
{ line: " if (low[to] > tin[v]) {", highlight: "normal" },
{ line: " // ЭТО МОСТ!", highlight: "normal" },
{ line: " }", highlight: "normal" },
{ line: " }", highlight: "normal" },
{ line: " }", highlight: "normal" },
{ line: "}", highlight: "normal" },
];

```

```

interface DfsStep {
 currentNode: number | null;
 visitedIndices: number[];
 tin: Record<number, number>;
 low: Record<number, number>;
 stack: number[];
 bridges: string[];
 log: string;
 activeEdge: [number, number] | null;
 backEdge: [number, number] | null;
 activeCodeLine: number[];
}

```

```

const GRAPH_NODES = [
 { id: 0, label: "A", x: 100, y: 120 },
 { id: 1, label: "B", x: 260, y: 120 },
 { id: 2, label: "C", x: 180, y: 200 },
 { id: 3, label: "D", x: 180, y: 300 },
 { id: 4, label: "E", x: 100, y: 370 },
 { id: 5, label: "F", x: 260, y: 370 },

```

```

tin: {0:1,1:2,6:3}, low: {0:1,1:2,6:3}, stack: [0,1
activeEdge: null, backEdge: null,
log: "У G нет соседей (кроме B). Возврат. low[G]=3 :
activeCodeLine: [13,14,15]
},
{
currentNode: 2, visitedIndices: [0,1,6,2],
tin: {0:1,1:2,6:3,2:4}, low: {0:1,1:2,6:3,2:4}, sta
activeEdge: [1,2], backEdge: null,
log: "Из B идём в C. tin[C]=4, low[C]=4.",
activeCodeLine: [4,5,7,10,11]
},
{
currentNode: 2, visitedIndices: [0,1,6,2],
tin: {0:1,1:2,6:3,2:4}, low: {0:1,1:2,6:3,2:1}, sta
activeEdge: null, backEdge: [2,0],
log: "Видим соседа A (уже посещён)! Обратное ребро
activeCodeLine: [7,8,9]
},
{
currentNode: 3, visitedIndices: [0,1,6,2,3],
tin: {0:1,1:2,6:3,2:4,3:5}, low: {0:1,1:2,6:3,2:1,3
activeEdge: [2,3], backEdge: null,
log: "Идём C→D. tin[D]=5, low[D]=5.",
activeCodeLine: [4,5,7,10,11]
},
{
currentNode: 4, visitedIndices: [0,1,6,2,3,4],
tin: {0:1,1:2,6:3,2:4,3:5,4:6}, low: {0:1,1:2,6:3,2
activeEdge: [3,4], backEdge: null,
log: "D→E. tin[E]=6, low[E]=6.",
activeCodeLine: [4,5,7,10,11]
},
{
currentNode: 5, visitedIndices: [0,1,6,2,3,4,5],
tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:2,6
activeEdge: [4,5], backEdge: null,
log: "E→F. tin[F]=7, low[F]=7.",

```

```

 currentNode: 0, visitedIndices: [0,1,6,2,3,4,5],
 tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:1,6:1,2:2,3:3,4:4,5:5},
 activeEdge: null, backEdge: null,
 log: "Возврат B→A. DFS завершён! Мосты: (B,G) и (C,D). Алгоритм завершён!",
 activeCodeLine: [11,13,16,17,18]
 },
 {
 currentNode: null, visitedIndices: [0,1,6,2,3,4,5],
 tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:1,6:1,2:2,3:3,4:4,5:5},
 activeEdge: null, backEdge: null,
 log: "Готово! Найдены мосты: (B,G) и (C,D). Алгоритм завершён!",
 activeCodeLine: []
 }
];

```

```

export function DfsBridgesSimulator() {
 const [dfsIdx, setDfsIdx] = useState(0);
 const [dfsAuto, setDfsAuto] = useState(false);
 const dfsTimer = useRef<NodeJS.Timeout | null>(null);

 useEffect(() => {
 if (dfsAuto) {
 dfsTimer.current = setInterval(() => {
 setDfsIdx(prev => {
 if (prev >= DFS_STEPS.length - 1) { setDfsAuto(false);
 return prev + 1;
 });
 }, 2500);
 }
 return () => { if (dfsTimer.current) clearInterval(dfsTimer.current);
 }, [dfsAuto]);

 const step = DFS_STEPS[dfsIdx];

 return (
 <div className="space-y-4">
 <div className="flex items-center justify-between">
 <h4 className="text-base font-bold text-white">

```

```

const activeBack = step.backEdge && {
 (step.backEdge[0]===e.u && step.backEdge[1]===e.v) ||
 (step.backEdge[0]===e.v && step.backEdge[1]===e.u)
};
let sc = "#475569", sw = 2, dash = "none";
if (bridge) { sc = "#eab308"; sw = 4; }
else if (activeTree) { sc = "#10b981"; sw = 2; }
else if (activeBack) { sc = "#f43f5e"; sw = 2; }
else if (step.visitedIndices.includes(e.u) || step.visitedIndices.includes(e.v)) {
 return <line key={e.key} x1={u.x} y1={u.y} x2={v.x} y2={v.y}
 stroke={sc} strokeWidth={sw} strokeDash={dash}
 className="transition-all duration-500" />
}
{step.currentNode !== null && step.currentNode.id === n.id}
const n = GRAPH_NODES.find(x => x.id === n.id);
if (!n) return null;
return <circle cx={n.x} cy={n.y} r={18} fill={cur} stroke={st}
 strokeWidth={2.5} className="transition-all duration-300" />
>;
}
}
{GRAPH_NODES.map(n => {
 const cur = step.currentNode === n.id;
 const vis = step.visitedIndices.includes(n.id);
 const st = step.stack.includes(n.id);
 return <g key={n.id}>
 <circle cx={n.x} cy={n.y} r={12} fill={cur} stroke={st}
 strokeWidth={2.5} className="transition-all duration-300" />
 <text x={n.x} y={n.y+3} textAnchor="middle">{n.label}</text>
 {vis && (
 <rect x={n.x-18} y={n.y-28} width={36} height={10}
 className="transition-all duration-300" />
 <text x={n.x} y={n.y-20} textAnchor="center">{
 l: {step.low[n.id]} || "?"</text>
 }
)}
 </g>
)}

```



```

 { u: 'D', v: 'F', w: 3 },
 { u: 'E', v: 'F', w: 1 },
];

 const adjList: { [key: string]: { v: string; w: number }[] } = {};
 nodes.forEach(n => (adjList[n.id] = []));
 edges.forEach(e => {
 adjList[e.u].push({ v: e.v, w: e.w });
 });

 const codeLines = [
 { line: 1, text: 'function dijkstra(graph, start):' },
 { line: 2, text: ' dist = {v: ∞}; dist[start] = 0' },
 { line: 3, text: ' pq = PriorityQueue(); pq.push(dist[start])' },
 { line: 4, text: ' while not pq.empty():' },
 { line: 5, text: ' d, u = pq.pop() // Извлекаем минимальный элемент' },
 { line: 6, text: ' if d > dist[u]: continue' },
 { line: 7, text: ' for v, weight in graph.neighbors(u):' },
 { line: 8, text: ' if dist[u] + weight < dist[v]:' },
 { line: 9, text: ' dist[v] = dist[u] + weight' },
 { line: 10, text: ' pq.push((dist[v], v))' },
 { line: 11, text: ' return dist // Все кратчайшие пути найдены' },
];

 const [steps, setSteps] = useState<Step[]>([]);
 const [currentStepIndex, setCurrentStepIndex] = useState(0);
 const [isPlaying, setIsPlaying] = useState(false);

 useEffect(() => {
 const simulationSteps: Step[] = [];
 const dist: Record<string, number> = { A: 0, B: 999 };
 const visited: Record<string, boolean> = { A: false, B: false };
 const prev: Record<string, string | null> = { A: null, B: null };

 simulationSteps.push({
 currentNode: null,
 checkingEdge: null,
 distances: { ...dist },
 visited: { ...visited },
 log: ' Дейкстра готов к доставке. Начинаем из Питера'
 });
 }, []);

```

```

 prev[v] = u;
 simulationSteps.push({
 currentNode: u, checkingEdge: { u, v },
 distances: { ...dist }, visited: { ...visited },
 log: ` Улучшение (Релаксация)! Новое кратчайшее расстояние: ${dist[v]},
 activeCodeLine: 9,
 });
 }
}

simulationSteps.push({
 currentNode: null, checkingEdge: null,
 distances: { ...dist }, visited: { ...visited },
 log: ` Дейкстра всё доставил! Все возможные кратчайшие расстояния найдены!
 activeCodeLine: 11,
});

setSteps(simulationSteps);
}, []));

useEffect(() => {
 let timer: any = null;
 if (isPlaying && currentStepIndex < steps.length - 1) {
 timer = setTimeout(() => {
 setCurrentStepIndex((prev) => prev + 1);
 }, 1500);
 } else if (currentStepIndex >= steps.length - 1) {
 setIsPlaying(false);
 }
 return () => clearTimeout(timer);
}, [isPlaying, currentStepIndex, steps.length]);

const currentStep = steps[currentStepIndex] || null;

if (!currentStep) return <div className="text-white">

return (

```



```
</div>
```

```
<div className="flex flex-col lg:flex-row gap-6 m-6" *
 {/* Граф */}
```

```
 <div className="w-full lg:w-2/3 bg-indigo-950/20
 justify-center min-h-[400px]">
```

```
 <div className="absolute top-3 left-3 bg-indigo-950/20
 font-bold shadow-sm">
```

```
 Шаг {currentStepIndex + 1} из {steps.length}
```

```
 </div>
```

```
 <svg className="w-full h-auto max-h-[500px]" *
 <defs>
```

```
 <marker id="arrow-dh-normal" markerWidth=
 <path d="M0,0 L6,3 L0,6 z" fill="#475569" />
 </marker>
```

```
 <marker id="arrow-dh-active" markerWidth=
 <path d="M0,0 L6,3 L0,6 z" fill="#f59e00" />
 </marker>
```

```
 <marker id="arrow-dh-opt" markerWidth="6"
 <path d="M0,0 L6,3 L0,6 z" fill="#10b981" />
 </marker>
```

```
 </defs>
```

```
 {/* Рёбра */}
```

```
 {edges.map((edge, idx) => {
 const uNode = nodes.find((n) => n.id === edge.u)
 const vNode = nodes.find((n) => n.id === edge.v)
 const isChecking =
 currentStep.checkingEdge &&
 (currentStep.checkingEdge.u === edge.u &&
 currentStep.checkingEdge.v === edge.v)

 const isOptimal = currentStep.previous[edge.u] === edge.v

 let marker = 'url(#arrow-dh-normal)';
 if (isChecking) marker = 'url(#arrow-dh-active)';
 else if (isOptimal) marker = 'url(#arrow-dh-opt)';

 return (

```

```

 stroke={isCurrent ? '#a5b4fc' : isV
 strokeWidth={isCurrent ? 4 : 2}
 className="transition-all duration-1
 />
 <text x={node.x} y={node.y + 5} fill=
 {node.id}
 </text>
 <text x={node.x} y={node.y - 28} fill=
 "middle" className="bg-indigo-950 px-1 py-0
 {dist === 999 ? '∞' : `d=${dist}`}
 </text>
 <text x={node.x} y={node.y + 35} fill=
 {node.name.split(' ')[0]}
 </text>
 </g>
);
 }}}
</svg>
<div className="w-full bg-slate-950/80 p-4 ro
 <p className="font-semibold text-amber-400
 <p className="min-h-[40px] flex items-cente
</div>
</div>

{/* Список смежности */}
<div className="w-full lg:w-1/3 bg-emerald-950/
 <h4 className="text-lg font-bold text-emerald
 <div className="space-y-2.5 flex-1 overflow-y
 {Object.keys(adjList).map((u) => {
 const isCurrentNode = currentStep.current
 return (
 <div key={u} className={`p-3 rounded-lg
 isCurrentNode ? 'bg-emerald-900/60
 t-slate-300'
 }`>
 <div className="flex items-center gap
 <span className={`px-2 py-0.5 round
 te-300'}`>

```

```

 </tr>
 </thead>
 <tbody className="text-sm">
 {nodes.map((n) => {
 const dist = currentStep.distances[n.id]
 const prev = currentStep.previous[n.id]
 const vis = currentStep.visited[n.id]
 const isCurr = currentStep.currentNode === n.id

 return (
 <tr key={n.id} className={`border-1
 isCurr ? 'bg-indigo-950/60 font-medium' : ''}>
 <td className="py-2.5 px-3 flex items-center">

 {n.name}
 </td>
 <td className="py-2.5 px-3 font-medium">
 {dist === 999 ? '∞' : dist}
 </td>
 <td className="py-2.5 px-3 text-sm">
 {prev || '-'}
 </td>
 </tr>
);
 })}
 </tbody>
 </table>
 </div>
</div>
</div>
</div>

{/* Код алгоритма */}
<div className="w-full lg:w-1/2 bg-slate-900/60 rounded-lg p-4">
 <div>
 <h4 className="text-lg font-bold text-slate-200">Алгоритм
 <div className="bg-slate-950/80 rounded-lg p-4">
 {codeLines.map((item) => {

```

```

const DP_CODE_LINES = [
 /* 1 */ "function fibMemo(n, memo = {}) {",
 /* 2 */ " if (n in memo) return memo[n];",
 /* 3 */ " if (n <= 2) return 1;",
 /* 4 */ " memo[n] = fibMemo(n - 1, memo) + fibMemo",
 /* 5 */ " return memo[n];",
 /* 6 */ "}";
];

export function DPVisualizer() {
 const [targetN, setTargetN] = useState<number>(5);
 const [steps, setSteps] = useState<DPStep[]>([]);
 const [currentStepIndex, setCurrentStepIndex] = useSt
 const [isPlaying, setIsPlaying] = useState<boolean>(f
 const [speed] = useState<number>(800);
 const [activeTab, setActiveTab] = useState<'table' |

 useEffect(() => {
 const generatedSteps: DPStep[] = [];
 let stepId = 0;
 const memoMap: { [key: number]: number } = {};

 function simulateFib(n: number): number {
 generatedSteps.push({
 id: stepId++,
 n,
 action: 'call',
 desc: `Вызов fibMemo(${n}). Проверяем наличие в
 codeLine: 2,
 memoState: { ...memoMap }
 });

 if (n in memoMap) {
 generatedSteps.push({
 id: stepId++,
 n,
 action: 'memo_hit',
 desc: `✂ Кэш-хит! fibMemo(${n}) уже посчитано

```

```

 });

 return memoMap[n];
 }

 const finalRes = simulateFib(targetN);

 generatedSteps.push({
 id: stepId++,
 n: targetN,
 action: 'done',
 desc: `  Расчет завершен! fibMemo(${targetN}) = $`,
 codeLine: 6,
 memoState: { ...memoMap }
 });

 setSteps(generatedSteps);
 setCurrentStepIndex(0);
 setIsPlaying(false);
}, [targetN]);

useEffect(() => {
 let timer: any = null;
 if (isPlaying && currentStepIndex < steps.length - 1) {
 timer = setTimeout(() => {
 setCurrentStepIndex(prev => prev + 1);
 }, speed);
 } else if (currentStepIndex >= steps.length - 1) {
 setIsPlaying(false);
 }
 return () => clearTimeout(timer);
}, [isPlaying, currentStepIndex, steps, speed]);

const currentStep = steps[currentStepIndex] || null;
const memoState = currentStep?.memoState || {};

return (
 <div className="bg-slate-900 rounded-2xl border bor

```

```

 <button
 onClick={() => { setIsPlaying(false); set
 disabled={currentStepIndex === 0}
 className="p-2 text-slate-400 hover:text-
 title="Шаг назад"
 >
 <ChevronLeft className="w-5 h-5" />
 </button>
 <button
 onClick={() => setIsPlaying(!isPlaying)}
 className={`flex items-center gap-1.5 px-
 isPlaying
 ? 'bg-amber-500 text-slate-950 hover:
 : 'bg-emerald-600 text-white hover:
 }`}
 >
 {isPlaying ? <Pause className="w-4 h-4" />
 {isPlaying ? 'Пауза' : 'Пуск'}}
 </button>
 <button
 onClick={() => { setIsPlaying(false); set
 disabled={currentStepIndex === steps.leng
 className="p-2 text-slate-400 hover:text-
 title="Шаг вперед"
 >
 <ChevronRight className="w-5 h-5" />
 </button>
 </div>
 </div>
 </div>

```

```

 { /* DP Table View */ }
 <div className="mb-8 bg-slate-950 p-4 md:p-6 round
 <h4 className="text-xs font-bold uppercase trac
 <div className="flex flex-wrap items-center gap
 {Array.from({ length: targetN }, (_, i) => i
 const isCached = num in memoState || num <=
 const val = num <= 2 ? 1 : memoState[num];

```

```

 <button
 onClick={() => setActiveTab('table')}
 className={`flex items-center gap-2 px-6 py-3
 activeTab === 'table'
 ? 'border-emerald-500 text-emerald-400 bg
 : 'border-transparent text-slate-400 hove
 }`}
 >
 <Eye className="w-4 h-4" />
 Стен-бай-стен состояние
 </button>
 <button
 onClick={() => setActiveTab('code')}
 className={`flex items-center gap-2 px-6 py-3
 activeTab === 'code'
 ? 'border-emerald-500 text-emerald-400 bg
 : 'border-transparent text-slate-400 hove
 }`}
 >
 <Code className="w-4 h-4" />
 Интерактивный код
 </button>
 </div>

 {/* Tab Content */}
 <div className="min-h-[320px] flex flex-col justify
 {activeTab === 'table' && {
 <div className="bg-slate-950 p-6 rounded-2xl
 <h5 className="font-bold text-white text-sm

 </h5>
 <p className="text-sm text-slate-400">
 В классической рекурсии для N={targetN} п
 вычисленное значение в таблице, мы момента
 </p>
 <div className="p-4 bg-slate-900 rounded-xl
 t-slate-300">
 Всего шагов симуляции: {steps.length}

```

```
 {/* Progress Bar */}
 <div className="mt-8 pt-6 border-t border-slate-800">
 <div className="flex items-center justify-between">
 Прогресс выполнения
 Шаг {currentStepIndex + 1} из {steps.length}
 </div>
 <div className="h-2 bg-slate-950 rounded-full overflow-hidden">
 <div
 className="h-full bg-gradient-to-r from-emerald-500 to-teal-500"
 style={{ width: `${((currentStepIndex + 1) / steps.length) * 100}%` }}
 ></div>
 </div>
 </div>
);
}
```

---

ФАЙЛ 36/83: src/components/EulerSimulator.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import { useState } from "react";
import { Undo } from "lucide-react";

const C1_NODES = [
 { id: 0, label: "A", x: 190, y: 60 },
 { id: 1, label: "B", x: 280, y: 140 },
 { id: 2, label: "C", x: 100, y: 140 },
 { id: 3, label: "D", x: 280, y: 250 },
 { id: 4, label: "E", x: 100, y: 250 }
];

const C1_EDGES = [
 { u: 0, v: 1 }, { u: 0, v: 2 },
 { u: 1, v: 3 }, { u: 2, v: 4 },
 { u: 1, v: 2 },
];
```



```

 const endDegree = C1_EDGES.filter(e => e.u === vId).length;
 const isCycle = startDegree % 2 === 0 && endDegree % 2 === 0;
 if (isCycle && nextPath[0] === vId) {
 setC1Msg(" ЭЙЛЕРОВ ЦИКЛ! Все рёбра пройдены, ф");
 } else {
 setC1Msg(` ЭЙЛЕРОВ ПУТЬ! Все рёбра пройдены! С`);
 }
 setC1Done(true);
 } else {
 setC1Msg(`Пройдено рёбер: ${nextEdges.length} / $`);
 }
};

```

```

return (
 <div className="space-y-4">
 <div className="flex items-center justify-between">
 <h4 className="text-base font-bold text-white">
 <button onClick={resetC1} className="flex items-center justify-center gap-2 border border-slate-700 transition-all">
 <Undo className="h-3.5 w-3.5" /> Сброс
 </button>
 </div>

 <div className="bg-slate-950 rounded-xl p-4 border border-slate-800">
 <div className="absolute inset-0 bg-[radial-gradient(circle,transparent_1px,slate_1px)] border border-slate-800">
 <svg className="w-full h-[260px] z-10 relative">
 {C1_EDGES.map((e, i) => {
 const u = C1_NODES.find(n => n.id === e.u)!;
 const v = C1_NODES.find(n => n.id === e.v)!;
 const key = `${Math.min(e.u, e.v)}-${Math.max(e.u, e.v)}`;
 const used = c1VisitedEdges.includes(key);
 return <line key={i} x1={u.x} y1={u.y} x2={v.x} y2={v.y}
 stroke={used ? "#6366f1" : "#475569"}
 strokeWidth={used ? 4 : 2}
 className="transition-all duration-300" />
 })}
 {C1_NODES.map(n => {

```

```
import React, { useState } from 'react';
import { CheckCircle2, XCircle, Award, RotateCcw, AlertTriangle } from 'lucide-react';

interface Question {
 id: number;
 question: string;
 options: string[];
 correctAnswer: number;
 explanation: string;
}

const quizQuestions: Question[] = [
 {
 id: 1,
 question: 'Что мы делаем в алгоритме Краскала, если ребро, красным) цвет?',
 options: [
 'Мы радуемся и добавляем его в минимальное остовное дерево',
 'Мы перекрашиваем их в синий цвет и делим граф по компонентам связности',
 'Мы безжалостно ВЫБРАСЫВАЕМ это ребро, иначе получим цикл',
 'Мы вызываем алгоритм Дейкстры для поиска нового минимального ребра'
],
 correctAnswer: 2,
 explanation: 'Верно! Если вершины уже в одном клане, добавление этого ребра создаст цикл. Это нарушит структуру дерева.'
 },
 {
 id: 2,
 question: 'Почему алгоритм Дейкстры впадает в ступор на графе с отрицательными ребрами?',
 options: [
 'Потому что он написан только для положительных весов',
 'Он жадный и считает, что каждый следующий шаг приведет к минимальному пути, не учитывая, что отрицательные ребра могут создать более короткий путь',
 'Отрицательные ребра создают гравитационный коллапс в алгоритме',
 'Дейкстра просто не любил отрицательные балансы на графах'
],
 correctAnswer: 1,
 explanation: 'Именно! Дейкстра уверен: если ты пришел к вершине, это лучший путь. Но отрицательные ребра могут опровергнуть это предположение.'
 }
];
```

```
 }
 };
```

```
export const ExpressQuiz: React.FC = () => {
 const [currentQIndex, setCurrentQIndex] = useState<number>(0);
 const [selectedOption, setSelectedOption] = useState<string>(null);
 const [isAnswered, setIsAnswered] = useState<boolean>(false);
 const [score, setScore] = useState<number>(0);
 const [showResults, setShowResults] = useState<boolean>(false);
```

```
 const currentQ = quizQuestions[currentQIndex];
```

```
 const handleSelectOption = (index: number) => {
 if (isAnswered) return;
 setSelectedOption(index);
 setIsAnswered(true);
 if (index === currentQ.correctAnswer) {
 setScore(prev => prev + 1);
 }
 };
};
```

```
 const handleNext = () => {
 if (currentQIndex + 1 < quizQuestions.length) {
 setCurrentQIndex(prev => prev + 1);
 setSelectedOption(null);
 setIsAnswered(false);
 } else {
 setShowResults(true);
 }
 };
};
```

```
 const handleRestart = () => {
 setCurrentQIndex(0);
 setSelectedOption(null);
 setIsAnswered(false);
 setScore(0);
 setShowResults(false);
 };
};
```

```

<div className="bg-slate-900/90 border border-slate
>
 <div className="flex items-center justify-between
 <div className="flex items-center gap-3">

 Экспресс-тест

 <h3 className="text-xl font-bold text-white">
 </div>

 Вопрос {currentQIndex + 1} / {quizQuestions.length}

 </div>

 <div className="mb-8">
 <h4 className="text-xl md:text-2xl font-bold text-white">
 {currentQ.question}
 </h4>

 <div className="space-y-4">
 {currentQ.options.map((option, idx) => {
 const isSelected = selectedOption === idx;
 const isCorrect = idx === currentQ.correctAnswer;

 let optionStyle = "bg-slate-800/80 hover:bg-slate-700/80";
 if (isAnswered) {
 if (isCorrect) {
 optionStyle = "bg-emerald-950/80 border border-emerald-500";
 } else if (isSelected) {
 optionStyle = "bg-rose-950/80 border border-rose-500";
 } else {
 optionStyle = "bg-slate-800/40 border-slate-700";
 }
 }
 return (
 <div
 key={idx}

```

```

 onClick={handleNext}
 disabled={!isAnswered}
 className={
 "px-8 py-3.5 font-bold rounded-xl transition
 (!isAnswered
 ? "bg-slate-800 text-slate-500 border border-slate-800
 : "bg-indigo-600 hover:bg-indigo-500 active:bg-indigo-700
)
 }
 >
 {currentQIndex + 1 < quizQuestions.length ? 'Следующий вопрос' : 'Завершено'}
 </button>
 </div>
 </div>
);
};

```

ФАЙЛ 38/83: src/components/FloydViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```

import { useState, useEffect } from 'react';

interface Step {
 k: number; i: number; j: number;
 matrix: number[][]; updated: boolean;
 log: string; activeCodeLine: number;
}

export default function FloydViz() {
 const [steps, setSteps] = useState<Step[]>([]);
 const [currentStepIndex, setCurrentStepIndex] = useState(0);
 const [isPlaying, setIsPlaying] = useState(false);

 const nodeNames = ['А 0', 'Б 1', 'В 2', 'Г 3', 'Д 4', 'Е 5', 'Ж 6', 'З 7', 'И 8', 'Й 9'];
 const nodesSvg = [
 { id: 0, name: 'Аэропорт 0', x: 100, y: 60 },
 { id: 1, name: 'Аэропорт 1', x: 200, y: 60 },
 { id: 2, name: 'Аэропорт 2', x: 300, y: 60 },
 { id: 3, name: 'Аэропорт 3', x: 400, y: 60 },
 { id: 4, name: 'Аэропорт 4', x: 500, y: 60 },
 { id: 5, name: 'Аэропорт 5', x: 600, y: 60 },
 { id: 6, name: 'Аэропорт 6', x: 700, y: 60 },
 { id: 7, name: 'Аэропорт 7', x: 800, y: 60 },
 { id: 8, name: 'Аэропорт 8', x: 900, y: 60 },
 { id: 9, name: 'Аэропорт 9', x: 1000, y: 60 },
];
}

```

```
useEffect(() => {
 const simulationSteps: Step[] = [];
 const dist = initialMatrix.map(row => [...row]);

 simulationSteps.push({
 k: -1, i: -1, j: -1, matrix: dist.map(r => [...r]),
 log: ' Инициализация матрицы прямых путей, ∞ (999)',
 activeCodeLine: 2,
 });

 const N = 7;
 for (let k = 0; k < N; k++) {
 simulationSteps.push({
 k, i: -1, j: -1, matrix: dist.map(r => [...r]),
 log: `+ Внешний цикл. Разрешаем пути через прогнанные,  $\infty$  (999)`,
 activeCodeLine: 3,
 });

 for (let i = 0; i < N; i++) {
 for (let j = 0; j < N; j++) {
 if (i === j || i === k || j === k) continue;

 const oldVal = dist[i][j];
 const viaVal = dist[i][k] + dist[k][j];
 if (dist[i][k] !== 999 && dist[k][j] !== 999 && viaVal < oldVal) {
 dist[i][j] = viaVal;
 simulationSteps.push({
 k, i, j, matrix: dist.map(r => [...r]),
 log: `  Улучшение пути [${i}]→[${j}] через [${k}],  $\infty$  (999)`,
 activeCodeLine: 7,
 });
 }
 }
 }
 }
}
```

```

<button onClick={() => { setIsPlaying(false);
 text-sm"> Сброс</button>
<button onClick={() => setIsPlaying(!isPlaying
 -amber-600' : 'bg-emerald-600'}`}>{isPlaying
<button onClick={() => { setIsPlaying(false);
 ; }} className="px-3 py-2 bg-emerald-600 te
</div>
</div>

<div className="flex flex-col lg:flex-row gap-6 m
 {/* Граф */}
 <div className="w-full lg:w-2/3 bg-indigo-950/2
 stify-center min-h-[400px]">
 <div className="absolute top-3 left-3 bg-indi
 ont-bold shadow-sm">
 {currentStep.k >= 0 && currentStep.k < 7 ?
 </div>

 <svg className="w-full h-auto max-h-[500px]"
 <defs>
 <marker id="arrow-fw-normal" markerWidth=
 <path d="M0,0 L6,3 L0,6 z" fill="#475569" />
 <marker id="arrow-fw-active" markerWidth=
 <path d="M0,0 L6,3 L0,6 z" fill="#10b981" />
 <marker id="arrow-fw-via" markerWidth="6"
 th d="M0,0 L6,3 L0,6 z" fill="#818cf8" /></
 </defs>
 {Object.keys(adjList).flatMap((uStr) => {
 const u = parseInt(uStr);
 return adjList[u].map((edge) => {
 const uNode = nodesSvg.find((n) => n.id
 const vNode = nodesSvg.find((n) => n.id
 const isTarget = currentStep.i === u &&
 const isVia = (currentStep.i === u && c
 let strokeColor = '#475569'; let marker
 if (isTarget) { strokeColor = '#10b981'
 else if (isVia) { strokeColor = '#818cf8
 return (

```

```

<div className="w-full lg:w-1/3 bg-emerald-950/
 <h4 className="text-lg font-bold text-emerald
 <div className="space-y-2 text-sm font-mono t
 {Object.keys(adjList).map((uStr) => {
 const u = parseInt(uStr);
 const isI = currentStep.i === u;
 const isK = currentStep.k === u;
 return {
 <div key={u} className={`flex flex-wrap
 isI ? 'bg-amber-900/40 border-ambe
 isK ? 'bg-indigo-900/40 border-ind
 }`}>
 <span className="font-bold text-whi
e-400">→
 {adjList[u].map((edge, idx) => {
 const isUpd = currentStep.i === u
 const isVia = currentStep.k !== -
 currentStep.j === edge.v);
 return {
 <span key={idx} className={`px-1
dow' : isVia ? 'bg-indigo-500 text-white bo
 {edge.v} ({edge.w})

 }
 })}
 </div>
 };
 })}
 </div>
</div>
</div>

```

```

<div className="flex flex-col lg:flex-row gap-6">
 /* Матрица расстояний */
 <div className="w-full lg:w-1/2 bg-rose-950/10
 <div className="flex justify-between items-ce
 <h4 className="text-lg font-bold text-rose-
 </div>
 </div>

```



```

 <div key={item.line} className={`py-1.5
e === item.line ? 'bg-blue-900/80 text-amber
 <span className="text-slate-600 w-5 s
 <span className="whitespace-pre flex-
 </div>
 }}}
</div>
</div>
</div>
</div>
</div>
);
}
```

-----  
ФАЙЛ 39/83: src/components/GraphTraversalViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

-----  
import { useState, useEffect, useMemo } from 'react';  
import { Play, Pause, SkipForward, Undo, RefreshCw } from 'lucide-react';

type Node = { id: string; label: string; x: number; y: number };  
type Edge = { u: string; v: string };

```
const nodes: Node[] = [
 { id: 'A', label: 'A', x: 200, y: 40 },
 { id: 'B', label: 'B', x: 100, y: 120 },
 { id: 'C', label: 'C', x: 300, y: 120 },
 { id: 'D', label: 'D', x: 50, y: 200 },
 { id: 'E', label: 'E', x: 150, y: 200 },
 { id: 'F', label: 'F', x: 250, y: 200 },
 { id: 'G', label: 'G', x: 350, y: 200 },
];
```

```
const edges: Edge[] = [
 { u: 'A', v: 'B' }, { u: 'A', v: 'C' },
 { u: 'B', v: 'D' }, { u: 'B', v: 'E' },
 { u: 'C', v: 'F' }, { u: 'C', v: 'G' },
 { u: 'D', v: 'E' }, { u: 'E', v: 'F' },
 { u: 'E', v: 'G' }, { u: 'F', v: 'G' },
];
```

```

 }
 }

 stack.pop();
 steps.push({ type: 'backtrack', node: v, desc:
 ueOrStack: [...stack], visitedNodes: Array.from(
 }

 dfs(start);
 return steps;
}

function generateBFSSteps(start: string) {
 const steps: Action[] = [];
 const vis = new Set<string>();
 const q: string[] = [];

 q.push(start);
 vis.add(start);
 steps.push({ type: 'visit', node: start, desc: `Начало
 ray.from(vis)] });

 while (q.length > 0) {
 const v = q[0];
 steps.push({ type: 'process', node: v, desc: `Действие
 q], visitedNodes: [...Array.from(vis)] });
 q.shift();

 for (const u of adj[v]) {
 if (!vis.has(u)) {
 vis.add(u);
 q.push(u);
 steps.push({ type: 'visit', node: u, desc: `Начало
 tack: [...q], visitedNodes: [...Array.from(
 }
 }
 }
 }
}

```

```

return (
 <div className="space-y-4">
 <div className="flex bg-slate-900 border border-slate-500" style={{ width: '100%' }}>
 <button onClick={() => setMode("dfs")}
 className={`px-4 py-2 text-xs font-medium text-slate-400 hover:text-slate-300`} `>
 DFS (В глубину)
 </button>
 <button onClick={() => setMode("bfs")}
 className={`px-4 py-2 text-xs font-medium text-slate-400 hover:text-slate-300`} `>
 BFS (В ширину / Волна)
 </button>
 </div>

 <div className="flex flex-col md:flex-row gap-4">
 { /* SVG Graph View */ }
 <div className="bg-slate-900 border border-slate-500 min-h-[300px]">
 <svg className="w-full h-full max-w-[600px] max-h-[300px]">
 { /* Edges */ }
 {edges.map((e, i) => {
 const u = nodes.find(n => n.id === e.u)
 const v = nodes.find(n => n.id === e.v)
 // Check if edge is part of the current path
 const edgeTraversed = isVisited(e.u, e.v)

 return (
 <line key={i} x1={u.x} y1={u.y} x2={v.x} y2={v.y}
 className={
 edgeTraversed ? 'stroke-emerald-500' : 'stroke-slate-400'
 }
 />
)
 })}

 { /* Nodes */ }
 {nodes.map(n => {
 const visited = isVisited(n.id)

```

```

 {/* Structure View (Stack / Queue) */}
 <div className="bg-slate-900 border border-emerald-500 p-2">
 <div className={`absolute -top-3 left-3 right-3 bottom-3
 ${mode === 'dfs' ? 'bg-slate-900 border border-emerald-500'}
 ${mode === 'dfs' ? 'Стек вызовов' : 'Очередь'}
 `}>
 <div className="flex flex-col gap-2">
 {step.queueOrStack && step.queueOrStack.map((item, idx) => {
 <div key={idx} className={`w-full flex items-center justify-between
 ${mode === 'dfs' ? 'bg-slate-900 border border-emerald-500' : 'bg-slate-900'}
 text-emerald-200`}
 <div>
 {item}
 <div>
 ${idx === (mode === 'dfs' ? step.queueOrStack.length - 1 : 0)
 ? `0_10px_rgba(245,158,11,0.2)` : ''}

 {item}

 </div>
 </div>
 })}
 </div>
 </div>
 </div>

 <div className="flex flex-col sm:flex-row gap-2">
 {/* Controls */}
 <div className="flex bg-slate-900 border border-emerald-500 p-2">
 <button onClick={() => {setAutoPlay(!autoPlay)}}
 "p-2 text-slate-400 hover:text-white hover:bg-slate-800"
 <Undo strokeWidth={3} size={16} />
 </button>
 </div>
 </div>

```

```
interface Patient {
 id: string;
 name: string;
 emoji: string;
 symptom: string;
 priority: number; // 1 to 99 (Severity)
 animState: 'in' | 'idle' | 'winner' | 'out';
}
```

// Max-heap helpers for Patients

```
function siftUp(arr: Patient[]): Patient[] {
 const a = arr.map(x => ({ ...x }));
 let idx = a.length - 1;
 while (idx > 0) {
 const parent = Math.floor((idx - 1) / 2);
 if (a[parent].priority < a[idx].priority) {
 [a[parent], a[idx]] = [a[idx], a[parent]];
 idx = parent;
 } else break;
 }
 return a;
}
```

```
function siftDown(arr: Patient[]): Patient[] {
 const a = arr.map(x => ({ ...x }));
 const n = a.length;
 let idx = 0;
 while (true) {
 let largest = idx;
 const l = 2 * idx + 1;
 const r = 2 * idx + 2;
 if (l < n && a[l].priority > a[largest].priority) largest = l;
 if (r < n && a[r].priority > a[largest].priority) largest = r;
 if (largest !== idx) {
 [a[largest], a[idx]] = [a[idx], a[largest]];
 idx = largest;
 } else break;
 }
```

```

start.forEach((p, i) => {
 arr = heapInsert(arr, { id: `init${i}`, ...p, animS
});
return arr;
})();

```

```
let uid = 300;
```

```

export const HeapViz: React.FC = () => {
 const [heap, setHeap] = useState<Patient[]>([]);
 const [customName, setCustomName] = useState('');
 const [customPriority, setCustomPriority] = useState(0);
 const [log, setLog] = useState<string>('');
 const [shakeEmpty, setShake] = useState(false);
 const [busy, setBusy] = useState(false);
 const [healingPatient, setHealing] = useState<Patient>({

```

```
 const addLog = (msg: string) => setLog(prev => [msg,
```

```
 // --- INSERT: Новый пациент поступает в приемный покой

```

```

 const insertPatient = useCallback((name: string, prio
 if (busy || heap.length >= 15) {
 addLog('⚠ Все палаты переполнены! Дождитесь выпис
 setShake(true);
 setTimeout(() => setShake(false), 400);
 return;
 }
 setBusy(true);

```

```

 const preset = PATIENT_PRESETS[Math.floor(Math.random() * PATIENT_PRESETS.length)];
 const finalName = name || preset.name.split(' ')[0];
 const finalEmoji = preset.emoji;
 const finalSymptom = preset.symptom;

```

```

 const newPatient: Patient = {
 id: `p${uid++}`,
 name: finalName,
 emoji: finalEmoji,

```

```

 setTimeout(() => setShake(false), 400);
 return;
 }
 setBusy(true);
 const topPatient = heap[0];
 setHealing(topPatient);

 addLog(`    СРОЧНО! Забираем самого тяжелого: ${topPatient.name}`);

 // Step 1: Root lights up as winner
 setHeap(prev => prev.map((x, i) => i === 0 ? { ...x, animState: 'winning' } : x));

 setTimeout(() => {
 // Step 2: Root exits the tree, tree reorganizes
 setHeap(prev => prev.map((x, i) => i === 0 ? { ...x, animState: 'exiting' } : x));

 setTimeout(() => {
 // Run binary heap extraction
 setHeap(prev => {
 if (prev.length === 0) return [];
 const a = prev.map(x => ({ ...x }));
 a[0] = { ...a[a.length - 1] };
 a.pop();
 return siftDown(a).map(x => ({ ...x, animState: 'sifting' }));
 });

 // Patient gets healed in the bed
 setTimeout(() => {
 setHealing(prev => prev ? { ...prev, animState: 'healed' } : null);
 addLog(`    Успех! Пациент ${topPatient.name} излечен!`);
 setTimeout(() => {
 setHealing(null);
 setBusy(false);
 }, 600);
 }, 800);

 }, 350);
 }, 650);

```

```

return (
 <div className="flex flex-col gap-6">
 {/* МНЕМОНИКА / ПРАВИЛО */}
 <div className="bg-emerald-950/40 border border-e
 <div className="text-3xl"> </div>
 <div>
 <h3 className="font-black text-emerald-400 te
 ty Queue)</h3>
 <p className="text-xs text-slate-300 mt-1 lea
 В реанимации не важно, кто пришел раньше, а
 остояние (максимальный приоритет).
 Куча (Heap) автоматически перестраивает пац
 оказывался на самом верху дерева (в корне).
 </p>
 </div>
 </div>

 {/* УПРАВЛЕНИЕ */}
 <div className="flex flex-col lg:flex-row items-s

 {/* Добавить случайного */}
 <button
 onClick={handleRandomInsert}
 disabled={busy || heap.length >= 15}
 className="flex-1 min-w-[150px] bg-gradient-t
 opacity-40 disabled:cursor-not-allowed text
 ve:scale-95 transition-all cursor-pointer f
 >
 Привезти по Скорой (INSERT)
 </button>

 {/* Добавить кастомного */}
 <form onSubmit={handleCustomInsert} className="
 <input
 type="text"
 required
 value={customName}
 onChange={e => setCustomName(e.target.value

```



```

 className="px-5 py-3.5 rounded-2xl bg-slate-800
 r-pointer border border-slate-700"
 >
 Сброс
 </button>
 </div>

```

```

 { /* ИНТЕРАКТИВНОЕ ОТДЕЛЕНИЕ */ }
 <div className="grid grid-cols-1 lg:grid-cols-12" >

```

```

 { /* ЛЕВАЯ КОЛОНКА: ДЕРЕВО ПАЦИЕНТОВ */ }
 <div className={`lg:col-span-8 bg-slate-900 rounded-2xl
 -[380px] overflow-hidden ${shakeEmpty ? 'animate-shake' : ''}` >
 <div className="absolute top-4 left-4 text-[14px] font-medium" >
 Сортировка по тяжести (Двоичная куча / Max-Heap)
 </div>

```

```

 <div className="relative w-full h-full min-h-[400px]" >
 <svg className="absolute inset-0 w-full h-full" >
 {edges}
 </svg>

```

```

 {heap.map((patient, idx) => {
 const pos = nodePos(idx);
 const isRoot = idx === 0;
 return (
 <div
 key={patient.id}
 className={`
 absolute flex flex-col items-center
 -translate-x-1/2 -translate-y-1/2 transition-all
 ${patient.animState === 'in' ? 'animate-in' : ''}
 ${patient.animState === 'winner' ? 'animate-winner' : ''}
 ${patient.animState === 'out' ? 'animate-out' : ''}
 ${isRoot ? 'bg-rose-950/80 border-rose-500' : ''}
 `
 style={{
 left: `${pos.x}%`,

```

```


 </div>
 <div>
 <h4 className="text-white font-black">
 Пациент: {healingPatient.name}
 </h4>
 <p className="text-emerald-400 text-x-small">
 Состояние: Стабилизация ({healingPatient.status})
 </p>
 <div className="flex justify-center gap-2">

 <Heart className="w-4 h-4 text-rose-500" />

 </div>
 </div>
 </div>
) : (
 <div className="text-center text-slate-600">
 Ждём
 <p className="text-xs font-bold font-mono">Ваш врач</p>
 <p className="text-[10px] mt-1">Ждём самовосстановления</p>
 </div>
)}
</div>

<div className="w-full h-3 bg-gradient-to-r from-emerald-400 to-emerald-500">
</div>

</div>

{/* ЛОГ ДЕЙСТВИЙ */}
<div className="bg-slate-900 border border-slate-800 p-4">
 <div className="text-[10px] font-mono text-slate-400">
 {log.map((entry, idx) => (
 <div
 key={idx}
 className={`text-xs font-mono flex items-center justify-between p-2`}
 >
 {idx === 0 ? Начало : }
 </div>
))}
 </div>
</div>

```

```

useEffect(() => {
 if (steps <= 1 || paused) return;
 const t = setInterval(() => setStep((s) => (s + 1)),
 period);
 return () => clearInterval(t);
}, [steps, period, paused]);

return step;
}

export const C = {
 idle: "#334155",
 idleStroke: "#475569",
 text: "#e2e8f0",
 dim: "#94a3b8",
 active: "#6366f1",
 done: "#10b981",
 warn: "#f59e0b",
 bad: "#f43f5e",
 edge: "#475569",
};

export const Svg: React.FC<{ children: React.ReactNode;
const paused = useContext(DemoPauseContext);
return (
 <div className="w-full">
 <svg viewBox={`0 0 ${W} ${H}`} className="w-full h-full">
 {children}
 </svg>
 {caption && (
 <div className="text-[10px] text-slate-400 text-center">
 {caption}
 </div>
)}
 <div className="text-[9px] text-slate-600 text-center">
 {paused ? "пауза — курсор на карточке" : "наведи"}
 </div>
 </div>

```

```
);
```

ФАЙЛ 42/83: src/components/hints/demosExtra.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```
import React from "react";
import { C, Edge, MOVE, Node, Svg, useLoop } from "../demo";

/** Флойд: внешний цикл по «разрешённой» промежуточной
export const FloydDemo: React.FC = () => {
 const step = useLoop(3, 1200);
 // i -> j напрямую 9; через k=1 получаем 3+4=7
 const pos: Record<string, [number, number]> = { i: [4, 1], j: [9, 1], k: [3, 4] };
 const viaOpen = step >= 1;
 const relaxed = step >= 2;
 return (
 <Svg
 caption={
 step === 0
 ? "k ещё запрещена: путь i → j = 9"
 : step === 1
 ? "Разрешаем пересадку через k: 3 + 4 = 7"
 : "7 < 9 → D[i][j] = 7"
 }
 >
 <Edge a={pos.i} b={pos.j} color={relaxed ? C.bad : C.done} />
 <Edge a={pos.i} b={pos.k} color={viaOpen ? C.done : C.bad} />
 <Edge a={pos.k} b={pos.j} color={viaOpen ? C.done : C.bad} />

 <text x={130} y={112} textAnchor="middle" fontSize=
 9
 </text>
 <text x={72} y={54} textAnchor="middle" fontSize=
 3
 </text>
```

```

 </Svg>
);
};

/** Что такое граф: вершины, рёбра, направление и вес.
export const GraphBasicsDemo: React.FC = () => {
 const step = useLoop(3, 1300);
 const pos: Record<string, [number, number]> = { A: [500, 100], B: [500, 300], C: [500, 500], D: [500, 700], E: [500, 900], F: [500, 1100], G: [500, 1300], H: [500, 1500], I: [500, 1700], J: [500, 1900], K: [500, 2100], L: [500, 2300], M: [500, 2500], N: [500, 2700], O: [500, 2900], P: [500, 3100], Q: [500, 3300], R: [500, 3500], S: [500, 3700], T: [500, 3900], U: [500, 4100], V: [500, 4300], W: [500, 4500], X: [500, 4700], Y: [500, 4900], Z: [500, 5100], AA: [500, 5300], AB: [500, 5500], AC: [500, 5700], AD: [500, 5900], AE: [500, 6100], AF: [500, 6300], AG: [500, 6500], AH: [500, 6700], AI: [500, 6900], AJ: [500, 7100], AK: [500, 7300], AL: [500, 7500], AM: [500, 7700], AN: [500, 7900], AO: [500, 8100], AP: [500, 8300], AQ: [500, 8500], AR: [500, 8700], AS: [500, 8900], AT: [500, 9100], AU: [500, 9300], AV: [500, 9500], AW: [500, 9700], AX: [500, 9900], AY: [500, 10100], AZ: [500, 10300], BA: [500, 10500], BB: [500, 10700], BC: [500, 10900], BD: [500, 11100], BE: [500, 11300], BF: [500, 11500], BG: [500, 11700], BH: [500, 11900], BI: [500, 12100], BJ: [500, 12300], BK: [500, 12500], BL: [500, 12700], BM: [500, 12900], BN: [500, 13100], BO: [500, 13300], BP: [500, 13500], BQ: [500, 13700], BR: [500, 13900], BS: [500, 14100], BT: [500, 14300], BU: [500, 14500], BV: [500, 14700], BW: [500, 14900], BX: [500, 15100], BY: [500, 15300], BZ: [500, 15500], CA: [500, 15700], CB: [500, 15900], CC: [500, 16100], CD: [500, 16300], CE: [500, 16500], CF: [500, 16700], CG: [500, 16900], CH: [500, 17100], CI: [500, 17300], CJ: [500, 17500], CK: [500, 17700], CL: [500, 17900], CM: [500, 18100], CN: [500, 18300], CO: [500, 18500], CP: [500, 18700], CQ: [500, 18900], CR: [500, 19100], CS: [500, 19300], CT: [500, 19500], CU: [500, 19700], CV: [500, 19900], CW: [500, 20100], CX: [500, 20300], CY: [500, 20500], CZ: [500, 20700], DA: [500, 20900], DB: [500, 21100], DC: [500, 21300], DD: [500, 21500], DE: [500, 21700], DF: [500, 21900], DG: [500, 22100], DH: [500, 22300], DI: [500, 22500], DJ: [500, 22700], DK: [500, 22900], DL: [500, 23100], DM: [500, 23300], DN: [500, 23500], DO: [500, 23700], DP: [500, 23900], DQ: [500, 24100], DR: [500, 24300], DS: [500, 24500], DT: [500, 24700], DU: [500, 24900], DV: [500, 25100], DW: [500, 25300], DX: [500, 25500], DY: [500, 25700], DZ: [500, 25900], EA: [500, 26100], EB: [500, 26300], EC: [500, 26500], ED: [500, 26700], EE: [500, 26900], EF: [500, 27100], EG: [500, 27300], EH: [500, 27500], EI: [500, 27700], EJ: [500, 27900], EK: [500, 28100], EL: [500, 28300], EM: [500, 28500], EN: [500, 28700], EO: [500, 28900], EP: [500, 29100], EQ: [500, 29300], ER: [500, 29500], ES: [500, 29700], ET: [500, 29900], EU: [500, 30100], EV: [500, 30300], EW: [500, 30500], EX: [500, 30700], EY: [500, 30900], EZ: [500, 31100], FA: [500, 31300], FB: [500, 31500], FC: [500, 31700], FD: [500, 31900], FE: [500, 32100], FF: [500, 32300], FG: [500, 32500], FH: [500, 32700], FI: [500, 32900], FJ: [500, 33100], FK: [500, 33300], FL: [500, 33500], FM: [500, 33700], FN: [500, 33900], FO: [500, 34100], FP: [500, 34300], FQ: [500, 34500], FR: [500, 34700], FS: [500, 34900], FT: [500, 35100], FU: [500, 35300], FV: [500, 35500], FW: [500, 35700], FX: [500, 35900], FY: [500, 36100], FZ: [500, 36300], GA: [500, 36500], GB: [500, 36700], GC: [500, 36900], GD: [500, 37100], GE: [500, 37300], GF: [500, 37500], GG: [500, 37700], GH: [500, 37900], GI: [500, 38100], GJ: [500, 38300], GK: [500, 38500], GL: [500, 38700], GM: [500, 38900], GN: [500, 39100], GO: [500, 39300], GP: [500, 39500], GQ: [500, 39700], GR: [500, 39900], GS: [500, 40100], GT: [500, 40300], GU: [500, 40500], GV: [500, 40700], GW: [500, 40900], GX: [500, 41100], GY: [500, 41300], GZ: [500, 41500], HA: [500, 41700], HB: [500, 41900], HC: [500, 42100], HD: [500, 42300], HE: [500, 42500], HF: [500, 42700], HG: [500, 42900], HH: [500, 43100], HI: [500, 43300], HJ: [500, 43500], HK: [500, 43700], HL: [500, 43900], HM: [500, 44100], HN: [500, 44300], HO: [500, 44500], HP: [500, 44700], HQ: [500, 44900], HR: [500, 45100], HS: [500, 45300], HT: [500, 45500], HU: [500, 45700], HV: [500, 45900], HW: [500, 46100], HX: [500, 46300], HY: [500, 46500], HZ: [500, 46700], IA: [500, 46900], IB: [500, 47100], IC: [500, 47300], ID: [500, 47500], IE: [500, 47700], IF: [500, 47900], IG: [500, 48100], IH: [500, 48300], II: [500, 48500], IJ: [500, 48700], IK: [500, 48900], IL: [500, 49100], IM: [500, 49300], IN: [500, 49500], IO: [500, 49700], IP: [500, 49900], IQ: [500, 50100], IR: [500, 50300], IS: [500, 50500], IT: [500, 50700], IU: [500, 50900], IV: [500, 51100], IW: [500, 51300], IX: [500, 51500], IY: [500, 51700], IZ: [500, 51900], JA: [500, 52100], JB: [500, 52300], JC: [500, 52500], JD: [500, 52700], JE: [500, 52900], JF: [500, 53100], JG: [500, 53300], JH: [500, 53500], JI: [500, 53700], JJ: [500, 53900], JK: [500, 54100], JL: [500, 54300], JM: [500, 54500], JN: [500, 54700], JO: [500, 54900], JP: [500, 55100], JQ: [500, 55300], JR: [500, 55500], JS: [500, 55700], JT: [500, 55900], JU: [500, 56100], JV: [500, 56300], JW: [500, 56500], JX: [500, 56700], JY: [500, 56900], JZ: [500, 57100], KA: [500, 57300], KB: [500, 57500], KC: [500, 57700], KD: [500, 57900], KE: [500, 58100], KF: [500, 58300], KG: [500, 58500], KH: [500, 58700], KI: [500, 58900], KJ: [500, 59100], KK: [500, 59300], KL: [500, 59500], KM: [500, 59700], KN: [500, 59900], KO: [500, 60100], KP: [500, 60300], KQ: [500, 60500], KR: [500, 60700], KS: [500, 60900], KT: [500, 61100], KU: [500, 61300], KV: [500, 61500], KW: [500, 61700], KX: [500, 61900], KY: [500, 62100], KZ: [500, 62300], LA: [500, 62500], LB: [500, 62700], LC: [500, 62900], LD: [500, 63100], LE: [500, 63300], LF: [500, 63500], LG: [500, 63700], LH: [500, 63900], LI: [500, 64100], LJ: [500, 64300], LK: [500, 64500], LL: [500, 64700], LM: [500, 64900], LN: [500, 65100], LO: [500, 65300], LP: [500, 65500], LQ: [500, 65700], LR: [500, 65900], LS: [500, 66100], LT: [500, 66300], LU: [500, 66500], LV: [500, 66700], LW: [500, 66900], LX: [500, 67100], LY: [500, 67300], LZ: [500, 67500], MA: [500, 67700], MB: [500, 67900], MC: [500, 68100], MD: [500, 68300], ME: [500, 68500], MF: [500, 68700], MG: [500, 68900], MH: [500, 69100], MI: [500, 69300], MJ: [500, 69500], MK: [500, 69700], ML: [500, 69900], MM: [500, 70100], MN: [500, 70300], MO: [500, 70500], MP: [500, 70700], MQ: [500, 70900], MR: [500, 71100], MS: [500, 71300], MT: [500, 71500], MU: [500, 71700], MV: [500, 71900], MW: [500, 72100], MX: [500, 72300], MY: [500, 72500], MZ: [500, 72700], NA: [500, 72900], NB: [500, 73100], NC: [500, 73300], ND: [500, 73500], NE: [500, 73700], NF: [500, 73900], NG: [500, 74100], NH: [500, 74300], NI: [500, 74500], NJ: [500, 74700], NK: [500, 74900], NL: [500, 75100], NM: [500, 75300], NN: [500, 75500], NO: [500, 75700], NP: [500, 75900], NQ: [500, 76100], NR: [500, 76300], NS: [500, 76500], NT: [500, 76700], NU: [500, 76900], NV: [500, 77100], NW: [500, 77300], NX: [500, 77500], NY: [500, 77700], NZ: [500, 77900], OA: [500, 78100], OB: [500, 78300], OC: [500, 78500], OD: [500, 78700], OE: [500, 78900], OF: [500, 79100], OG: [500, 79300], OH: [500, 79500], OI: [500, 79700], OJ: [500, 79900], OK: [500, 80100], OL: [500, 80300], OM: [500, 80500], ON: [500, 80700], OO: [500, 80900], OP: [500, 81100], OQ: [500, 81300], OR: [500, 81500], OS: [500, 81700], OT: [500, 81900], OU: [500, 82100], OV: [500, 82300], OW: [500, 82500], OX: [500, 82700], OY: [500, 82900], OZ: [500, 83100], PA: [500, 83300], PB: [500, 83500], PC: [500, 83700], PD: [500, 83900], PE: [500, 84100], PF: [500, 84300], PG: [500, 84500], PH: [500, 84700], PI: [500, 84900], PJ: [500, 85100], PK: [500, 85300], PL: [500, 85500], PM: [500, 85700], PN: [500, 85900], PO: [500, 86100], PP: [500, 86300], PQ: [500, 86500], PR: [500, 86700], PS: [500, 86900], PT: [500, 87100], PU: [500, 87300], PV: [500, 87500], PW: [500, 87700], PX: [500, 87900], PY: [500, 88100], PZ: [500, 88300], QA: [500, 88500], QB: [500, 88700], QC: [500, 88900], QD: [500, 89100], QE: [500, 89300], QF: [500, 89500], QG: [500, 89700], QH: [500, 89900], QI: [500, 90100], QJ: [500, 90300], QK: [500, 90500], QL: [500, 90700], QM: [500, 90900], QN: [500, 91100], QO: [500, 91300], QP: [500, 91500], QQ: [500, 91700], QR: [500, 91900], QS: [500, 92100], QT: [500, 92300], QU: [500, 92500], QV: [500, 92700], QW: [500, 92900], QX: [500, 93100], QY: [500, 93300], QZ: [500, 93500], RA: [500, 93700], RB: [500, 93900], RC: [500, 94100], RD: [500, 94300], RE: [500, 94500], RF: [500, 94700], RG: [500, 94900], RH: [500, 95100], RI: [500, 95300], RJ: [500, 95500], RK: [500, 95700], RL: [500, 95900], RM: [500, 96100], RN: [500, 96300], RO: [500, 96500], RP: [500, 96700], RQ: [500, 96900], RR: [500, 97100], RS: [500, 97300], RT: [500, 97500], RU: [500, 97700], RV: [500, 97900], RW: [500, 98100], RX: [500, 98300], RY: [500, 98500], RZ: [500, 98700], SA: [500, 98900], SB: [500, 99100], SC: [500, 99300], SD: [500, 99500], SE: [500, 99700], SF: [500, 99900], SG: [500, 100100], SH: [500, 100300], SI: [500, 100500], SJ: [500, 100700], SK: [500, 100900], SL: [500, 101100], SM: [500, 101300], SN: [500, 101500], SO: [500, 101700], SP: [500, 101900], SQ: [500, 102100], SR: [500, 102300], SS: [500, 102500], ST: [500, 102700], SU: [500, 102900], SV: [500, 103100], SW: [500, 103300], SX: [500, 103500], SY: [500, 103700], SZ: [500, 103900], TA: [500, 104100], TB: [500, 104300], TC: [500, 104500], TD: [500, 104700], TE: [500, 104900], TF: [500, 105100], TG: [500, 105300], TH: [500, 105500], TI: [500, 105700], TJ: [500, 105900], TK: [500, 106100], TL: [500, 106300], TM: [500, 106500], TN: [500, 106700], TO: [500, 106900], TP: [500, 107100], TQ: [500, 107300], TR: [500, 107500], TS: [500, 107700], TT: [500, 107900], TU: [500, 108100], TV: [500, 108300], TW: [500, 108500], TX: [500, 108700], TY: [500, 108900], TZ: [500, 109100], UA: [500, 109300], UB: [500, 109500], UC: [500, 109700], UD: [500, 109900], UE: [500, 110100], UF: [500, 110300], UG: [500, 110500], UH: [500, 110700], UI: [500, 110900], UJ: [500, 111100], UK: [500, 111300], UL: [500, 111500], UM: [500, 111700], UN: [500, 111900], UO: [500, 112100], UP: [500, 112300], UQ: [500, 112500], UR: [500, 112700], US: [500, 112900], UT: [500, 113100], UJ: [500, 113300], UV: [500, 113500], UW: [500, 113700], UX: [500, 113900], UY: [500, 114100], UZ: [500, 114300], VA: [500, 114500], VB: [500, 114700], VC: [500, 114900], VD: [500, 115100], VE: [500, 115300], VF: [500, 115500], VG: [500, 115700], VH: [500, 115900], VI: [500, 116100], VJ: [500, 116300], VK: [500, 116500], VL: [500, 116700], VM: [500, 116900], VN: [500, 117100], VO: [500, 117300], VP: [500, 117500], VQ: [500, 117700], VR: [500, 117900], VS: [500, 118100], VT: [500, 118300], VU: [500, 118500], VV: [500, 118700], VW: [500, 118900], VX: [500, 119100], VY: [500, 119300], VZ: [500, 119500], WA: [500, 119700], WB: [500, 119900], WC: [500, 120100], WD: [500, 120300], WE: [500, 120500], WF: [500, 120700], WG: [500, 120900], WH: [500, 121100], WI: [500, 121300], WJ: [500, 121500], WK: [500, 121700], WL: [500, 121900], WM: [500, 122100], WN: [500, 122300], WO: [500, 122500], WP: [500, 122700], WQ: [500, 122900], WR: [500, 123100], WS: [500, 123300], WT: [500, 123500], WU: [500, 123700], WV: [500, 123900], WW: [500, 124100], WX: [500, 124300], WY: [500, 124500], WZ: [500, 124700], XA: [500, 124900], XB: [500, 125100], XC: [500, 125300], XD: [500, 125500], XE: [500, 125700], XF: [500, 125900], XG: [500, 126100], XH: [500, 126300], XI: [500, 126500], XJ: [500, 126700], XK: [500, 126900], XL: [500, 127100], XM: [500, 127300], XN: [500, 127500], XO: [500, 127700], XP: [500, 127900], XQ: [500, 128100], XR: [500, 128300], XS: [500, 128500], XT: [500, 128700], XU: [500, 128900], XV: [500, 129100], XW: [500, 129300], XX: [500, 129500], XY: [500, 129700], XZ: [500, 129900], YA: [500, 130100], YB: [500, 130300], YC: [500, 130500], YD: [500, 130700], YE: [500, 130900], YF: [500, 131100], YG: [500, 131300], YH: [500, 131500], YI: [500, 131700], YJ: [500, 131900], YK: [500, 132100], YL: [500, 132300], YM: [500, 132500], YN: [500, 132700], YO: [500, 132900], YP: [500, 133100], YQ: [500, 133300], YR: [500, 133500], YS: [500, 133700], YT: [500, 133900], YU: [500, 134100], YV: [500, 134300], YW: [500, 134500], YX: [500, 134700], YY: [500, 134900], YZ: [500, 135100], ZA: [500, 135300], ZB: [500, 135500], ZC: [500, 135700], ZD: [500, 135900], ZE: [500, 136100], ZF: [500, 136300], ZG: [500, 136500], ZH: [500, 136700], ZI: [500, 136900], ZJ: [500, 137100], ZK: [500, 137300], ZL: [500, 137500], ZM: [500, 137700], ZN: [500, 137900], ZO: [500, 138100], ZP: [500, 138300], ZQ: [500, 138500], ZR: [500, 138700], ZS: [500, 138900], ZT: [500, 139100], ZU: [500, 139300], ZV: [500, 139500], ZW: [500, 139700], ZX: [500, 139900], ZY: [500, 140100], ZZ: [500, 140300], AA: [500, 140500], AB: [500, 140700], AC: [500, 140900], AD: [500, 141100], AE: [500, 141300], AF: [500, 141500], AG: [500, 141700], AH: [500, 141900], AI: [500, 142100], AJ: [500, 142300], AK: [500, 142500], AL: [500, 142700], AM: [500, 142900], AN: [500, 143100], AO: [500, 143300], AP: [500, 143500], AQ: [500, 143700], AR: [500, 143900], AS: [500, 144100], AT: [500, 144300], AU: [500, 144500], AV: [500, 144700], AW: [500, 144900], AX: [500, 145100], AY: [500, 145300], AZ: [500, 145500], BA: [500, 145700], BB: [500, 145900], BC: [500, 146100], BD: [500, 146300], BE: [500, 146500], BF: [500, 146700], BG: [500, 146900], BH: [500, 147100], BI: [500, 147300], BJ: [500, 147500], BK: [500, 147700], BL: [500, 147900], BM: [500, 148100], BN: [500, 148300], BO: [500, 148500], BP: [500, 148700], BQ: [500, 148900], BR: [500, 149100], BS: [500, 149300], BT: [500, 149500], BU: [500, 149700], BV: [500, 149900], BW: [500, 150100], BX: [500, 150300], BY: [500, 150500], BZ: [500, 150700], CA: [500, 150900], CB: [500, 151100], CC: [500, 151300], CD: [500, 151500], CE: [500, 151700], CF: [500, 151900], CG: [500, 152100], CH: [500, 152300], CI: [500, 152500], CJ: [500, 152700], CK: [500, 152900], CL: [500, 153100], CM: [500, 153300], CN: [500, 153500], CO: [500, 153700], CP: [500, 153900], CQ: [500, 154100], CR: [500, 154300], CS: [500, 154500], CT: [500, 154700], CU: [500, 154900], CV: [500, 155100], CW: [500, 155300], CX: [500, 155500], CY: [500, 155700], CZ: [500, 155900], DA: [500, 156100], DB: [500, 156300], DC: [500, 156500], DD: [500, 156700], DE: [500, 156900], DF: [500, 157100], DG: [500, 157300], DH: [500, 157500], DI: [500, 157700], DJ: [500, 157900], DK: [500, 158100], DL: [500, 158300], DM: [500, 158500], DN: [500, 158700], DO: [500, 158900], DP: [500, 159100], DQ: [500, 159300], DR: [500, 159500], DS: [500, 159700], DT: [500, 159900], DU: [500, 160100], DV: [500, 160300], DW: [500, 160500], DX: [500, 160700], DY: [500, 160900], DZ: [500, 161100], EA: [500, 161300], EB: [500, 161500], EC: [500, 161700], ED: [500, 161900], EE: [500, 162100], EF: [500, 162300], EG: [500, 162500], EH: [500, 162700], EI: [500, 162900], EJ: [500, 163100], EK: [500, 163300], EL: [500, 163500], EM: [500, 163700], EN: [500, 163900], EO: [500, 164100], EP: [500, 164300], EQ: [500, 164500], ER: [500, 164700], ES: [500, 164900], ET: [500, 165100], EU: [500, 165300], EV: [500, 165500], EW: [500, 165700], EX: [500, 165900], EY: [500, 166100], EZ: [500, 166300], FA: [500, 166500], FB: [500, 166700], FC: [500, 166900], FD: [500, 167100], FE: [500, 167300], FF: [500, 167500], FG: [500, 167700], FH: [500, 167900], FI: [500, 168100], FJ: [500, 168300], FK: [500,
```

```

 <g key={v}>
 <rect x={30 + i * 52} y={62} width={46} height={46} fill={color} />
 <text x={53 + i * 52} y={77} textAnchor="middle">
 {v}
 </text>
 </g>
)})
 {step >= 2 &&
 raw.map(({v, i}) => {
 <g key={`c${i}`}>
 <rect x={startX + i * (boxW + 4)} y={98} width={boxW} height={boxH} fill={color} />
 <text x={startX + i * (boxW + 4) + boxW / 2} y={113} textAnchor="center">
 {sorted.indexOf(v)}
 </text>
 </g>
 })
 </Svg>
);
};

/* ----- Splay: три случая поворота ----- */

type Pt = [number, number];

const SplayFrames: React.FC<{
 frames: { pos: Record<string, Pt>; edges: [string, string][];
 captions: string[];
 ms?: number;
}> = ({ frames, captions, ms = 1500 }) => {
 const step = useLoop(frames.length, ms);
 const f = frames[step];
 const color = (id: string) =>
 id === "x" ? "#6366f1" : id === "p" ? "#0ea5e9" : id === "c" ? "#f1c40f" : "#2ecc71";

 return (
 <Svg caption={captions[step]}>
 {f.edges.map(([a, b], i) => {

```

```

frames=[
 { pos: { g: [176, 24], p: [126, 70], x: [76, 112]
 { pos: { p: [130, 26], x: [78, 78], g: [186, 78]
 { pos: { x: [102, 24], p: [152, 70], g: [202, 112]
]}
/>
);

/** Zig-Zag – x и p с разных сторон, крутим сначала ниж
export const ZigZagDemo: React.FC = () => {
 <SplayFrames
 captions=["Зигзаг: g влево, x вправо", "Поворот НИ
 frames=[
 { pos: { g: [176, 24], p: [110, 70], x: [152, 112]
 { pos: { g: [176, 24], x: [110, 70], p: [64, 112]
 { pos: { x: [130, 30], p: [80, 100], g: [180, 100]
]}
 />
);

```

ФАЙЛ 43/83: src/components/hints/demosGraph.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРО

```

import React from "react";
import { C, Edge, Node, Svg, useLoop } from "../demoKit"

const GRAPH_POS: Record<string, [number, number]> = {
 A: [32, 65], B: [92, 30], C: [92, 100], D: [152, 65],
};
const GRAPH_EDGES: [string, string][] = [
 ["A", "B"], ["A", "C"], ["B", "D"], ["C", "D"], ["D",
];

export const BfsDemo: React.FC = () => {
 const order = [["A"], ["B", "C"], ["D"], ["E", "F"]];

```

```

const nodes = [
 { id: "трусы", x: 52, y: 32 }, { id: "штаны", x: 148, y: 32 },
 { id: "носки", x: 52, y: 98 }, { id: "боты", x: 148, y: 98 }
];
const edges: [string, string][] = [
 ["трусы", "штаны"],
 ["носки", "боты"]
];
const order = ["трусы", "носки", "штаны", "боты"];
const step = useLoop(order.length + 1, 800);
const placed = order.slice(0, step);
const pos = Object.fromEntries(nodes.map((n) => [n.id, n.x, n.y]));
return (
 <Svg caption={`Порядок: ${placed.join(" → ") || "..."} `}
 {edges.map(([u, v], i) => (
 <Edge key={i} a={pos[u]} b={pos[v]} color={placed[i]} />
))}
 {nodes.map((n) => {
 const idx = placed.indexOf(n.id);
 return (
 <g key={n.id}>
 <rect x={n.x - 34} y={n.y - 12} width={68} height={24} />
 <text x={n.x} y={n.y} textAnchor="middle" dx={-34} dy={-12}>
 {idx >= 0 && <circle cx={n.x + 30} cy={n.y - 12} r={12} />
 {idx >= 0 && {
 <text x={n.x + 30} y={n.y - 12} textAnchor="middle">
 {placed[idx]}
 </text>
 }}
 </g>
 </g>
);
 })}
 <text x={224} y={62} textAnchor="middle" fontSize={16}>
 Порядок: {placed.join(" → ") || "..."}
 </text>
 <text x={224} y={74} textAnchor="middle" fontSize={16}>
 {placed[0]}
 </text>
 </Svg>
);
};

```

```

export const RelaxDemo: React.FC = () => {
 const step = useLoop(3, 1100);
 const dV = [12, 12, 7][step];

```



```

const step = useLoop(2, 1300);
const pos: Record<string, [number, number]> = {
 A: [36, 40], B: [36, 96], C: [130, 68], D: [224, 40]
};
const edges: [string, string][] = [["A", "B"], ["A",
const cut = step === 1;
return (
 <Svg caption={cut ? "Убрали вершину C → две отдельные"
 {edges.map(([u, v], i) => (cut && (u === "C" || v
 {Object.entries(pos).map(([k, [x, y]]) =>
 cut && k === "C" ? (
 <circle key={k} cx={x} cy={y} r={12} fill="no
) : (
 <Node key={k} x={x} y={y} label={k} fill={k =
 } />
)
)}
 </Svg>
);
};

```

```

export const MstDemo: React.FC = () => {
 const pos: Record<string, [number, number]> = {
 A: [40, 35], B: [130, 22], C: [220, 45], D: [70, 10
];
 const edges = [
 { u: "A", v: "B", w: 2 }, { u: "B", v: "C", w: 3 },
 { u: "D", v: "E", w: 4 }, { u: "B", v: "E", w: 6 },
];
 const chosen = ["A-D", "A-B", "B-C", "D-E"];
 const step = useLoop(chosen.length + 1, 800);
 const taken = new Set(chosen.slice(0, step));
 return (
 <Svg caption={`Жадно берём самые лёгкие рёбра без ц
 {edges.map((e, i) => {
 const on = taken.has(`${e.u}-${e.v}`);
 return (
 <g key={i}>

```

```

import { C, Edge, Node, Svg, useLoop } from "../demoKit"

export const HeapDemo: React.FC = () => {
 const states = [[4, 8, 9, 12, 2], [4, 2, 9, 12, 8], [
 const step = useLoop(states.length, 1000);
 const heap = states[step];
 const pos: [number, number][] = [[130, 24], [80, 68],
 const moving = step === 0 ? 4 : step === 1 ? 1 : 0;
 return (
 <Svg caption="Новый элемент всплывает наверх, пока"
 <Edge a={pos[0]} b={pos[1]} /><Edge a={pos[0]} b=
 <Edge a={pos[1]} b={pos[3]} /><Edge a={pos[1]} b=
 {heap.map((v, i) => (
 <Node key={i} x={pos[i][0]} y={pos[i][1]} label=
))}
 </Svg>
);
};

export const StackDemo: React.FC = () => {
 const states = ["A"], ["A", "B"], ["A", "B", "C"], [
 const step = useLoop(states.length, 800);
 const items = states[step];
 const popping = step >= 3;
 return (
 <Svg caption={popping ? "pop: забираем ВЕРХНИЮЮ тапе
 <rect x={90} y={22} width={80} height={96} rx={6}
 {items.map((v, i) => (
 <g key={v} style={{ transition: "all .3s" }}>
 <rect x={94} y={100 - i * 26} width={72} height=
 fill={i === items.length - 1 ? (popping ? C
 <text x={130} y={111 - i * 26} textAnchor="mi
 </g>
))}
 <text x={130} y={16} textAnchor="middle" fontSize=
 </Svg>
);
};

```

```

 fill="none" stroke={rootColor[root(i)]} str
) : null
 })
 {pos.map(([x, y], i) => (<Node key={i} x={x} y={y}
 </Svg>
);
};

```

```

const TRIE_NODES = [
 { id: 0, x: 130, y: 22, ch: "•", parent: null as number },
 { id: 1, x: 72, y: 62, ch: "a", parent: 0 },
 { id: 2, x: 188, y: 62, ch: "b", parent: 0 },
 { id: 3, x: 42, y: 104, ch: "b", parent: 1 },
 { id: 4, x: 102, y: 104, ch: "c", parent: 1 },
 { id: 5, x: 188, y: 104, ch: "c", parent: 2 },
];

```

```

export const TrieDemo: React.FC = () => {
 const step = useLoop(TRIE_NODES.length + 1, 700);
 return (
 <Svg caption="Путь от корня по буквам = слово; общий"
 {TRIE_NODES.filter((n) => n.parent !== null).map(
 const p = TRIE_NODES[n.parent as number];
 return <Edge key={n.id} a={[p.x, p.y]} b={[n.x,
]}
 {TRIE_NODES.map((n) => (<Node key={n.id} x={n.x}
 </Svg>
);
};

```

```

export const TrieBfsDemo: React.FC = () => {
 const levels = [[0], [1, 2], [3, 4, 5]];
 const step = useLoop(levels.length + 1, 950);
 const doneIds = new Set(levels.slice(0, step).flat());
 const wave = step > 0 && step <= levels.length ? leve
 return (
 <Svg caption={`BFS по бору: уровень ${Math.max(0, M
 {TRIE_NODES.filter((n) => n.parent !== null).map(

```

```

const nodes = [
 { x: 130, y: 24, w: 226, label: "[0..7]", lvl: 0 },
 { x: 72, y: 60, w: 110, label: "[0..3]", lvl: 1 },
 { x: 188, y: 60, w: 110, label: "[4..7]", lvl: 1 },
 { x: 44, y: 96, w: 52, label: "[0..1]", lvl: 2 },
 { x: 100, y: 96, w: 52, label: "[2..3]", lvl: 2 },
 { x: 160, y: 96, w: 52, label: "[4..5]", lvl: 2 },
 { x: 216, y: 96, w: 52, label: "[6..7]", lvl: 2 },
];
return (
 <Svg caption="Запрос собирается из $O(\log n)$ готовых"
 {nodes.map((n, i) => (
 <g key={i}>
 <rect x={n.x - n.w / 2} y={n.y - 10} width={n.w} height={n.h}
 fill={n.lvl === step ? C.active : C.idle} stroke={C.dim} />
 <text x={n.x} y={n.y} textAnchor="middle" domClass={C.dim} />
 </g>
))}
 </Svg>
);
};

```

```

export const PrefixSumDemo: React.FC = () => {
 const a = [3, 1, 4, 1, 5, 9];
 const pref = a.reduce<number[]>((acc, v) => [...acc, v], []);
 const step = useLoop(4, 1200);
 const l = 1, r = 3;
 const show = step >= 1;
 return (
 <Svg caption={show ? `a[${l}..${r}] = P[${r + 1}] - P[${l}]` :
 "о префиксные суммы P"}>
 <text x={4} y={18} fontSize={8} fill={C.dim}>a</text>
 {a.map((v, i) => (
 <g key={i}>
 <rect x={20 + i * 39} y={24} width={34} height={12}
 fill={show && i >= l && i <= r ? C.active : C.idle} stroke={C.dim} />
 <text x={37 + i * 39} y={36} textAnchor="middle" fill={C.dim}>{v}</text>
 </g>
))}
 </Svg>
);
};

```

```

const step = useLoop(costs.length + 1, 550);
const total = costs.slice(0, step).reduce((a, b) => a + b, 0);
const avg = step ? (total / step).toFixed(2) : "-";
return (
 <Svg caption={`Сделано ${step} операций, суммарно $
 {costs.map((c, i) => (
 <rect key={i} x={14 + i * 30} y={104 - c * 10} width={30} height={10}
 fill={i < step ? (c > 2 ? C.bad : C.done) : C.done}
)}
 <line x1={8} y1={106} x2={252} y2={106} stroke={C.done}
 <text x={130} y={20} textAnchor="middle" fontSize={14}
 </Svg>
);
};

```

```

export const NpDemo: React.FC = () => {
 const step = useLoop(2, 1400);
 return (
 <Svg caption={step === 0 ? "Найти решение – долго (1000 лет)" : "Решение найдено"}
 <ellipse cx={130} cy={64} rx={112} ry={48} fill={step === 0 ? "#ccc" : "#333"}
 <ellipse cx={92} cy={64} rx={52} ry={32} fill="#333"}
 <text x={92} y={64} textAnchor="middle" dominantBaseline="middle"
 <text x={206} y={34} textAnchor="middle" fontSize={14}
 <circle cx={196} cy={84} r={13} fill={step === 1 ? "#ccc" : "#333"}
 <text x={196} y={84} textAnchor="middle" dominantBaseline="middle"
 ?"/>
 <text x={130} y={124} textAnchor="middle" fontSize={14}
 </Svg>
);
};

```

```

export const PrefixFunctionDemo: React.FC = () => {
 const s = "abacaba";
 const pi = [0, 0, 1, 0, 1, 2, 3];
 const step = useLoop(s.length + 1, 700);
 const i = Math.max(0, step - 1);
 const len = step > 0 ? pi[i] : 0;
 return (

```

```

 </g>
);
 })
 })
 <text x={130} y={122} textAnchor="middle" fontSize=
 каждая клетка считается один раз и переиспользу
 </text>
</Svg>
);
};

```

ФАЙЛ 45/83: src/components/hints/MiniDemo.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```

import React, { useState } from "react";
import { DemoPauseContext } from "../demoKit";
import { ArticulationDemo, BfsDemo, BridgeDemo, DfsDemo,
import {
 AmortizedDemo, DsuDemo, HeapDemo, NpDemo, PrefixSumDemo,
 SparseTableDemo, StackDemo, SuffixLinkDemo, TrieBfsDemo,
} from "../demosStruct";
import {
 ComponentsDemo, CoordCompressDemo, FloydDemo, GraphBa
 ZigDemo, ZigZagDemo, ZigZigDemo,
} from "../demosExtra";

export type DemoKind =
 | "bfs" | "dfs" | "heap" | "stack" | "queue" | "dsu"
 | "suffix-link" | "toposort" | "relax" | "prefix-sum"
 | "segment-tree" | "bridge" | "articulation" | "mst"
 | "prefix-function" | "dp" | "floyd" | "components" |
 | "zig" | "zig-zig" | "zig-zag";

const REGISTRY: Record<DemoKind, React.FC> = {
 bfs: BfsDemo,

```

```
 onMouseEnter={() => setPaused(true)}
 onMouseLeave={() => setPaused(false)}
 >
 <Cmp />
 </div>
 </DemoPauseContext.Provider>
);
};
```

---

ФАЙЛ 46/83: src/components/hints/SimulatorModal.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

---

```
import React, { useEffect } from "react";
import { createPortal } from "react-dom";
import { X } from "lucide-react";
import { getViz } from "../vizRegistry";
```

```
interface Props {
 vizId: string | null;
 onClose: () => void;
}
```

/\*\* Модальное окно с полноценным симулятором – вызывает

export const SimulatorModal: React.FC<Props> = ({ vizId

```
 useEffect(() => {
 if (!vizId) return;
 const onKey = (e: KeyboardEvent) => {
 if (e.key === "Escape") onClose();
 };
 window.addEventListener("keydown", onKey);
 document.body.style.overflow = "hidden";
 return () => {
 window.removeEventListener("keydown", onKey);
 document.body.style.overflow = "";
 };
 });
```

```
import React, { useEffect, useLayoutEffect, useRef, use
import { createPortal } from "react-dom";
import { Play, X } from "lucide-react";
import { GLOSSARY_BY_ID } from "../../data/glossary";
import { MiniDemo } from "../MiniDemo";
import { getViz } from "../vizRegistry";

export interface HintAnchor {
 termId: string;
 rect: DOMRect;
}

interface Props {
 anchor: HintAnchor | null;
 onClose: () => void;
 onOpenSimulator: (vizId: string) => void;
 onCardEnter: () => void;
 onCardLeave: () => void;
}

const CARD_W = 320;
const GAP = 10;

export const TermPopover: React.FC<Props> = ({ anchor,
 const cardRef = useRef<HTMLDivElement>(null);
 const [pos, setPos] = useState<{ top: number; left: number}>({
 top: 0, left: 0,

 useLayoutEffect(() => {
 if (!anchor) {
 setPos(null);
 return;
 }
 const vw = window.innerWidth;
 const vh = window.innerHeight;
 const width = Math.min(CARD_W, vw - 16);
 const height = cardRef.current?.offsetHeight ?? 300
```



```

 visibility: pos ? "visible" : "hidden",
 }}
 role="dialog"
 >
 <div className="bg-slate-900 border border-indigo-500" >
 <div className="flex items-start gap-2 px-3 pt-3" >
 <h4 className="text-sm font-bold text-indigo-400" >Подсказка
 <button
 onClick={onClose}
 className="text-slate-500 hover:text-white"
 aria-label="Заккрыть подсказку"
 >
 <X className="w-4 h-4" />
 </button>
 </div>

 <div className="p-3 space-y-2.5 max-h-[60vh] overflow-y-auto" >
 <p className="text-[13px] leading-relaxed text-slate-300">
 {entry.demo && <MiniDemo kind={entry.demo} />}

 {entry.formal && (
 <p className="text-[11.5px] leading-relaxed text-slate-300">
 {entry.formal}
 </p>
)}

 {entry.complexity && (
 <div className="text-[11px] font-mono text-slate-300">
 Сложность: {entry.complexity}
 </div>
)}

 {viz && simulatorId && (
 <button
 onClick={() => onOpenSimulator(simulatorId)}
 className="w-full flex items-center justify-center text-slate-300
 rounded-lg py-2 transition-colors"
 >

```

```

];

 const isNodeVisible = (id: string) => {
 if (id === 'S' && (step === 0 || step === 7)) return true;
 };

 const getPotential = (id: string) => {
 if (step >= 2 && step < 7) {
 if (id === 'S') return 0;
 if (id === 'A') return 0;
 if (id === 'B') return -2;
 if (id === 'C') return -1;
 }
 return null;
 };

 const getEdgeColorClass = (e: any) => {
 if (e.u === 'S') {
 if (step === 0 || step === 7) return "opacity-0";
 if (step === 1) return "stroke-amber-500 stroke-dasharray-4";
 if (step === 2) return "stroke-pink-500 stroke-dasharray-4";
 return "stroke-slate-700 stroke-dasharray-4";
 }

 if (e.id === 'AB') {
 if (step === 4) return "stroke-amber-400 stroke-dasharray-4";
 if (step > 4) return "stroke-emerald-500";
 }
 if (e.id === 'BC') {
 if (step === 5) return "stroke-amber-400 stroke-dasharray-4";
 if (step > 5) return "stroke-emerald-500";
 }
 if (e.id === 'CA') {
 if (step === 6) return "stroke-amber-400 stroke-dasharray-4";
 if (step > 6) return "stroke-emerald-500";
 }
 };

```

```

 if (step === 6) return `${e.weight} + (${-1
 return e.newWeight;
 }
 return e.weight;
};

const isEdgeTextHighlighted = (e: any) => {
 if (e.id === 'AB' && step === 4) return true;
 if (e.id === 'BC' && step === 5) return true;
 if (e.id === 'CA' && step === 6) return true;
 return false;
};

const isEdgeTextEmerald = (e: any) => {
 if (e.id === 'AB' && step > 4) return true;
 if (e.id === 'BC' && step > 5) return true;
 if (e.id === 'CA' && step > 6) return true;
 return false;
};

const getDesc = () => {
 switch (step) {
 case 0: return (
 <div>
 Исходный граф. Имеется ребро
 Если мы запустим быстрый алгоритм Д
 </div>
);
 case 1: return (
 <div>
 Шаг 1: Точка отсчета.

 Добавляем фиктивную вершину S. Соеди
ми.
 Это наша база для расчета потенциал
 </div>
);
 case 2: return (
 <div>

```

```

 Все новые веса ребер в графе стали :
 При этом старые кратчайшие пути оста
шин!
 </div>
);
 default: return "";
}
};

return (
 <div className="bg-slate-800 p-4 rounded-xl border
 <h4 className="text-sm md:text-base font-bo
 <span className="w-2.5 h-2.5 bg-amber-400
 Визуализация: Перевзвешивание алгоритмов
 </h4>

 <div className="flex flex-col md:flex-row gap-4
 <div className="bg-[#0f172a] flex-1 min-h-100
 nter p-4">
 <svg width="400" height="300" class="
 <defs>
 <marker id="arrow-slate" viewBox="0 0 10 5"
 start-reverse">
 <path d="M 0 0 L 10 5 L 5 10" />
 </marker>
 <marker id="arrow-slate-dim" viewBox="0 0 10 5"
 uto-start-reverse">
 <path d="M 0 0 L 10 5 L 5 10" />
 </marker>
 <marker id="arrow-amber" viewBox="0 0 10 5"
 start-reverse">
 <path d="M 0 0 L 10 5 L 5 10" />
 </marker>
 <marker id="arrow-pink" viewBox="0 0 10 5"
 tart-reverse">
 <path d="M 0 0 L 10 5 L 5 10" />
 </marker>
 <marker id="arrow-emerald" viewBox="0 0 10 5"
 end-reverse">
 <path d="M 0 0 L 10 5 L 5 10" />
 </marker>
 </defs>
 </div>
 </div>
 </div>
 </div>
);
);

```



```
 </div>
 </div>
 </div>
 </div>
);
}
```

---

ФАЙЛ 49/83: src/components/KosarajuViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import React, { useState, useEffect } from "react";
```

```
import {
 Play,
 Pause,
 RotateCcw,
 Layers,
} from "lucide-react";
```

```
interface GNode {
 id: number;
 x: number;
 y: number;
 label: string;
}
```

```
interface GEdge {
 u: number;
 v: number;
}
```

```
const initialNodes: GNode[] = [
 { id: 0, x: 150, y: 100, label: "0" },
 { id: 1, x: 300, y: 100, label: "1" },
 { id: 2, x: 225, y: 220, label: "2" }, // 0->1->2->0
 { id: 3, x: 450, y: 160, label: "3" },
```

```

 desc: string,
 currentNode: number | null,
 transposed: boolean,
 currentSccId: number,
) => {
 generatedSteps.push({
 phase,
 desc,
 visited: new Set(visited),
 currentNode,
 order: [...order],
 transposed,
 sccMap: { ...sccMap },
 currentSccId,
 activeEdges: transposed
 ? initialEdges.map((e) => ({ u: e.v, v: e.u }
 : [...initialEdges],
));
};

recordStep(
 "init",
 "Инициализация алгоритма Косарайю. Начинаем первый",
 null,
 false,
 -1,
);

// Adj list
const adj: number[][] = Array.from({ length: 5 }, () => []);
initialEdges.forEach((e) => adj[e.u].push(e.v));

// DFS 1: post-order list
const dfs1 = (u: number) => {
 visited.add(u);
 recordStep(
 "dfs1",
 `DFS 1: зашли в вершину ${u}, помечаем посещенной`
);
};

```

```
// Transpose adj
const adjT: number[][] = Array.from({ length: 5 },
initialEdges.forEach((e) => adjT[e.v].push(e.u));

// Reset visited for phase 2
visited.clear();
let sccId = 0;

const dfs2 = (u: number, currentScc: number) => {
 visited.add(u);
 sccMap[u] = currentScc;
 recordStep(
 "dfs2",
 `DFS 2: заходим в ${u} на транспонированном графе`,
 u,
 true,
 currentScc,
);

 for (const to of adjT[u]) {
 if (!visited.has(to)) {
 dfs2(to, currentScc);
 }
 }
};

// Run DFS2 using the order (reversed from back to front)
for (let i = order.length - 1; i >= 0; i--) {
 const u = order[i];
 if (!visited.has(u)) {
 sccId++;
 recordStep(
 "dfs2",
 `DFS 2: берем следующую непосещенную из стека`,
 u,
 true,
 sccId,
);
 }
}
```



```

 if (isPlaying && currentStepIdx < steps.length - 1) {
 timer = setInterval(() => {
 setCurrentStepIdx((prev) => prev + 1);
 }, 1500);
 } else {
 setIsPlaying(false);
 }
 return () => clearInterval(timer);
 }, [isPlaying, currentStepIdx, steps.length]);

const togglePlay = () => setIsPlaying(!isPlaying);
const reset = () => {
 setIsPlaying(false);
 setCurrentStepIdx(0);
};

const getScColor = (id: number) => {
 if (id === 1) return "fill-emerald-600 stroke-emera
 if (id === 2) return "fill-indigo-600 stroke-indigo
 return "fill-slate-800 stroke-slate-500";
};

return (
 <div className="bg-slate-900 rounded-xl border bord
 <div className="bg-slate-800 p-4 border-b border-
 <h3 className="font-bold text-white flex items-
 <Layers className="w-5 h-5 text-emerald-400"
 Визуализатор Алгоритма Косарайю (Поиск SCC)
 </h3>

 <div className="flex items-center gap-2 bg-slat
 <button
 onClick={togglePlay}
 className="flex items-center gap-2 px-3 py-
 d-500 text-white"
 >
 {isPlaying ? (
 <Pause className="w-4 h-4 ml-1" />

```

```

 markerWidth="6"
 markerHeight="6"
 orient="auto-start-reverse"
 >
 <path d="M 0 1 L 10 5 L 0 9 z" fill="#1f77b4" />
 </marker>
 <marker
 id="arrow-transposed"
 viewBox="0 0 10 10"
 refX="22"
 refY="5"
 markerWidth="6"
 markerHeight="6"
 orient="auto-start-reverse"
 >
 <path d="M 0 1 L 10 5 L 0 9 z" fill="#66bb6a" />
 </marker>
</defs>

```

```

{/* Edges */}
{initialEdges.map((edge, idx) => {
 const u = initialNodes.find((n) => n.id === edge.u);
 const v = initialNodes.find((n) => n.id === edge.v);

 // If transposed, draw reversed coordinates
 const x1 = step.transposed ? v.x : u.x;
 const y1 = step.transposed ? v.y : u.y;
 const x2 = step.transposed ? u.x : v.x;
 const y2 = step.transposed ? u.y : v.y;

 const isSccEdge =
 step.sccMap[edge.u] &&
 step.sccMap[edge.v] &&
 step.sccMap[edge.u] === step.sccMap[edge.v];
 let stroke = "#475569";
 let markerId = "arrow";

 if (step.transposed) {

```

```

 (step.phase === "dfs1" && step.currentNode === node) {
 const sccId = step.sccMap[node.id];

 let classes = "fill-slate-800 stroke-slate-800";
 if (sccId) {
 classes = getSccColor(sccId);
 } else if (isCurrent) {
 classes = "fill-amber-600 stroke-amber-600";
 } else if (isVisited && step.phase === "dfs2") {
 classes = "fill-emerald-700 stroke-emerald-700";
 }

 return (
 <g key={node.id}>
 <circle
 cx={node.x}
 cy={node.y}
 r="20"
 className={`transition-all duration-200 ${classes}`}
 strokeWidth="3"
 />
 <text
 x={node.x}
 y={node.y}
 dy=".3em"
 textAnchor="middle"
 fill="white"
 fontSize="13px"
 fontWeight="bold"
 className="select-none"
 >
 {node.label}
 </text>

 {sccId && (
 <text
 x={node.x}
 y={node.y - 28}
 />
)}
 </g>
);
 }
 }
}

```

```

 СТЕК ВЫХОДА (DFS 1 ORDER):

 <div className="flex flex-wrap gap-1 bg-slate-600 text-white" >
 {step.order.length === 0 ? (

 Нет шагов

) : (
 [...step.order].reverse().map(({val, idx}) => {
 <div
 key={idx}
 className="bg-indigo-950 border border-white p-1"
 ex items-center gap-1"
 >
 {val}
 </div>
 })
)}
 </div>
 </div>

 {/* List of actions log */}
 <div>

 ЛОГ ОПЕРАЦИЙ:

 <div className="space-y-1.5 max-h-[150px]" >
 {steps
 .slice(0, currentStepIdx + 1)
 .reverse()
 .slice(0, 5)
 .map(({s, idx}) => {
 <div
 key={idx}
 className={`text-[11px] p-1.5 rounded`
 >
 {s.desc}
 </div>
 })
)}
 </div>
 </div>

```

```

 ide-react';

interface Vertex {
 id: number;
 label: string;
 x: number;
 y: number;
 color: string; // 'none', 'red', 'blue', 'purple', 'm
}

interface Edge {
 id: string;
 u: number;
 v: number;
 weight: number;
 status: 'pending' | 'inspecting' | 'included' | 'reje
 color: string;
}

interface StepLog {
 title: string;
 description: string;
 type: 'info' | 'success' | 'warning' | 'error';
}

// 9 Vertices layout
const initialVertices: Vertex[] = [
 { id: 0, label: 'A', x: 20, y: 20, color: 'none' },
 { id: 1, label: 'B', x: 50, y: 13, color: 'none' },
 { id: 2, label: 'C', x: 80, y: 20, color: 'none' },
 { id: 3, label: 'D', x: 18, y: 50, color: 'none' },
 { id: 4, label: 'E', x: 50, y: 50, color: 'none' },
 { id: 5, label: 'F', x: 82, y: 50, color: 'none' },
 { id: 6, label: 'G', x: 20, y: 80, color: 'none' },
 { id: 7, label: 'H', x: 50, y: 87, color: 'none' },
 { id: 8, label: 'I', x: 80, y: 80, color: 'none' },
];

```

```

 setLogs(prev => [{
 title: 'Шаг 1: Берем самое дешевое ребро A-B (в',
 description: 'Оно соединяет две бесцветные верши',
 type: 'info'
 }, ...prev]));
 },
 // Step 2: Include A-B
 () => {
 setVertices(prev => prev.map(v => v.id === 0 || v
 setEdges(prev => prev.map(e => e.id === '0-1' ? {
 setLogs(prev => [{
 title: 'Успех: Ребро A-B в осто́ве',
 description: 'Вершины A and B окрашены в красны',
 type: 'success'
 }, ...prev]));
 },
 // Step 3: Inspect D-E (3)
 () => {
 setEdges(prev => prev.map(e => e.id === '3-4' ? {
 setLogs(prev => [{
 title: 'Шаг 2: Берем второе ребро D-E (вес 3)',
 description: 'Оно соединяет две другие бесцветн',
 type: 'info'
 }, ...prev]));
 },
 // Step 4: Include D-E
 () => {
 setVertices(prev => prev.map(v => v.id === 3 || v
 setEdges(prev => prev.map(e => e.id === '3-4' ? {
 setLogs(prev => [{
 title: 'Успех: Ребро D-E в осто́ве',
 description: 'Вершины D и E окрашены в синий цв',
 type: 'success'
 }, ...prev]));
 },
 // Step 5: Inspect B-C (4)
 () => {
 setEdges(prev => prev.map(e => e.id === '1-2' ? {

```

```

 setEdges(prev => prev.map(e => e.id === '1-4' ? {
 setLogs(prev => [{
 title: 'Шаг 5: Берем ребро В-Е (вес 6)',
 description: 'Н0: это ребро соединяет вершины В
 type: 'warning'
 }, ...prev]));
 },
// Step 10: Reject В-Е
() => {
 setEdges(prev => prev.map(e => e.id === '1-4' ? {
 setLogs(prev => [{
 title: 'Отказ: Цикл обнаружен!',
 description: 'Мы выбрасываем ребро В-Е. Иначе п
 type: 'error'
 }, ...prev]));
},
// Step 11: Inspect E-F (7)
() => {
 setEdges(prev => prev.map(e => e.id === '4-5' ? {
 setLogs(prev => [{
 title: 'Шаг 6: Берем ребро E-F (вес 7)',
 description: 'Оно соединяет фиолетовую вершину
 type: 'info'
 }, ...prev]));
},
// Step 12: Include E-F
() => {
 setVertices(prev => prev.map(v => v.id === 5 ? {
 setEdges(prev => prev.map(e => e.id === '4-5' ? {
 setLogs(prev => [{
 title: 'Успех: Ребро E-F в осто́ве',
 description: 'Вершина F окрашена в фиолетовый ц
 type: 'success'
 }, ...prev]));
},
// Step 13: Inspect D-G (8)
() => {
 setEdges(prev => prev.map(e => e.id === '3-6' ? {

```

```

 title: 'Шаг 9: Берем ребро G-H (вес 10)',
 description: 'Соединяет фиолетовую G и бесцветную H',
 type: 'info'
 }, ...prev]);
},
// Step 18: Include G-H
() => {
 setVertices(prev => prev.map(v => v.id === 7 ? {
 setEdges(prev => prev.map(e => e.id === '6-7' ? {
 setLogs(prev => [{
 title: 'Успех: Ребро G-H в остоле',
 description: 'Вершина H перекрашена в фиолетовую',
 type: 'success'
 }, ...prev]);
},
// Step 19: Inspect E-H (11) - Cycle!
() => {
 setEdges(prev => prev.map(e => e.id === '4-7' ? {
 setLogs(prev => [{
 title: 'Шаг 10: Берем ребро E-H (вес 11)',
 description: 'Оба конца E и H уже фиолетовые. Соединим их.',
 type: 'warning'
 }, ...prev]);
},
// Step 20: Reject E-H
() => {
 setEdges(prev => prev.map(e => e.id === '4-7' ? {
 setLogs(prev => [{
 title: 'Отказ: Цикл обнаружен!',
 description: 'Выбрасываем ребро E-H.',
 type: 'error'
 }, ...prev]);
},
// Step 21: Inspect H-I (12)
() => {
 setEdges(prev => prev.map(e => e.id === '7-8' ? {
 setLogs(prev => [{
 title: 'Шаг 11: Берем ребро H-I (вес 12)',

```



```

 }, ...prev]));
},
// Step 3: Include D-E, add D's edges to heap
() => {
 setVertices(prev => prev.map(v => v.id === 3 ? {
 setEdges(prev => prev.map(e => e.id === '3-4' ? {
 ..e, status: 'in_heap' } : e));
 setHeapQueue(['A-D (5)', 'B-E (6)', 'E-F (7)', 'D
 setLogs(prev => [{
 title: 'Успех: Плесень захватила вершину D',
 description: 'В кучу добавлены новые пути из D:
 type: 'success'
 }, ...prev]));
},
// Step 4: Pop A-D (5)
() => {
 setEdges(prev => prev.map(e => e.id === '0-3' ? {
 setLogs(prev => [{
 title: 'Шаг 3: Куча выдает ребро A-D (вес 5)',
 description: 'Плесень расширяется в сторону вер
 type: 'info'
 }, ...prev]));
},
// Step 5: Include A-D, add A's edges to heap
() => {
 setVertices(prev => prev.map(v => v.id === 0 ? {
 setEdges(prev => prev.map(e => e.id === '0-3' ? {
 eap' } : e));
 setHeapQueue(['A-B (2)', 'B-E (6)', 'E-F (7)', 'D
 setLogs(prev => [{
 title: 'Успех: Плесень поглотила вершину A',
 description: 'Новое ребро A-B(2) добавлено в ку
 type: 'success'
 }, ...prev]));
},
// Step 6: Pop A-B (2)
() => {
 setEdges(prev => prev.map(e => e.id === '0-1' ? {

```

```

 },
 // Step 10: Pop B-E (6) - Cycle!
 () => {
 setEdges(prev => prev.map(e => e.id === '1-4' ? {
 setLogs(prev => [{
 title: 'Шаг 6: Куча выдает ребро B-E (вес 6)',
 description: 'ВНИМАНИЕ! Плесень проверяет верши
 type: 'warning'
 }, ...prev]));
 },
 // Step 11: Reject B-E
 () => {
 setEdges(prev => prev.map(e => e.id === '1-4' ? {
 setHeapQueue(['E-F (7)', 'D-G (8)', 'C-F (9)', 'E
 setLogs(prev => [{
 title: 'Отказ: Игнорируем внутреннее ребро',
 description: 'Ребро B-E отброшено. Плесень не р
 type: 'error'
 }, ...prev]));
 },
 // Step 12: Pop E-F (7)
 () => {
 setEdges(prev => prev.map(e => e.id === '4-5' ? {
 setLogs(prev => [{
 title: 'Шаг 7: Куча выдает ребро E-F (вес 7)',
 description: 'Плесень из центра Токио (E) тянет
 type: 'info'
 }, ...prev]));
 },
 // Step 13: Include E-F, add F's edges
 () => {
 setVertices(prev => prev.map(v => v.id === 5 ? {
 setEdges(prev => prev.map(e => e.id === '4-5' ? {
 eap' } : e));
 setHeapQueue(['D-G (8)', 'C-F (9 - цикл)', 'E-H (
 setLogs(prev => [{
 title: 'Успех: Вершина F захвачена',
 description: 'Ребро F-I(13) добавлено в кучу. Р

```

```

 title: 'Отказ: Игнорируем ребро C-F',
 description: 'Выбрасываем из кучи.',
 type: 'error'
 }, ...prev]));
},
// Step 18: Pop G-H (10)
() => {
 setEdges(prev => prev.map(e => e.id === '6-7' ? {
 setLogs(prev => [{
 title: 'Шаг 10: Куча выдает ребро G-H (вес 10)',
 description: 'Плесень ползет к вершине H.',
 type: 'info'
 }, ...prev]));
},
// Step 19: Include G-H, add H's edges
() => {
 setVertices(prev => prev.map(v => v.id === 7 ? {
 setEdges(prev => prev.map(e => e.id === '6-7' ? {
 eap' } : e));
 setHeapQueue(['E-H (11 - цикл)', 'H-I (12)', 'F-I
 setLogs(prev => [{
 title: 'Успех: Вершина H захвачена',
 description: 'В кучу добавлено H-I(12).',
 type: 'success'
 }, ...prev]));
},
// Step 20: Pop E-H (11) - Cycle!
() => {
 setEdges(prev => prev.map(e => e.id === '4-7' ? {
 setLogs(prev => [{
 title: 'Шаг 11: Куча выдает E-H (вес 11)',
 description: 'Обе вершины E и H уже в плесени.'
 type: 'warning'
 }, ...prev]));
},
// Step 21: Reject E-H
() => {
 setEdges(prev => prev.map(e => e.id === '4-7' ? {

```

```

// Node 3(D) -> D-E(3)
// Node 4(E) -> D-E(3)
// Node 5(F) -> E-F(7)
// Node 6(G) -> D-G(8)
// Node 7(H) -> G-H(10)
// Node 8(I) -> H-I(12)
setEdges(prev => prev.map(e =>
 ['0-1', '3-4', '1-2', '4-5', '3-6', '6-7', '7-8'
]));
setLogs(prev => [{
 title: 'Первая пятилетка: Каждая деревня выбирает
description: 'Каждая из 9 деревень одновременно
 А выбирает А-В(2), D выбирает D-E(3), а G вы
 !',
 type: 'info'
}, ...prev]);
},
// Step 2: Concurrently merge into Kolkhozes (Components)
() => {
 // Concurrently build chosen roads. Merge into:
 // Component 1: {A, B, C} (colored red)
 // Component 2: {D, E, F, G, H, I} (colored gold/olive)
 setVertices(prev => prev.map(v =>
 [0, 1, 2].includes(v.id) ? { ...v, color: 'red' } : v
));
 setEdges(prev => prev.map(e => {
 if (['0-1', '1-2'].includes(e.id)) {
 return { ...e, status: 'included', color: 'red' };
 }
 if (['3-4', '4-5', '3-6', '6-7', '7-8'].includes(e.id)) {
 return { ...e, status: 'included', color: 'blue' };
 }
 return e;
 }));
 setLogs(prev => [{
 title: 'Слияние: Деревни объединились в колхозы
description: 'Все выбранные дороги построены ра
 оз {D, E, F, G, H, I}.'
 }]);
}

```

```

 }
 };

 const currentSteps =
 activeAlgo === 'kruskal' ? kruskalSteps :
 activeAlgo === 'prim' ? primSteps :
 boruvkaSteps;

 const handleNextStep = () => {
 if (stepIndex < currentSteps.length) {
 currentSteps[stepIndex]();
 setStepIndex(stepIndex + 1);
 }
 };

 const handleReset = (algo: 'kruskal' | 'prim' | 'boruvka') => {
 setActiveAlgo(algo);
 setVertices(initialVertices);
 setEdges(initialEdges);
 setStepIndex(0);
 setHeapQueue([]);
 if (algo === 'kruskal') {
 setLogs([
 {
 title: 'Введение в раскраску (Краскал)',
 description: 'Представь, что все 9 вершин графа
уже покрашены, и мы начинаем раскраску.',
 type: 'info',
 },
]);
 } else if (algo === 'prim') {
 setLogs([
 {
 title: 'Введение в Плесень (Прима)',
 description: 'Логика «Плесени»: выбираем стар
тую вершину (Heap) и захватываем соседей!',
 type: 'info',
 },
]);
 }
 };

```

```

return (
 <div className="space-y-12">

 {/* 3-in-1 MST Simulator Box */}
 <div className="bg-slate-900/90 border border-slate-800" >
 <div className="flex flex-col lg:flex-row justify-between" >
 <div>
 <div className="flex items-center gap-2 mb-2" >

 Супер-Интерактив 3 в 1

 <h3 className="text-2xl font-bold text-white">
 MST
 </h3>
 </div>
 <p className="text-slate-400 text-sm">
 Наблюдай за работой Краскала (Раскраска),
 </p>
 </div>

 {/* Algorithm Tabs */}
 <div className="flex flex-wrap items-center gap-2" >
 <div className="flex items-center bg-slate-800" >
 <button
 onClick={() => handleReset('kruskal')}
 className={
 "flex items-center gap-1.5 px-3 py-2"
 +
 (activeAlgo === 'kruskal'
 ? 'bg-indigo-600 text-white shadow-md'
 : 'text-slate-400 hover:text-slate-300')
 }
 >
 <GitGraph className="w-3.5 h-3.5" />
 Краскал (Билет 18)
 </button>
 <button
 onClick={() => handleReset('prim')}

```

```

 className={
 "flex items-center justify-center gap-2
initial cursor-pointer " +
 (stepIndex >= currentSteps.length
 ? "bg-slate-800 text-slate-500 border
: activeAlgo === 'kruskal'
 ? "bg-indigo-600 hover:bg-indigo-500
: activeAlgo === 'prim'
 ? "bg-emerald-600 hover:bg-emerald-500
 : "bg-rose-600 hover:bg-rose-500 text
)
 }
 >
 <SkipForward className="w-4 h-4" />
 {stepIndex === 0 ? 'Начать' : stepIndex}
 </button>
 </div>
</div>

<div className="grid grid-cols-1 lg:grid-cols-3
 /* Graph Canvas */
 <div className="lg:col-span-2 bg-slate-950 rounded-ful
 active min-h-[480px] overflow-hidden">

 /* Legend */
 <div className="absolute top-4 left-4 bg-slate-950
 -center gap-3 text-xs">
 <div className="flex items-center gap-1.5">
 <div className="w-3 h-3 rounded-full bg-slate-800" />
 </div>
 {activeAlgo === 'kruskal' ? (
 <>
 <div className="flex items-center gap-1.5">
 <div className="w-3 h-3 rounded-full bg-slate-800" />
 </div>
 <div className="flex items-center gap-1.5">
 <div className="w-3 h-3 rounded-full bg-slate-800" />
 </div>
 <div className="flex items-center gap-1.5">
 <div className="w-3 h-3 rounded-full bg-slate-800" />
 </div>
 </>
) : (
 <div className="flex items-center gap-1.5">
 <div className="w-3 h-3 rounded-full bg-slate-800" />
 </div>
)
 }
 </div>
 <div className="lg:col-span-1 bg-slate-950 rounded-ful
 active min-h-[480px] overflow-hidden">
 /* Legend */
 <div className="absolute top-4 left-4 bg-slate-950
 -center gap-3 text-xs">
 <div className="flex items-center gap-1.5">
 <div className="w-3 h-3 rounded-full bg-slate-800" />
 </div>
 {activeAlgo === 'kruskal' ? (
 <>
 <div className="flex items-center gap-1.5">
 <div className="w-3 h-3 rounded-full bg-slate-800" />
 </div>
 <div className="flex items-center gap-1.5">
 <div className="w-3 h-3 rounded-full bg-slate-800" />
 </div>
 <div className="flex items-center gap-1.5">
 <div className="w-3 h-3 rounded-full bg-slate-800" />
 </div>
 </>
) : (
 <div className="flex items-center gap-1.5">
 <div className="w-3 h-3 rounded-full bg-slate-800" />
 </div>
)
 }
 </div>
</div>

```

```

const strokeColor = getEdgeStroke(edge);
const isInspecting = edge.status === 'inspect';
const isRejected = edge.status === 'rejected';
const isInHeap = edge.status === 'in_heap';

return (
 <g key={edge.id}>
 <line
 x1={u.x + "%"}
 y1={u.y + "%"}
 x2={v.x + "%"}
 y2={v.y + "%"}
 stroke={strokeColor}
 strokeWidth={isInspecting ? 4.5 : 1}
 strokeDasharray={isRejected ? '4,4' : ''}
 className={"transition-all duration-300"}
 />
 <text
 x={({u.x + v.x} / 2 + "%")}
 y={({u.y + v.y} / 2 + "%")}
 fill={isInspecting ? '#f59e0b' : '#94a3b8'}
 : '#94a3b8'}
 fontSize="4.5"
 fontFamily="monospace"
 fontWeight="bold"
 textAnchor="middle"
 dominantBaseline="middle"
 className="select-none filter drop-shadow"
 dy="-1.5"
 >
 {edge.weight}
 </text>
 </g>
);
}}}
</svg>

{/* HTML Overlay for Vertices */}

```



```

{activeAlgo === 'prim' && {
 <div className="p-3 bg-slate-900/60 border-1" >
 <p className="text-[11px] font-bold text-white">
 ✂ Приоритетная очередь (Min-Heap):
 </p>
 <div className="flex flex-wrap gap-1.5">
 {heapQueue.length === 0 ? {

 Очередь пуста

 } : {
 heapQueue.map((item, idx) => {

 {item}

 })
 }}
 </div>
 </div>
)}

<div className="flex-1 p-4 overflow-y-auto">
 {logs.map((log, idx) => {
 <div
 key={idx}
 className={
 "p-4 rounded-xl border transition-all"
 (log.type === 'success'
 ? 'bg-emerald-950/40 border-emerald-500'
 : log.type === 'warning'
 ? 'bg-amber-950/40 border-amber-500'
 : log.type === 'error'
 ? 'bg-rose-950/40 border-rose-500'
 : 'bg-slate-900/60 border-slate-700')
 >
 <div className="flex items-center gap-2">
 {log.type === 'success' && <CheckCi

```

```

 className={"px-3 py-1.5 rounded text-xs font-weight-medium"}
 == 'boruvka' ? "bg-rose-600 text-white" : "bg-slate-950 text-slate-300"
 >
 Борувка (Билет 20)
 </button>
 </div>
 </div>

{activeCheatTab === 'kruskal' && (
 <div className="grid grid-cols-1 lg:grid-cols-2 gap-4">
 <div className="lg:col-span-1 space-y-4">
 <div className="bg-slate-950 p-4 rounded-3xl">
 <p className="text-slate-400 text-xs font-weight-medium">
 <Clock className="w-3.5 h-3.5 text-indigo-400" />
 </p>
 <p className="text-2xl font-black text-white">Крускал</p>
 <p className="text-slate-400 text-xs font-weight-medium">Минимальное количество ребер, соединяющих все вершины графа, называется минимальным остовным деревом.
 </p>
 </div>
 <div className="bg-slate-950 p-4 rounded-3xl">
 <p className="text-slate-400 text-xs font-weight-medium">
 <Award className="w-3.5 h-3.5 text-indigo-400" />
 </p>
 <p className="text-2xl font-black text-white">Награда</p>
 <p className="text-slate-400 text-xs font-weight-medium">Награда за решение задачи.
 </p>
 </div>
 <div className="bg-slate-950 p-4 rounded-3xl">
 <p className="text-slate-400 text-xs font-weight-medium">
 <Code className="w-3.5 h-3.5 text-indigo-400" />
 </p>
 <p className="text-slate-400 text-xs font-weight-medium">
 <pre>
 graph LR
 A((A)) --- B((B))
 B --- C((C))
 C --- D((D))
 D --- E((E))
 E --- F((F))
 F --- G((G))
 G --- H((H))
 H --- I((I))
 I --- J((J))
 J --- K((K))
 K --- L((L))
 L --- M((M))
 M --- N((N))
 N --- O((O))
 O --- P((P))
 P --- Q((Q))
 Q --- R((R))
 R --- S((S))
 S --- T((T))
 T --- U((U))
 U --- V((V))
 V --- W((W))
 W --- X((X))
 X --- Y((Y))
 Y --- Z((Z))
 Z --- AA((A))
 </pre>
 </p>
 </div>
 </div>
 </div>
)

```

```

 <Award className="w-3.5 h-3.5 text-emerald-400" />
 </p>
 <p className="text-2xl font-black text-slate-900" />
 <p className="text-slate-400 text-xs mt-2" />
 </div>
 <div className="bg-slate-950 p-4 rounded-lg" />
 <p className="text-slate-400 text-xs font-medium" />
 ложении:</p>
 <p className="text-xs text-slate-300 leading-relaxed" />
 ие чистые вершины через самое дешевое доступное ребро.</p>
 </div>
 </div>
 <div className="lg:col-span-2" />
 <div className="bg-slate-950 p-4 rounded-lg" />
 <p className="text-slate-400 text-xs font-medium" />
 <Code className="w-3.5 h-3.5 text-emerald-400" />
 </p>
 <pre className="text-xs text-emerald-400 font-mono" />
80 flex-1">
{`import heapq

def prim(graph, start):
 visited = [False] * len(graph)
 heap = [(0, start, -1)] # (weight, node, parent)
 mst = []

 while heap:
 w, u, prev = heapq.heappop(heap)
 if visited[u]: continue
 visited[u] = True
 if prev != -1: mst.append((prev, u, w))

 for next_w, v in graph[u]:
 if not visited[v]:
 heapq.heappush(heap, (next_w, v, u))
 return mst`}
 </pre>
 </div>

```

```
mst = []
while len(mst) < n - 1:
 # Для каждого колхоза ищем самый дешёвый мост
 cheapest = [-1] * n
 for u, v, w in edges:
 ru, rv = find(u), find(v)
 if ru != rv:
 if cheapest[ru] == -1 or cheapest[ru][2] > w:
 cheapest[ru] = [u, v, w]
 if cheapest[rv] == -1 or cheapest[rv][2] > w:
 cheapest[rv] = [u, v, w]
 # Объединяем все колхозы по выбранным мостам
 for bridge in cheapest:
 if bridge != -1 and union(bridge[0], bridge[1]):
 mst.append(bridge)
 }
}
```

---

ФАЙЛ 51/83: src/components/MnemonicCards.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

---

```
import dijkstraImg from '../assets/dijkstra-kind.jpg';
import bellmanImg from '../assets/bellman-ford-truck.jpg';
import floydImg from '../assets/floyd-network.jpg';
```

```
const cards = [
 {
 img: dijkstraImg,
 alt: 'Добрый робот Дейкстра',
 },
 {
 img: bellmanImg,
 alt: 'Добрый робот Беллмана',
 },
 {
 img: floydImg,
 alt: 'Добрый робот Флойда',
 },
];
```

```
export default function MnemonicCards() {
 return (
 <div className="grid grid-cols-1 md:grid-cols-3 gap-4">
 {cards.map((c, i) => (
 <div key={i} className={`bg-slate-800 rounded-2xl p-4">
 <div className="h-48 overflow-hidden">
 <img src={c.img} alt={c.alt}
 className="w-full h-full object-cover group-hover:scale-110 transition duration-300"/>
 </div>
 <div>
 <h3 className="text-lg font-bold mb-1 ${c.isCurrent ? 'text-white' : 'text-slate-400'}">{c.title}</h3>
 <p className="text-slate-400 text-sm">{c.desc} {c.index}
 </p>
 <div className={`mt-3 text-xs font-mono px-2 ${c.isCurrent ? 'text-white' : 'text-slate-400'}>{c.content}</div>
 </div>
))}
 </div>
);
}
```

---

ФАЙЛ 52/83: src/components/MobileToc.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import React, { useEffect, useState } from "react";
import { List, X } from "lucide-react";
import { Sidebar } from "../Sidebar";

interface Props {
 selectedChapterId: string;
 setSelectedChapterId: (id: string) => void;
 /** Заголовок текущей темы – показываем в кнопке, что это текущая */
 currentTitle: string;
 index: number;
```

```

 <span className="block text-[13px] font-bol

 </button>
 </div>

 {open && (
 <div className="lg:hidden fixed inset-0 z-[90]">
 <div className="absolute inset-0 bg-slate-950"
 <div className="absolute inset-y-0 left-0 w-[100%] h-[100%] flex flex-direction-column justify-center items-center text-white text-[13px] font-bold">
 <div className="flex items-center justify-between p-2">

 <button
 onClick={() => setOpen(false)}
 className="p-2 -mr-2 text-slate-400 hover:text-white"
 aria-label="Заккрыть содержание"
 >
 <X className="w-5 h-5" />
 </button>
 </div>
 <div className="flex-1 min-h-0">
 <Sidebar
 selectedChapterId={selectedChapterId}
 setSelectedChapterId={setSelectedChapterId}
 onNavigate={() => setOpen(false)}
 />
 </div>
 </div>
 </div>
 </div>
)}
</>
);
};

```

Подготовка к экзаменам: Алгоритмы, Структуры

</p>

</div>

</div>

<div className="flex items-center w-full lg:w-auto">

<button

id="tab-guide-btn"

onClick={() => setActiveTab('guide')}

className={`flex-1 lg:flex-none flex items-center`}

duration-200 whitespace-nowrap \${

activeTab === 'guide'

? 'bg-indigo-600 text-white shadow-lg stroke-indigo-500'

: 'text-slate-400 hover:text-white'

}`}

>

<BookOpen className="w-4 h-4" /> Учебное пособие

</button>

<button

id="tab-simulator-btn"

onClick={() => setActiveTab('simulator')}

className={`flex-1 lg:flex-none flex items-center`}

duration-200 whitespace-nowrap \${

activeTab === 'simulator'

? 'bg-indigo-600 text-white shadow-lg stroke-indigo-500'

: 'text-slate-400 hover:text-white'

}`}

>

<Cpu className="w-4 h-4" /> Тренажёры

</button>

<a

id="download-pdf-btn"

href="/export/full\_code\_all\_pages.pdf"

download="full\_code\_all\_pages.pdf"

target="\_blank"

rel="noreferrer"

className="flex items-center justify-center"

```

export function PlanarityDemo() {
 return (
 <div className="space-y-4">
 <h4 className="text-base font-bold text-white">K5
 <p className="text-xs text-slate-400">Просто смотри
 <div className="grid grid-cols-1 md:grid-cols-2 gap-4">
 { /* K5 */ }
 <div className="bg-slate-950 rounded-xl p-4 border border-slate-800">
 <div className="absolute top-2 left-2 bg-slate-900/30 w-auto z-20">K5 – НЕПЛАНАРЕН</div>
 <svg className="w-full h-full absolute inset-0">
 {(() => {
 const pts = [
 {id:0,x:160,y:40},
 {id:1,x:270,y:100},
 {id:2,x:230,y:230},
 {id:3,x:90,y:230},
 {id:4,x:50,y:100}
];
 const edges = [
 [0,1],[0,2],[0,3],[0,4],
 [1,2],[1,3],[1,4],
 [2,3],[2,4],
 [3,4]
];
 function findPt(id:number) { return pts.find(p => p.id === id); }
 function intersect(a:number,b:number,c:number,d:number) {
 if (a===c||a===d||b===c||b===d) return true;
 const p1=findPt(a),p2=findPt(b),p3=findPt(c),p4=findPt(d);
 function ccw(ax:number,ay:number,bx:number,by:number,cx:number,cy:number) {
 return (cy-ay)*(bx-ax)>(by-ay)*(cx-ax);
 }
 return ccw(p1.x,p1.y,p3.x,p3.y,p4.x,p4.y)<
 && ccw(p1.x,p1.y,p2.x,p2.y,p3.x,p3.y);
 }
 })() }
 <const crosses: number[] = [];>

```



```

 {id:0,x:60,y:60},{id:1,x:160,y:60},{id:2,x:260,y:60},
 {id:3,x:60,y:220},{id:4,x:160,y:220},{id:5,x:260,y:220},
];
 const edges = [
 [0,3],[0,4],[0,5],
 [1,3],[1,4],[1,5],
 [2,3],[2,4],[2,5]
];
 function findPt(id:number) { return pts.find(p => p.id === id); }
 function intersect(a:number,b:number,c:number,d:number) {
 if (a===c||a===d||b===c||b===d) return true;
 const p1=findPt(a),p2=findPt(b),p3=findPt(c),p4=findPt(d);
 if (!p1||!p2||!p3||!p4) return false;
 function ccw(ax:number,ay:number,bx:number,by:number,cx:number,cy:number) {
 return (cy-ay)*(bx-ax)>(by-ay)*(cx-ax);
 }
 return ccw(p1.x,p1.y,p3.x,p3.y,p4.x,p4.y)<0
 && ccw(p1.x,p1.y,p2.x,p2.y,p3.x,p3.y)<0;
 }
 const crosses:number[]=[];
 for(let i=0;i<edges.length;i++) for(let j=i+1;j<edges.length;j++) {
 if (intersect(edges[i][0],edges[i][1],edges[j][0],edges[j][1])) {
 if (!crosses.includes(i)) crosses.push(i);
 if (!crosses.includes(j)) crosses.push(j);
 }
 }
 const lines = []; const circles = [];
 for(let i=0;i<edges.length;i++) {
 const p1=findPt(edges[i][0]),p2=findPt(edges[i][1]);
 const xing=crosses.includes(i);
 lines.push(<line key={`l${i}`} x1={p1.x} y1={p1.y} x2={p2.x} y2={p2.y}
 stroke={xing?"#f43f5e":"#4f46e5"} stroke-width={2}/>);
 }
 const labels = ["\u04141","\u04142","\u04143"];
 for(const p of pts) {
 circles.push(<g key={`c${p.id}`}>
 <circle cx={p.x} cy={p.y} r={8} fill="#4f46e5" stroke="#4f46e5" stroke-width={2}/>
 <text x={p.x} y={p.y+3} textAnchor="middle">{labels[p.id-1]}/>
 </g>);
 }

```

```

const PRESETS = [
 { name: 'Студент Вася', emoji: '🍌', color: 'from-blue',
 text: 'голода после матана!' },
 { name: 'Кодер Семён', emoji: '🍜', color: 'from-violet',
 text: 'ищу ИИ за порцию борща!' },
 { name: 'Кот Борис', emoji: '🐈', color: 'from-amber',
 text: 'есть сметаны, плиз.' },
 { name: 'Бабушка Люба', emoji: '👵', color: 'from-rose',
 text: 'есть сметаны, плиз.' },
 { name: 'Инопланетянин', emoji: '👽', color: 'from-emerald',
 text: 'есть сметаны, плиз.' },
 { name: 'Бизнесмен', emoji: '💼', color: 'from-slate',
 text: 'есть сметаны, плиз.' },
];
```

```
let counter = 3;
```

```
export const QueueViz: React.FC = () => {
 const [queue, setQueue] = useState<Customer[]>([
 { id: 'q1', name: 'Студент Вася', emoji: '🍌', color: 'from-blue',
 text: 'голода после матана!', animState: 'idle' },
 { id: 'q2', name: 'Кодер Семён', emoji: '🍜', color: 'from-violet',
 text: 'ищу ИИ за порцию борща!', animState: 'idle' },
 { id: 'q3', name: 'Кот Борис', emoji: '🐈', color: 'from-amber',
 text: 'есть сметаны, плиз.', animState: 'idle' },
]);
```

```
 const [servedHistory, setServedHistory] = useState<string[]>([]);
 const [isServing, setIsServing] = useState(false);
 const [shakeEmpty, setShake] = useState(false);
 const [log, setLog] = useState<string[]>([]);
```

```
 const addLog = (msg: string) => setLog(prev => [...prev, msg]);
```

```
 // ENQUEUE: Встать в конец очереди (хвост / Tail)
```

```
 const enqueue = useCallback(() => {
 if (isServing) return;
 if (queue.length >= 6) {
```

```

// Step 1: Head guest gets eating animation
setQueue(prev => prev.map((c, idx) => idx === 0 ? {

setTimeout(() => {
 // Step 2: Guest leaves with happy face, queue sh
 setQueue(prev => prev.map((c, idx) => idx === 0 ?

 setServedHistory(prev => [`${headGuest.emoji} ${h

 setTimeout(() => {
 setQueue(prev => prev.slice(1));
 setIsServing(false);
 addLog(` ${headGuest.name} поел и ушел сытым.
 }, 400);
}, 900);
}, [queue, isServing]));

const reset = () => {
 counter = 3;
 setQueue([
 { id: 'r1', name: 'Студент Вася', emoji: ' ', c
 т голода после матана!', animState: 'in' },
 { id: 'r2', name: 'Кодер Семён', emoji: ' ', c
 апишу ИИ за порцию борща!', animState: 'in' },
 { id: 'r3', name: 'Кот Борис', emoji: ' ', col
 без сметаны, плиз.', animState: 'in' },
]);
 setServedHistory([]);
 setIsServing(false);
 setLog([' Столовая сброшена. Снова голодная толпа!
 setTimeout(() => {
 setQueue(prev => prev.map(c => ({ ...c, animState
 }, 500);
});

return (
 <div className="flex flex-col gap-6">
 /* МНЕМОНИКА / ПРАВИЛО */

```

```

 >
 Сброс
 </button>
</div>

{/* ИНТЕРАКТИВНАЯ КУХНЯ */}
<div className="grid grid-cols-1 md:grid-cols-12" >

 {/* ЛЕВАЯ КОЛОНКА: ОЧЕРЕДЬ ЗА СУПОМ */}
 <div className={`md:col-span-9 bg-slate-900 rounded-2xl
 -[300px] overflow-hidden ${shakeEmpty ? 'animate-shake' : ''}`>
 <div className="absolute top-4 left-4 text-[1.2em] font-mono">
 Очередь голодных гостей (Буфер FIFO / BFS)
 </div>

 <div className="flex-1 flex flex-col justify-between">
 <div className="flex items-end justify-between">

 {/* ПОВАРИХА И КОТЕЛ С СУПОМ */}
 <div className="flex flex-col items-center">
 <div className="relative">
 <div className="absolute -top-10 left-1/2 transform translate-x-1/2">
 <div className="w-1.5 h-6 bg-slate-400 rounded">
 <div className="w-2 h-4 bg-slate-400 rounded">
 </div>

 </div>
 <div className="text-center">

 ПОВАР

 </div>
 </div>
 </div>
 </div>
 </div>

 {/* РАЗДЕЛИТЕЛЬНАЯ СТРЕЛКА */}
 <div className="flex flex-col items-center">

```

```

 { /* Персонаж */ }
 <div className={`
 w-14 h-14 rounded-2xl border-1
 relative
 ${customer.color}
 ${isHead ? 'ring-4 ring-amber-500' : ''}
 `}>
 {customer.name}

 { /* Индексы HEAD/TAIL */ }
 {(isHead || isTail) && (
 <span className={`
 absolute -bottom-2 text-[0.8em]
 ${isHead ? 'bg-amber-500' : 'bg-slate-900'}
 `}>
 {isHead && isTail ? 'h+t' : isHead ? 'h' : 't'}

)}
 </div>
 </div>
);
))
}
</div>

</div>

{ /* Пол кухни */ }
<div className="w-full h-3 bg-gradient-to-r from-slate-900 to-slate-800">
</div>

{ /* ПРАВАЯ КОЛОНКА: СЫТЫЕ И ДОВОЛЬНЫЕ */ }
<div className="md:col-span-3 bg-slate-900 rounded-2xl p-4">
 <div className="absolute top-4 left-4 text-[1.2em] font-medium">
 Сытые гости (Архив FIFO)
 </div>

```



```

 if (newNodes[curr].children[c] === undefined) {
 count++;
 newNodes[curr].children[c] = count;
 newNodes[count] = {
 id: count,
 char: c,
 parent: curr,
 children: {},
 fail: 0,
 term_link: 0,
 is_terminal: false,
 words: [],
 depth: newNodes[curr].depth + 1
 };
 }
 curr = newNodes[curr].children[c];
 }
 if (!newNodes[curr].words.includes(word)) {
 newNodes[curr].words.push(word);
 newNodes[curr].is_terminal = true;
 }
});

let currentX = 0;
const paddingX = 70;
const paddingY = 80;

const layoutDFS = (u: number, depth: number) => {
 const childKeys = Object.values(newNodes[u].children);
 newNodes[u].y = 40 + depth * paddingY;

 if (childKeys.length === 0) {
 newNodes[u].x = 40 + currentX * paddingX;
 currentX++;
 } else {
 let leftX = -1;
 let rightX = -1;
 childKeys.forEach(v => {

```

```

const removeWord = (wordToRemove: string) => {
 if (words.length <= 1) {
 setSpeechText('Оставь хотя бы одно слово, иначе мы не сможем продолжить!');
 return;
 }
 const updated = words.filter((w) => w !== wordToRemove);
 setWords(updated);
 buildInitialTrie(updated);
};

// Запуск BFS
const startBFS = () => {
 const rootChildren = Object.values(nodes[0].children);
 const queue = [...rootChildren];
 const updatedNodes = { ...nodes };
 rootChildren.forEach((childId) => {
 updatedNodes[childId].fail = 0;
 });

 setNodes(updatedNodes);
 setBfsQueue(queue);
 setSimulationStep(1);
 setCurrentNode(null);
 setSpeechText(
 'Начинаем обход в ширину (BFS)! Все вершины на главном уровне суффикса просто нет. Жми «Следующий шаг»!'
);
};

// Один шаг BFS
const stepBFS = () => {
 if (bfsQueue.length === 0) {
 setSimulationStep(2);
 setCurrentNode(null);
 setSpeechText(
 'Ура! Очередь пуста! Мы успешно построили полную таблицу переходов!'
);
 }
};

```



```

 setSpeechText(logs);
 };

 return (
 <div className="bg-slate-800/90 rounded-2xl p-6 border-1px border-slate-700" >
 {/* Шапка виджета */}
 <div className="flex flex-wrap items-center justify-between" >
 <div className="flex items-center space-x-3" >

 <div>
 <h4 className="text-2xl font-bold text-white" >
 Интерактивный конструктор: Говорящий Эхо-
 </h4>
 <p className="text-sm text-slate-400" >
 Построение дерева и расчет суффиксных ссы
 </p>
 </div>
 </div>
 <div className="flex items-center gap-2" >
 <button
 onClick={() => buildInitialTrie(words)}
 className="flex items-center gap-1 bg-slate-700 text-white px-3 py-2 rounded-md transition-colors"
 title="Сбросить автомат"
 >
 <RotateCcw className="w-4 h-4" /> Сбросить
 </button>
 </div>
 </div>

 {/* Верхняя панель: Говорящий лосось и диалоговое
 <div className="grid grid-cols-1 md:grid-cols-3 gap-4" >
 {/* Аватар Лосося (Сеня) с анимацией моргания и
 <div className="flex flex-col items-center justify-center" >
 <div className="w-40 h-40 relative flex items-center justify-center" >
 full border border-blue-500/30 shadow-inner
 {/* SVG Лосося */}
 <svg

```

```

 <path d="M 165 105 Q 155 120 165 125 Q
) : (
 <path d="M 165 110 Q 155 115 168 118" s
)}

 {/* Улыбка / жабры */}
 <path d="M 130 80 Q 125 100 130 120" stro
 <path d="M 120 85 Q 115 100 120 115" stro
</svg>

 {/* Декоративные пузыри */}
 <div className="absolute -top-1 right-4 tex
 <div className="absolute top-8 right-1 text
</div>
 <span className="mt-3 bg-rose-600/30 border b
 Эхо-Лосось Сеня

</div>

{/* Пузырь с текстом (Баббл) */}
<div className="md:col-span-2 bg-slate-800 bord
 tween min-h-[140px]">
 {/* Треугольник баббла */}
 <div className="absolute -left-3 top-1/2 -tra
 slate-800 border-b-[12px] border-b-transpar

 <div className="flex items-center space-x-2 t
 <Volume2 className={`w-4 h-4 ${isTalking ?
 Прямое вещание из аквариума:
 </div>

 <p className="text-slate-200 text-base leading
 {speechText}
 </p>

 <div className="mt-4 pt-3 border-t border-sla
 <div className="text-xs text-slate-400 flex
 <Info className="w-3.5 h-3.5 text-blue-40

```

```

 { /* SVG Canvas for Automaton Tree */ }
 <div className="mb-8 bg-slate-900 border border-s
 <h5 className="text-white font-bold mb-3 flex i
 Дерево и Суффиксные (fail) ссы
 </h5>

 {(() => {
 let maxX = 0;
 let maxY = 0;
 Object.values(nodes).forEach(n => {
 if (n.x && n.x > maxX) maxX = n.x;
 if (n.y && n.y > maxY) maxY = n.y;
 });
 const svgWidth = Math.max(maxX + 80, 400);
 const svgHeight = Math.max(maxY + 60, 200);

 return (
 <svg viewBox={`0 0 ${svgWidth} ${svgHeight}`}
 <defs>
 <marker id="arrowhead" markerWidth="6"
 <polygon points="0 0, 6 3, 0 6" fill=
 </marker>
 <marker id="fail-arrow" markerWidth="6"
 <polygon points="0 0, 6 3, 0 6" fill=
 </marker>
 </defs>

 { /* Draw Edges */ }
 {Object.values(nodes).map(node => {
 if (node.id === 0) return null;
 const parent = nodes[node.parent];
 if (!parent.x || !parent.y || !node.x ||

 const isCurrent = currentNode === node.

 return (
 <g key={`edge-${node.id}`}>
 <line

```

```

- 16}` }
 fill="none"
 stroke={isCurrent ? '#f43f5e' : 'rgb(0,0,0)'}
 strokeWidth={isCurrent ? 2 : 1.5}
 strokeDasharray="4 4"
 markerEnd="url(#fail-arrow)"
 />
);
}}

```

```

{/* Draw Nodes */}
Object.values(nodes).map(node => {
 if (!node.x || !node.y) return null;
 const isRoot = node.id === 0;
 const isCurrent = currentNode === node.id;
 const isInQueue = bfsQueue.includes(node.id);

```

```

 let fill = '#1e293b';
 let stroke = '#475569';

```

```

 if (isCurrent) {
 fill = '#2563eb';
 stroke = '#60a5fa';
 } else if (isInQueue) {
 fill = '#b45309';
 stroke = '#fbbf24';
 } else if (node.is_terminal) {
 fill = '#059669';
 stroke = '#34d399';
 }

```

```

 return (
 <g key={`node-${node.id}`}>
 <circle
 cx={node.x}
 cy={node.y}
 r="16"
 fill={fill}

```

```

<div className="flex flex-wrap gap-2 mb-4">
 {words.map((w) => {
 <span
 key={w}
 className="bg-slate-800 border border-
er gap-1 group"
 >
 {w}
 <button
 onClick={() => removeWord(w)}
 className="text-slate-500 hover:tex
 title="Удалить"
 >
 ×
 </button>

 })}
</div>
</div>

```

```

<form onSubmit={handleAddWord} className="fle
 <input
 type="text"
 value={newWord}
 onChange={(e) => setNewWord(e.target.valu
 placeholder="НОВОЕ СЛОВО..."
 maxLength={8}
 className="bg-slate-800 border border-sla
 outline-none focus:border-blue-500"
 />
 <button
 type="submit"
 className="bg-slate-700 hover:bg-slate-60
 ition-colors"
 >
 <Plus className="w-3.5 h-3.5" /> Добавить
 </button>
</form>

```

```

 ? 'bg-slate-800/80'
 : 'hover:bg-slate-800/40'
 }`}
>
<td className="py-2.5 px-3 flex items-center" >
 <span
 className={`w-2 h-2 rounded-full
 ${node.id === 0
 ? 'bg-purple-500'
 : isCurrent
 ? 'bg-blue-400 animate-pulse'
 : isInQueue
 ? 'bg-amber-400'
 : 'bg-slate-600'}
 }`}
 >
 #{node.id}
 {node.id === 0 &&
 </td>
<td className="py-2.5 px-3">

 {node.char}

</td>
<td className="py-2.5 px-3 text-slate-500" >
 {node.id === 0 ? '-' : `#${node.id}</td>
<td className="py-2.5 px-3" >
 {node.id === 0 ? (

) : (

 #{node.fail}

)}
</td>
<td className="py-2.5 px-3" >
 {node.id === 0 || node.term line

```

-----  
ФАЙЛ 57/83: src/components/SegmentTreeVisualizer.tsx  
КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 10  
-----

```
import React, { useState, useEffect, useCallback, useMemo } from 'react';
import { Play, Pause, RotateCcw, ChevronLeft, ChevronRight } from 'lucide-react';
```

```
// ===== TYPES =====
```

```
interface Step {
 id: number;
 desc: string;
 codeLine: number;
 treeState: Record<number, number>; // node index to value
 highlightNodes: number[]; // current highlighted nodes
 mergeChildren?: [number, number]; // pair being merged
 arrayHighlight?: [number, number]; // [L, R] range in array
 result?: number; // partial result
}
```

```
// ===== PURE LOGIC: build tree =====
```

```
function buildTreeFull(arr: number[]): Record<number, number> {
 const n = arr.length;
 const tree: Record<number, number> = {};
 function go(v: number, l: number, r: number) {
 if (l === r) { tree[v] = arr[l]; return; }
 const m = (l + r) >> 1;
 go(2 * v, l, m);
 go(2 * v + 1, m + 1, r);
 tree[v] = (tree[2 * v] ?? 0) + (tree[2 * v + 1] ?? 0);
 }
 go(1, 0, n - 1);
 return tree;
}
```

```
// ===== STEP GENERATORS =====
```

```
// 1. Build steps (recursive top-down)
```

```
function genBuildSteps(arr: number[]): Step[] {
```

```

 go(1, 0, n - 1);
 steps.push({ id: id++, desc: ` Дерево построено! Корень: 1
 Highlight: [0, n - 1] });
 return steps;
 }

```

// 2. Top-Down Query steps

```

function genQueryTopDownSteps(arr: number[], qL: number, qR: number) {
 const n = arr.length;
 const tree = buildTreeFull(arr);
 const steps: Step[] = [];
 let id = 0;

```

```

 function go(v: number, l: number, r: number, ql: number, qr: number) {
 if (ql > r || qr < l) {
 steps.push({ id: id++, desc: `query(${v}, [${l}..${r}]`,
 codeLine: 2, treeState: tree, highlightNodes: [v] });
 return 0;
 }

```

```

 if (ql <= l && r <= qr) {
 steps.push({ id: id++, desc: `query(${v}, [${l}..${r}]`,
 codeLine: 3, treeState: tree, highlightNodes: [v] });
 return tree[v];
 }

```

```

 const m = (l + r) >> 1;
 steps.push({ id: id++, desc: `query(${v}, [${l}..${r}]`,
 codeLine: 6, treeState: tree, highlightNodes: [v] });
 const left = go(2 * v, l, m, ql, qr);
 const right = go(2 * v + 1, m + 1, r, ql, qr);
 const res = left + right;
 steps.push({ id: id++, desc: `Возврат в узел ${v}: ${res}`,
 codeLine: 7, treeState: tree, highlightNodes: [v],
 rangeChildren: [2 * v, 2 * v + 1], arrayHighlight: [left, right] });
 return res;
 }

```

```

 const ans = go(1, 0, n - 1, qL, qR);
 steps.push({ id: id++, desc: `Запрос sum([${qL}..${qR}]`,
 codeLine: 8, treeState: tree, highlightNodes: [1], arrayHighlight: [qL, qR], result: ans });
 return steps;
}

```



```

 steps.push({ id: id++, desc: `Поднимаемся: l = ${
 , arrayHighlight: [qL, qR], result: res }));
 }
}
steps.push({ id: id++, desc: ` Запрос sum([${qL}..${qR}],
 ighlightNodes: [1], arrayHighlight: [qL, qR], result: res });
return steps;
}

```

// ===== TREE RENDERING HELPER =====

```
interface VNode { v: number; l: number; r: number; val: number; }
```

```

function buildVisualTree(treeState: Record<number, number>) {
 if (l === r) return { v, l, r, val: treeState[v] ?? null };
 const m = (l + r) >> 1;
 return { v, l, r, val: treeState[v] ?? null, left: buildVisualTree(
 v + 1, m + 1, r) };
}

```

// ===== CODE SNIPPETS =====

```

const BUILD_CODE = [
 "build(v, l, r) {",
 " if (l === r) { tree[v] = A[l]; return; }",
 " // -----",
 " // -----",
 " mid = (l + r) >> 1;",
 " build(2*v, l, mid);",
 " build(2*v+1, mid+1, r);",
 " tree[v] = tree[2*v] + tree[2*v+1];",
 "}"
]

```

```
];
```

```

const QUERY_TD_CODE = [
 "query(v, l, r, ql, qr) {",
 " if (ql > r || qr < l) return 0;",
 " // -----",
 " if (ql <= l && r <= qr) return tree[v];",
 " // -----",
 " mid = (l + r) >> 1;",

```

```

 -slate-900' }, amber: { btn: 'bg-amber-600 hover:bg-amber-500 bg-amber-600', childRing: 'ring-rose-500' },
 [color]);

 return { step, idx, setIdx, playing, setPlaying, speed };
 }

// ===== SIMULATION PANEL =====
function SimPanel({ title, icon, steps, code, color, arr: number[] }) {
 const p = Player({ steps, color });
 if (!p) return null;
 const { step, idx, setIdx, playing, setPlaying, speed } = p;
 const n = arr.length;

 const vTree = useMemo(() => buildVisualTree(step.tree, n), [step]);

 const renderNode = useCallback((nd: VNode): React.ReactNode => {
 const isHigh = step.highlightNodes.includes(nd.v);
 const isChild = step.mergeChildren?.includes(nd.v);
 const has = nd.val !== null;

 let cls = 'bg-slate-800 border-slate-700 text-slate-200';
 if (isHigh) cls = `${clr.ring} text-white ring-2 shadow-2xl`;
 else if (isChild) cls = `${clr.childRing} text-amber-400`;
 else if (has) cls = `${clr.filled} text-slate-200`;

 return (
 <div key={nd.v} className="flex flex-col items-center justify-center">
 <div className={`flex flex-col items-center justify-center gap-2 ${cls}`}>
 {nd.val}
 [{nd.left, nd.right}]
 {nd.val}
 </div>
 {(nd.left || nd.right) && (
 <div className="flex flex-col items-center mt-2">
 <div className="w-px h-2 bg-slate-700" />

```

```
</div>
```

```
{/* Code */}
```

```
<pre className="px-3 py-2 text-[9px] md:text-[10px] font-mono">
```

```
 {code.map((line, i) => {
 const ln = i + 1;
 const active = step.codeLine === ln;
 return (
 <div key={ln} className={`px-2 py-0.5 rounded-2 border-indigo-500 text-indigo-200` : color === 'dark' ? 'bg-amber-600/20 border-l-2 border-amber-600' : 'bg-slate-900/60 border-l-2 border-slate-900'}>
 {ln}
 </div>
);
 })}
</pre>
```

```
{/* Description + result */}
```

```
<div className="px-3 py-2.5 bg-slate-900/60 border-t-1 border-slate-900">

 {step.desc}
 {step.result !== undefined && (
 {step.result}
)}
</div>
```

```
{/* Controls */}
```

```
<div className="px-3 py-2.5 bg-slate-950 border-t-1 border-slate-950">
 <div className="flex items-center bg-slate-900/60 border-t-1 border-slate-900">
 <button onClick={() => { setPlaying(false); setStep(0) }}>

 </button>
 <button onClick={() => { setPlaying(false); setStep(0) }}>

 </button>
 </div>
</div>
```

```

 }, [arr]));

const handleApply = (e: React.FormEvent) => {
 e.preventDefault();
 const parsed = customInput.split(',').map(s => parseInt(s));
 if (parsed.length >= 2 && parsed.length <= 16) {
 setArr(parsed);
 }
};

const randomize = () => {
 const size = arr.length;
 const a = Array.from({ length: size }, () => Math.floor(Math.random() * 100));
 setArr(a);
 setCustomInput(a.join(', '));
};

const buildSteps = useMemo(() => genBuildSteps(arr), [arr]);
const queryTDSteps = useMemo(() => genQueryTopDownSteps(arr), [arr]);
const queryBUSteps = useMemo(() => genQueryBottomUpSteps(arr), [arr]);

const totalSum = arr.reduce((a, b) => a + b, 0);

return (
 <div className="bg-slate-900 rounded-2xl border border-slate-800 p-4">
 {/* ---- TOP BAR: array editor + query range ---- */}
 <div className="mb-6 space-y-4">
 <div className="flex flex-col lg:flex-row gap-4">
 {/* Array input */}
 <form onSubmit={handleApply} className="flex flex-col">
 <div className="flex items-center justify-between">
 <label className="text-[10px] font-bold uppercase">Array</label>
 <button type="button" onClick={randomize}>Randomize</button>
 </div>
 <input type="text" value={customInput} />
 </form>
 <div className="flex gap-2">
 <input type="text" value={queryRange} />
 <button type="button" onClick={handleQuery}>Query</button>
 </div>
 </div>
 <div className="mt-4">
 <div>Build Steps</div>
 {buildSteps}
 </div>
 <div>Query Steps</div>
 {queryTDSteps}
 {queryBUSteps}
 <div>Total Sum: {totalSum}</div>
 </div>
 </div>
);

```

```

 type="number"
 min={0}
 max={arr.length - 1}
 value={qR}
 onChange={e => setQR(Math.min(Number(
 className="w-14 bg-slate-900 border b
 ocus:border-indigo-500"
 />
 </div>
 </div>
 <div className="text-[10px] text-slate-500 p
 Запрос: sum(A[{qL}..{qR}]) = {arr.slice(q
 </div>
</div>
</div>
</div>

{/* ---- TABS ---- */}
<div className="flex flex-wrap items-center gap-1
 <button onClick={() => setView('build')} classN
 -colors ${view === 'build' ? 'border-indigo-5
 -slate-200'}`}>
 <Hammer className="w-3.5 h-3.5" /> Построить
 </button>
 <button onClick={() => setView('queries')} clas
 on-colors ${view === 'queries' ? 'border-emerg
 er:text-slate-200'}`}>
 <Search className="w-3.5 h-3.5" /> Запрос: св
 </button>
 <button onClick={() => setView('theory')} class
 n-colors ${view === 'theory' ? 'border-blue-5
 te-200'}`}>
 <Info className="w-3.5 h-3.5" /> Почему / Спра
 </button>
</div>

{/* ---- TAB CONTENT ---- */}
{view === 'build' && (

```

```

<h4 className="text-base font-bold text-white">
 во разбилось именно так для $N=\{arr.length\}$?
<p className="text-sm text-slate-300 leading-relaxed">
 Каждый узел берёт <code className="bg-slate-900 text-emerald-500">
 к получает <code className="bg-slate-900 text-emerald-500">
 -slate-900 px-1 rounded font-mono text-emerald-500
 евая часть забирает на 1 больше (округление)
</p>
<div className="bg-slate-900 p-4 rounded-lg">
 <div className="text-slate-400">Корень: L
 <div className="text-slate-300">mid = (0
 <div className="text-indigo-300">→ Левый:
 <div className="text-emerald-300">→ Правый:
 h - 1) >> 1)) эл.)</div>
 <div className="text-slate-500 mt-2">Всего
 } внутренних.</div>
</div>
</div>

{/* Comparison cards */}
<div className="grid grid-cols-1 xl:grid-cols-2">
 <div className="bg-slate-950 p-5 rounded-xl">
 <div className="absolute -top-3 left-4 bg-emerald-500
 e border border-emerald-500 shadow-md">
 1 Запрос Сверху-вниз (Рекурсия)
 </div>
 <p className="text-xs text-slate-300 mb-3">
 Начинаем с корня. Если отрезок узла полн
 наче спускаемся в обоих детей.
 </p>
 <div className="space-y-1.5 text-xs">
 <div className="flex justify-between border-b">
 0(
 <div className="flex justify-between border-b">

 <div className="flex justify-between"><
 ✓ легко</div>
 </div>
 </div>
 </div>
 </div>
 </div>
 </div>
 </div>
 <div className="bg-slate-950 p-5 rounded-xl">
 <div className="absolute -top-3 left-4 bg-emerald-500
 e border border-emerald-500 shadow-md">
 2 Запрос Снизу-вверх (Итерация)
 </div>
 <p className="text-xs text-slate-300 mb-3">
 Начинаем с листьев. Если отрезок узла полн
 наче поднимаемся к родителям.
 </p>
 <div className="space-y-1.5 text-xs">
 <div className="flex justify-between border-b">
 0(
 <div className="flex justify-between border-b">

 <div className="flex justify-between"><
 ✓ легко</div>
 </div>
 </div>
 </div>
 </div>
 </div>
 </div>
 </div>
</div>

```

ФАЙЛ 58/83: src/components/Sidebar.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```
import React, { useMemo, useState } from "react";
import { ChevronRight, Compass, Search, Sparkles, X } from "lucide-react";
import { chapters } from "../data/content";
import { VIZ_REGISTRY } from "../vizRegistry";
```

```
interface SidebarProps {
 selectedChapterId: string;
 setSelectedChapterId: (id: string) => void;
 /** Мобильный режим: список живёт в выдвижной панели */
 onNavigate?: () => void;
}
```

```
export const Sidebar: React.FC<SidebarProps> = ({ selectedChapterId, setSelectedChapterId, onNavigate }) => {
 const [query, setQuery] = useState("");
```

```
 const groups = useMemo(() => {
 const q = query.trim().toLowerCase();
 const filtered = q ? chapters.filter((c) => c.title.toLowerCase().includes(q)) : chapters;
 const map = new Map<string, typeof chapters>();
 for (const c of filtered) {
 const key = c.category?.trim() || "Прочие темы";
 if (!map.has(key)) map.set(key, []);
 (map.get(key) as typeof chapters).push(c);
 }
 return Array.from(map.entries());
 }, [query]);
```

```
 return (
 <aside className="w-full bg-slate-900/80 lg:bg-transparent" >
 <div className="p-4 pb-3 shrink-0" >
 <h3 className="text-xs font-bold text-slate-400" >
 <Compass className="w-4 h-4 text-indigo-400" />
 </h3>
 <div className="relative" >
```

```

 className={`w-full text-left px-3 py-3
 isSelected
 ? "bg-indigo-600/15 border-indigo-600/15"
 : "bg-slate-800/40 border-transparent"}
 `}
 >
 {hasViz && {
 <span
 className="w-1.5 h-1.5 rounded-full border-1px solid indigo-600"
 title="Есть интерактивная визуализация"
 />
 }}

 <ChevronRight
 className={`w-4 h-4 shrink-0 mt-0.5
 isSelected ? "text-indigo-400" : "text-white"
 `}
 />
 </button>
);
 }}}
</div>
</div>
)}}
</div>

<div className="shrink-0 m-3 mt-0 bg-gradient-to-r from-indigo-600/15 to-transparent
] text-slate-300 space-y-1.5 hidden lg:block">
 <div className="font-bold text-indigo-400 flex items-center">
 <Sparkles className="w-3.5 h-3.5" /> Как пользоваться?
 </div>
 <p className="leading-relaxed">
 Подчёркнутые термины в тексте раскрываются по клику.
 </p>
 <p className="text-slate-400 italic">Стрелки ← и → указывают на
 </div>
</aside>

```



```

));
 const [popMessage, setPopMessage] = useState<string | null>('');

 // Queue state
 const [queue, setQueue] = useState<Person[]>([
 { id: '1', name: 'Студент Вася', icon: '👤' },
 { id: '2', name: 'Голодный кодер', icon: '👨‍💻' },
 { id: '3', name: 'Кот Борис', icon: '🐈' },
]);
 const [dequeueMessage, setDequeueMessage] = useState<string | null>('');

 // Heap state
 const [heap, setHeap] = useState<Hypothesis[]>([
 { id: '1', text: 'Кандидат: "Привет, я искусственный интеллект"' },
 { id: '2', text: 'Кандидат: "Привет, я испек вкусный хлеб"' },
 { id: '3', text: 'Кандидат: "Привет, я испанский инжениер"' },
]);
 const [newCustomText, setNewCustomText] = useState('');
 const [newCustomProb, setNewCustomProb] = useState(0.5);
 const [heapMessage, setHeapMessage] = useState<string | null>('');

 // Stack handlers
 const addPlate = () => {
 const presets = [
 { name: 'Сковорода от омлета', icon: '🍳' },
 { name: 'Кастрюля от супа', icon: '🍲' },
 { name: 'Чашка от кофе', icon: '☕' },
 { name: 'Миска с остатками салата', icon: '🥗' },
 { name: 'Тарелка от тако', icon: '🌮' },
];
 const randomPreset = presets[Math.floor(Math.random() * presets.length)];
 const newPlate = {
 id: Date.now().toString(),
 name: randomPreset.name,
 icon: randomPreset.icon,
 };
 setStack(prev => [...prev, newPlate]);
 setPopMessage(`Положили поверх стопки: ${newPlate.name}`);
 };
 };

```

```

const servePerson = () => {
 if (queue.length === 0) {
 setDequeueMessage("⚠ Очередь пуста! Суп ждет нов
 return;
 }
 const firstPerson = queue[0];
 setQueue(prev => prev.slice(1));
 setDequeueMessage(` Выдан суп первому в очереди (F
});

const resetQueue = () => {
 setQueue([
 { id: '1', name: 'Студент Вася', icon: ' ' },
 { id: '2', name: 'Голодный кодер', icon: ' ' },
 { id: '3', name: 'Кот Борис', icon: ' ' },
]);
 setDequeueMessage(" Очередь восстановлена.");
};

// Heap handlers
const addHypothesis = (e: React.FormEvent) => {
 e.preventDefault();
 if (!newCustomText.trim()) return;
 const item: Hypothesis = {
 id: Date.now().toString(),
 text: `Кандидат: "${newCustomText}"`,
 probability: Number(newCustomProb)
 };
 setHeap(prev => [...prev, item]);
 setNewCustomText('');
 setHeapMessage(` Добавлена гипотеза с вероятностью
});

const popHeap = () => {
 if (heap.length === 0) {
 setHeapMessage("⚠ Куча пуста! Нет кандидатов для
 return;
 }

```

```

<div className="grid grid-cols-1 md:grid-cols-3 gap-3">
 <button
 onClick={() => setActiveTab('stack')}
 className={`flex items-center justify-between border-1
 activeTab === 'stack'
 ? 'bg-blue-600/20 border-blue-500 text-blue-500'
 : 'bg-slate-800/60 border-slate-700/60 text-slate-200'
 }`>
 <div className="flex items-center gap-3">
 <Layers className={`w-5 h-5 ${activeTab === 'stack' ? 'bg-blue-600' : 'bg-slate-800'} border-1 border-current`}>
 <div className="text-left">
 <div className="font-bold text-sm text-white">LIFO
 <div className="text-xs opacity-80">LIFO
 </div>
 </div>

 DFS

 </button>

 <button
 onClick={() => setActiveTab('queue')}
 className={`flex items-center justify-between border-1
 activeTab === 'queue'
 ? 'bg-indigo-600/20 border-indigo-500 text-indigo-500'
 : 'bg-slate-800/60 border-slate-700/60 text-slate-200'
 }`>
 <div className="flex items-center gap-3">
 <AlignLeft className={`w-5 h-5 ${activeTab === 'queue' ? 'bg-indigo-600' : 'bg-slate-800'} border-1 border-current`}>
 <div className="text-left">
 <div className="font-bold text-sm text-white">FIFO
 <div className="text-xs opacity-80">FIFO
 </div>
 </div>

 BFS

 </button>
 </div>
 </div>

```

```

<div className="flex flex-wrap items-center"
 <button
 onClick={addPlate}
 className="flex-1 md:flex-none flex item
rounded-xl text-sm font-bold shadow-lg tra
 >
 <Plus className="w-4 h-4" /> Поставить
 </button>
 <button
 onClick={popPlate}
 className="flex-1 md:flex-none flex item
border-blue-500/40 px-4 py-2.5 rounded-xl
 >
 Помыть верхнюю
 </button>
 <button
 onClick={resetStack}
 title="Сбросить"
 className="p-2.5 bg-slate-800 hover:bg-
tion-colors cursor-pointer"
 >
 <RotateCcw className="w-5 h-5" />
 </button>
</div>
</div>

```

```

{popMessage && (
 <div className="bg-blue-950/60 border borde
imate-fade-in">
 <span className="inline-block w-2 h-2 roun
 {popMessage}
 </div>
)}

```

```

{/* Visualization */}
<div className="bg-slate-950/60 p-6 rounded-2
 >
 {stack.length === 0 ? (

```

```

 });
 }
 }
 </div>
)}
<div className="mt-6 text-xs text-slate-500" >
 ПОЛОЖИЛ ПОСЛЕДНЕЙ → ПОМЫЛ ПЕРВОЙ (LIFO) •
</div>
</div>
</div>
)}

{/* QUEUE TAB */}
{activeTab === 'queue' && (
 <div className="space-y-6">
 <div className="bg-slate-800/80 p-5 rounded-xl" >
 <div>
 <h3 className="text-lg font-bold text-white">
 Симуляция очереди за супом

 /span>
 </h3>
 <p className="text-slate-400 text-xs mt-1">
 Кто первым пришел, тот первым получил суп
 </p>
 </div>
 <div className="flex flex-wrap items-center">
 <button
 onClick={addPerson}
 className="flex-1 md:flex-none flex items-center justify-center gap-2.5 rounded-xl text-sm font-bold shadow-lg" >
 <Plus className="w-4 h-4" /> Встать в очередь
 </button>
 <button
 onClick={servePerson}
 className="flex-1 md:flex-none flex items-center justify-center gap-2.5 rounded-xl border-indigo-500/40 px-4 py-2.5 rounded"

```



```

 className="p-2.5 bg-slate-800 hover:bg-
tion-colors cursor-pointer"
 >
 <RotateCcw className="w-5 h-5" />
 </button>
 </div>
 </div>

 { /* Add Form */ }
 <form onSubmit={addHypothesis} className="bg-
s-12 gap-4 items-end">
 <div className="md:col-span-6">
 <label className="block text-xs font-bold">
 <input
 type="text"
 value={newCustomText}
 onChange={e => setNewCustomText(e.target.value)}
 placeholder='например: "Привет, я лучший!"'
 className="w-full bg-slate-900 border border-
rder-emerald-500 transition-colors"
 />
 </div>
 <div className="md:col-span-3">
 <label className="block text-xs font-bold">
 Вероятность:
 {newCustomProb}

 </label>
 <input
 type="range"
 min="0.01"
 max="0.99"
 step="0.01"
 value={newCustomProb}
 onChange={e => setNewCustomProb(Number(e.target.value))}
 className="w-full accent-emerald-500 cursor-pointer"
 />
 </div>
 <div className="md:col-span-3">
 <button

```

```

 return (
 <div
 key={item.id}
 className={`flex items-center justify-between
 ${isTop ? 'bg-gradient-to-r from-emerald-500 to-slate-900/10 scale-[1.01]' : 'bg-slate-900/80 border-slate-800'}
 `}
 >
 <div className="flex items-center" >
 <span className={`flex items-center justify-between
 ${isTop ? 'bg-emerald-500 text-white' : 'bg-slate-900/80 text-slate-400'}
 `}>
 #{index + 1}

 <div>
 <div className={`font-bold text-white ${isTop ? 'text-white' : 'text-slate-400'}
 {isTop && (

 Вершина Кучи (Root) • Будущее

)}
 `}
 </div>
 </div>
 </div>

 <div className="flex items-center justify-between" >
 { /* Custom progress bar */ }
 <div className="hidden sm:block" >
 <div
 className={`h-full rounded-full ${isTop ? 'bg-emerald-500' : 'bg-slate-400'}
 style={{ width: `${percent}%` }}
 ></div>
 </div>
 <span className={`font-mono font-medium text-white ${isTop ? 'text-white' : 'text-slate-400'}
 `} >
 {percent}%

 </div>
)
 }
 }
 </div>
)

```



```

/**
 * Каталог всех интерактивных тренажёров.
 *
 * Раньше вкладка «Симулятор» показывала один случайный
 * теперь это витрина: видно всё, что вообще можно поты
 */
export const SimulatorHub: React.FC<Props> = ({ onOpen,
 const [query, setQuery] = useState("");

 const items = useMemo(() => {
 const titleById = new Map(chapters.map((c) => [c.id,
 const all = Object.entries(VIZ_REGISTRY).map(([id,
 id,
 entry,
 chapterTitle: titleById.get(id),
]));
 const q = query.trim().toLowerCase();
 if (!q) return all;
 return all.filter(
 (i) =>
 i.entry.title.toLowerCase().includes(q) ||
 (i.entry.hint ?? "").toLowerCase().includes(q)
 (i.chapterTitle ?? "").toLowerCase().includes(q)
);
 }, [query]);

 return (
 <div>
 <div className="mb-6">
 <h2 className="text-xl sm:text-2xl font-extrabold">
 <p className="text-sm text-slate-400 leading-relaxed">
 Все интерактивные демонстрации в одном месте.
 здесь их можно открыть напрямую.
 </p>
 </div>

 <div className="relative mb-6 max-w-md">
 <Search className="w-4 h-4 text-slate-500 absolute

```

```

 {chapterTitle && {
 <button
 onClick={() => onGoChapter(id)}
 title={chapterTitle}
 className="flex items-center gap-1.5
0 hover:text-white transition-colors min-w-1
 >
 <BookOpen className="w-3.5 h-3.5 shrink-0" />

 </button>
 }}
 </div>
 </div>
)}
</div>
</div>
);
};

```

ФАЙЛ 61/83: src/components/SplayPlayground.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```

import React, { useState } from "react";
import { SplayRotationsViz } from "../SplayRotationsViz";
import SplayWalkViz from "../SplayWalkViz";
import { SplayRotationSandbox } from "../SplayRotationSandbox";

/**
 * Страница splay – три вкладки по нарастанию сложности
 * 1. «Механика» – покадровый разбор Zig / Zig-Zig / Zig-Zag
 * (прицел → поворот, призраки старых позиций, обход)
 * 2. «Большое дерево» – те же случаи внутри настоящего дерева
 * поворот – кадры «прицел → отстёгиваем среднее по
 * 3. «Собери сам» – песочница: подними узел кликами,
 */

```

```

 {tab === "big" && <SplayWalkViz />}
 {tab === "play" && <SplayRotationSandbox />}

 {tab === "play" && {
 <p className="text-xs text-slate-500 mt-3">
 Порядок поворотов в песочнице сверяется с прав
 </p>
 }}
 </div>
);
};

export default SplayPlayground;

```

ФАЙЛ 62/83: src/components/SplayRotationSandbox.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import React, { useCallback, useMemo, useState } from "react";
import { CheckCircle2, Dices, Eye, EyeOff, RotateCcw, Undo } from "lucide-react";

/**
 * Песочница поворотов.
 *
 * Здесь ничего не подсказывается: дано настоящее дерево,
 * поднять отмеченный узел в корень. Клик по узлу = один шаг.
 * Порядок поворотов выбираешь сам; правильный ли он по
 * только после того, как узел окажется наверху (или по
 */

export interface TNode {
 key: number;
 left: TNode | null;
 right: TNode | null;
}

```

```

 target: 5,
 build: () => {
 const n5 = leaf(5);
 const n6: TNode = { key: 6, left: n5, right: leaf(7) };
 const n4: TNode = { key: 4, left: leaf(3), right: n6 };
 return { key: 8, left: n4, right: leaf(9) };
 },
 },
];

/* ----- операции над деревом -----

export const findPath = (root: TNode | null, key: number): TNode[] => {
 const path: TNode[] = [];
 let cur = root;
 while (cur) {
 path.push(cur);
 if (key === cur.key) return path;
 cur = key < cur.key ? cur.left : cur.right;
 }
 return [];
};

/** Поднять узел key на один уровень (поворот с родителем)
export function rotateUp(root: TNode, key: number): TNode {
 const path = findPath(root, key);
 if (path.length < 2) return root;

 const x = path[path.length - 1];
 const p = path[path.length - 2];
 const gg = path.length >= 3 ? path[path.length - 3] : null;

 if (p.left === x) {
 p.left = x.right;
 x.right = p;
 } else {
 p.right = x.left;
 x.left = p;
 }

 if (gg) {
 gg.left = x;
 gg.right = p;
 }

 return root;
}

```

```

const walk = (n: TNode | null, depth: number) => {
 if (!n) return;
 walk(n.left, depth + 1);
 nodes.push({ key: n.key, x: order++, y: depth, depth });
 if (n.left) edges.push([n.key, n.left.key]);
 if (n.right) edges.push([n.key, n.right.key]);
 walk(n.right, depth + 1);
};
walk(root, 0);

const cols = Math.max(1, order);
const rows = Math.max(1, Math.max(...nodes.map((n) => n.depth)));
const colW = 46;
const rowH = 62;
return {
 nodes: nodes.map((n) => ({ ...n, x: n.x * colW + colW / 2, y: n.y * rowH })),
 edges,
 width: cols * colW,
 height: rows * rowH + 20,
};
}

/* ----- КОМПОНЕНТ -----

export const SplayRotationSandbox: React.FC = () => {
 const [presetIdx, setPresetIdx] = useState(0);
 const preset = PRESETS[presetIdx];

 const [tree, setTree] = useState<TNode>(() => preset.root);
 const [history, setHistory] = useState<TNode[]>([]);
 const [moves, setMoves] = useState<{ node: number; count: number }>({});
 const [showAnswer, setShowAnswer] = useState(false);

 const target = preset.target;
 const solved = tree.key === target;

 const reset = useCallback(
 (idx = presetIdx) => {

```

```

 <h4 className="text-sm sm:text-base font-bold">
 <p className="text-[12px] text-slate-400 lead">
 Клик по узлу = один поворот с его родителем

 поворотов выбираешь сам, разбор покажем в к
 </p>
 </div>
 <button
 onClick={() => reset((presetIdx + 1) % PRESETS.length)}
 className="flex items-center gap-1.5 px-3 py-1.5 text-sm font-bold transition-colors shrink-0"
 >
 <Dices className="w-3.5 h-3.5" /> Другое дерево
 </button>
 </div>

 <div className="flex gap-2 mb-3 overflow-x-auto">
 {PRESETS.map((p, i) => {
 <button
 key={p.name}
 onClick={() => reset(i)}
 className={`px-3 py-1.5 rounded-lg text-[11px] ${
 i === presetIdx ? "bg-indigo-600 text-white" : "bg-white text-slate-600"
 }`}
 >
 {p.name}
 </button>
 })}
 </div>

 <div className="bg-slate-900 rounded-xl border border-slate-800">
 <svg
 viewBox={`0 0 ${width} ${height}`}
 className="h-auto block mx-auto"
 style={{ minWidth: Math.min(width * 1.5, 420) }}
 role="img"
 >
 {edges.map(([a, b], i) => {

```

```

 strokeWidth={2}
 />
 <text textAnchor="middle" dominantBaseline="middle">
 {n.key}
 </text>
 </g>
);
}
</svg>
</div>

<div className="flex flex-wrap items-center gap-2">
 <button
 onClick={undo}
 disabled={history.length === 0}
 className="flex items-center gap-1.5 px-3 py-1.5 text-white disabled:opacity-40 text-xs font-bold"
 >
 <Undo2 className="w-3.5 h-3.5" /> Отменить
 </button>
 <button
 onClick={() => reset()}
 className="flex items-center gap-1.5 px-3 py-1.5 text-white font-bold transition-colors"
 >
 <RotateCcw className="w-3.5 h-3.5" /> Заново
 </button>
 <button
 onClick={() => setShowAnswer((s) => !s)}
 className={`flex items-center gap-1.5 px-3 py-1.5 text-white border
 ${showAnswer
 ? "bg-amber-600/20 border-amber-500 text-amber-500"
 : "bg-slate-800 border-slate-700 text-slate-200"}
 `}
 >
 {showAnswer ? <EyeOff className="w-3.5 h-3.5" /> : "Скрыть разбор"}
 </button>

```

```


 })
 {wrong === 0 && moves.length > minimal && (
 <div className="opacity-80">Порядок верный,
 })
 </div>
 })
</div>
);
};

```

ФАЙЛ 63/83: src/components/SplayRotationsViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 10

```

import React, { useEffect, useMemo, useState } from "react";
import { Gauge, Pause, Play, RotateCcw } from "lucide-react";

```

```

/**
 * Разбор трёх случаев операции Splay: Zig, Zig-Zig, Zig-Zag.
 *
 * Что сделано ради понятности:
 * * у узлов настоящие числа (20 / 40 / 60), а не абстракции;
 * * сразу видно, что дерево остаётся деревом поиска;
 * * под деревом печатается обход слева направо: он ОДНОУРОВНЕВНЫЙ;
 * * это и есть доказательство, что поворот ничего не ломает;
 * * кадр «прицел» ничего не двигает, только подсвечивает;
 * * на кадре «результат» остаются призраки – пунктирные узлы в
 * местах и стрелки «откуда → куда»;
 * * по умолчанию анимация НЕ крутится сама: пользователь
 * идёт в своём темпе. Автопрокрутка включается кнопкой.
 */

```

```

type NodeId = "x" | "p" | "g" | "A" | "B" | "C" | "D";
type Pos = Partial<Record<NodeId, [number, number]>>;

```



```

const ZIG: Case = {
 key: "zig",
 label: "Zig",
 when: "родитель x — уже корень, деда нет",
 rotations: "1 поворот",
 keys: { x: 20, p: 40 },
 inorder: ["A", "20", "B", "40", "C"],
 ranges: "A < 20 · B между 20 и 40 · C > 40",
 frames: [
 {
 ...ZIG_BEFORE,
 kind: "start",
 title: "Было: 20 висит под корнем 40",
 text: "Нам нужен ключ 20, а он на глубине 1. Деду",
 ms: RESULT_MS,
 },
 {
 ...ZIG_BEFORE,
 kind: "aim",
 title: "Прицел: крутим ребро 40 — 20",
 text: "Ничего пока не двигается. Смотри на подсве",
 focus: ["p", "x"],
 ms: AIM_MS,
 },
 {
 pos: { x: [160, 50], A: [80, 130], p: [245, 130],
 edges: [["x", "A"], ["x", "p"], ["p", "B"], ["p",
 kind: "result",
 title: "Стало: 20 — корень",
 text: "40 стало правым сыном двадцатки. Поддерев
 ева от 40 — это одно и то же место.",
 moved: ["x", "p", "B"],
 ms: RESULT_MS,
 },
],
};

```

```

 title: "Поворот 1: 40 поднялось над 60",
 text: "40 встало в корень, 60 опустилось к нему п
 убине 1, а не 2.",
 moved: ["p", "g", "C"],
 ms: RESULT_MS,
 },
 {
 ...ZZ_S1,
 kind: "aim",
 title: "Прицел 2: ребро 40 – 20",
 text: "А теперь обычный одиночный поворот – тот ж
 focus: ["p", "x"],
 ms: AIM_MS,
 },
 {
 pos: { x: [100, 45], A: [45, 118], p: [188, 118],
 edges: [["x", "A"], ["x", "p"], ["p", "B"], ["p",
 kind: "result",
 title: "Стало: 20 в корне, бамбук стал кустом",
 text: "Сравни с первым кадром: А и В были на глуб
 дерево, а не только достаёт ключ.",
 moved: ["x", "p", "A", "B"],
 ms: RESULT_MS,
 },
],
};

```

/\* ----- Zig-Zag -----

```

const ZG_S0 = {
 pos: { g: [258, 40], D: [328, 112], p: [148, 112], A:
 edges: [["g", "p"], ["g", "D"], ["p", "A"], ["p", "x"]
} as unknown as Pick<Frame, "pos" | "edges">;

```

```

const ZG_S1 = {
 pos: { g: [258, 40], D: [328, 112], x: [148, 112], p:
 edges: [["g", "x"], ["g", "D"], ["x", "p"], ["x", "C"]
} as unknown as Pick<Frame, "pos" | "edges">;

```

```

 focus: ["g", "x"],
 ms: AIM_MS,
 },
 {
 pos: { x: [180, 50], p: [90, 130], g: [275, 130],
 edges: [["x", "p"], ["x", "g"], ["p", "A"], ["p",
 kind: "result",
 title: "Стало: 20 и 60 — два сына сорока",
 text: "Дерево стало идеально симметричным: все че
 очку». ",
 moved: ["x", "p", "g", "C"],
 ms: RESULT_MS,
 },
],
};

```

```

const CASES: Case[] = [ZIG, ZIGZIG, ZIGZAG];

```

```

const COLOR: Record<string, { fill: string; stroke: str
 x: { fill: "#6366f1", stroke: "#a5b4fc" },
 p: { fill: "#0ea5e9", stroke: "#7dd3fc" },
 g: { fill: "#f59e0b", stroke: "#fcd34d" },
 sub: { fill: "#1e293b", stroke: "#475569" },
};

```

```

const ROLE_RU: Partial<Record<NodeId, string>> = {
 x: "x — ищем его",
 p: "p — родитель",
 g: "g — дед",
};

```

```

const isSubtree = (id: NodeId) => id === "A" || id ===

```

```

const Tree: React.FC<{ frame: Frame; prev: Pos; keys: C
 const { pos, edges, focus, moved } = frame;
 const dim = (id: NodeId) => (focus ? !focus.includes(
 const ghosts = frame.kind === "result" ? (moved ?? [])

```

```

 const pb = pos[b];
 if (!pa || !pb) return null;
 const hot = Boolean(focus && focus.includes(a));
 return (
 <line
 key={` ${a} - ${b} - ${i} `}
 x1={pa[0]}
 y1={pa[1]}
 x2={pb[0]}
 y2={pb[1]}
 stroke={hot ? "#fbbf24" : "#475569"}
 strokeWidth={hot ? 5 : 2}
 opacity={focus && !hot ? 0.25 : 1}
 style={{ transition: "all 1200ms cubic-bezier(0.16, 0.66, 0.84, 0.34)" }}
 />
);
 }
}

```

```

{Object.keys(pos) as NodeId[]}.map((id) => {
 const p = pos[id];
 if (!p) return null;
 const sub = isSubtree(id);
 const c = COLOR[sub ? "sub" : id];
 const r = sub ? 15 : 20;
 const hot = Boolean(focus && focus.includes(id));
 return (
 <g
 key={id}
 style={{
 transform: `translate(${p[0]}px, ${p[1]}px)`,
 transition: MOVE,
 opacity: dim(id) ? 0.28 : 1,
 }}
 >
 {hot && <circle r={r + 7} fill="none" stroke="red" />}
 <circle r={r} fill={c.fill} stroke={c.stroke} />
 <text
 textAnchor="middle"

```

```

 if (!playing) return;
 const t = setTimeout(() => setFrame((f) => (f + 1) % CASES.length), duration);
 return () => clearTimeout(t);
 }, [playing, frame, duration, total]);

return (
 <div className="w-full bg-slate-950 rounded-2xl border border-slate-800">
 <div className="flex gap-2 mb-4 overflow-x-auto">
 {CASES.map((c, i) => (
 <button
 key={c.key}
 onClick={() => setCaseIdx(i)}
 className={`px-3 sm:px-4 py-2 rounded-lg text-white
 ${i === caseIdx ? "bg-indigo-600 text-white" : "bg-slate-800 text-slate-300"}
 `}
 >
 {c.label}
 </button>
))}
 </div>

 <div className="mb-3 text-xs sm:text-[13px] text-slate-400">
 Корпус
 ({active.rotation}°)
 </div>

 {/* шаг + пояснение стоят НАД картинкой: сначала шаг, потом пояснение */}
 <div className="mb-3 rounded-xl border border-slate-800">
 <div className="flex items-center gap-2 mb-1 flex-wrap">
 <span
 className={`text-[10px] font-bold uppercase
 ${current.kind === "aim"
 ? "bg-amber-500/20 text-amber-300"
 : current.kind === "start"
 ? "bg-slate-700 text-slate-300"
 : "bg-indigo-500/20 text-indigo-300"}
 `}
 >

```

```

 playing
 ? { animation: `demoProgress ${duration}ms`
 : { width: "100%", background: "#334155"
 }
 />
</div>

<div className="flex flex-wrap items-center gap-2" >
 <button
 onClick={() => {
 setPlaying(false);
 setFrame((f) => (f - 1 + total) % total);
 }}
 disabled={idx === 0}
 className="px-3 py-2 rounded-lg bg-slate-800 text-white
 dark:hover:text-slate-300 text-xs font-bold transition-colors"
 >
 ← Назад
 </button>
 <button
 onClick={() => {
 setPlaying(false);
 setFrame((f) => (f + 1) % total);
 }}
 className="px-4 py-2 rounded-lg bg-indigo-600 text-white
 w-indigo-900/40"
 >
 {last ? "⏮ Сначала" : "Дальше →"}
 </button>
 <button
 onClick={() => setPlaying((p) => !p)}
 className="flex items-center gap-1.5 px-3 py-2 text-white
 text-xs font-bold transition-colors"
 >
 {playing ? <Pause className="w-3.5 h-3.5" /> : "Пуск"}
 {playing ? "Стоп" : "Авто"}
 </button>
 <button

```

---

ФАЙЛ 64/83: src/components/SplayWalkViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import React, { useEffect, useMemo, useRef, useState } from 'react';

/**
 * Splay на БОЛЬШОМ дереве – пошагово.
 *
 * Проблема, которую решает: на игрушечных A/B/C-схемах
 * «двух зигов подряд», а мгновенный splay в большом дереве
 * стало». Здесь каждый ОДИНОЧНЫЙ поворот – атомарный шаг.
 * поворот → текст, какие рёбра развернулись и какие поворачиваются.
 * Порядок поворотов виден явно: Zig-Zig – ВЕРХНЕЕ ребро поворачивается
 * НИЖНЕЕ первым.
 *
 * Раскладка – слоты in-order: ключи не двигаются по горизонтали.
 * поворот меняет только глубину. Поэтому «что переехало» не важно.
 *
 * Чистые функции (splaySteps, rotateUp, collectEdges) :
 */

export interface SN {
 key: number;
 left: SN | null;
 right: SN | null;
 parent: SN | null;
}

export function cloneWithParents(t: SN | null): SN | null {
 if (!t) return null;
 const n: SN = { key: t.key, left: null, right: null, parent: null };
 const l = t.left ? cloneWithParents(t.left) : null;
 const r = t.right ? cloneWithParents(t.right) : null;
 if (l) {
```

```

 if (child.right) child.right.parent = p;
 child.right = p;
 p.parent = child;
 } else {
 p.right = child.left;
 if (child.left) child.left.parent = p;
 child.left = p;
 p.parent = child;
 }
 child.parent = g;
 if (!g) return;
 if (g.left === p) g.left = child;
 else g.right = child;
}

/** Рёбра в направленном виде "p→c": для diff между шага
export function collectEdges(t: SN | null): string[] {
 const es: string[] = [];
 const walk = (n: SN | null) => {
 if (!n) return;
 if (n.left) es.push(`${n.key}→${n.left.key}`);
 if (n.right) es.push(`${n.key}→${n.right.key}`);
 walk(n.left);
 walk(n.right);
 };
 walk(t);
 return es.sort();
}

const diffEdges = (before: string[], after: string[]):
 const rm = before.filter((e) => !after.includes(e));
 const ad = after.filter((e) => !before.includes(e));
 const parts: string[] = [];
 for (const a of ad) {
 const [p, c] = a.split("→");
 if (rm.includes(`${c}→${p}`)) parts.push(`ребро ${p}
 else parts.push(`поддеревцо ${c} переехало к ${p}`);
 }

```



```

 }
 cur = key < cur.key ? cur.left : cur.right;
}
const x = found ? cur : last;
steps.push({
 kind: "search",
 root: cloneWithParents(root)!,
 pathKeys: [...pathKeys],
 note: isInsert
 ? `Вставили ${key} обычным BST-спуском. Теперь по
 : found
 ? `Спуск по ключу ${key}: путь ${pathKeys.join(
 : `Ключа ${key} нет. Спуск упёрся в ${last.key}
 rotCount: 0,
});

// --- подъём ---
let rotCount = 0;
let target: SN = x as SN;
const fixRoot = () => {
 while (root && root.parent) root = root.parent;
};
/** Один поворот = два атомарных кадра: cut (что пере
const emitRotation = (child: SN, label: string) => {
 const p = child.parent!;
 const mid = p.left === child ? child.right : child.
 steps.push({
 kind: "cut",
 root: cloneWithParents(root)!,
 cutEdge: [child.key, mid ? mid.key : child.key],
 cutLabel: mid ? `поддерево ${mid.key}` : "пустое
 caseLabel: label,
 note: mid
 ? `Отстёгиваем среднее поддерево ${mid.key}: он
 сторону к ${p.key}.`
 : `Среднего поддерева нет: ${child.key} поднима
 rotCount,
 });

```

```

 aimEdge: [g.key, p.key],
 caseLabel: "Zig-Zig",
 note: `Zig-Zig (${side}, линия ${g.key}—${p.key}
 а ${target.key} пока не трогаем.`,
 rotCount,
 });
 emitRotation(p, "Zig-Zig");
 steps.push({
 kind: "aim",
 root: cloneWithParents(root)!,
 aimEdge: [target.parent.key, target.key],
 caseLabel: "Zig-Zig",
 note: `Теперь обычный зиг: ребро ${target.parent
 rotCount,
 });
 emitRotation(target, "Zig-Zig");
} else {
 const side = `${g.key} → ${pIsLeft ? "влево" : "в
 steps.push({
 kind: "aim",
 root: cloneWithParents(root)!,
 aimEdge: [p.key, target.key],
 caseLabel: "Zig-Zag",
 note: `Zig-Zag (эмейка: ${side}). Порядок ОБРАТ
 rotCount,
 });
 emitRotation(target, "Zig-Zag");
 steps.push({
 kind: "aim",
 root: cloneWithParents(root)!,
 aimEdge: [target.parent.key, target.key],
 caseLabel: "Zig-Zag",
 note: `Вроде ребро: ${target.parent.key}—${tar
 rotCount,
 });
 emitRotation(target, "Zig-Zag");
}
}

```

```

 { line: 5, text: " elif p,g,x на одной линии: # Z
 { line: 6, text: " rotate(p over g) # BE
 { line: 7, text: " rotate(x over p)" },
 { line: 8, text: " else: # Zi
 { line: 9, text: " rotate(x over p) # HI
 { line: 10, text: " rotate(x over g)" },
];

```

```

/* ----- рендер -----

```

```

const VIEW_W = 760;
const SLOT_W = 88;
const LEVEL_H = 76;

```

```

const layout = (root: SN | null) => {
 const pos = new Map<number, { px: number; py: number }>
 let slot = 0;
 const walk = (n: SN | null, d: number) => {
 if (!n) return;
 walk(n.left, d + 1);
 pos.set(n.key, { px: 56 + slot * SLOT_W, py: 40 + d * LEVEL_H });
 slot += 1;
 walk(n.right, d + 1);
 };
 walk(root, 0);
 return pos;
};

```

/\*\* Врезка «механика одного поворота»: как вправляют вы

```

const MechanismInset: React.FC<{ step: SplayStep | null }> = ({ step }) => {
 const isCut = step?.kind === "cut";
 const isRot = step?.kind === "rot";
 return (
 <div className="bg-slate-950/80 rounded-lg border border-slate-700">
 <h4 className="text-xs font-bold text-slate-300 mb-2">Механика одного поворота
 <p className="text-[10px] text-slate-500 mb-2">Как вправляют вы
 <svg viewBox="0 0 340 190" className="w-full">
 <g transform="translate(10,6)">

```

```

 <text x="112" y="90" fill={isRot ? "#10b981"
 }</text>
 <text x="28" y="160" fill="#64748b" fontSize=
 </g>
 </svg>
 <p className="text-[10px] text-slate-500 leading-
 Порядок всегда один: отстегнуть β у x → поднять
 </p>
 </div>
);
};

```

```

const SplayWalkViz: React.FC = () => {
 const initial = useMemo(buildInitial, []);
 const [steps, setSteps] = useState<SplayStep[] | null>();
 const [idx, setIdx] = useState(0);
 const [playing, setPlaying] = useState(false);
 const [input, setInput] = useState("");
 const [treeVers, setTreeVers] = useState(0);
 const baseRef = useRef<SN>(initial);
 const timer = useRef<ReturnType<typeof setTimeout> | null>();

 const runKey = (key: number, isInsert = false) => {
 const { steps: st } = splaySteps(baseRef.current, key);
 setSteps(st);
 setIdx(0);
 setPlaying(true);
 };

 useEffect(() => {
 if (!playing || !steps) return;
 if (idx >= steps.length - 1) {
 setPlaying(false);
 // фиксируем результат в базовом дереве
 baseRef.current = steps[steps.length - 1].root ??
 setTreeVers((v) => v + 1);
 return;
 }
 }, [playing, steps, idx]);
}

```

```
const dNow = steps ? depthOf(viewRoot, steps[steps.length - 1]) : 0;
const hBefore = useMemo(() => (steps ? heightOf(steps[steps.length - 1], viewRoot) : 0));
const hNow = heightOf(viewRoot);
```

```
const doInsert = () => {
 const v = parseInt(input, 10);
 if (Number.isNaN(v)) return;
 const withNew = bstInsert(baseRef.current, v);
 baseRef.current = withNew;
 setTreeVers((t) => t + 1);
 runKey(v, true);
 setInput("");
};
```

```
const stepColor = (k: number): string => {
 if (!step) return "#0f172a";
 if (step.kind === "search" && step.pathKeys?.includes(k)) return "#f59e0b";
 if (step.aimEdge && step.aimEdge.includes(k)) return "#f59e0b";
 return "#0f172a";
};
```

```
const strokeColor = (k: number): string => {
 if (!step) return "#64748b";
 if (step.kind !== "search" && step.pathKeys?.includes(k)) return "#f59e0b";
 if (step.aimEdge?.includes(k)) return "#f59e0b";
 if (step.pathKeys?.includes(k)) return "#38bdf8";
 return "#64748b";
};
```

```
return (
 <div className="bg-slate-800/90 p-6 rounded-2xl border border-slate-700" >
 { /* Wanka */ }
 <div className="flex flex-wrap items-center justify-center" >
 <div className="flex items-center gap-3" >
 <div className="p-2.5 bg-indigo-600 text-white rounded-md" >

 </div>
 <div>
 <h3 className="text-2xl font-bold text-white" >
```

```

 }}
 className="px-3 py-2 rounded-lg text-xs font-medium"
 >
 Перемешать
 </button>
 </div>
 </div>

 { /* Плеер */ }
 { steps && {
 <div className="flex flex-wrap items-center gap-2" >
 <button onClick={() => { setIdx(0); setPlaying(true); }} >
 В начало
 </button>
 <button onClick={() => { setPlaying(false); setIdx(0); }} >
 Пауза
 </button>
 <button onClick={() => setPlaying(!playing)} >
 { playing ? "▶ Пауза" : "▶ Пуск" }
 </button>
 <button onClick={() => { setPlaying(false); setIdx(0); }} >
 В конец
 </button>

 Шаг { idx + 1 } из { steps.length } • поворотов

 { step?.caseLabel && {

 { step.caseLabel }

 } }
 </div>
 } }
}
```

```

 }}}
 {nodes.map((n) => {
 const isAimNode = step?.aimEdge?.includes(n)
 return (
 <g
 key={`n-${n.key}-${treeVers}`}
 transform={`translate(${n.px}, ${n.py})`}
 onClick={() => {
 if (playing) return;
 const has = inorderKeys(baseRef.current)
 if (has) runKey(n.key);
 }}
 className="cursor-pointer"
 >
 <circle
 r={isAimNode ? 24 : 20}
 fill={stepColor(n.key)}
 stroke={strokeColor(n.key)}
 strokeWidth={isAimNode ? 4 : 2}
 className="transition-all duration-300"
 />
 <text y={4} fill="#ffffff" fontSize="13">
 {n.key}
 </text>
 </g>
);
 })}
 </svg>

```

```

 {/* Лог шага + механика */}
 <div className="w-full flex flex-col lg:flex-row">
 <div className="flex-1 bg-slate-950/80 p-4 rounded">
 <p className="font-semibold text-amber-400 mb-2">
 {step?.kind === "aim" ? " Прицел:" : step?.kind === "search" ? " Спуск:" : "Стрельба:"}
 </p>
 <p className="min-h-[40px] flex items-center">

```





```

const addLog = (msg: string) => setLog(prev => [msg,

// PUSH: Положить грязную тарелку сверху стопки
const pushPlate = useCallback(() => {
 if (isWashing) return;
 if (dirtyStack.length >= 7) {
 addLog('⚠ Стопка слишком высокая, тарелки могут р
 setShake(true);
 setTimeout(() => setShake(false), 400);
 return;
 }

 const preset = PLATE_PRESETS[counter % PLATE_PRESET
 counter++;

 const newPlate: Plate = {
 id: Date.now().toString(),
 ...preset,
 isDirty: true,
 animState: 'in',
 };

 setDirtyStack(prev => [...prev, newPlate]);
 addLog(` PUSH: Положили ${newPlate.emoji} сверху с

 setTimeout(() => {
 setDirtyStack(prev => prev.map(p => p.id === newP
 }, 500);
}, [dirtyStack, isWashing]);

// POP: Помыть верхнюю тарелку (LIFO)
const popPlate = useCallback(() => {
 if (isWashing) return;
 if (dirtyStack.length === 0) {
 setShake(true);
 addLog('⚠ POP: Нечего мыть! Все тарелки чистые.')
 setTimeout(() => setShake(false), 400);

```

```

 setCleanRack([]);
 setIsWashing(false);
 setShowSoap(false);
 setLog([' Стол сброшен. Посуда снова грязная!']);
 });

 const topIndex = dirtyStack.length - 1;

 return (
 <div className="flex flex-col gap-6">
 {/* МНЕМОНИКА / ПРАВИЛО */}
 <div className="bg-blue-950/40 border border-blue-950/40">
 <div className="text-3xl"> </div>
 <div>
 <h3 className="font-black text-blue-400 text-4xl">ПРАВИЛО</h3>
 <p className="text-xs text-slate-300 mt-1 leading-4">
 Ты складываешь грязные тарелки друг на друга.
 А когда приходит время мыть, ты берёшь именно самую верхнюю.
 Попытаешься вытащить нижнюю – всё рухнет!
 </p>
 </div>
 </div>

 {/* УПРАВЛЕНИЕ */}
 <div className="flex items-center gap-3 flex-wrap">
 <button
 onClick={pushPlate}
 disabled={isWashing}
 className="flex-1 min-w-[150px] bg-gradient-to-r from-blue-950/40 to-blue-950/0
 opacity-40 disabled:cursor-not-allowed text-white text-sm font-medium
 rounded-95 transition-all cursor-pointer flex items-center justify-center">
 Положить грязную (PUSH)
 </button>

 <button
 onClick={popPlate}
 disabled={isWashing || dirtyStack.length === 0}
 className="flex-1 min-w-[150px] bg-gradient-to-r from-blue-950/40 to-blue-950/0
 opacity-40 disabled:cursor-not-allowed text-white text-sm font-medium
 rounded-95 transition-all cursor-pointer flex items-center justify-center">
 Вытащить (POP)
 </button>
 </div>
 </div>
);
}

export default DishWasher;

```

```

 <div className="absolute w-4 h-4 rounded-
"></div>
 <div className="absolute w-5 h-5 rounded-
"></div>
 <div className="absolute text-4xl anim-cr
</div>
)}

{dirtyStack.length === 0 ? (
 <div className="flex-1 flex flex-col items-
 <span className="text-6xl mb-3 animate-pu
 <p className="text-white font-black text-
 <p className="text-slate-500 text-xs mt-1
 </div>
) : (
 <div className={`flex-1 flex flex-col justifi

 {/* Указатель TOP */}
 <div
 className="absolute right-8 flex items-
 style={{ bottom: `${24 + topIndex * 44}
 >
 <span className="text-xs font-mono text
er gap-1">
 <ArrowUp className="w-3.5 h-3.5" />
 ВЕРШИНА (TOP)

 <span className="text-blue-400 animate-p
 </div>

 {/* Тарелки (отрисовываем в обратном поряд
 {dirtyStack.slice().reverse().map(({plate,
 const idx = dirtyStack.length - 1 - rev
 const isTop = idx === topIndex;
 return (
 <div
 key={plate.id}
 className={`

```

```

{cleanRack.length === 0 ? {
 <div className="flex-1 flex flex-col item

 <p className="text-xs font-bold font-mo
 <p className="text-[10px] mt-1">Здесь 6
 </div>
) : {
 <div className="flex flex-col gap-2 w-full
 {cleanRack.map(plate => {
 <div
 key={plate.id}
 className="w-full h-8 rounded-full
items-center justify-center gap-2 opacity-1
 >
 {plate.n
 <span className="text-[10px] text-e
 </div>
 })}
 </div>
})}
</div>

{/* Столешница сушилки */}
<div className="w-full h-4 bg-gradient-to-r f
</div>
</div>

{/* ЛОГ ДЕЙСТВИЙ */}
<div className="bg-slate-900 border border-slate-:
 <div className="text-[10px] font-mono text-slate
 {log.map((entry, idx) => {
 <div
 key={idx}
 className={`text-xs font-mono flex items-ce
 >
 {idx === 0 ? <span className="text-blue-500
 {entry}
 </div>
 })}
 </div>
}

```

```

 ска увеличивается.` }));
 j++;
 } else {
 steps.push({ i, j, pi: [...pi], highlight:
 овпадения нет.` }));
 }

 pi[i] = j;
 steps.push({ i, j, pi: [...pi], desc: `Записывае

 if (j === pattern.length) {
 steps.push({ i, j, pi: [...pi], found: true
 }
}
return { s, steps, patternLength: pattern.length };
}

```

```

function generateZSteps(pattern: string, text: string)
 if (!pattern) pattern = " ";
 const s = pattern + "#" + text;
 const n = s.length;
 const z = new Array(n).fill(0);
 const steps: any[] = [];

 let l = 0, r = 0;
 steps.push({ i: 0, l, r, z: [...z], desc: `Начало.

 for (let i = 1; i < n; i++) {
 steps.push({ i, l, r, z: [...z], desc: `Шаг i=$

 if (i <= r) {
 steps.push({ i, l, r, z: [...z], desc: `Вни
 !` }));
 z[i] = Math.min(r - i + 1, z[i - l]);
 steps.push({ i, l, r, z: [...z], desc: `Пре
 }. Копипаста сработала.` }));
 }
 }
}

```



```

// KMP Logic
if (mode === "kmp") {
 if (step.highlight?.include
 borderColor = "border-a
 bgColor = step.match ===
-900/40");
 }
}

// Z Logic
if (mode === "z") {
 if (index >= step.l && index
 bgColor = "bg-emerald-9
 borderColor = "border-e
 }
 if (step.zTarget === index
 borderColor = "border-a
 bgColor = step.match ===
-900/40");
 }
}

return {
 <div key={index} className=
 {/* L/R markers for Z */
 {mode === "z" && step.l
 <div className="abs
 }}
 {mode === "z" && step.r
 <div className="abs
 }}

 {/* Index */}
 <span className={`text-

 {/* Char Box */}
 <div className={`w-8 h-
xtColor} font-mono text-lg transition-color

```

```

 </div>
 })
 {mode === "z" && step.z}
 <div className="abs
 <span classNam
 </div>
 })

 {/* Found flash effect
 {step.found && mode ===
 <div className="abs
ulse z-0 bg-emerald-500/20 shadow-[0_0_15px
 })
 {step.found && mode ===
 <div className="abs
ulse z-0 bg-emerald-500/20 shadow-[0_0_15px
 })
 </div>
)
 }}}}
 </div>
</div>

<div className="flex flex-col sm:flex-row g
 {/* Controls */}
 <div className="flex bg-slate-900 borde
 <button onClick={prevStep} disabled:
rounded-lg disabled:opacity-30 disabled:hov
 <Undo strokeWidth={3} size={16}
 </button>
 <button onClick={playPause} classNam
x items-center justify-center min-w-[80px]"
 {autoplay ? <><Pause size={16}
 </button>
 <button onClick={nextStep} disabled:
r:bg-slate-800 rounded-lg disabled:opacity-
 <SkipForward strokeWidth={3} si
 </button>

```



```

</div>
<pre className="p-4 text-xs font-mono text-gray-800">
{mode === 'kmp' ? `vector<int> pi(s.length());
for (int i = 1; i < s.length(); i++) {
 int j = pi[i-1];
 while (j > 0 && s[i] != s[j]) j = pi[j-1];
 if (s[i] == s[j]) j++;
 pi[i] = j;
}` : `int l = 0, r = 0;
for (int i = 1; i < n; i++) {
 if (i <= r) z[i] = min(r - i + 1, z[i - l]);
 while (i + z[i] < n && s[z[i]] == s[i + z[i]]) z[i]++;
 if (i + z[i] - 1 > r) {
 l = i;
 r = i + z[i] - 1;
 }
}`}
</pre>
</div>
</div>
);
}

```

ФАЙЛ 67/83: src/components/TopologicalSortViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```

import React, { useState, useEffect } from "react";
import {
 Play,
 Pause,
 RotateCcw,
 Clock,
} from "lucide-react";

interface TNode {

```

```

const [steps, setSteps] = useState<TStep[]>([]);
const [currentStepIdx, setCurrentStepIdx] = useState(0);
const [isPlaying, setIsPlaying] = useState(false);

useEffect(() => {
 const listSteps: TStep[] = [];
 const visited = new Set<number>();
 const fullyProcessed = new Set<number>();
 const topoList: number[] = [];
 const dfsStack: number[] = [];

 const recordStep = (desc: string, currentNode: number) => {
 listSteps.push({
 desc,
 visited: new Set(visited),
 fullyProcessed: new Set(fullyProcessed),
 currentNode,
 topoList: [...topoList],
 dfsStack: [...dfsStack],
 });
 };

 recordStep(
 "Начало топологической сортировки. Используем кла",
 null,
);

 // Adj list
 const adj: number[][] = Array.from({ length: 5 }, () => []);
 dagEdges.forEach((e) => adj[e.u].push(e.v));

 const dfs = (u: number) => {
 visited.add(u);
 dfsStack.push(u);
 recordStep(
 `DFS: зашли в вершину ${u} ("${dagNodes[u].name",
 u,
);
 };

```

```

 setSteps(listSteps);
 setCurrentStepIdx(0);
 }, []);

const step = steps[currentStepIdx] || {
 desc: "Заняк...",
 visited: new Set(),
 fullyProcessed: new Set(),
 currentNode: null,
 topoList: [],
 dfsStack: [],
};

useEffect(() => {
 let timer: NodeJS.Timeout;
 if (isPlaying && currentStepIdx < steps.length - 1)
 timer = setInterval(() => {
 setCurrentStepIdx((prev) => prev + 1);
 }, 1600);
 } else {
 setIsPlaying(false);
 }
 return () => clearInterval(timer);
}, [isPlaying, currentStepIdx, steps.length]);

const togglePlay = () => setIsPlaying(!isPlaying);
const reset = () => {
 setIsPlaying(false);
 setCurrentStepIdx(0);
};

const getTopoListString = () => {
 return [...step.topoList]
 .reverse()
 .map((id) => dagNodes[id].name)
 .join(" → ");
};

```

```

 id="dag-arrow"
 viewBox="0 0 10 10"
 refX="24"
 refY="5"
 markerWidth="6"
 markerHeight="6"
 orient="auto-start-reverse"
 >
 <path d="M 0 1 L 10 5 L 0 9 z" fill="#4
 </marker>
 <marker
 id="dag-arrow-active"
 viewBox="0 0 10 10"
 refX="24"
 refY="5"
 markerWidth="6"
 markerHeight="6"
 orient="auto-start-reverse"
 >
 <path d="M 0 1 L 10 5 L 0 9 z" fill="#8
 </marker>
</defs>

{/* Edges */}
{dagEdges.map(({edge, idx}) => {
 const u = dagNodes.find((n) => n.id === e
 const v = dagNodes.find((n) => n.id === e

 const isActive =
 (step.currentNode === edge.u || step.cu
 step.visited.has(edge.v));

 return (
 <line
 key={`edge-${idx}`}
 x1={u.x}
 y1={u.y}
 x2={v.x}

```

```

 y={node.y - 22}
 width="110"
 height="44"
 rx="8"
 fill={fill}
 stroke={stroke}
 strokeWidth="2"
 className="transition-all duration-100"
 />
 <text
 x={node.x}
 y={node.y - 4}
 textAnchor="middle"
 fill="white"
 fontSize="11px"
 fontWeight="bold"
 className="pointer-events-none"
 >
 ({node.label}) {node.name.split(" ").join(" ")}
 </text>
 <text
 x={node.x}
 y={node.y + 12}
 textAnchor="middle"
 fill="#94a3b8"
 fontSize="9px"
 className="pointer-events-none"
 >
 {node.name.split(" ").slice(1).join(" ")}
 </text>
 </g>
);
}
}
</svg>
</div>

```

```

{/* Console column */}
<div className="lg:col-span-4 bg-slate-950 p-5"

```

```


 </div>
 <div className="flex items-center gap-2">

 Черные (готово)

 </div>
 </div>
 </div>
</div>

<div className="mt-4 pt-3 border-t border-slate-200">

 Шаг {currentStepIdx + 1} из {steps.length}

 <div className="flex gap-1">
 <button
 disabled={currentStepIdx === 0}
 onClick={() => setCurrentStepIdx((prev) => prev - 1)}
 className="bg-slate-900 border border-slate-700 px-2 py-1 text-white"
 >
 Назад
 </button>
 <button
 disabled={currentStepIdx >= steps.length - 1}
 onClick={() => setCurrentStepIdx((prev) => prev + 1)}
 className="bg-slate-900 border border-slate-700 px-2 py-1 text-white"
 >
 Вперед
 </button>
 </div>
</div>
</div>
</div>
</div>
);

```

```

 right: TNode | null;
}

export const POINTS: Pair[] = [
 { key: 45, grade: 8 },
 { key: 3, grade: 6 },
 { key: 6, grade: 5 },
 { key: 2, grade: 4 },
 { key: 9, grade: 3 },
 { key: 7, grade: 2 },
 { key: 12, grade: 0 },
 { key: 1, grade: -4 },
];

/** Порядок ввода (как в черновике) – намеренно НЕ отсортирован */
export const INPUT_ORDER: Pair[] = [
 { key: 3, grade: 5 },
 { key: 45, grade: 8 },
 { key: 3, grade: 6 },
 { key: 1, grade: -4 },
 { key: 7, grade: 2 },
 { key: 6, grade: 5 },
 { key: 2, grade: 4 },
 { key: 9, grade: 3 },
 { key: 12, grade: 0 },
];

// (3,5) дубль ключа/приоритета – уберём: ввод без повторов
export const INPUT_CLEAN: Pair[] = INPUT_ORDER.filter((p, i) => !INPUT_ORDER.some((p2, j) => j < i && p2.key === p.key));

const sortedByY = (ps: Pair[]): Pair[] => [...ps].sort((a, b) => a.grade - b.grade);

/* ----- чистые алгоритмы (экспортируются для тестов) ----- */

export function insert(root: TNode | null, p: Pair): TNode {
 if (!root) return { id: pairId(p.key, p.grade), key: p.key, grade: p.grade, left: null, right: null };
 if (p.key <= root.key) root.left = insert(root.left, p);
 else root.right = insert(root.right, p);
 return root;
}

```

```

export function mergeTree(a: TNode | null, b: TNode | null): TNode {
 const trace: MergeResult["trace"] = [];
 const go = (a: TNode | null, b: TNode | null): TNode {
 if (!a || !b) {
 trace.push({ aId: a?.id ?? null, bId: b?.id ?? null, note: `Обе ветви пустые, возвращаем целиком.` });
 return a ?? b;
 }
 if (a.grade > b.grade) {
 trace.push({ aId: a.id, bId: b.id, chosenId: a.id, note: `Выбираем a, так как его левое поддереву с (${b.key}, ${b.grade}) больше, чем его правое поддереву с (${a.key}, ${a.grade}).` });
 a.right = go(a.right, b);
 return a;
 }
 trace.push({ aId: a.id, bId: b.id, chosenId: b.id, note: `Выбираем b, так как его левое поддереву с (${a.key}, ${a.grade}) больше, чем его правое поддереву с (${b.key}, ${b.grade}).` });
 b.left = go(a, b.left);
 return b;
 };
 return { root: go(a, b), trace };
}

```

```

export function eraseNode(root: TNode | null, key: number): TNode {
 const trace: { atId: NodeId | null; note: string }[] = [];
 const go = (t: TNode | null): TNode | null => {
 if (!t) return null;
 if (key < t.key) {
 trace.push({ atId: t.id, note: `Удаляем элемент с ключом ${key}, так как он меньше, чем ${t.key}.` });
 t.left = go(t.left);
 return t;
 }
 if (key > t.key) {
 trace.push({ atId: t.id, note: `Удаляем элемент с ключом ${key}, так как он больше, чем ${t.key}.` });
 t.right = go(t.right);
 return t;
 }
 trace.push({ atId: t.id, note: `Нашли элемент с ключом ${key}, удаляем его.` });
 const m = mergeTree(t.left, t.right);
 return m;
 };
 return { root: go(root), trace };
}

```



```

 slot += 1;
 walk(n.right, depth + 1);
 };
 walk(root, 0);
 return pos;
};

const treeEdges = (root: TNode | null): { from: NodeId;
const es: { from: NodeId; to: NodeId }[] = [];
const walk = (n: TNode | null) => {
 if (!n) return;
 if (n.left) { es.push({ from: n.id, to: n.left.id });
 if (n.right) { es.push({ from: n.id, to: n.right.id });
};
walk(root);
return es;
};

interface TreeViewProps {
 root: TNode | null;
 width?: number;
 height?: number;
 nodeColor?: (id: NodeId) => string;
 nodeStroke?: (id: NodeId) => string;
 edgeColor?: (from: NodeId, to: NodeId) => string;
 edgeDash?: (from: NodeId, to: NodeId) => boolean;
 pulseId?: NodeId | null;
 subtitle?: string;
}

const TreeView: React.FC<TreeViewProps> = ({ root, width
const pos = useMemo(() => layoutTree(root), [root]);
const nodes = useMemo(() => [...pos.entries()].map(([
const edges = useMemo(() => treeEdges(root), [root]);
const autoW = Math.max(420, ...nodes.map((n) => n.px +
const autoH = Math.max(260, ...nodes.map((n) => n.py +
const w = width ?? autoW;
const h = height ?? autoH;

```

```

 const [a, b] = id.split(":").map(Number);
 return [a, b];
 };

 /* ----- плоскость (клетчатая бумага) -----

 const PL_W = 780;
 const PL_H = 300;
 const PX = (key: number) => 42 + (key - 1) * ((PL_W - 60) / 10);
 const PY = (grade: number) => 24 + (8 - grade) * ((PL_H - 10) / 8);

 interface PlaneProps {
 points: Pair[];
 drawnEdges: { a: Pair; b: Pair }[];
 pending: Pair | null;
 interactive?: boolean;
 selected?: Pair | null;
 cursor?: { x: number; y: number } | null;
 badEdge?: { a: Pair; b: Pair } | null;
 onPointDown?: (p: Pair) => void;
 onPointUp?: (p: Pair) => void;
 onMove?: (e: React.PointerEvent<SVGSVGElement>) => void;
 }

 const Plane: React.FC<PlaneProps> = ({ points, drawnEdges, pending,
 }) => {
 const byPair = (p: Pair) => ({ x: PX(p.key), y: PY(p.grade) });
 return (
 <svg
 viewBox={`0 0 ${PL_W} ${PL_H}`}
 className={`w-full ${interactive ? "touch-none select-none" : ""}`}
 onPointerMove={onMove}
 style={interactive ? { cursor: "crosshair" } : undefined}
 >
 <defs>
 <pattern id="pl-grid" width="14" height="14" patternUnits="userSpaceOnUse">
 <path d="M 14 0 L 0 0 0 14" fill="none" stroke="black" stroke-width="1"/>
 </pattern>
 </defs>

```



Универсальное пособие • полный исходный код

```

export function gameVerdict(edges: GEdge[], all: Pair[])
 const heap = edges.every((e) => e.a.grade > e.b.grade)
 const oneParent = new Set(edges.map((e) => gpid(e.b)))
 const acyclic = !edges.some((e) => reaches(edges, e.a, e.b))
 const bst = forestIsBst(edges, all);
 const roots = all.filter((p) => !edges.some((e) => gp(e.a) === p.key))
 const complete = edges.length === all.length - 1 && roots.length === 1
 return { heap, oneParent, acyclic, bst, complete, win: complete }
```

/\*\* Попытка хода: почему нельзя — или ОК. \*/

```
export function tryConnect(edges: GEdge[], a: Pair, b: Pair): boolean {
 if (sameP(a, b)) return { ok: false, reason: "Это одна точка" }
 if (a.grade === b.grade)
 return { ok: false, reason: "«!» равны — «кто выше?»" }
 const parent = a.grade > b.grade ? a : b;
 const child = a.grade > b.grade ? b : a;
 if (parentOf(edges, gpid(child)))
 return { ok: false, reason: `У точки ${child.key} уже есть родительская линия.` };
 if (reaches(edges, parent, child))
 return { ok: false, reason: "Цикл: линия замкнёт саму себя" };
 const next = [...edges, { a: parent, b: child }];
 if (!forestIsBst(next, POINTS))
 return { ok: false, reason: "Линия вниз по у допустима только если не образует цикла" };
 return { ok: true };
}
```

/\* ===== РЕЖИМ BUILD ===== \*/

```
type BuildPhase = "sort" | "connect" | "game";
type SortAlgo = "none" | "quicksort" | "counting";
type ConnectOrder = "y-desc" | "left-right";
```

```
const parsePairs = (raw: string): { pairs: Pair[]; errors: string[] } => {
 const pairs: Pair[] = [];
 const errors: string[] = [];
 const seen = new Set<string>();
```

```

 if ((pid(e.a) === pid(x) && pid(e.b) === pid(y)) ||
 return { ok: false, reason: "такая линия уже пров
 }
 if (x.grade === y.grade) return { ok: false, reason:
 " };
 const [parent, child] = x.grade > y.grade ? [x, y] :
 if (edges.some((e) => pid(e.b) === pid(child)))
 return { ok: false, reason: `у точки (${child.key});
 // цикл: родитель уже сидит в компоненте ребёнка (реб
 const childComp = new Set<string>([pid(child)]);
 let grew = true;
 while (grew) {
 grew = false;
 for (const e of edges) {
 if (childComp.has(pid(e.b)) && !childComp.has(pid
 childComp.add(pid(e.a));
 grew = true;
 }
 }
 }
 if (childComp.has(pid(parent))) return { ok: false, r
 return {
 ok: true,
 edge: { a: parent, b: child },
 reason:
 x.grade > y.grade ? undefined : "линия в treap вс
 };
}

```

```

export function checkGameState(
 points: Pair[],
 edges: GameEdge[],
): { heapOk: boolean; singleParent: boolean; connected:
 const n = points.length;
 if (n < 2) return { heapOk: true, singleParent: true,
 const heapOk = edges.every((e) => e.a.grade > e.b.gra
 const parentCount = new Map<string, number>();
 for (const e of edges) parentCount.set(pid(e.b), (par

```

```

 let left: Pair | null = null;
 let right: Pair | null = null;
 if (sorted.length === 2) {
 if (sorted[0].key === sorted[1].key || sorted[0].key === sorted[1].key) {
 bstOk = false; // двое детей с равным x или с равным y
 return;
 }
 left = sorted[0];
 right = sorted[1];
 } else if (sorted.length === 1) {
 // одиночный ребёнок: сторона важна – определяет направление
 if (sorted[0].key <= p.key) left = sorted[0];
 else right = sorted[0];
 }
 if (left) walk(left);
 inorder.push(p.key);
 if (right) walk(right);
};
walk(roots[0]);
if (seen.size !== n) bstOk = false;
for (let i = 1; i < inorder.length; i++)
 if (inorder[i] < inorder[i - 1]) bstOk = false;
}
return { heapOk, singleParent, connected, countOk, bstOk };
}

```

/\*\* Дерево из проведённых линий – для показа результата

```

export function edgesToTree(edges: GameEdge[]): TNode | null {
 const parentOf = new Map<string, GameEdge>();
 for (const e of edges) parentOf.set(pid(e.b), e);
 const roots = edges.filter((e) => !parentOf.has(pid(e.a)));
 if (!roots.length && edges.length) return null;
 const nodes = new Map<string, TNode>();
 const make = (p: Pair): TNode => ({
 id: pid(p),
 key: p.key,
 grade: p.grade,
 left: null,
 right: null,
 });
 for (const e of edges) {
 const parent = parentOf.get(pid(e.b));
 if (parent) {
 const node = make(e);
 parent.left = node;
 }
 }
 return roots.length ? make(roots[0]) : null;
}

```

```

 const walk = (node: TNode | null) => {
 if (!node) return;
 if (node.left) { es.push({ a: { key: node.key, gr
 left}); }
 if (node.right) { es.push({ a: { key: node.key, g
 de.right}); }
 };
 walk(userTree);
 return es;
 }, [userTree]);
 const leftRightEdges = useMemo(() => [...canonEdges].

 useEffect(() => {
 if (!playing || phase === "game") return;
 const t = setTimeout(() => {
 if (phase === "sort") {
 if (algo === "none") { setPlaying(false); return;
 if (sortIdx < 3) setSortIdx(sortIdx + 1);
 else setPlaying(false);
 } else {
 const total = order === "y-desc" ? canonEdges.l
 if (connIdx < total) setConnIdx(connIdx + 1);
 else setPlaying(false);
 }
 }, 900);
 return () => clearTimeout(t);
 }, [playing, phase, sortIdx, connIdx, algo, order, ca

 const startConnect = (ord: ConnectOrder) => {
 setOrder(ord);
 setPhase("connect");
 setConnIdx(0);
 setPlaying(true);
 };

 const comparisons = algo === "quicksort" ? (n > 1 ? M
 const connEdgesAll = order === "y-desc" ? canonEdges
 const connEdges = connEdgesAll.slice(0, connIdx);

```

```

 }
 };
 const pointDown = (p: Pair) => {
 if (phase !== "game") return;
 if (!sel) {
 setSel(p);
 setGameMsg(`Выбрана (${p.key}; ${p.grade}) – теперь
 } else if (sel.key === p.key && sel.grade === p.grade) {
 setSel(null);
 setGameMsg(null);
 } else {
 attemptConnect(sel, p);
 setSel(null);
 }
 };
 const pointUp = (p: Pair) => {
 if (phase !== "game" || !sel) return;
 if (sel.key === p.key && sel.grade === p.grade) return;
 attemptConnect(sel, p);
 setSel(null);
 setCursor(null);
 };
 const planeMove = (e: React.PointerEvent<SVGSVGElement>) => {
 if (!sel) return;
 const rect = (e.currentTarget as SVGSVGElement).getBoundingClientRect();
 setCursor({ x: ((e.clientX - rect.left) / rect.width) * 100 });
 };

 const undoEdge = () => {
 setGameEdges((prev) => prev.slice(0, -1));
 setGameMsg(null);
 };

 const showSolution = () => {
 setGameEdges(canonEdges.map((e) => ({ a: e.a, b: e.b })));
 setHintUsed(true);
 setGameMsg("Решение показано – собери сам в следующий раз");
 };

```



```

<h4 className="text-base font-bold text-emera
<div className="flex items-center gap-2">
 {phase === "sort" ? (
 <>
 <span className="text-xs text-slate-400
 {["quicksort", "counting"] as const }.m
 <button
 key={a}
 onClick={() => { setAlgo(a); setSor
Math.round(n * Math.log2(n)) : 0} сравнений
 disabled={n < 2}
 className={ `px-3 py-1.5 rounded-lg
600 text-white" : "bg-slate-700 text-slate-
 >
 {a === "quicksort" ? "quicksort (ср
 </button>
)}}
 </>
) : (
 <>
 <span className="text-xs text-slate-400
 <button
 onClick={() => startConnect("y-desc")
 className={ `px-3 py-1.5 rounded-lg te
bg-emerald-600 text-white" : "bg-slate-700
 >
 по y ↓ (алгоритм)
 </button>
 <button
 onClick={() => startConnect("left-right
 className={ `px-3 py-1.5 rounded-lg te
"bg-emerald-600 text-white" : "bg-slate-700
 >
 слева направо
 </button>
 <button
 onClick={startGame}
 disabled={n < 2}

```

```

) : phase === "game" ? (
 gameWon ? (
 <> Готово! {hintUsed ? "Решение подсказано": "Решение найдено"}
 ево: линии вниз по y, обход по x отсортировано
) : gameMsg ? (
 <>△ {gameMsg}</>
) : (
 <>Кликни точку-родителя (выше по «!»), за
)
) : connDone ? (
 <> Проведено {connEdges.length} линий в по
 : точки прибиты, дерево одно. А теперь
) : (
 <>Линий проведено: {connIdx} из {connEdgesA
)}
 </div>
</div>

{/* Чек-лист инвариантов */}
<div className={`rounded-xl p-4 border transition
 er-slate-700 bg-slate-900/40`}>
 <h4 className="text-sm font-bold text-slate-300">
 <ul className="space-y-1.5 text-sm">
 {phase === "game" ? (
 <>
 {verdict.heapOk ? " " : " "} каждое р
 {verdict.singleParent ? " " : " "} у
 {verdict.connected ? " " : " "} все т
 {verdict.countOk ? " " : " "} линий р
 {verdict.bstOk ? " " : " "} обход сле
 </>
) : (
 <>
 {connDone ? " " : " "} каждое ребро в
 {connDone ? " " : " "} обход слева-на
 {connDone ? " " : " "} линий ровно
 </>
)}
 </div>

```

```

const fullTree = useMemo(() => buildTree(POINTS), [])
const split = useMemo(() => splitTree(buildTree(POINTS), stepIdx))
const steps = split.trace;
const idx = Math.min(stepIdx, steps.length - 1);
const done = idx >= steps.length - 1;

const inL = new Set<NodeId>();
const inR = new Set<NodeId>();
steps.slice(0, idx + 1).forEach((s) => (s.goLeft ? inL.add(s.id) : inR.add(s.id)));

const lTree = split.l;
const rTree = split.r;

return (
 <div className="space-y-4">
 <div className="flex flex-wrap items-center gap-3">
 Разрез
 <select value={x0} onChange={(e) => { setX0(Number(e.target.value)); }}>
 <option value="2">2
 <option value="3">3
 <option value="6">6
 <option value="7">7
 <option value="9">9
 <option value="12">12
 <option value="45">45
 </select>
 <button onClick={() => setStepIdx(Math.max(0, stepIdx - 1))}>←
 <button onClick={() => setStepIdx(Math.min(stepIdx, steps.length - 1))}>→
 шаг {stepIdx + 1} из {steps.length}
 </div>

 <div className="bg-slate-900/40 rounded-xl p-4 border border-slate-700">
 <h4 className="text-sm font-bold text-slate-300">Розовые узлы
 , розовые → R
 <TreeView
 root={fullTree}
 nodeColor={(id) => (inL.has(id) ? "#1e3a8a" : "#f43f00")}
 nodeStroke={(id) => (inL.has(id) ? "#60a5fa" : "#f43f00")}
 subtitle={done ? "Каждый узел покрашен ровно один раз" : ""}
 />
 </div>
 </div>
)

```

```

<div className="space-y-4">
 <div className="flex items-center gap-3">
 <button onClick={() => setStepIdx(Math.max(0, i))}
 bg-slate-700 text-white disabled:opacity-30">
 <button onClick={() => setStepIdx(Math.min(step, i))}
 font-bold bg-indigo-600 text-white disabled:opacity-30">
 шаг {i} из {step}
 </div>

 <div className="grid grid-cols-1 md:grid-cols-2 gap-3">
 <div className="bg-sky-950/20 rounded-xl p-3 border border-slate-700">
 <h5 className="text-sm font-bold text-sky-300">Левое дерево
 <TreeView root={l} width={480} height={220} nodeRenderer={nodeRenderer} />
 </div>
 <div className="bg-rose-950/20 rounded-xl p-3 border border-slate-700">
 <h5 className="text-sm font-bold text-rose-300">Правое дерево
 <TreeView root={r} width={480} height={220} nodeRenderer={nodeRenderer} />
 </div>
 </div>

 <div className="text-xs text-slate-300 bg-slate-900 p-3 border border-slate-700">
 {cur?.chosenId ? (
 <div>⊗ {cur.note}</div>
) : (
 <div>Готово: {done ? "деревья склеены обратно в одно" : "деревья не склеены"}</div>
)}
 </div>

 {done && (
 <div className="bg-emerald-950/20 rounded-xl p-3 border border-slate-700">
 <h5 className="text-sm font-bold text-emerald-300">Слияние
 <TreeView root={merged.root} subtitle="merge({l, r})" />
 </div>
)}

 <div className="text-xs text-slate-400 bg-slate-900 p-3 border border-slate-700">
 Грамотно про «слияние»: у корней сравнить значения, если они

```

```

 {!done ? (
 <div className="bg-slate-900/40 rounded-xl p-4" >
 <TreeView
 root={fullTree}
 nodeColor={({id} => (pathSet.has(id) ? "#783"
 nodeStroke={({id} => (pathSet.has(id) ? "#f5
 subtitle="Куча сама умеет удалять только ве
 />
 </div>
) : (
 <div className="bg-slate-900/40 rounded-xl p-4" >
 <h4 className="text-sm font-bold text-slate-300" >
 <TreeView root={res.root} subtitle="Валидность" >
 <ul className="mt-2 space-y-1 text-sm" >
 обход слева-направо по-прежнему сортир
 каждое ребро вниз по y (куча целая) —

 </div>
))

 <div className="text-xs text-slate-300 bg-slate-900/40 p-2" >
 {steps[idx]?.note}
 </div>

 <div className="text-xs text-slate-400 bg-slate-900/40 p-2" >
 Ответ на «убрать можно только самый верхний
 ягивает merge двух детей. Удаляется любая<
 </div>
 </div>
);
};

/* ===== РЕЖИМ LAYERS (2k/2k+1) =====

const LayersMode: React.FC = () => {
 const [pick, setPick] = useState<"heap" | "treap">("h
 // Полное дерево из 7 узлов (куча): индексы 1..7

```

```

 <p className="text-xs text-slate-400">Полное
 → $2i/2i+1$ попадает в узел.</p>
 </div>
) : (
 <div className="bg-slate-900/40 rounded-xl p-4" >
 <h4 className="text-sm font-bold text-rose-300" >
 >
 <TreeView root={full} subtitle="Попробуй «пос
 " />
 <div className="flex gap-2 overflow-x-auto mt-2" >
 {[1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13]
 .map((i) => {
 const id = slots[i];
 return (
 <div key={i} className={`px-2.5 py-2 rounded-
 der-slate-700` : "bg-rose-950/40 border-rose-
 <div className="text-slate-500 text-[0.8em]
 {id ? <div className="text-white font-
 </div>
 } : null} />
 </div>
);
 })}
 </div>
 <p className="text-xs text-rose-300 mt-2">Смо
 то левая ветка ушла на 4 уровня и заняла да
 казателями (left/right), а не арифметикой.</p>
 </div>
)}

 <div className="text-xs text-slate-400 bg-slate-900/40
 Грабля «дети в массиве по $2k/2k+1$ »: фор
 ивым: слои дырявые, и «адрес» ребёнка посчита
 </div>
</div>
);
};

/* ===== РЕЖИМ SEARCH (поиск ≠ сортировка) =====
const SearchMode: React.FC = () => {

```

```

<div className="flex gap-2">
 {stage < 2 && {
 <button onClick={() => { setStage(stage === 0
 digo-600 text-white">
 {stage === 0 ? "Найти у для x=7 бинарным по
 </button>
 }}
 {stage === 2 && {
 <button onClick={() => setBidx(Math.min(bsStep
 -1.5 rounded-lg text-xs font-bold bg-indigo
 Шаг поиска →
 </button>
 }}
 <button onClick={() => { setStage(0); setBidx(0
 C6poc</button>
</div>

```

```

<div className="flex gap-1.5 overflow-x-auto">
 {(stage === 2 ? sortedByKey : shuffled).map((p,
 const highlight = stage === 1 && i === wrongM
 const inBs = stage === 2 && (() => { const s :
 const isMid = stage === 2 && (() => { const s :
 const isFound = stage === 2 && bidx >= bsStep
 return (
 <div key={` ${p.key}:${p.grade}-${i}`} class=
 nsition-all ${highlight ? "border-rose-500 l
 r-amber-500 bg-amber-950/30" : inBs ? "bord
 <div className="text-white font-bold">{p.
 <div className="text-slate-400 text-[10px
 </div>
);
 }}}
</div>

```

```

<div className="text-xs text-slate-300 bg-slate-9
 {stage === 0 && <>Массив пар не отсортирован
 а беспорядок.</>}
 {stage === 1 && <> Бинарный поиск посмотрел на

```





```

 {v.oneParent ? " " : " "} у каждой точки
 {v.acyclic ? " " : " "} циклов нет
 {v.bst ? " " : " "} обход слева-направо –
 {v.complete ? " " : " "} линий $n-1 = \{POI$

 {win && {
 <p className="mt-3 text-sm font-bold text-emerald-500">
 ения просто не существует.</p>
 }}
 </div>

 <div className="text-xs text-slate-300 bg-slate-900">
 </div>
);
};

/* ===== КОРПУС ===== */

type Tab = "build" | "split" | "merge" | "erase" | "layers" | "search" | "game";

const TABS: { id: Tab; label: string }[] = [
 { id: "build", label: "Собрать" },
 { id: "split", label: "Split $\times$ " },
 { id: "merge", label: "Merge " },
 { id: "erase", label: "Erase " },
 { id: "layers", label: "Миф: $2k/2k+1$ " },
 { id: "search", label: "Поиск $\neq$ сортировка" },
 { id: "game", label: "Игра " },
];

export const TreapBuildViz = () => {
 const [tab, setTab] = useState<Tab>("build");

 return (
 <div className="bg-slate-800/90 p-6 rounded-2xl border">
 <div className="flex flex-wrap items-center justify-center">
 <div className="flex items-center gap-3">
 <div className="p-2.5 bg-emerald-600 text-white">

```

```
import React from "react";

import { ArticulationPointsViz } from "./ArticulationPointsViz";
import BellmanFordViz from "./BellmanFordViz";
import BfsViz from "./BfsViz";
import { DPVisualizer } from "./DPVisualizer";
import { DfsBridgesSimulator } from "./DfsBridgesSimulator";
import DijkstraViz from "./DijkstraViz";
import { EulerSimulator } from "./EulerSimulator";
import FloydViz from "./FloydViz";
import { GraphTraversalViz } from "./GraphTraversalViz";
import { HeapViz } from "./HeapViz";
import { JohnsonViz } from "./JohnsonViz";
import { KosarajuViz } from "./KosarajuViz";
import { KruskalSimulator } from "./KruskalSimulator";
import MnemonicCards from "./MnemonicCards";
import { PlanarityDemo } from "./PlanarityDemo";
import { QueueViz } from "./QueueViz";
import { SalmonAutomatonWidget } from "./SalmonAutomatonWidget";
import { SegmentTreeVisualizer } from "./SegmentTreeVisualizer";
import SplayPlayground from "./SplayPlayground";
import { StackViz } from "./StackViz";
import { StringAlgorithmsViz } from "./StringAlgorithmsViz";
import { TopologicalSortViz } from "./TopologicalSortViz";
import { WaterfallAnimationWidget } from "./WaterfallAnimationWidget";
import ChapterImage from "./ChapterImage";
import { Simulator } from "./Simulator";
import { PrefixSumViz } from "./visualizers/PrefixSumViz";
import { SparseTableViz } from "./visualizers/SparseTableViz";
import { TreapBuildViz } from "./TreapBuildViz";

export interface VizEntry {
 /** Заголовок панели/модалки. */
 title: string;
 /** Короткое пояснение, что здесь можно потыкать. */
 hint?: string;
 Component: React.FC;
```

```

treap: {
 title: "Treap: плоскость → дерево → Split/Merge/Era
 hint: "7 вкладок: Собрать (сортировка + соединение
 обери дерево мышкой, судья-инварианты объясняет к
 Component: TreapBuildViz,
},
"prefix-sums-2d": {
 title: "Префиксные суммы (1D и 2D)",
 hint: "Наведите на число — увидите, из чего оно сло
 Component: PrefixSumViz,
},
"dynamic-programming": {
 title: "Динамическое программирование",
 hint: "Таблица подзадач заполняется на глазах.",
 Component: DPVisualizer,
},
"splay-tree": {
 title: "Splay: механика → большое дерево → собери с
 hint: "Три вкладки: покадровая механика поворотов в
 песочница «подними узел сам» с проверкой порядка.
 Component: SplayPlayground,
},
"splay-rotations": {
 title: "Splay: механика → большое дерево → собери с
 hint: "Три вкладки: покадровая механика поворотов в
 песочница «подними узел сам» с проверкой порядка.
 Component: SplayPlayground,
},
"graph-dfs-bfs": { title: "DFS и BFS на графе", Component:
"top-sort": { title: "Топологическая сортировка", Component:
"scc-kosaraju": { title: "Компоненты сильной связности", Component:
"graph-articulation": { title: "Точки сочленения", Component:
"bridges-code": { title: "Мосты: DFS + tin/low", Component:
"euler-path-vs-cycle": { title: "Эйлеров путь и цикл", Component:
"planarity-euler-formula": { title: "Планарность: K5 и K3,3", Component:
dijkstra: {
 title: "Дейкстра",
 hint: "Жадно забираем ближайшую вершину и релаксируем

```

```
 </div>
),
 },
 intro: { title: "Мнемокарточки", Component: MnemonicCard },
 "everyday-basics": {
 title: "Бытовой тренажёр: стек, очередь, куча",
 hint: "Тарелки, очередь в столовой и мешок гипотез",
 Component: Simulator,
 },
 },
};
```

```
export const getViz = (id?: string): VizEntry | undefined;
```

---

ФАЙЛ 70/83: src/components/WaterfallAnimationWidget.tsx  
КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import React, { useState } from 'react';
import { ArrowRight, CheckCircle2, AlertCircle, Lightbulb } from 'lucide-react';

// Структура заранее подготовленного красивого бора для Waterfall
// Словарь: ["HE", "SHE", "HIS", "HERS"]
interface WNode {
 id: number;
 label: string; // путь от корня
 char: string;
 x: number; // Координаты для SVG водопада
 y: number;
 depth: number;
 fail: number;
 term_link: number;
 is_terminal: boolean;
 words: string[];
}

const WATERFALL_NODES: { [key: number]: WNode } = {
```

```

const [isSearchDone, setIsSearchDone] = useState<boolean>(false);
const [searchLog, setSearchLog] = useState<string>('Режим поиска: Введите текст и жмите «Следующий шаг»');
const [jumpPath, setJumpPath] = useState<{ from: number; to: number; }>(null);
const [foundMatches, setFoundMatches] = useState<{ index: number; text: string; }[]>([]);
const [activeSearchCodeLine, setActiveSearchCodeLine] = useState<number>(1);

// === СОСТОЯНИЯ РЕЖИМА ОБУЧЕНИЯ (TRAINING) ===
// Шаги обучения от 0 до 8 (по числу вершин в боре, и т.д.)
const [trainStep, setTrainStep] = useState<number>(0);

const handleTextChange = (e: React.ChangeEvent<HTMLInputElement>) => {
 const clean = e.target.value.toUpperCase().replace(/ /g, '');
 setTextToScan(clean);
 resetSearchSimulation();
};

const resetSearchSimulation = () => {
 setCurrentIndex(0);
 setCurrentNode(0);
 setIsSearchDone(false);
 setSearchLog('Симуляция сброшена. Лосось вернулся на старт');
 setJumpPath(null);
 setFoundMatches([]);
 setActiveSearchCodeLine(1);
};

const stepSearchSimulation = () => {
 if (currentIndex >= textToScan.length) {
 setIsSearchDone(true);
 setSearchLog('Мы проплыли весь текст! Лосось доволен');
 setActiveSearchCodeLine(4);
 return;
 }

 const char = textToScan[currentIndex];
 let curr = currentNode;

```

```

 if (WATERFALL_NODES[temp].is_terminal) {
 WATERFALL_NODES[temp].words.forEach((word) :
 currentMatches.push({ index: currentIndex, word: word });
 matchedWordsForLog.push(word);
 });
 }
 temp = WATERFALL_NODES[temp].term_link;
 }

 if (matchedWordsForLog.length > 0) {
 logMsg += `    БИНГО! Найдено слово: ${matchedWordsForLog[0].word}\n`;
 setActiveSearchCodeLine(4);
 }

 setCurrentNode(curr);
 setCurrentIndex(currentIndex + 1);
 setSearchLog(logMsg);
 if (currentMatches.length > 0) {
 setFoundMatches((prev) => [...prev, ...currentMatches]);
 }
};

// ПОЛНАЯ ПОШАГОВАЯ ИНФОРМАЦИЯ О ПОСТРОЕНИИ ДЕРЕВА (В ДАННОМ ПРИМЕРЕ)
const TRAINING_STEPS_INFO = [
 {
 targetNodeId: 0,
 title: 'Шаг 0 (Depth 0): Вершина #0 (ROOT)',
 desc: 'Начало алгоритма BFS. Корень (ROOT) не обработан. Другие вершины сразу инициализируются с fail = ROOT',
 codeLine: 1,
 scoutPos: 0,
 inspectedParentFail: 0,
 checkTargetChar: 'ø',
 checkingBranchesFrom: null,
 },
 {
 targetNodeId: 1,
 title: 'Шаг 1 (Depth 1): Вершина #1 (H)',

```

```

 checkTargetChar: 'I',
 checkingBranchesFrom: 0,
},
{
 targetNodeId: 8,
 title: 'Шаг 5 (Depth 2): Вершина #8 (SH) — КАК СТАТЬ РЫБНЫМ?
 desc: ' ВАЖНЫЙ ВОПРОС! Обрабатываем #8 (SH) на Depth 2.
). Рыба плывет в ROOT и ищет ветку "H". У корня
 суффикс "H"!',
 codeLine: 4,
 scoutPos: 0,
 inspectedParentFail: 0,
 checkTargetChar: 'H',
 checkingBranchesFrom: 0,
},
{
 targetNodeId: 3,
 title: 'Шаг 6 (Depth 3): Вершина #3 (HER)',
 desc: 'Переходим на Depth 3 к #3 (HER). Родитель #3 (HER)
 е подходят. fail[HER] = ROOT.',
 codeLine: 3,
 scoutPos: 0,
 inspectedParentFail: 0,
 checkTargetChar: 'R',
 checkingBranchesFrom: 0,
},
{
 targetNodeId: 6,
 title: 'Шаг 7 (Depth 3): Вершина #6 (HIS)',
 desc: 'Обрабатываем #6 (HIS) на Depth 3. Родитель #6 (HIS)
 но ветка "S" → #7 (S) совпадает! fail[HIS] = #7 (S)',
 codeLine: 4,
 scoutPos: 0,
 inspectedParentFail: 0,
 checkTargetChar: 'S',
 checkingBranchesFrom: 0,
},
{

```

```

 Визуализация пошагового построения всех f
 </p>
 </div>
</div>

{/* Переключатель режимов */}
<div className="bg-slate-900 p-1.5 rounded-xl b
 <button
 onClick={() => setActiveMode('training')}
 className={`flex items-center space-x-1.5 p
 activeMode === 'training'
 ? 'bg-indigo-600 text-white shadow-lg s
 : 'text-slate-400 hover:text-white'
 `}
 >
 <GitBranch className="w-4 h-4" />
 Режим 1: Полное обучение (BFS от корн
 </button>
 <button
 onClick={() => setActiveMode('search')}
 className={`flex items-center space-x-1.5 p
 activeMode === 'search'
 ? 'bg-blue-600 text-white shadow-lg sha
 : 'text-slate-400 hover:text-white'
 `}
 >
 <Play className="w-4 h-4" />
 Режим 2: Поиск в тексте
 </button>
</div>
</div>

{/* ПАНЕЛЬ УПРАВЛЕНИЯ В ЗАВИСИМОСТИ ОТ РЕЖИМА */}
{activeMode === 'training' ? {
 /* ПАНЕЛЬ РЕЖИМА ОБУЧЕНИЯ */
 <div className="grid grid-cols-1 lg:grid-cols-3
 {/* Управление шагами обучения */}
 <div className="bg-slate-900/90 p-5 rounded-x

```



```

 disabled={trainStep === TRAINING_STEPS_
 className="flex-1 bg-indigo-600 hover:b
 ition-all shadow-lg shadow-indigo-900/50"
 >
 {trainStep === TRAINING_STEPS_INFO.leng
 </button>
 </div>
 </div>

```

```

 {/* Блок с кодом построения fail-ссылки и опи
 <div className="lg:col-span-2 bg-slate-900/90
 >
 <div>
 <h5 className="text-white font-bold mb-3
 <span className="flex items-center gap-
 ❌ Код BFS построения fail

 <span className="text-xs bg-indigo-950
 Вершина: #{targetTrainingNode.id} ({t

 </h5>
 <div className="bg-slate-950 p-4 rounded-
 <div className={`p-1.5 rounded flex ite
 -l-4 border-indigo-500 text-indigo-300 font
 1. let u = trie[r][char]; // Те
 {currentTrainInfo.codeLine === 1 && <
 тут}
 </div>
 <div className={`p-1.5 rounded flex ite
 -l-4 border-indigo-400 text-indigo-200 font
 2. let v = fail[r]; // Подъем к
 ainInfo.inspectedParentFail].label}}
 {currentTrainInfo.codeLine === 2 && <
 к родителю}
 </div>
 <div className={`p-1.5 rounded flex ite
 l-4 border-amber-500 text-amber-300 font-bo
 3. while (v !== root && trie[v]

```

```

<p className="text-xs text-slate-400 mb-4">
 Введи текст, чтобы лосось просканировал
</p>
<input
 type="text"
 value={textToScan}
 onChange={handleTextChange}
 maxLength={15}
 placeholder="ВВЕДИТЕ ТЕКСТ..."
 className="w-full bg-slate-800 border border-blue-500 focus:outline-none focus:border-blue-500"
/>
</div>

<div>
 <div className="mb-4 flex flex-wrap gap-1">

 {textToScan.split('').map((char, idx) =>
 <span
 key={idx}
 className={`w-7 h-8 flex items-center justify-center
 ${idx === currentIndex ? 'bg-blue-600 text-white shadow' :
 idx < currentIndex ? 'bg-emerald-950/60 text-emerald-500' :
 'bg-slate-800 text-slate-500'}
 `}
 >
 {char}

)}

 </div>
 {currentIndex >= textToScan.length && (

 ГОТОВО ✓

)}
</div>

```

```

 ем по течению}
 </div>
 <div className={`p-1.5 rounded flex item
l-4 border-emerald-500 text-emerald-300 font
 4. if (is_terminal[curr]) report
 {activeSearchCodeLine === 4 &&
овпадение!}
 </div>
 </div>
 </div>

 <div>
 <h5 className="text-slate-300 font-bold m
 <AlertCircle className="w-3.5 h-3.5 tex
 </h5>
 <div className="bg-slate-800 p-3.5 rounded
tems-center">
 {searchLog}
 </div>
 </div>
</div>
})

{/* SVG Анимация водопада */}
<div className="bg-gradient-to-b from-blue-950 vi
 -hidden shadow-2xl">
 {/* Водопадные фоновые эффекты */}
 <div className="absolute inset-0 opacity-10 bg-
 o-900 to-transparent pointer-events-none"></d
 <div className="absolute top-0 left-1/2 -transl
 <div className="w-8 bg-blue-400 h-full animat
 <div className="w-12 bg-blue-300 h-full anima
 <div className="w-6 bg-blue-500 h-full animat
 </div>

 <div className="flex items-center justify-between
 <h5 className="text-white font-bold text-base

```

```

 {level.label}
 </text>
 </g>
)))

 /* Рендер прямых переходов (потоков воды) */
 Object.entries(TRIE_TRANSITIONS).map(([fromId, toId]) => {
 const fromNode = WATERFALL_NODES[Number(fromId)];
 return Object.entries(children).map(([character, children]) => {
 const toNode = WATERFALL_NODES[toId];

 // Подсветка в режиме поиска
 let isHighlighted = activeMode === 'search' ?
 type === 'normal';

 // Подсветка в режиме обучения (когда ищем слово)
 let isTrainingExamined = activeMode === 'train' ?
 let isMatchBranch = isTrainingExamined && character === toNode.label;

 return (
 <g key={`edge-${fromNode.id}-${toNode.id}-${character}`} >
 /* Линия течения воды */
 <line
 x1={fromNode.x}
 y1={fromNode.y}
 x2={toNode.x}
 y2={toNode.y}
 stroke={isHighlighted ? '#3b82f6' : '#ccc'}
 strokeWidth={isHighlighted || isTrainingExamined ? 2 : 1}
 className={isHighlighted || isTrainingExamined ? 'highlighted' : ''}
 />
 /* Символ перехода */
 <circle cx={{fromNode.x + toNode.x} / 2} cy={{fromNode.y + toNode.y} / 2}
 r={10} fill={isTrainingExamined ? (isMatchBranch ? '#10b981' : '#f43f5a') : '#ccc'}
 />
 <text
 x={{(fromNode.x + toNode.x) / 2}}
 y={{(fromNode.y + toNode.y) / 2} + 15}
 fill={isTrainingExamined ? (isMatchBranch ? '#10b981' : '#f43f5a') : 'black'}
 font-size={12}
 />
 </g>
);
 });
 });

```

```

// Вычисляем изогнутую кривую для красоты
const dx = targetNode.x - node.x;
const dy = targetNode.y - node.y;
const cx = node.x + dx / 2 + (node.id % 2 === 0 ? -1 : 1) * 30;
const cy = node.y + dy / 2 - 30;

return (
 <g key={`fail-${node.id}`}>
 <path
 d={`M ${node.x} ${node.y} Q ${cx} ${cy} ${targetNode.x} ${targetNode.y}`}
 fill="none"
 stroke={isHighlighted ? '#f43f5e' : '#10b98f'}
 strokeWidth={isHighlighted || isJustEstablished ? 6 : 2}
 strokeDasharray="6,6"
 className={isHighlighted || isJustEstablished ? 'highlighted' : ''}
 opacity={isHighlighted || isJustEstablished ? 1 : 0.5}
 />
 {isJustEstablished && (
 <text x={cx} y={cy - 10} fill="#10b98f">
 fail[{node.label}] = {targetNode.label}
 </text>
)}
 </g>
);
}}}

```

```

{/* Рендер вершин (бассейнов водопада) */}
Object.values(WATERFALL_NODES).map((node) => {
 const isCurrent = activeFishPosNode.id === node.id;
 const isTargetChild = activeMode === 'target' && node.id === targetNode.id;
 const hasWords = node.words.length > 0;

```

```

return (
 <g key={`node-${node.id}`} className={isCurrent ? 'current' : ''}>
 {/* Внешнее свечение для активной вершины */}
 {isCurrent && (
 <circle cx={node.x} cy={node.y} r="

```

```

 { /* Индикатор словарных слов */ }
 { hasWords && {
 <g transform={`translate(${node.x +
 <circle cx="0" cy="0" r="9" fill=
 <text x="0" y="3" fontSize="10" f
 *
 </text>
 </g>
 }}
 </g>
);
}}
}}}

{ /* ОДИН ПЛАВНЫЙ АНИМИРОВАННЫЙ ЭХО-ЛОСОСЬ *
<g>
 { /* Круг-подложка под рыбу */ }
 <circle
 cx={activeFishPosNode.x + 15}
 cy={activeFishPosNode.y - 25}
 r="18"
 fill={activeMode === 'training' ? '#8b5
 stroke="#ffffff"
 strokeWidth="2"
 style={{ transition: 'cx 0.8s cubic-bez
 className="shadow-lg"
 />
 { /* Сама рыба */ }
 <text
 x={activeFishPosNode.x + 15}
 y={activeFishPosNode.y - 19}
 fontSize="18"
 textAnchor="middle"
 style={{ transition: 'x 0.8s cubic-bezi
 className="animate-float select-none"
 >
 {activeMode === 'training' ? ' ♂ ' :
 </text>
</g>

```

---

## РАЗДЕЛ: ВИЗУАЛИЗАТОРЫ (ВЛОЖЕННЫЕ)

---

ФАЙЛ 71/83: src/components/visualizers/PrefixSumViz.tsx

КАТЕГОРИЯ: Визуализаторы (вложенные) | СТРОК: 330 | РАЗМ: 1000

---

```
import React, { useMemo, useState } from "react";

/**
 * Префиксные суммы: 1D и 2D.
 *
 * Главная фишка – наведение на число:
 * * в 1D наводим на P[i] – подсвечивается кусок массива
 * * в 1D наводим на a[i] – видно, в какие префиксы он входит
 * * в 2D наводим на ячейку – подсвечивается прямоугольник
 * а при выбранном запросе показывается формула включения-исключения
 */

const A1 = [3, 1, 4, 1, 5, 9, 2, 6];

const A2 = [
 [1, 2, 3, 4],
 [5, 6, 7, 8],
 [9, 1, 2, 3],
 [4, 5, 6, 7],
];

const cellBase =
 "flex items-center justify-center rounded-md font-mono text-sm" +
 "w-[clamp(30px,9vw,46px)] h-[clamp(30px,9vw,46px)] text-center";

export const PrefixSumViz: React.FC = () => {
 const [mode, setMode] = useState<"1d" | "2d">("1d");
 return (
```

```

: hover?.kind === "a"
 ? { from: hover.i, to: hover.i }
 : null;

const sum = pref[range.r + 1] - pref[range.l];

return (
 <div className="space-y-5">
 <div className="overflow-x-auto pb-1">
 <div className="inline-block min-w-full">
 /* индексы */
 <div className="flex gap-1.5 mb-1 pl-[52px]">
 {A1.map((_, i) => (
 <div key={i} className={`${cellBase} !h-5`}
 {i}
 </div>
))}
 </div>

 /* массив a */
 <div className="flex gap-1.5 items-center mb-1">
 <div className="w-[46px] shrink-0 text-right">
 {A1.map((v, i) => {
 const inCover = covered && i >= covered.f
 const inRange = i >= range.l && i <= range.r
 return (
 <div
 key={i}
 onMouseEnter={() => setHover({ kind: "a", from: i, to: i })}
 onMouseLeave={() => setHover(null)}
 onClick={() => setRange((r) => (i < r ? i : r))}
 className={`${cellBase} cursor-pointer`}
 inCover={inCover}
 inRange={inRange}
 >
 {v}
 </div>
)
 })}
 </div>
 </div>
 </div>
 </div>
)
)

```



```

 { /* Пояснение под курсором */ }
 <div className="min-h-[62px] bg-slate-900 border
 {hover?.kind === "pref" ? {
 hover.i === 0 ? {
 <p className="text-slate-300">
 <b className="text-emerald-400">P[0] = 0
 отрезка, начинающегося с нуля.
 </p>
 } : {
 <p className="text-slate-300">
 <b className="text-emerald-400">
 P[{hover.i}] = {pref[hover.i]}
 {" "}
 – это сумма всего, что набежало от начала

 a[0..{hover.i - 1}] = {A1.slice(0, hove

 </p>
 }
 } : hover?.kind === "a" ? {
 <p className="text-slate-300">
 <b className="text-indigo-400">
 a[{hover.i}] = {A1[hover.i]}
 {" "}
 входит во все префиксы, начиная с{" "}

 подвинуть границу запроса.

 </p>
 } : {
 <p className="text-slate-500">
 Наведите курсор на любое число: у <b classN
 у <b className="text-indigo-400">a – ку
 </p>
 }
 }
 </div>

 { /* Запрос */ }

```

```

const Cc = P[rect.r2 + 1][rect.c1];
const Aa = P[rect.r2 + 1][rect.c2 + 1];
const total = Aa - B - Cc + D;

const cornerOf = (i: number, j: number) => {
 if (i === rect.r2 + 1 && j === rect.c2 + 1) return
 if (i === rect.r1 && j === rect.c2 + 1) return "B";
 if (i === rect.r2 + 1 && j === rect.c1) return "C";
 if (i === rect.r1 && j === rect.c1) return "D";
 return null;
};

return (
 <div className="space-y-5">
 <div className="grid gap-5 lg:grid-cols-2">
 {/* Исходная матрица */}
 <div className="overflow-x-auto">
 <div className="text-xs text-slate-400 mb-2">
 <div className="inline-block">
 {A2.map((row, i) => {
 <div key={i} className="flex gap-1.5 mb-1">
 {row.map((v, j) => {
 const inRect = i >= rect.r1 && i <= r
 return (
 <div
 key={j}
 onClick={() =>
 setRect((r) =>
 i < r.r1 || j < r.c1 ? { r1:
 }
)
 className={` ${cellBase} cursor-po
 inRect ? "bg-indigo-500 text-wh
 }`}
 >
 {v}
 </div>
)
 }
)
 </div>
 </div>
 </div>
 </div>
);

```

```

 }}
 </div>
);
 }}}
</div>
 }}}
</div>
</div>
</div>

<div className="min-h-[54px] bg-slate-900 border border-indigo-500"
 {hover ? {
 <p className="text-slate-300">
 <b className="text-amber-400">
 $P[\{hover.i\}][\{hover.j\}] = \{P[\{hover.i\}][\{hover.j\}] +$
 {" "}
 – сумма всего прямоугольника от левого верхнего угла до
 </p>
 } : {
 <p className="text-slate-500">Наведите курсор на элемент
 }}
</div>

<div className="bg-slate-900 border border-indigo-500"
 <p className="font-mono text-sm sm:text-base text-slate-300">
 сумма = A + <span
 C + <span
 {
 </p>
 <p className="text-[11px] text-slate-500 mt-1">
 Вычли два «хвоста» и вернули дважды вычитенный
 </p>
</div>
</div>
);
};

```

```

 [39, 81, 62, 28, 51, 42, 85, 14]
];

const ST2D: number[][][][] = Array.from({length: 8}, () =>
 Array.from({length: 8}, () =>
 Array.from({length: 4}, () =>
 Array(4).fill(0)
)
)
);

for (let r = 0; r < 8; r++) {
 for (let c = 0; c < 8; c++) {
 ST2D[r][c][0][0] = val2D[r][c];
 }
}

for (let kx = 0; kx <= 3; kx++) {
 for (let ky = 0; ky <= 3; ky++) {
 if (kx === 0 && ky === 0) continue;
 for (let r = 0; r + (1 << kx) <= 8; r++) {
 for (let c = 0; c + (1 << ky) <= 8; c++) {
 if (kx > 0) {
 ST2D[r][c][kx][ky] = Math.min(
 ST2D[r][c][kx-1][ky],
 ST2D[r + (1 << (kx-1))][c][kx-1][ky]
);
 } else {
 ST2D[r][c][kx][ky] = Math.min(
 ST2D[r][c][kx][ky-1],
 ST2D[r][c + (1 << (ky-1))][kx][ky-1]
);
 }
 }
 }
 }
}

```

```

 </div>
);
};

```

```

const Viz1D: React.FC = () => {
 const [step, setStep] = useState(0);
 const [hover, setHover] = useState<{ i: number; j: number }>({ i: 0, j: 0 });

 // Total steps: we animate computing each element for each j
 const computeSteps: { i: number; j: number }[] = [];
 for (let j = 1; j < LOG; j++) {
 for (let i = 0; i + (1 << j) <= N; i++) {
 computeSteps.push({ i, j });
 }
 }
}

```

```

const maxStep = computeSteps.length;
const covered = hover ? { from: hover.i, to: hover.i + (1 << hover.j) } : null;

```

```

return (
 <div className="flex flex-col gap-5">
 <div className="flex items-center justify-between">
 <div>
 <h3 className="text-lg font-bold text-indigo-600">
 Построение Sparse Table (Min)
 </h3>
 <p className="text-sm text-slate-400">
 Формула: $ST[i][j] = \min(ST[i][j-1], ST[i + (1 \ll (j-1))][j-1])$
 </p>
 </div>
 <div className="flex gap-2">
 <button
 onClick={() => setStep(Math.max(0, step - 1))}
 className="p-2 bg-slate-800 hover:bg-slate-700 disabled:opacity-50"
 disabled={step === 0}
 >
 <ChevronLeft size={20} />
 </button>

```

```

 <p className="text-slate-300">
 <b className="text-sky-400">
 ST[{hover.i}][{hover.j}] = {stID[hover.i]}
 {" "}
 – это минимум на отрезке{" "}

 [{covered.from}, {covered.to}]
 {" "}
 длины $2^{\{hover.j\}}$ = {1 << hover.j}:{ " "}}

 min({arrID.slice(covered.from, covered.to)}

 </p>
) : (
 <p className="text-slate-500">
 Наведите курсор (или тапните) на любое число
 </p>
)}
</div>
</div>

<div className="overflow-x-auto pb-2">
 <div className="inline-block min-w-full bg-slate-500">
 <div className="grid grid-cols-[auto_repeat(4,1fr)]">
 {/* Headers */}
 <div className="text-slate-500 font-mono text-sm">
 i \ j
 </div>
 <div className="text-center font-bold text-slate-500">
 2^0 (L=1)
 </div>
 <div className="text-center font-bold text-slate-500">
 2^1 (L=2)
 </div>
 <div className="text-center font-bold text-slate-500">
 2^2 (L=3)
 </div>
 <div className="text-center font-bold text-slate-500">
 2^3 (L=4)
 </div>
 </div>
 </div>
</div>

```

```

// We are asking if THIS cell [i][j]
if (currentStep.j - 1 === j) {
 if (currentStep.i === i) isSrc1 =
 if (currentStep.i + (1 << (current
 isSrc2 = true;
 }
}

return (
 <div key={j} className="relative fl
 {!isValid ? (
 <div className="w-[clamp(30px,9
) : (
 <motion.div
 onMouseEnter={() => isVisible
 onMouseLeave={() => setHover(
 onClick={() => isVisible && s
 initial={{ opacity: 0, scale:
 animate={{
 opacity: isVisible ? 1 : 0,
 scale: isVisible ? 1 : 0.8,
 }}
 className={`w-[clamp(30px,9vw
clamp(11px,3vw,17px)] font-bold transition-
 hover && hover.i === i && h
 ? "bg-sky-500 text-white
 : isActive
 ? "bg-indigo-500 text-whi
 : isSrc1
 ? "bg-emerald-500 text-r
 : isSrc2
 ? "bg-amber-500 text-r
 : isVisible
 ? "bg-slate-800 tex
 : "bg-transparent t
 }`}
 >
 {st1D[i][j]}

```

```

 const c = computeSteps[step - 1];
 const half = 1 << (c.j - 1);
 return (
 <p className="text-lg text-slate-300">

 ST[{c.i}][{c.j}]
 {" "}
 = min(

 ST[{c.i}][{c.j - 1}]

 ,

 ST[{c.i + half}][{c.j - 1}]

) → min(
 {st1D

 {st1D[c.i + half][c.j - 1]}

) ={" "}

 {st1D[c.i][c.j]}

 </p>
);
 })()
) : (
 <p className="text-slate-500 italic">
 Нажмите далее, чтобы увидеть, как блоки сво
 </p>
)}
 </div>
</div>
);
};

```

```

const Viz2DBuild: React.FC = () => {

```



```

return (
 <React.Fragment key={step}>
 {idx > 0 && <div className="text-slate-800">
 <div className={`flex flex-col items-center justify-center gap-4`}>
 <h4 className={`font-bold mb-3 flex items-center gap-2`}>
 Матрица блоков {stepL}x{stepH}

 {stepL}x{stepH}

 </h4>
 <div className="max-w-full overflow-hidden">
 <div className={`grid gap-[2px] p-[10px] bg-slate-800/90 shadow-[0_0_25px_rgba(99,102,241,0.5)]`} style={{width: '100%', height: '100%', gridTemplateColumns: `repeat(${stepSize}, max-content)`}}>
 {Array.from({length: stepSize * stepSize}, (_, i) => {
 const r = Math.floor(idx / stepSize);
 const c = idx % stepSize;

 let highlightClass = isCurr ? 'bg-indigo-300' : 'bg-slate-800/50 cursor-pointer';
 let zIndex = 'z-0';

 const isActive = !!activeCell && (activeCell.r === r && activeCell.c === c);

 if (isActive) {
 if (isCurr) {
 highlightClass = 'bg-indigo-300';
 zIndex = 'z-10';
 } else {
 const prevL = 1 << (k - 1);
 const isTop = r < activeCell.r;
 const isLeft = c < activeCell.c;

 if (isTop && isLeft) {
 highlightClass = 'bg-rose-300';
 md:scale-[1.08]';

```

```

 }}}}
 </div>
</div>

<div className="flex-1 min-w-0 flex flex-col gap-4" style={{flex: 1}}>
 <div className="bg-slate-900 p-4 sm:p-6 rounded" style={{background-color: '#1e293b', color: 'white'}}>
 <h3 className="text-lg font-bold text-slate-300" style={{color: 'white'}}>
 {k === 0 ? (
 <div className="text-slate-400 text-sm leading-6" style={{color: 'white'}}>
 <p className="mb-4">При
 ы имеют размер
 <div className="bg-[#0a0f1e] rounded-xl p-4" style={{background-color: '#0a0f1e', color: 'white'}}>
 // Ба
 ST
 </div>
 <p className="mt-6 italic text-indigo-400" style={{color: 'white'}}>
 <Play size={14} className="fill-indigo-400" style={{color: 'white'}}>
 </p>
 </div>
) : (
 <div className="flex flex-col gap-4 animate-in" style={{background-color: '#1e293b', color: 'white'}}>
 <div className="bg-[#0a0f1e] rounded-xl p-4" style={{background-color: '#0a0f1e', color: 'white'}}>
 // Те
 int<
 ="text-amber-300">1}}

 // Кв
 ST

 e="text-amber-300">1], <span

 ext-amber-300">1][k-<span className=
 pan>

 <span className="text-emer
 ="text-amber-300">1][k-<span classNa
 span>

 <span className="text-ambe
 white">c + pL][k-<span classNa="te

```

```

 <span className="text
>{ST2D[activeCell.r + prevL][activeCell.c][
 </div>
 <div className="flex ite
 <span className="flex
shadow-[0_0_8px_#f59e0b]"></div> ST[{active
 <span className="text
[activeCell.r + prevL][activeCell.c + prevL
 </div>
 </>
)
 }}{()}}
</div>

 <div className="pt-3 mt-3 border-t
) = <span className="text-white
0">{ST2D[activeCell.r][activeCell.c][k][k]}
 </div>
 <p className="mt-4 text-[12px] font
список матриц (слева) вниз, чтобы увидеть,
 </div>
) : {
 <div className="bg-slate-950/40 border
mt-2 flex items-center justify-center anim
 Наведите курсор на матрицу {L}x{L
 </div>
 }}
</div>
 }}
</div>
</div>
</div>
);
};

```

```

const Viz2DQuery: React.FC = () => {
 const [r1, setR1] = useState(1);
 const [c1, setC1] = useState(1);

```

```

 className="absolute border-[3px
ow-[0_0_15px_rgba(244,63,94,0.3)] border-da
style={{
 top: `${(r1 / 8) * 100}%`,
 left: `${(c1 / 8) * 100}%`,
 marginTop: '8px',
 marginLeft: '8px',
 height: 'calc(' + (H/8*100) +
 width: 'calc(' + (W/8*100) +
 }}
/>

```

```

 /* Показываем 4 угла в самой матрице
 <div className="absolute pointer-events-none
shadow-[0_0_10px_#f43f5e]" style={{ top: `calc(
lc(`${(Ly / 8) * 100}% - 16px)`, height: `calc(
 <div className="absolute pointer-events-none
shadow-[0_0_10px_#3b82f6]" style={{ top: `calc(
width: `calc(`${(Ly / 8) * 100}% - 16px)`, height:
 <div className="absolute pointer-events-none
ld-400 shadow-[0_0_10px_#10b981]" style={{ top: `calc(
8px)`, width: `calc(`${(Ly / 8) * 100}% - 16px)`, height:
 <div className="absolute pointer-events-none
00 shadow-[0_0_10px_#f59e0b]" style={{ top: `calc(
00}% + 8px)`, width: `calc(`${(Ly / 8) * 100}% - 16px)`, height:
 </div>

```

```

 <div className="bg-slate-950 p-4 rounded-2xl
er-slate-800 shadow-[inset_0_2px_10px_rgba(0,0,0,0.5)]"
 <label className="flex items-center justify-between"
 Параметры
 <div className="flex gap-2">
 <input type="number" min={0} max={100} value={r1} />
14 bg-slate-900 border border-slate-700 py-2 px-2
 <input type="number" min={0} max={100} value={c1} />
14 bg-slate-900 border border-slate-700 py-2 px-2
 </div>
 </label>

```

```

 // 2.
 int<

В запроса)

r1][

во, чтобы правый край уперся в c2)

r1][

px, чтобы нижний край уперся в r2)

nded">r2 - Lx + 1][

и влево от c2)

">r2 - Lx + 1][
 {'}}''));
 </div>

 <div className="w-full flex justify-center">
 <div className="flex flex-col items-center">
 <h4 className="text-slate-400 font-bold">
 Откуда берутся эти 4 числа: матрица
border-slate-700 shadow-sm">ST[][][{{kx}}][{{ky}}]
 </h4>
 <div className="grid gap-[2px] p-2.5">
Columns: `repeat(${matCols}, 1fr)`>
 {Array.from({length: matRows * matCols}, (v, idx) => {
 const r = Math.floor(idx / matCols);
 const c = idx % matCols;
 let isTL = r === r1 && c === c1;
 let isTR = r === r1 && c === c2;

```

```
 span>

 + 1][c1][kx][ky], ST2D[r2 - Lx + 1][c2 - Ly]
 </div>
 </div>
 </div>
 </div>
)
}
```

---

## РАЗДЕЛ: HTML-РАЗМЕТКА (LAYOUT)

---

---

ФАЙЛ 73/83: src/layout/footer.html

КАТЕГОРИЯ: HTML-разметка (layout) | СТРОВОК: 137 | РАЗМЕР

---

```
 </div>
 </main>
</div>

<script>
 function setMode(mode) {
 document.getElementById('btn-mode-normal').className =
 ? 'px-3 py-1 rounded text-sm font-bold text-white'
 : 'px-3 py-1 rounded text-sm font-bold text-black';

 document.getElementById('btn-mode-wtf').className =
 ? 'px-3 py-1 rounded text-sm font-bold text-white'
 : 'px-3 py-1 rounded text-sm font-bold text-black';
 }
}
```

```

 document.getElementById('btn-theme-dark').classList.add('text-white transition-colors' : 'px-3 py-1 rounded transition-colors');
 document.getElementById('btn-theme-light').classList.add('text-white transition-colors' : 'px-3 py-1 rounded transition-colors');
 document.getElementById('btn-theme-notebook').classList.add('text-white transition-colors' : 'px-3 py-1 rounded transition-colors');
 }

 // Initialize theme and mode
 setTheme('dark');

 // Setup intersection observer for TOC highlighting
 document.addEventListener('DOMContentLoaded', () => {
 const mobileMenuBtn = document.getElementById('mobile-menu-btn');
 const mobileDrawer = document.getElementById('mobile-drawer');
 const closeDrawerBtn = document.getElementById('close-drawer');
 const drawerOverlay = document.getElementById('drawer-overlay');

 function toggleDrawer() {
 const isClosed = mobileDrawer.classList.contains('hidden');
 if (isClosed) {
 mobileDrawer.classList.remove('hidden');
 if (drawerOverlay) {
 drawerOverlay.classList.remove('hidden');
 // Timeout allows display:block to take effect
 setTimeout(() => drawerOverlay.classList.remove('hidden'), 10);
 }
 document.body.style.overflow = "hidden";
 } else {
 mobileDrawer.classList.add('hidden');
 if (drawerOverlay) {
 drawerOverlay.classList.add('hidden');
 setTimeout(() => drawerOverlay.classList.add('hidden'), 10);
 }
 document.body.style.overflow = "auto";
 }
 }

 mobileMenuBtn.addEventListener('click', toggleDrawer);
 closeDrawerBtn.addEventListener('click', toggleDrawer);
 drawerOverlay.addEventListener('click', toggleDrawer);
 });

```

```
 link.classList.remove('link-active');
 }
 });
 }, observerOptions);

 document.querySelectorAll('section[id^="question-"]')
 .forEach(section => observer.observe(section));
});
</script>
</body>
</html>
```

---

ФАЙЛ 74/83: src/layout/header.html

КАТЕГОРИЯ: HTML-разметка (layout) | СТРОК: 1169 | РАЗМЕР: 10.5 КБ

---

```
<!DOCTYPE html>
<html lang="ru" class="scroll-smooth">
<head>
 <meta charset="UTF-8">
 <meta name="viewport" content="width=device-width, initial-scale=1.0">
 <title>Пособие по алгебре: От пиццы к предикатам</title>
 <script src="https://cdn.tailwindcss.com"></script>
 <script src="https://polyfill.io/v3/polyfill.min.js"></script>
 <script id="MathJax-script" async src="https://cdn.jsdelivr.net/npm/mathjax@3.2.2/es5/tex-mml-chtml.js"></script>
 <script>
 window.MathJax = {
 tex: {
 inlineMath: [['$', '$'], ['\\(', '\\)']],
 displayMath: [['$$', '$$'], ['\\[', '\\]']],
 processEscapes: true,
 },
 };
 </script>
```



```

 overflow-y: hidden;
 max-width: 100%;
 scrollbar-width: none;
 -ms-overflow-style: none;
 -webkit-overflow-scrolling: touch;
 }
 mjsx-container[display="true"]::-webkit-scrollbar
 display: none;
 }

 @media (max-width: 640px) {
 html { font-size: 13px; }

 .uno-card { width: 40px; height: 60px; font-size: 12px; }
 .uno-card::before, .uno-card::after { font-size: 12px; }
 .uno-card::before { top: 1px; left: 2px; }
 .uno-card::after { bottom: 1px; right: 2px; }

 .uno-card.mini { width: 28px !important; height: 28px !important; font-size: 4px; }
 .uno-card.mini::before, .uno-card.mini::after { font-size: 4px; }

 .uno-equation .text-2xl { font-size: 1.2rem; }
 .uno-equation.gap-4 { gap: 0.5rem !important; }

 /* Allow horizontal scroll for equations in containers
 .formula-scroll { flex-wrap: nowrap !important; }
 .formula-scroll { width: 100%; height: 100%; }
 }

 .uno-inner {
 position: absolute; width: 110%; height: 75%;
 border-radius: 50%; transform: rotate(-30deg);
 box-shadow: inset 0 0 5px rgba(0,0,0,0.3);
 }

 .uno-val { position: relative; z-index: 2; }
 .card-red { background-color: #e52521; } .card-blue { background-color: #004dcf; } .card-green { background-color: #339e35; } .card-yellow { background-color: #f1c40f; }

```

```

 late-700 {
 background-color: rgba(255, 255, 255, 0.7)
 border-color: #cbd5e1 !important;
 box-shadow: 2px 2px 5px rgba(0,0,0,0.05);
 }
body.theme-notebook .text-slate-200, body.theme-
 ant; }
body.theme-notebook .text-slate-400 { color: #5
body.theme-notebook .border-slate-600, body.the
 : #94a3b8 !important; }
body.theme-notebook .font-mono { font-family: "
body.theme-notebook #top-controls { background-

/* CREEPY BLINKING */
@keyframes creepy-blink {
 0%, 95%, 98%, 100% { transform: scaleY(1);
 96.5% { transform: scaleY(0.1); }
}
.creepy-eye {
 transform-origin: center;
 transform-box: fill-box;
 animation: creepy-blink 4s infinite;
}
.creepy-eye-alt {
 transform-origin: center;
 transform-box: fill-box;
 animation: creepy-blink 5.2s infinite;
 animation-delay: 1.5s;
}
.creepy-emoji {
 display: inline-block;
 transform-origin: center;
 animation: creepy-blink 3.8s infinite;
}
</style>
</head>
<body class="bg-slate-900 text-slate-200 font-sans lead

```

```

 <path d="M0,0 L6,3 L0,6 Z" fill="#2dd4b1" stroke="#2dd4b1" stroke-width="1" />
 </marker>
 <marker id="arrow-indigo" markerWidth="6" markerHeight="6" viewBox="0 0 6 6" orient="auto" />
 <path d="M0,0 L6,3 L0,6 Z" fill="#818cf8" stroke="#818cf8" stroke-width="1" />
 </marker>
 <marker id="arrow-lime" markerWidth="6" markerHeight="6" viewBox="0 0 6 6" orient="auto" />
 <path d="M0,0 L6,3 L0,6 Z" fill="#a3e63d" stroke="#a3e63d" stroke-width="1" />
 </marker>
 <marker id="arrow-green" markerWidth="6" markerHeight="6" viewBox="0 0 6 6" orient="auto" />
 <path d="M0,0 L6,3 L0,6 Z" fill="#4ade80" stroke="#4ade80" stroke-width="1" />
 </marker>

 <!-- ЛЕГО БЛОКИ -->
 <g id="lego-yellow">
 <rect x="0" y="0" width="40" height="20" fill="yellow" />
 <rect x="6" y="-5" width="8" height="6" fill="yellow" />
 <rect x="26" y="-5" width="8" height="6" fill="yellow" />
 <line x1="2" y1="2" x2="38" y2="2" stroke="black" stroke-width="1" />
 </g>
 <g id="lego-red">
 <rect x="0" y="0" width="40" height="20" fill="red" />
 <rect x="6" y="-5" width="8" height="6" fill="red" />
 <rect x="26" y="-5" width="8" height="6" fill="red" />
 <line x1="2" y1="2" x2="38" y2="2" stroke="black" stroke-width="1" />
 </g>
 <g id="lego-rem">
 <rect x="0" y="0" width="40" height="10" fill="red" />
 <rect x="6" y="-5" width="8" height="6" fill="red" />
 <rect x="26" y="-5" width="8" height="6" fill="red" />
 <line x1="2" y1="2" x2="38" y2="2" stroke="black" stroke-width="1" />
 </g>
 <!-- МЯСОРУБКА -->
 <g id="meat-grinder">
 <path d="M 10 40 L 15 20 Q 25 15 35 20" fill="black" stroke="black" stroke-width="1" />
 <rect x="5" y="40" width="40" height="20" fill="black" stroke="black" stroke-width="1" />
 <!-- Ручка -->
 <path d="M 45 50 Q 60 50 60 30 L 75 30" fill="black" stroke="black" stroke-width="1" />
 <circle cx="75" cy="30" r="4" fill="black" stroke="black" stroke-width="1" />
 </g>

```

```
 <h3 class="font-black text-transparent text-sm">
 Содержание
 </h3>
 <button id="close-drawer-btn" class="
 <svg xmlns="http://www.w3.org/20
 <path stroke-linecap="round
 </svg>
 </button>
 </div>
 <nav class="space-y-3 text-sm font-medium">

 <details class="group" open>
 <summary class="list-none cursor-pointer text-blue-500 pl-3 font-bold text-blue-300">
 1. Алгебраические структуры

 </summary>
 <div class="pl-6 space-y-2 text-sm">

 </div>
 </details>

 <details class="group">
 <summary class="list-none cursor-pointer text-indigo-500 pl-3 font-bold text-indigo-300">
 2. Комплексные числа

 </summary>
 <div class="pl-6 space-y-2 text-sm">


```

```


 </summary>
 <div class="pl-6 space-y-2 text
 <a href="#q5-def" class="block
 <a href="#q5-group" class="bloc
 <a href="#q5-prim" class="block
 <a href="#q5-crit" class="block
 <a href="#q5-cyclic" class="blo
 </div>
 </details>

 <details class="group">
 <summary class="list-none curso
 <a href="#question-6" class:
er-lime-500 pl-3 font-bold text-lime-300">
 6. Делимость, НОД и Евклид

 </summary>
 <div class="pl-6 space-y-2 text
 <a href="#q6-1" class="block ho
 <a href="#q6-2" class="block ho
 <a href="#q6-3" class="block ho
 <a href="#q6-4" class="block ho
 <a href="#q6-5" class="block ho
 <a href="#q6-6" class="block ho
 </div>
 </details>

 <details class="group">
 <summary class="list-none curso
 <a href="#question-7" class:
er-pink-500 pl-3 font-bold text-pink-300">
 7. Взаимно простые числ

 </summary>
 <div class="pl-6 space-y-2 text
 <a href="#q7-1" class="bloc
```

```
</details>

<details class="group">
 <summary class="list-none cursor-pointer text-rose-500 pl-3 font-bold text-rose-400">
 10. Делимость и Схема Г

</summary>
<div class="pl-6 space-y-2 text-rose-400">

a>

 </div>
</details>

<details class="group">
 <summary class="list-none cursor-pointer text-cyan-500 pl-3 font-bold text-cyan-400">
 11. НОД и НОК многочленов

</summary>
<div class="pl-6 space-y-2 text-cyan-400">

нственности НОД

ПИРАЛЬ)

вращается!)
 </div>
</details>

<details class="group">
```

```


 deg)

 ition-colors border border-rose-500/30"> K
 </div>
 </details>

 <details class="group">
 <summary class="list-none cursor-pointer">

 der-cyan-500 pl-3 font-bold text-cyan-400">
 15. Производная. Критер

 </summary>
 <div class="pl-6 mt-2 space-y-2">

 </div>
 </details>

 <details class="group">
 <summary class="list-none cursor-pointer">

 der-red-500 pl-3 font-bold text-red-400">
 16. Неприводимые многоч

 </summary>
 <div class="pl-6 mt-2 space-y-2">

 </div>
 </details>

 <div class="mt-4 mb-2 text-xs font-l
 <a href="#question-17" class="block
```

```

 });
 }
 });
});

// Scroll spy logic
document.addEventListener('DOMContentLoaded', () => {
 const sections = document.querySelectorAll(
 'h2, h3, h4, h5, h6, h7, h8, h9, h10, h11, h12, h13, h14, h15, h16, h17, h18, h19, h20, h21, h22, h23, h24, h25, h26, h27, h28, h29, h30, h31, h32, h33, h34, h35, h36, h37, h38, h39, h40, h41, h42, h43, h44, h45, h46, h47, h48, h49, h50, h51, h52, h53, h54, h55, h56, h57, h58, h59, h60, h61, h62, h63, h64, h65, h66, h67, h68, h69, h70, h71, h72, h73, h74, h75, h76, h77, h78, h79, h80, h81, h82, h83, h84, h85, h86, h87, h88, h89, h90, h91, h92, h93, h94, h95, h96, h97, h98, h99, h100'
);
 const navLinks = document.querySelectorAll('a[href="#q5-"], div[id^="q6-"], div[id^="q7-"], div[id^="q8-"], div[id^="q9-"], div[id^="q10-"], div[id^="q11-"], div[id^="q12-"], div[id^="q13-"], div[id^="q14-"], div[id^="q15-"], div[id^="q16-"], div[id^="q17-"], div[id^="q18-"], div[id^="q19-"], div[id^="q20-"], div[id^="q21-"], div[id^="q22-"], div[id^="q23-"], div[id^="q24-"], div[id^="q25-"], div[id^="q26-"], div[id^="q27-"], div[id^="q28-"], div[id^="q29-"], div[id^="q30-"], div[id^="q31-"], div[id^="q32-"], div[id^="q33-"], div[id^="q34-"], div[id^="q35-"], div[id^="q36-"], div[id^="q37-"], div[id^="q38-"], div[id^="q39-"], div[id^="q40-"], div[id^="q41-"], div[id^="q42-"], div[id^="q43-"], div[id^="q44-"], div[id^="q45-"], div[id^="q46-"], div[id^="q47-"], div[id^="q48-"], div[id^="q49-"], div[id^="q50-"], div[id^="q51-"], div[id^="q52-"], div[id^="q53-"], div[id^="q54-"], div[id^="q55-"], div[id^="q56-"], div[id^="q57-"], div[id^="q58-"], div[id^="q59-"], div[id^="q60-"], div[id^="q61-"], div[id^="q62-"], div[id^="q63-"], div[id^="q64-"], div[id^="q65-"], div[id^="q66-"], div[id^="q67-"], div[id^="q68-"], div[id^="q69-"], div[id^="q70-"], div[id^="q71-"], div[id^="q72-"], div[id^="q73-"], div[id^="q74-"], div[id^="q75-"], div[id^="q76-"], div[id^="q77-"], div[id^="q78-"], div[id^="q79-"], div[id^="q80-"], div[id^="q81-"], div[id^="q82-"], div[id^="q83-"], div[id^="q84-"], div[id^="q85-"], div[id^="q86-"], div[id^="q87-"], div[id^="q88-"], div[id^="q89-"], div[id^="q90-"], div[id^="q91-"], div[id^="q92-"], div[id^="q93-"], div[id^="q94-"], div[id^="q95-"], div[id^="q96-"], div[id^="q97-"], div[id^="q98-"], div[id^="q99-"], div[id^="q100-"]');
 let isScrolling = false;

 document.querySelectorAll('.group').forEach((group) => {
 group.addEventListener('click', (e) => {
 isScrolling = true;
 setTimeout(() => isScrolling = false, 1000);
 });
 });

 const observer = new IntersectionObserver((entries) => {
 if (isScrolling) return;

 let visibleId = null;
 entries.forEach(entry => {
 if (entry.isIntersecting) {
 visibleId = entry.target.id;
 }
 });

 // Highlight active link
 document.querySelectorAll('a').forEach((a) => {
 a.style.fontWeight = 'normal';
 if (a.href.endsWith(visibleId)) {
 a.style.fontWeight = 'bold';
 }
 });

 // Open the parent link
 const correspondingLink = document.querySelector(`a[href="#${visibleId}"]`);
 if (correspondingLink) {
 correspondingLink.click();
 }
 });
});

```



```
box.style.width = '80%';
box.style.height = '80%';
box.style.backgroundColor = '#1a1a1a';
box.style.padding = '20px';
box.style.borderRadius = '10px';
box.style.display = 'flex';
box.style.flexDirection = 'column';
box.style.color = '#fff';

const header = document.createElement('h2');
header.innerText = 'Экспорт в PDF';
header.style.marginBottom = '10px';

const subHeader = document.createElement('h3');
subHeader.innerText = 'Из-за особенностей браузеров, экспорт в PDF осуществляется в новой вкладке (через меню)';
subHeader.style.marginBottom = '10px';
subHeader.style.color = '#f87171';
subHeader.style.fontSize = '0.9em';

const textarea = document.createElement('div');
textarea.value = contentRaw;
textarea.style.flex = '1';
textarea.style.width = '100%';
textarea.style.color = '#000';
textarea.style.padding = '10px';
textarea.style.fontFamily = 'monospace';

const btnContainer = document.createElement('div');
btnContainer.style.display = 'flex';
btnContainer.style.gap = '10px';
btnContainer.style.marginTop = '10px';

const copyBtn = document.createElement('button');
copyBtn.innerText = 'Копировать';
copyBtn.style.padding = '10px';
copyBtn.style.backgroundColor = '#fff';
copyBtn.style.color = '#fff';
```

```

 let clone = document.querySelector('#clone');

 // Smart markdown parsing based on marked.js
 // Smart markdown parsing based on marked.js
 clone.querySelectorAll('h2').forEach(el => {
 clone.querySelectorAll('h3').forEach(el => {
 clone.querySelectorAll('.analog').forEach(el => {
 clone.querySelectorAll('li').forEach(el => {
 clone.querySelectorAll('svg').forEach(el => {
 let caption = '';
 if (el.nextElementSibling && el.nextElementSibling.tagName === 'CAPTION') {
 caption = el.nextElementSibling.textContent;
 }
 el.textContent = '\n\n[Изображение: ' + caption + ']';
 });
 clone.querySelectorAll('.img-caption').forEach(el => {
 let text = clone.innerHTML;
 text = text.replace(/\n{3,}/g, '\n\n');

 if (window !== window.top) {
 fallbackModal(text, 'Markdown');
 return;
 }
 const blob = new Blob([text], { type: 'text/html' });
 const url = URL.createObjectURL(blob);
 const link = document.createElement('a');
 link.href = url;
 link.download = 'vyisshaya_algebra.html';
 document.body.appendChild(link);
 link.click();
 document.body.removeChild(link);
 });
 }

 function setTheme(theme) {
 const body = document.body;
 }

```

```

body.bg-white .info-block {
 background-color: #94a3b8;
 padding: 10px;
}

body.bg-white .analogy-block {
 background-color: #94a3b8;
 padding: 10px;
}

body.bg-white .img-caption {
 background-color: #94a3b8;
 padding: 10px;
}

/* Tweak card blocks */
body.bg-white .card-red {
 background-color: #f08080;
 padding: 10px;
}

body.bg-white aside {
 background-color: #334155;
 color: white;
 padding: 10px;
}

body.bg-white .text-white {
 color: white;
}

/* Ensure active links are white */
body.bg-white aside a.active {
 color: white;
}
body.bg-white aside a.active {
 color: white;
}
body.bg-white aside a.active {
 color: white;
}
body.bg-white aside a.active {
 color: white;
}
body.bg-white aside a.active {
 color: white;
}
body.bg-white aside a.active {
 color: white;
}
body.bg-white aside a:active {
 color: white;
}

body.bg-white table tr {
 background-color: #f0f0f0;
}
body.bg-white table tr {
 background-color: #f0f0f0;
}
body.bg-white .text-teal {
 color: #20b2aa;
}
body.bg-white .text-rose {
 color: #e91e63;
}
body.bg-white .text-emerald {
 color: #2e8b57;
}
body.bg-white .text-cyan {
 color: #00bfff;
}
body.bg-white .text-amber {
 color: #ffc107;
}
body.bg-white .text-fuchsia {
 color: #ff00ff;
}
body.bg-white .text-lime {
 color: #90ee90;
}
body.bg-white .text-blue {
 color: #1e90ff;
}
body.bg-white .text-green {
 color: #32cd32;
}
body.bg-white .text-indigo {
 color: #673ab7;
}

`
;
} else if (theme === 'terminal') {
 body.classList.add('bg-black');

```

```

 <button onclick="downloadMD()"
- bold flex items-center justify-center gap-1
 Markdown
 </button>
 <button onclick="downloadHTML()"
t-sm font-bold flex items-center justify-center
 HTML
 </button>
 </div>

 <!-- Смена темы -->
 <div>
 <div class="text-xs text-slate-500">
 <div class="flex items-center gap-2">
 <button id="theme-dark" onclick="toggleTheme('dark')"
justify-center text-amber-300 hover:text-white
line-amber-500" title="Темная тема"> </button>
 <button id="theme-light" onclick="toggleTheme('light')"
r justify-center text-amber-50 hover:text-white
">* </button>
 <button id="theme-terminal"
- center justify-center text-green-400 hover:text-green-500
терминала"> </button>
 </div>
 </div>
 </div>
</aside>

<!-- ОСНОВНОЙ КОНТЕНТ -->
<main class="flex-1 bg-slate-800 p-4 sm:p-8 md:p-12"
- base min-w-0">
 <header class="text-center mb-12 border-b border-slate-700">
 <h1 class="text-4xl md:text-5xl font-bl
4">
 ВЫСШАЯ АЛГЕБРА
 </h1>

```

```

 <!-- Column headers -->
 <div class="bg-yellow-900/40 bo
оля и кольца, целые числа</div>
 <div class="bg-orange-900/40 bo
ольцо полиномов</div>
 <div class="bg-cyan-900/40 bord
сные числа</div>

 <!-- Row 1: База -->
 <a href="#question-1" onclick="setM
-4 border-yellow-500 hover:bg-slate-700 tra
 <div class="text-xs text-yellow
 <div class="font-bold text-slate

 <a href="#question-9" onclick="setM
-4 border-orange-500 hover:bg-slate-700 tra
 <div class="text-xs text-orange
 <div class="font-bold text-slate

 <a href="#question-2" onclick="setM
-4 border-cyan-500 hover:bg-slate-700 trans
 <div class="text-xs text-cyan-5
 <div class="font-bold text-slate

 <!-- Row 2: Базовые операции / Евклид -->
 <a href="#question-6" onclick="setM
-4 border-yellow-500 hover:bg-slate-700 tra
 <div class="text-xs text-yellow
 <div class="font-bold text-slate

 <a href="#question-10" onclick="setM
-1-4 border-orange-500 hover:bg-slate-700 t
 <div class="text-xs text-orange
 <div class="font-bold text-slate

 <a href="#question-3" onclick="setM
-4 border-cyan-500 hover:bg-slate-700 trans

```

```

- l-4 border-orange-500 hover:bg-slate-700 t
 <div class="text-xs text-orange
 <div class="font-bold text-slate

 <a href="#question-4" onclick="setM
- 4 border-cyan-500 hover:bg-slate-700 trans.
 <div class="text-xs text-cyan-5
 <div class="font-bold text-slate

 <!-- Row 6: Кратность -->
 <div class="hidden md:block"></div>
 <a href="#question-14" onclick="setl
- l-4 border-orange-500 hover:bg-slate-700 t
 <div class="text-xs text-orange
 <div class="font-bold text-slate

 <div class="hidden md:block"></div>

 <!-- Row 7: Производная -->
 <div class="hidden md:block"></div>
 <a href="#question-15" onclick="setl
- l-4 border-orange-500 hover:bg-slate-700 t
 <div class="text-xs text-orange
 <div class="font-bold text-slate

 <div class="hidden md:block"></div>

 <!-- КЛАСТЕР 3: НЕПРИВОДИМОСТЬ -->
 <div class="col-span-1 md:col-span-1
b-2 border-b border-emerald-500/30 pb-2">Кл

 <a href="#question-17" onclick="setl
- l-4 border-teal-500 hover:bg-slate-700 tra
 <div class="text-xs text-teal-5
 <div class="font-bold text-slate

 <a href="#question-18" onclick="setl

```

```
<!-- ЛИКБЕЗ SECTION INSERTED HERE -->
<section id="proof-algorithms" class="m
 <div class="flex items-center mb-6">
 <div class="bg-red-500/20 p-3 r
 <svg class="w-8 h-8 text-re
rg/2000/svg">
 <path stroke-linecap="r
110 4m-6 8a2 2 0 100-4m0 4a2 2 0 110-4m0 4
 </svg>
 </div>
 <h2 class="text-3xl pr-4 border
 </div>

 <div class="prose prose-invert max-
 <p class="text-lg">
 Хватит тупить. Доказательств
есть для того, чтобы жать на кнопки. Вот жес
 </p>

 <h2 class="text-2xl font-bold b
(Меметика)</h2>

 <!-- Прямое -->
 <div class="bg-slate-800/80 p-6
 <h3 class="text-xl text-blur
 <p class="text-sm text-slate
 <ul class="list-disc pl-5 sp
 <span class="text-w
k$).

 <span class="text-w
 <span class="text-w
ово.

 </div>

 <!-- От противного -->
 <div class="bg-slate-800/80 p-6
 <h3 class="text-xl text-fuc
```

```
<!-- Двойная делимость -->
<div class="bg-slate-800 p-6" style="background-color: #2d3748; color: white; padding: 10px;">
 <h3 class="text-xl text-orange-400">Двойная делимость
 <p class="text-sm text-slate-200">
 <ul class="list-disc pl-5" style="list-style-type: none;">

 вверх по уравнениям).

 рху вниз, показывая, что $\Delta \vdots r_i$

 х. Твой d больше.

 </div>
```

```
<h2 class="text-2xl font-bold text-slate-200">Алгоритмы экзекуции</h2>
```

```
<div class="grid grid-cols-1 md:grid-cols-2" style="background-color: #2d3748; color: white; padding: 10px;">
 <div class="bg-slate-800 p-6" style="background-color: #2d3748; color: white; padding: 10px;">
 <h4 class="text-red-400">Алгоритм 1
 <p class="text-sm text-slate-200">
 <p class="text-xs font-sans text-slate-200">
 со следующим. Если в конце 0 – пациент мертв
 </div>
```

```
<div class="bg-slate-800 p-6" style="background-color: #2d3748; color: white; padding: 10px;">
 <h4 class="text-red-400">Алгоритм 2
 <p class="text-sm text-slate-200">
 <p class="text-xs font-sans text-slate-200">
 c . Пока получается 0 – корень жив. Как то
 корня минус один.</p>
 </div>
```

```
<div class="bg-slate-800 p-6" style="background-color: #2d3748; color: white; padding: 10px;">
 <h4 class="text-red-400">Алгоритм 3
 <p class="text-sm text-slate-200">
 <p class="text-xs font-sans text-slate-200">
```



```
 <li class="bg-slate-800/50 p-4">

 <div>
 <h4 class="text-cyan">
 <p class="text-sm">
 есть сложение – есть ноль. Если есть умножение
 <code class="text-x">
 </div>

 <li class="bg-slate-800/50 p-4">

 <div>
 <h4 class="text-cyan">
 <p class="text-sm">
 авь их драться. Умножение всегда врывается
 <code class="text-x">
 </div>

 </div>
 </section>
 </div>

 <div id="questions-container">
 <!-- ===== ВОПРОС 1 =====>
```

---

РАЗДЕЛ: УТИЛИТЫ

---

```

 if (!href) continue;
 try {
 const res = await fetch(href);
 if (res.ok) parts.push(await res.text());
 } catch {
 /* чужой домен – пропускаем */
 }
 }
 return parts.join("\n");
}

const EXTRA_CSS = `
/* – правки для автономного файла – */
html { color-scheme: dark; }
body { background: #020617; color: #e2e8f0; font-family:
a.term-hint { text-decoration: none; }
.export-shell { max-width: 62rem; margin: 0 auto; padding:
.export-head { border-bottom: 1px solid #1e293b; padding:
.export-note { background: #0f172a; border: 1px solid #
 border-radius: .5rem; padding: .85rem 1rem; font-size:
.export-note a { color: #a5b4fc; }
.export-gloss { margin-top: 3rem; border-top: 1px solid
.export-gloss h2 { color: #fff; font-size: 1.25rem; font
.export-term { background: #0f172a; border: 1px solid #
.export-term h3 { color: #c7d2fe; font-size: .95rem; font
.export-term p { margin: 0 0 .3rem; font-size: .85rem;
.export-term .formal { color: #94a3b8; font-size: .8rem;
.export-term .cx { color: #6ee7b7; font-size: .78rem; font
.export-foot { margin-top: 3rem; border-top: 1px solid #
@media print {
 body { background: #fff; color: #0f172a; }
 .export-note, details { break-inside: avoid; }
 details { display: block; }
 details > div, details > p { display: block !important; }
 pre { white-space: pre-wrap; }
}
`;

```

```

 if (!termIds.length) return "";
 const items = termIds
 .map((id) => {
 const e = GLOSSARY_BY_ID.get(id);
 if (!e) return "";
 return `<div class="export-term" id="term-${escapeHtml(
 <h3>${escapeHtml(e.title)}</h3>
 <p>${escapeHtml(e.short)}</p>
 ${e.formal ? `<p class="formal">${escapeHtml(e.formal)}</p>
 ${e.complexity ? `<p class="cx">Сложность: ${escapeHtml(e.complexity)}</p>` : ""}
 </div>`;
 })
 .join("\n");

 return `<section class="export-gloss">
 <h2>Словарик терминов из этой темы</h2>
 <p style="font-size:.8rem;color:#94a3b8;margin:0 0 1rem 0">
 ны сюда.</p>
 ${items}
 </section>`;
}

```

```

function wrap(opts: { title: string; subtitle: string; css: string; }) {
 const stamp = new Date().toLocaleString("ru-RU");
 return `<!doctype html>
<html lang="ru">
<head>
<meta charset="utf-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<title>${escapeHtml(opts.title)}</title>
<style>${opts.css}</style>
<style>${EXTRA_CSS}</style>
</head>
<body>
<div class="export-shell">
 <header class="export-head">
 <div style="font-size:.7rem;text-transform:uppercase">
 <h1 style="color:#fff;font-size:1.6rem;font-weight:bold">

```



```

const ROOT = path.resolve(path.dirname(fileURLToPath(import.meta.url)),
const TMP = path.join(ROOT, "tmp", "audit");

fs.mkdirSync(TMP, { recursive: true });
const bundle = path.join(TMP, "content.bundle.mjs");

await build({
 entryPoints: [path.join(ROOT, "src/data/content.ts")],
 bundle: true,
 format: "esm",
 platform: "node",
 outfile: bundle,
 logLevel: "error",
});

const { chapters } = await import(`file://${bundle}`);

// Список интерактивов берём из реестра визуализаторов
const registry = fs.readFileSync(path.join(ROOT, "src/c
const registryBody = registry.slice(registry.indexOf("V
const vizIds = new Set([...registryBody.matchAll(/^\s{2

const rows = chapters.map((c) => {
 const html = c.content ?? "";
 const analogies = (html.match(/Аналогия/gi) ?? []).length;
 return {
 id: c.id,
 title: c.title,
 num: Number.parseInt(c.title.match(/^(\\d+)\\.\\/)?.[1]
 analogies,
 hasAnalogy: analogies > 0,
 inlineSvg: /<svg[\\s>]/i.test(html),
 image: /<img[\\s>]/i.test(html),
 interactive: vizIds.has(c.id),
 chars: html.length,
 };
});

```

```
/**
 * Проверка покрытия глав всплывающими подсказками:
 * в каждой ли теме находятся термины из глоссария.
 *
 * node scripts/audit-terms.mjs
 */
import { build } from "esbuild";
import path from "node:path";

const ROOT = process.cwd();
await build({
 entryPoints: ["src/data/content.ts", "src/data/glossary.ts"],
 bundle: true, format: "esm", platform: "node",
 outdir: "tmp/audit-terms", outExtension: { ".js": ".mjs" },
 external: ["react", "react-dom", "motion", "lucide-react"],
});

const { chapters } = await import(path.join(ROOT, "tmp/audit-terms.mjs"));
const { GLOSSARY_LABELS, GLOSSARY } = await import(path.join(ROOT, "tmp/audit-terms.mjs"));

const esc = (s) => s.replace(/[\.*\+\?\^\$\{\}\(\)|[\]\[\]]/g, "\\");
const re = new RegExp(
 `(?<![\\p{L}\\p{N}_-])(?=${GLOSSARY_LABELS.map((l) => l.label})` +
 "giu"
);

const map = new Map(GLOSSARY_LABELS.map((l) => [l.label, l]));

// те же лимиты, что и в src/hooks/useTermHints.ts
const MAX_PER_SURFACE = 2;
const MAX_PER_TERM = 8;

const bad = [];
const used = new Set();
console.log("тем | подсв. | глава");
for (const c of chapters) {
 const text = c.content
```

```
/**
 * ШАГ 3: PNG -> PDF
 *
 * Склеивает отрендеренные страницы tmp/export-pages/pages/
 * в единый документ public/export/full_code_all_pages.pdf
 */
import fs from "node:fs";
import path from "node:path";
import PDFDocument from "pdfkit";
import { ROOT, OUT_DIR, PAGES_DIR, PDF_PATH, ensureDir, }

// A4 в пунктах PDF
const A4 = { width: 595.28, height: 841.89 };

async function main() {
 if (!fs.existsSync(PAGES_DIR)) {
 throw new Error(`Нет ${path.relative(ROOT, PAGES_DIR)}`);
 }

 const pages = fs
 .readdirSync(PAGES_DIR)
 .filter((f) => f.endsWith(".png"))
 .sort();

 if (pages.length === 0) throw new Error("Не найдено ни одной страницы");

 ensureDir(OUT_DIR);

 const doc = new PDFDocument({
 size: [A4.width, A4.height],
 margin: 0,
 autoFirstPage: false,
 info: {
 Title: "Универсальное пособие – полный исходный код",
 Author: "CI: rebuild-pdf",
 Subject: "Экспорт всего кода проекта (код -> txt)",
 CreationDate: new Date(),
 },
 },
```

## Универсальное пособие • полный исходный код

```

* Список файлов берётся из git (чтобы ничего не забыть
* с фоллбэком на обход файловой системы, если git недо
*/
```

```
import fs from "node:fs";
import path from "node:path";
import { execFileSync } from "node:child_process";
import { ROOT, OUT_DIR, TXT_PATH, HR, SUBHR, ensureDir,
```

```
/** Расширения, которые считаем «кодом» и включаем в да
const CODE_EXT = new Set([
 ".ts", ".tsx", ".js", ".jsx", ".mjs", ".cjs",
 ".css", ".scss", ".html", ".json", ".md", ".yaml", ".y
]);
```

```
/** Явные исключения: генерируемое, бинарное и просто ш
const EXCLUDE_FILES = new Set(["package-lock.json", "bu
const EXCLUDE_DIRS = ["node_modules", "dist", "build",
```

```
/** Человекочитаемые категории для оглавления. */
const CATEGORIES = [
 [/^src\/data\/tickets\/$/, "Билеты (учебный контент)"],
 [/^src\/data\/$/, "Контент и данные пособия"],
 [/^src\/components\/visualizers\/$/, "Визуализаторы (в
 [/^src\/components\/$/, "Интерактивные компоненты и си
 [/^src\/layout\/$/, "HTML-разметка (layout)"],
 [/^src\/utils\/$/, "Утилиты"],
 [/^src\/$/, "Ядро приложения"],
 [/^scripts\/$/, "Скрипты сборки и экспорта"],
 [/^\.github\/$/, "CI / автоматизация"],
 [/^[^\/]+\$/ , "Конфигурация проекта"],
];
```

```
const categoryOf = (rel) => CATEGORIES.find(([re]) => re
```

```
/** Порядок разделов в документе. */
const ORDER = [
 "Конфигурация проекта",
 "Ядро приложения",
```



```

 .filter((rel) => fs.existsSync(path.join(ROOT, rel))
 .sort((a, b) => {
 const ia = ORDER.indexOf(categoryOf(a));
 const ib = ORDER.indexOf(categoryOf(b));
 return ia !== ib ? ia - ib : a.localeCompare(b);
 }));
}

/** Достаём заголовки глав из данных пособия (без запус
function extractChapters(files) {
 const chapters = [];
 const seen = new Set();
 for (const rel of files.filter((f) => f.startsWith("s
 const src = fs.readFileSync(path.join(ROOT, rel), "
 const re = /id:\s*"([^"]+)"\s*,\s*\n?\s*title:\s*"
 let m;
 while ((m = re.exec(src)) !== null) {
 const [, id, title] = m;
 if (seen.has(id)) continue;
 seen.add(id);
 chapters.push({ id, title, file: rel });
 }
 }
 return chapters.sort((a, b) => {
 const na = parseInt(a.title.match(/(\d+)/)?.[0] ??
 const nb = parseInt(b.title.match(/(\d+)/)?.[0] ??
 return na - nb;
 }));
}

function main() {
 const files = listFiles();
 const chapters = extractChapters(files);
 ensureDir(OUT_DIR);

 const out = [];
 const push = (s = "") => out.push(s);

```

```

current = null;
files.forEach((rel, i) => {
 const cat = categoryOf(rel);
 if (cat !== current) {
 current = cat;
 push(HR);
 push(`РАЗДЕЛ: ${cat.toUpperCase()}`);
 push(HR);
 push();
 }
 const content = fs.readFileSync(path.join(ROOT, rel));
 const lines = content.split("\n");
 push(SUBHR);
 push(`ФАЙЛ ${i + 1}/${files.length}: ${rel}`);
 push(`КАТЕГОРИЯ: ${cat} | СТРОК: ${lines.length} |`);
 push(SUBHR);
 push();
 push(content.replace(/\n+$/, ""));
 push();
 push();
});

push(HR);
push(`КОНЕЦ ДОКУМЕНТА • ${files.length} файлов • сгенерировано`);
push(HR);

const txt = out.join("\n") + "\n";
fs.writeFileSync(TXT_PATH, txt, "utf8");

console.log("[1/3] код -> txt");
console.log(`файлов: ${files.length}`);
console.log(`глав: ${chapters.length}`);
console.log(`строк: ${txt.split("\n").length}`);
console.log(`выход: ${path.relative(ROOT, TXT_PATH)}`);
}

main();

```

```
 /** Максимум символов в строке (далее – жёсткий пере
 cols: 138,
 /** Строк содержимого на странице (+2 служебные: шапка
 rows: 76,
};

export const HR = "=".repeat(PAGE.cols);
export const SUBHR = "-".repeat(PAGE.cols);

export function ensureDir(dir) {
 fs.mkdirSync(dir, { recursive: true });
}

export function humanSize(bytes) {
 if (bytes < 1024) return `${bytes} B`;
 if (bytes < 1024 * 1024) return `${(bytes / 1024).toFixed(2)} KB`;
 return `${(bytes / 1024 / 1024).toFixed(2)} MB`;
}

/** ImageMagick 7 (`magick`) или 6 (`convert`). */
export function imageMagickCmd() {
 for (const cmd of ["magick", "convert"]) {
 try {
 execFileSync(cmd, ["-version"], { stdio: "ignore" });
 return cmd;
 } catch {}
 /* пробуем следующий */
 }
 throw new Error(
 "ImageMagick не найден. Установите его: sudo apt-get install imagemagick
);
}
```

---

```
function paginate(txt) {
 const source = txt.replace(/\r\n/g, "\n").split("\n");
 const pages = [];
 for (let i = 0; i < source.length; i += PAGE.rows) {
 pages.push(source.slice(i, i + PAGE.rows));
 }
 return pages;
}

async function main() {
 if (!fs.existsSync(TXT_PATH)) {
 throw new Error(`Нет ${path.relative(ROOT, TXT_PATH)}`);
 }

 const im = imageMagickCmd();
 const pages = paginate(fs.readFileSync(TXT_PATH, "utf8"));

 fs.rmSync(PAGES_DIR, { recursive: true, force: true });
 ensureDir(PAGES_DIR);

 const total = pages.length;
 const jobs = pages.map((lines, idx) => {
 const num = String(idx + 1).padStart(4, "0");
 const header = `Универсальное пособие • полный исходный код
 Math.max(1, PAGE.cols - 44 - `стр. ${idx + 1} / ${total}`);
 const body = [header, "-".repeat(PAGE.cols), ...lines];
 const txtFile = path.join(PAGES_DIR, `page-${num}.txt`);
 const pngFile = path.join(PAGES_DIR, `page-${num}.png`);
 fs.writeFileSync(txtFile, body + "\n", "utf8");
 return { txtFile, pngFile, num };
 });

 const args = (job) => [
 "-density", String(PAGE.density),
 "-page", PAGE.size,
 "-font", fs.existsSync(PAGE.fontFile) ? PAGE.fontFile : "serif",
 "-pointsize", String(PAGE.pointsize),
```

Универсальное пособие • полный исходный код

```

main().catch((err) => {
 console.error("Ошибка рендера страниц:", err.message)
 process.exit(1);
});
```

=====

РАЗДЕЛ: CI / АВТОМАТИЗАЦИЯ

=====

-----

ФАЙЛ 83/83: .github/workflows/rebuild-pdf.yml

КАТЕГОРИЯ: CI / автоматизация | СТРОК: 128 | РАЗМЕР: 4.0 КБ

-----

name: Rebuild code PDF

# Пайплайн пересборки PDF при каждом коммите:

```

исходный код → full_code_all_pages.txt (script)

|
|→ page-0001.png ... page-NNNN.png (images)

|
|→ full_code_all_page.pdf (pdf)
```

# Готовые TXT и PDF коммитятся обратно в ветку (их отдаёт GitHub Actions)  
# промежуточные PNG сохраняются как artifacts запуска.

on:

push:

branches:

- "\*\*"

paths-ignore:

# чтобы коммит самого workflow не запускал бесконечно

- "public/export/\*\*"

- "\*\*/\*.md"

workflow\_dispatch:

inputs:

- ```
for policy in /etc/ImageMagick-6/policy.xml /
  if [ -f "$policy" ]; then
    sudo sed -i 's/rights="none" pattern="\{P
  fi
done
convert -version
```
- name: Install dependencies
run: npm ci --no-audit --no-fund
 - name: Step 1 – код → txt
run: npm run export:txt
 - name: Step 2 – txt → png
env:
 EXPORT_DENSITY: \${github.event.inputs.densi
run: npm run export:png
 - name: Step 3 – png → pdf
run: npm run export:pdf
 - name: Type-check приложения
run: npx tsc --noEmit
continue-on-error: true
 - name: Summary
run: |
 {
 echo "### Экспорт пересобран"
 echo ""
 echo "| Артефакт | Размер |"
 echo "| --- | --- |"
 echo "| \`public/export/full_code_all_pages
 echo "| \`public/export/full_code_all_pages
 echo "| PNG-страниц | \$(ls tmp/export-pages
 } >> "\$GITHUB_STEP_SUMMARY"
 - name: Upload artifacts (PDF + TXT)